AD14N AD15N AD17N AD18N AC104N

发行版本 *v1.3*

liujie chenzhuohao mawancheng liuhaokun

2023 年 09 月 26 日

文档版本说明:

1 文档版本说明 **3**
- 1.1 v1.3 说明 . . . 3
- 1.2 v1.2 说明 . . . 4
- 1.3 v1.1 说明 . . . 5
- 1.4 v1.0 说明 . . . 5

2 新版 SDK 命名说明 **7**

3 环境介绍 **9**
- 3.1 硬件环境介绍 . . . 9
  - 3.1.1 AD14N AC104N 硬件开发环境 . . . 10
  - 3.1.2 AD15N 硬件开发环境 . . . 12
  - 3.1.3 AD17N 硬件开发环境 . . . 14
  - 3.1.4 AD18N 硬件开发环境 . . . 16
- 3.2 SDK 工程与配置 . . . 18
  - 3.2.1 应用配置文件《app_config.c》介绍 . . . 20
  - 3.2.2 应用配置文件《app_config.h》介绍 . . . 24
  - 3.2.3 应用 APP 配置文件《app_modules.h》介绍 . . . 27
  - 3.2.4 CodeBlocks 工程宏定义介绍 . . . 29
- 3.3 新版本 SDK 工程裁切（2023 年 7 月发布的 SDK 起始） . . . 30
- 3.4 旧版本 SDK 工程裁切 . . . 30
  - 3.4.1 如何将解码格式从工程中移除 . . . 30
  - 3.4.2 如何将压缩格式从工程中移除 . . . 32

4 下载目录 & 工具使用环境介绍 **35**
- 4.1 下载目录介绍 . . . 36
  - 4.1.1 dowload_bat.c 文件介绍 . . . 36
  - 4.1.2 isd_config.ini 文件介绍 . . . 38
- 4.2 开发工具介绍 . . . 41
  - 4.2.1 资源打包工具 . . . 41
  - 4.2.2 音频转换工具 . . . 43
- 4.3 多算法授权简单介绍 . . . 46

5 时钟驱动 **47**
- 5.1 AD14N / AC104N 系统时钟超频说明 . . . 48
- 5.2 内置系统 flash spi 时钟说明 . . . 48
  - 5.2.1 AD14N / AC104N SPI 时钟限制 . . . 49

5.2.2 AD15N SPI 时钟限制 . . . 50
5.2.3 AD17N SPI 时钟限制 . . . 51
5.3 时钟相关函数 . . . 51
5.3.1 函数 void pll_sel(u32 pll_clock, _PLL_DIV pll_div, _PLL_B_DIV pll_b_div) . . . 51
5.3.2 函数 int clk_get(const char *name); . . . 52
5.3.3 函数 int clk_set(const char *name, int clk) . . . 52

6 内置系统 flash 写保护 53
6.1 A0 芯片如何添加写保护参数 . . . 53
6.1.1 flash 写保护工具说明 . . . 53
6.1.2 新增 flash 写保护参数与指令 . . . 54
6.1.3 写保护参数下载说明 . . . 60
6.1.4 相关函数说明 . . . 61
6.2 内封 flash 芯片如何添加写保护参数（旧） . . . 61
6.2.1 配置烧写器获取内封 flash 写保护参数 . . . 61
6.2.2 配置强制下载工具获取内封 flash 写保护参数 . . . 62
6.2.3 相关函数说明 . . . 62
6.3 内封 flash 芯片如何添加写保护参数（新） . . . 62
6.4 程序中开启与关闭写保护功能 . . . 62
6.4.1 相关函数说明 . . . 63

7 中断系统 65
7.1 中断的注册与注销 . . . 66
7.1.1 函数 void HWI_Install(unsigned char index, unsigned int isr, unsigned char priority) . . . 67
7.1.2 函数 void HWI_Uninstall(unsigned char index) . . . 67
7.2 软中断说明 . . . 68
7.2.1 函数 void bit_set_swi(unsigned char index) . . . 69
7.2.2 函数 void bit_clr_swi(unsigned char index) . . . 69
7.3 软件不可屏蔽中断 . . . 69
7.3.1 函数 void _OS_ENTER_CRITICAL(u32 bit_list) . . . 70
7.3.2 函数 void _OS_ENTER_CRITICAL(u32 bit_list_l, u32 bit_list_h) . . . 70
7.3.3 函数 void _OS_EXIT_CRITICAL(void) . . . 70
7.3.4 函数 void vm_isr_response_index_register(u8 index) . . . 70
7.3.5 函数 void vm_isr_response_index_unregister(u8 index) . . . 70
7.3.6 函数 u32 get_vm_isr_response_index_l(void) . . . 71
7.3.7 函数 u32 get_vm_isr_response_index_h(void) . . . 71
7.4 硬件不可屏蔽中断 . . . 71
7.4.1 函数 void irq_unmask_set(u8 index) . . . 71
7.4.2 函数 void irq_unmask_disable(u8 index) . . . 71

8 GPIO 接口说明 73
8.1 DACP，DACN 口设置（AC104/AD14/15/17） . . . 73
8.2 普通 IO 口 (AC104/AD14/15/17/18) . . . 74
8.3 模块 IO 映射 . . . 76
8.3.1 AC104/AD14/AD15 . . . 76
8.3.2 AD17 . . . 77
8.3.3 AD18 . . . 78
8.4 常见问题和注意事项 . . . 78
8.4.1 AC104/AD14/AD15/AD17 生产不良率高 . . . 78
8.4.2 AD18 部分 IO 下拉使用寄存器设置无效 . . . 78

9 设备管理 79
9.1 设备注册表 . . . 80
9.2 设备函数 . . . 83
9.2.1 函数 int devices_init() . . . 83

9.2.2 函数 bool dev_online(const char* name) . . . 83
9.2.3 函数 void* dev_open(const char* name, void* arg) . . . 83
9.2.4 函数 int dev_bulk_read(void* _device, void* buf, u32 offset, u32 len) . . . 84
9.2.5 函数 int dev_bulk_write(void* _device, void* buf, u32 offset, u32 len) . . . 84
9.2.6 函数 int dev_byte_read(void* _device, void* buf, u32 offset, u32 len) . . . 84
9.2.7 函数 int dev_byte_write(void* _device, void* buf, u32 offset, u32 len) . . . 85
9.2.8 函数 int dev_ioctl(void* device, int cmd, u32 arg) . . . 85
9.2.9 函数 int dev_close(void* device) . . . 86

10 文件系统管理 87
10.1 文件系统介绍 . . . 88
10.1.1 sydfs 文件系统 . . . 89
10.1.2 norfs 文件系统 . . . 90
10.1.3 fat/simple_fat 文件系统 . . . 91
10.2 文件系统使用一般流程 . . . 91
10.3 通用文件系统管理函数 . . . 92
10.3.1 函数 void vfs_init(void) . . . 92
10.3.2 函数 u32 vfs_mount(void **ppvfs, void *device, void *type) . . . 92
10.3.3 函数 u32 vfs_openbypath(void *pvfs, void **ppvfile, const char *path) . . . 93
10.3.4 函数 u32 vfs_openbyindex(void *pvfs, void **ppvfile, u32 index) . . . 94
10.3.5 函数 u32 vfs_createfile(void *pvfs, void **ppvfile, u32 *pindex) . . . 94
10.3.6 函数 u32 vfs_read(void *pvfile, void *buf, u32 len) . . . 95
10.3.7 函数 u32 vfs_write(void *pvfile, void *buf, u32 len) . . . 95
10.3.8 函数 u32 vfs_seek(void *pvfile, u32 offset, u32 mode) . . . 95
10.3.9 函数 u32 vfs_file_close(void **ppvfile) . . . 96
10.3.10 函数 u32 vfs_fs_close(void **ppvfs) . . . 96
10.3.11 函数 u32 vfs_file_name(void *pvfile, void *name, u32 len) . . . 97
10.3.12 函数 int vfs_get_attrs(void *pvfile, void *pvfs_attr) . . . 97
10.3.13 函数 int vfs_ioctl(void *pvfile, int cmd, int arg) . . . 97
10.3.14 函数 int vfs_file_crc(void *pvfile) . . . 98
10.4 FAT 文件系统管理函数 . . . 98
10.4.1 函数 int vfs_ftell(void *pvfile, void *parm) . . . 99
10.4.2 函数 int vfs_get_fsize(void *pvfile, void *parm) . . . 99
10.4.3 函数 struct vfscan *vfs_fscan_new(void *pvfs, const char *path, const char *arg, u8 max_deepth, int (*callback)(void), strcut vfscan *fsn, struct vfscan_reset_info *info) . . . 99
10.4.4 函数 void vfs_fscan_release(void *pvfs, struct vfscan *fs) . . . 100
10.4.5 函数 int vfs_get_folderinfo(void **pvfile, struct vfscan *fs, int *start_num, int *end_num) . . . 100
10.4.6 函数 int vfs_mk_dir(void *pvfs, char *folder, u8 mode) . . . 100
10.4.7 函数 int vfs_get_encfolder_info(void *pvfs, char *folder, char *ext, u32 *last_num, u32 *total_num) . . . 100
10.4.8 函数 u32 vfs_file_delete(void *pvfile) . . . 101
10.4.9 函数 int vfs_select(void *pvfs, void **ppvfile, struct vfscan *fs, int sel_mode, int arg) . . . 101

11 解码接口 103
11.1 解码使用一般流程 . . . 104
11.2 数据流获取 . . . 104
11.3 解码管理接口函数 . . . 105
11.3.1 函数 dec_obj *decoder_io(void *pfile, u32 dec_ctl, dp_buff * dbuff,u8 loop) . . . 105
11.3.2 函数 void irq_decoder_ret(dec_obj *obj,u32 ret) . . . 106
11.4 解码功能控制函数 . . . 107
11.4.1 函数 void decoder_pause(dec_obj *obj) . . . 107
11.4.2 函数 void decoder_stop(dec_obj *obj, DEC_STOP_WAIT wait) . . . 108
11.5 各个格式的解码函数 . . . 108
11.5.1 统一介绍 decode_api . . . 108

12 F1X 格式解码流程说明 **109**
12.1 简介 . . . 109
12.2 结构体 / 函数说明 . . . 109
12.2.1 结构体 f1x_ctl . . . 109
12.2.2 函数 void *f1x_play_api(void *pfile, u8 *loop_tab, u32 size, u8 index, u32 addr) . . . 110
12.2.3 函数 int f1x_parsing(void *pfile, u32 *buff, u32 sum) . . . 110
12.2.4 函数 u32 set_play_file(void *priv, u32 *startpos, u32 *endpos) . . . 110
12.3 解码流程 . . . 110

13 编码接口 **113**
13.1 编码应用流程 . . . 114
13.2 编码功能控制接口函数 . . . 115
13.2.1 void encoder_io(u32(* fun)(void*), void* pfile) . . . 115
13.2.2 void stop_encode(void* pfile, u32 dlen) . . . 115

14 JL_MIO 功能 **117**
14.1 JLmio 一般流程 . . . 117
14.1.1 使用 JLmio.exe 创建 mio 文件 . . . 117
14.1.2 使用打包工具将音频文件与所制成的 mio 文件放于同一目录下 . . . 118
14.1.3 在 download.bat 中添加相应 mio 文件夹 . . . 118
14.1.4 在 app_music.c 中添加相应 mio 文件夹 . . . 119
14.1.5 在 mio_api.c 中将相应宏开关打开 . . . 120
14.2 JLmio 接口介绍 . . . 121
14.2.1 void mio_a_pwm_init(u32 chl) . . . 121
14.2.2 void mio_a_pwm_run(u32 chl，u32 pwm_var) . . . 122
14.2.3 void mio_a_io_init(u32 mask) . . . 122
14.2.4 void mio_a_io_run(u32 mask，u32 io_ver) . . . 122

15 玩具应用篇（新工程 - voice_toy） **123**
15.1 简介 . . . 123
15.1.1 模式相关函数 . . . 124
15.2 toy_music 多路解码模式 . . . 124
15.2.1 模式说明 . . . 124
15.2.2 解码播放函数 . . . 125
15.3 toy_midi midi 解码模式 . . . 127
15.3.1 模式说明 . . . 127
15.3.2 相关函数 . . . 127
15.4 toy_midi_keyboard midi 琴模式 . . . 128
15.4.1 模式说明 . . . 128
15.4.2 相关函数 . . . 128
15.5 toy_speaker 扩音模式 . . . 131
15.5.1 模式说明 . . . 131
15.5.2 相关函数 . . . 131
15.6 toy_linein linein 模式 . . . 131
15.6.1 模式说明 . . . 131
15.7 toy_record 录音模式 . . . 131
15.7.1 模式说明 . . . 131
15.7.2 相关函数 . . . 132

16 玩具应用篇（旧工程 - app_mg） **133**
16.1 简介 . . . 133
16.2 模式初始化函数 . . . 134
16.2.1 函数 int app_mg_init(APP_T app,void* param) . . . 134
16.2.2 函数 void app(void) . . . 134
16.3 模式切换函数 . . . 135

16.3.1 函数 int app_switch(APP_T app,void* param) . . . 135
16.3.2 函数 int app_switch_next(void) . . . 135
16.3.3 函数 int app_switch_priv(void) . . . 135
16.3.4 函数 u8 app_get(void) . . . 135
16.4 常用工程配置文件 . . . 135
16.4.1 “app_config.c”文件 . . . 135
16.4.2 函数 u8 app_get(void) . . . 135
16.4.3 code block 工程配置 . . . 136

17 小音箱应用 137

18 AUDIO DAC 139
18.1 AUDIO DAC 配置参数 . . . 140
18.1.1 AD14N / AD15N / AC104N DAC 驱动模式介绍 . . . 140
18.1.2 AD17N APA 驱动模式介绍 . . . 142
18.1.3 AD18N AUDIO 驱动模式介绍 . . . 144
18.2 AUDIO DAC 接口介绍 . . . 148
18.2.1 函数 void dac_mode_init(void) . . . 148
18.2.2 函数 void dac_init_api(u32 sr) . . . 148
18.2.3 函数 void dac_power_on(u32 sr, bool delay_flag) . . . 148
18.2.4 函数 void dac_off_api(void) . . . 148
18.2.5 函数 bool dac_mute(bool mute) . . . 149
18.2.6 函数 u8 dac_vol(char set, u8 vol) . . . 149
18.2.7 函数 bool regist_dac_channel(void *psound, void *kick) . . . 149
18.2.8 函数 bool unregist_dac_channel(void *psound) . . . 149
18.2.9 函数 void fill_dac_fill(u8 *buf, u32 len) . . . 149
18.2.10 函数 void single_apa_startup_mute_cb(void) . . . 150
18.2.11 函数 void single_apa_mute(u8 mute) . . . 150

19 AUDIO ADC（AD14N / AC104N） 151
19.1 1 AUDIO ADC 接口介绍 . . . 151
19.1.1 1.1 函数 void audio_adc_mode_init(void) . . . 151
19.1.2 1.2 函数 u32 audio_adc_init_api(u32 sr, ADC_MODE mode, u32 ch) . . . 152
19.1.3 1.3 函数 void audio_adc_off_api(void) . . . 152
19.1.4 1.4 函数 bool regist_audio_adc_channel(void *psound, void *kick) . . . 152
19.1.5 1.5 函数 bool unregist_audio_adc_channel(void *psound) . . . 152
19.1.6 1.6 函数 void fill_audio_adc_fill(u8 *buf, u32 len) . . . 152

20 音量 155
20.1 底层音量接口 . . . 155
20.1.1 函数 void dac_mode_init(u16 vol, void *phy_vol_set_func) . . . 156
20.1.2 函数 u8 dac_vol(char set,u8 vol) . . . 156
20.2 音量应用介绍 . . . 156

21 混响（echo） 159
21.1 1 配置参数说明 . . . 159
21.2 2 相关函数 . . . 160
21.2.1 2.1 函数 void *link_echo_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr) . . . 160
21.2.2 2.2 函数 void *echo_api(void *obuf, u32 sr, void **ppsound) . . . 160

22 移频啸叫抑制（pitchshifter_howling） 161
22.1 1 配置参数说明 . . . 161
22.2 2 相关函数 . . . 162

22.2.1 2.1 函数 void *link_pitchshift_howling_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr) . . . 162
22.2.2 2.2 函数 void *pitchshift_howling_api(void *obuf, u32 sr, void **ppsound) . . . 162

23 自适应陷波啸叫抑制（notch_howling） 163
23.1 1 配置参数说明 . . . 163
23.2 2 相关函数 . . . 164
23.2.1 2.1 函数 void *link_notch_howling_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr) . . . 164
23.2.2 2.2 函数 void *notch_howling_api(void *obuf, u32 sr, void **ppsound) . . . 164

24 均衡器（PCM_EQ） 165
24.1 1 配置参数说明 . . . 165
24.2 2 相关函数 . . . 166
24.2.1 2.1 函数 void *link_pcm_eq_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr, u32 channel) . . . 166
24.2.2 2.2 函数 void *pcm_eq_api(void *obuf, u32 sr, u32 channel, void **ppsound) . . . 166
24.3 3 EQ 工具使用 . . . 166
24.3.1 3.1 EQ 工具使用介绍 . . . 166
24.3.2 3.2 SDK 配置说明 . . . 168

25 变音音效（vo_pitch） 171
25.1 可选音效 . . . 171
25.2 配置参数说明 . . . 172
25.3 相关结构体及函数 . . . 172
25.3.1 结构体 VOICE_PITCH_PARA_STRUCT . . . 172
25.3.2 函数 void *link_voice_pitch_sound(sound_out_obj *p_curr_sound, void *p_dac_cbuf, void **pp_effect, u32 cmd) . . . 173
25.3.3 函数 void *voice_pitch_api(void *obuf, u32 cmd, void **ppsound) . . . 173
25.3.4 函数 void rap_callback(void *priv, int pos) . . . 173
25.3.5 函数 void rap_reopen(void) . . . 173
25.4 RAP 和 RAP_REALTIME 模式文件替换说明 . . . 173

26 混合变音音效（voicechanger） 177
26.1 可选音效 . . . 177
26.2 配置参数说明 . . . 178
26.3 相关函数接口 . . . 178
26.3.1 函数 void *link_voice_changer_sound(void *p_sound_out, void p_dac_cbuf, void **pp_effect, u32 in_sr) . . . 178
26.3.2 函数 void *voice_changer_api(void *obuf, u32 sr, void **ppsound) . . . 179
26.3.3 函数 void update_voice_changer_parm(VOICECHANGER_AV_PARM *new_vc_parm, VOICESYN_AV_PARM *new_vsyn_ctrol) . . . 179

27 变速变调（speed） 181
27.1 配置参数说明 . . . 181
27.2 相关函数 . . . 182
27.2.1 函数 void *speed_api(void *obuf, u32 insample, void **ppsound) . . . 182
27.2.2 函数 int music_play_speed_set_para(music_play_obj *hdl, SPEED_PITCH_PARA_STRCUT *sp_parm) . . . 182

28 软关机 Soft off 和低功耗 Power down 183
28.1 关机与唤醒相关参数配置 . . . 184
28.1.1 A0 芯片 flash 使用 VDDIO 供电配置 . . . 184
28.1.2 VDDIO 电压档位配置 . . . 184
28.1.3 低功耗唤醒源配置 . . . 185

28.2 Soft off 及 Power down 相关函数 . . . 188
28.2.1 函数 void sys_softoff() . . . 188
28.2.2 函数 void sys_power_down(u32 usec) . . . 188
28.2.3 函数 void board_set_soft_poweroff(void) . . . 188
28.3 Soft off 及 Power down 唤醒源获取及其他函数 . . . 189
28.3.1 AD14N / AD15N / AC104N 相关接口 . . . 189
28.3.2 AD17N 与 AD18N 相关接口 . . . 190

29 VDDIO 电压配置说明 **191**
29.1 VDDIO 作用域 . . . 191
29.2 如何配置每个阶段的 VDDIO 档位 . . . 191
29.2.1 上电过程： . . . 191
29.2.2 Boot-Loader： . . . 192
29.2.3 SDK： . . . 192
29.3 常见问题介绍 . . . 193
29.3.1 VDDIO 偏差 . . . 193

30 VM 掉电存储说明 **195**
30.1 VM 掉电存储作用和数据存储 . . . 195
30.2 VM 掉电存储使用说明 . . . 196
30.3 VM 掉电存储版本简单说明 . . . 196
30.4 新旧 VM 文件组成及版本选择 . . . 197
30.5 VM 相关参数介绍 . . . 199
30.5.1 ID 号（vm_api.h） . . . 200
30.5.2 读/写入数据长度 . . . 200
30.5.3 vm 存储区域大小 . . . 201
30.5.4 新版 VM 格式整理时缓存空间设置 new_vm_buff（new_vm.h） . . . 201
30.5.5 新版 vm 格式整理后是否立即擦除旧半区区域功能 . . . 202
30.5.6 新版 vm 是否支持多段读功能 . . . 202
30.6 VM 掉电记忆存储接口 . . . 203
30.6.1 函数 u32 syscfg_vm_init(u32 addr, u32 size) . . . 203
30.6.2 函数 u32 vm_write(u32 id,u8 *buf,u32 len) . . . 203
30.6.3 函数 u32 vm_read(u32 id ,u8 *buf,u32 len) . . . 203
30.6.4 函数 u32 vm_pre_erase(void) . . . 204
30.6.5 函数 nvm_pre_erasure_next(NEW_VM_OBJ *p_nvm, u16 using_next, u16 idle_next) . . . 204
30.7 VM 常见问题介绍 . . . 204
30.7.1 VM 简单使用 demo： . . . 204
30.7.2 调整 VM 区域大小设置： . . . 205
30.7.3 新版 VM 在关机或者掉电后可能存在数据丢失 BUG： . . . 205
30.7.4 VM 格式整理原理： . . . 206
30.7.5 VM 卡音问题原因和解决办法： . . . 206

31 消息与事件 **207**
31.1 1 消息与事件使用一般流程 . . . 208
31.2 2 消息与事件函数 . . . 209
31.2.1 2.1 void event_init(void) . . . 209
31.2.2 2.2 void message_init() . . . 209
31.2.3 2.3 u32 event2msg_api(u32 event) . . . 209
31.2.4 2.4 int post_event(int event) . . . 210
31.2.5 2.5 int post_msg(int argc, …) . . . 211
31.2.6 2.6 bool get_event_status(u32 event) . . . 212
31.2.7 2.7 函数 int get_msg(int len, int *msg) . . . 213
31.2.8 2.8 函数 void clear_all_message(void) . . . 213

32 按键系统 key **215**

32.1 按键类型介绍 . . . 216
32.1.1 IO 按键 . . . 216
32.1.2 AD 按键 . . . 217
32.1.3 IR 按键 . . . 218
32.1.4 MATRIX 按键 . . . 218
32.1.5 touch 按键： . . . 218
32.2 按键接口介绍： . . . 221
32.2.1 函数 void key_init(void) . . . 221
32.2.2 函数 void key_table_sel(void *msg_filter) . . . 221
32.2.3 函数 void key_scan(void) . . . 221

33 Reserved_Area 区域说明 223
33.1 1 认证码使用说明 . . . 223
33.2 2 认证码文件说明 . . . 224

34 多录音分区（AD14N） 225
34.1 1 多录音分区一般流程 . . . 225
34.1.1 1.1 创建录音分区初始化 . . . 226
34.1.2 1.2 创建录音切换分区消息事件并设置键位 . . . 227
34.1.3 1.3 在 app 录音任务中实现录音分区切换操作 . . . 228

35 虚拟 RTC 229
35.1 功能接口介绍 . . . 229
35.1.1 int rtc_simulate_init(const struct vir_rtc_data *arg) . . . 229
35.1.2 void read_sys_time(struct sys_time *curr_time); . . . 229
35.1.3 void write_sys_time(struct sys_time *curr_time); . . . 230
35.1.4 void read_alarm(struct sys_time *alarm_time); . . . 230
35.1.5 void write_alarm(struct sys_time *alarm_time); . . . 230
35.1.6 void vir_set_alarm_ctrl(u8 set_alarm); . . . 230
35.1.7 u8 vir_get_alarm_ctrl(void); . . . 230

36 双备份升级 231
36.1 支持双备份升级的芯片以及对应 SDK 版本 . . . 231
36.2 SDK 相关配置 . . . 231
36.3 相关接口说明 . . . 232
36.3.1 函数 u32 jlfs_get_idle_bank_info(u32 *bank_addr, u32 *bank_size) . . . 232
36.3.2 函数 u32 jlfs_updata_dual_bank_info(u32 bank_addr, u16 data_crc) . . . 233
36.3.3 函数 u32 jlfs_check_dual_bank_info(u32 update_bank_addr) . . . 233
36.4 常见客户问题 . . . 233

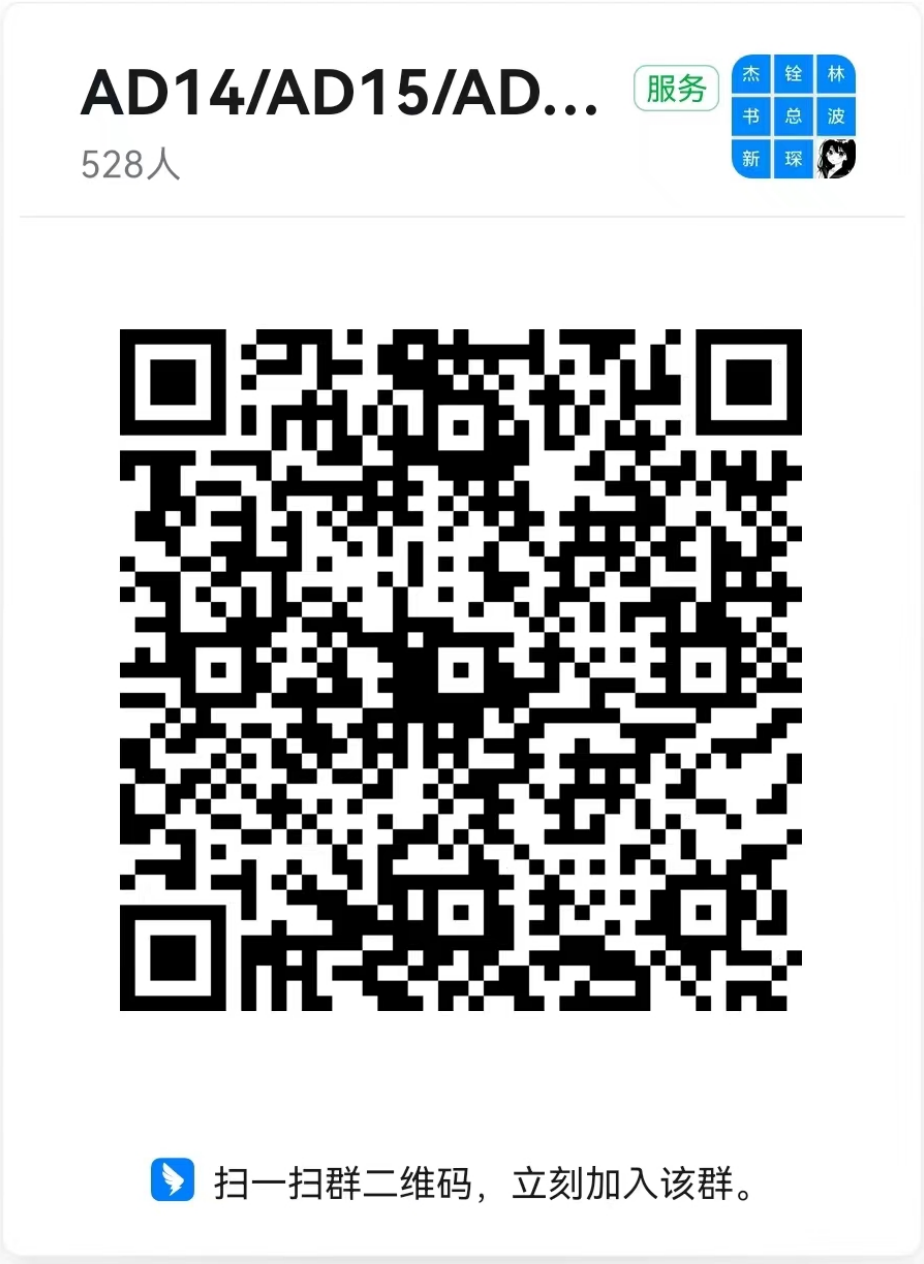
37 Indices and tables 235

开源社区仓库链接：

https://gitee.com/Jieli-Tech/fw-AD15N

https://github.com/Jieli-Tech/fw-AD15N

开源社区交流群：



开源社区钉钉群二维码

作者：刘杰、陈卓豪、马万程、刘昊坤

CHAPTER 1

文档版本说明

本页介绍历史版本

1.1 v1.3 说明

日期：2023 年 9 月 11 日

对应 SDK 版本：AD1x-4578_AC104-release_v1.8.0

1、新增 AD18N 芯片相关内容：

- 添加环境介绍-> “*AD18N 硬件开发环境*”；
- 更新 SDK 工程与配置说明-> “*SDK 工程与配置*”；
- 更新外挂资源 flash 下载说明-> “*外挂资源 flash 下载*”；
- 更新 isd_config.ini 配置说明-> “*isd_config.ini 文件介绍*”；
- 更新 AUDIO_DAC 相关说明-> “*AUDIO DAC*”；
- 更新软关机和低功耗相关说明-> “*软关机 Soft off 和低功耗 Power down*”；
- 更新内封 flash 写保护说明（使用了新策略）-> “*内封 flash 芯片如何添加写保护参数（新）*”；

2、新增章节说明：

- 新增双备份升级相关内容介绍-> “*双备份升级*”；
- 新增了中断说明-> “*中断系统*”；
- 新增变音音效 (vo_pitch)rap 文件替换说明-> “*RAP 和 RAP_REALTIME 模式文件替换说明*”；
- 新增 GPIO 接口章节-> “*GPIO 接口说明*”；

- 新增 F1X 解码格式说明-> “*F1X 格式解码流程说明*”;
- 新增 VDDIO 章节说明-> “*VDDIO 电压配置说明*”;

3、更新章节说明：

- 修改虚拟 RTC 章节说明，概述添加注意事项-> “*虚拟 RTC*”;
- 新增各芯片启动时间指标-> “*软关机 Soft off 和低功耗 Power down*”;
- 新增 AD17N 单端推功放原理图-> “*AD17N APA 驱动模式介绍*”;
- 删除 AD15N 单端推功放原理图，该方式底噪较大;
- 更新 flash 单双线原理图以及软件配置-> “*isd_config.ini 文件介绍*”;
- 更新 VM 常见问题介绍-> “*VM 常见问题介绍*”;
- 更新 speed 变速变调参数限制说明-> “*变速变调（speed）*”;

1.2 v1.2 说明

日期：2023 年 7 月 11 日

对应 SDK 版本：AD1x-457_AC104-release_v1.7.0

1、新增 AD17N 芯片相关内容：

- 添加环境介绍->” *AD17N 硬件开发环境* “
- 更新 SDK 工程与配置说明->” *SDK 工程与配置* “;
- 添加应用配置文件《app_moduels.h》说明->” *应用 APP 配置文件《app_modules.h》介绍* “;
- 更新工程裁切说明->” *新版本 SDK 工程裁切（2023 年 7 月发布的 SDK 起始）* “;
- 更新外挂资源 flash 下载说明->” *外挂资源 flash 下载* “;
- 更新 isd_config.ini 配置说明->” *isd_config.ini 文件介绍* “;
- 更新 AUDIO_DAC 相关说明->” *AUDIO DAC* “;
- 更新软关机和低功耗相关说明->” *软关机 Soft off 和低功耗 Power down* “;

2、新增发声玩具应用工程 voice_toy 相关内容->” *玩具应用篇（新工程 - voice_toy）* “;

3、新增 new vm 相关内容->” *VM 掉电存储说明* “;

4、新增 speed 变速变调说明->” *变速变调（speed）* “;

5、新增 clock 时钟说明->” *时钟驱动* “;

6、新增虚拟 RTC 章节说明-> “*虚拟 RTC*” ;

7、新增内置系统 flash 写保护说明-> “*内置系统 flash 写保护*” ;

1.3 v1.1 说明

日期：2023 年 3 月 30 日

1、将 MP3、WAV 的说明加入到《解码接口》的章节中

1.4 v1.0 说明

日期：2023 年 3 月 18 日

建立文档，文档内容包括:

```
环境介绍
下载目录&工具使用环境介绍
设备管理
文件系统管理
解码接口
编码接口
JL_MIO功能
玩具应用篇
小音箱应用篇
AUDIO DAC
AUDIO ADC (AD14N)
音量
混响 (echo)
移频啸叫抑制 (pitchshifter_howling)
自适应陷波啸叫抑制 (notch_howling)
均衡器 (PCM_EQ)
变音音效 (vo_pitch)
vm虚拟存储
消息与事件
按键系统key
Reserved_Area区域说明
多录音分区 (AD14N)
软关机Soft off和低功耗Power down
```

CHAPTER 2

新版 SDK 命名说明

2023 年 9 月发布的 SDK 整合了 AD14N / AC104N / AD15N / AD17N / AD18N 的工程，SDK 命名为 *AD1x-4578_AC104-release_v1.8.0* ，其含义为：

- AD1x-4578：代表 AD14、AD15、AD17、AD18N
- AC104：代表 AC104
- v1.8.0：表示当前 SDK 版本号



新版本 SDK 名称说明

CHAPTER 3

环境介绍

本章节主要介绍软硬件开发环境。

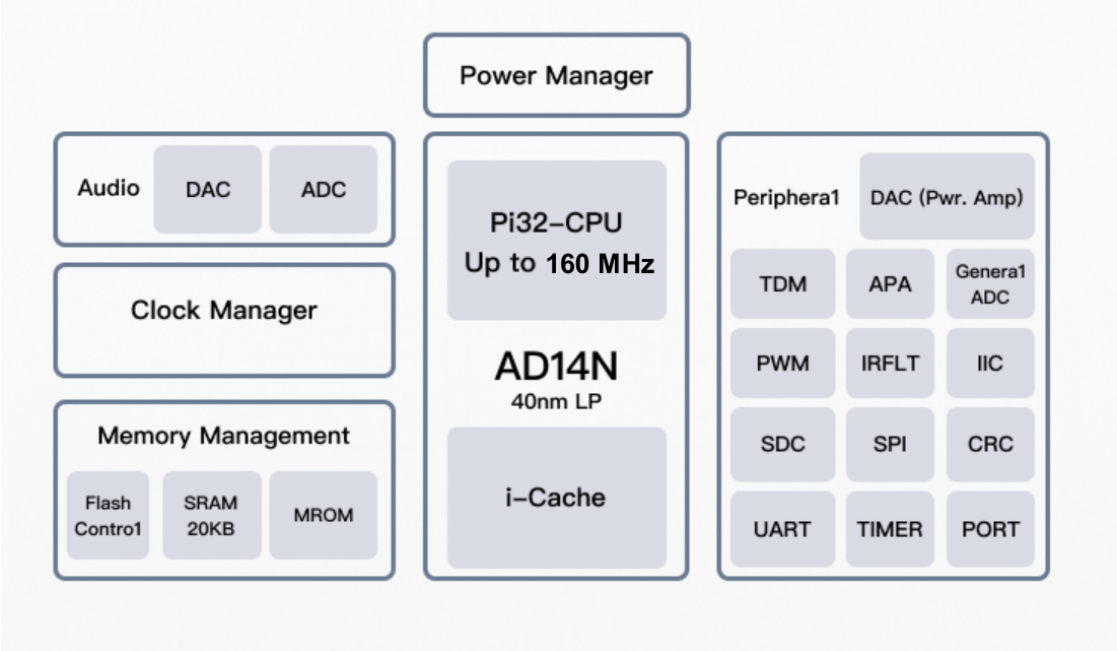
3.1 硬件环境介绍

AD 系列硬件开发环境使用不同的开发板，包括不同底板和顶板。以下芯片适用于 AD 系列在线文档说明。

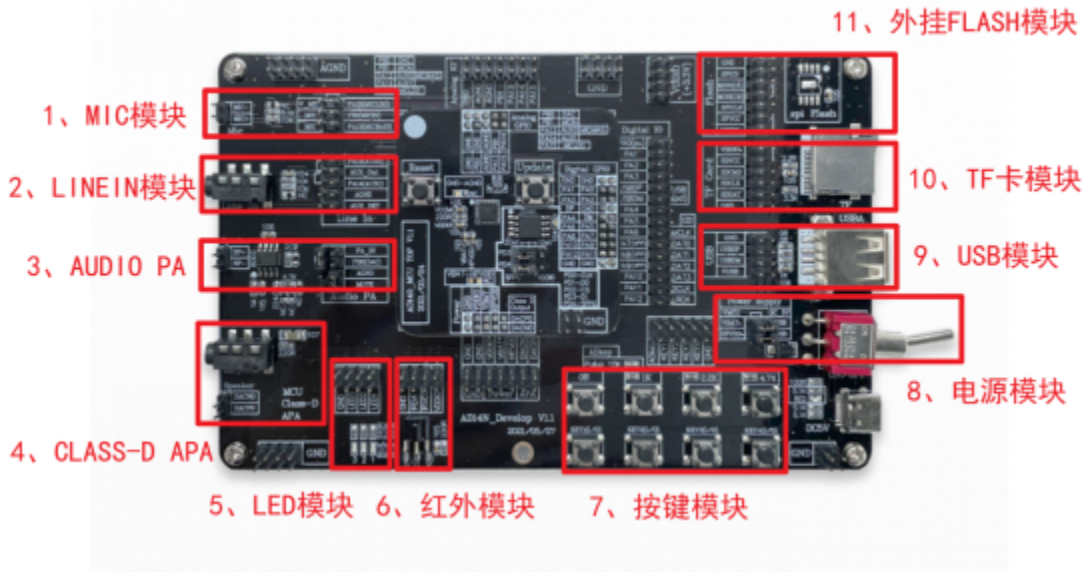
适用本文的芯片	AD14N / AC104N	AD15N	AD17N	AD18N

3.1.1 AD14N AC104N 硬件开发环境

AD14N 芯片框图如下图所示：



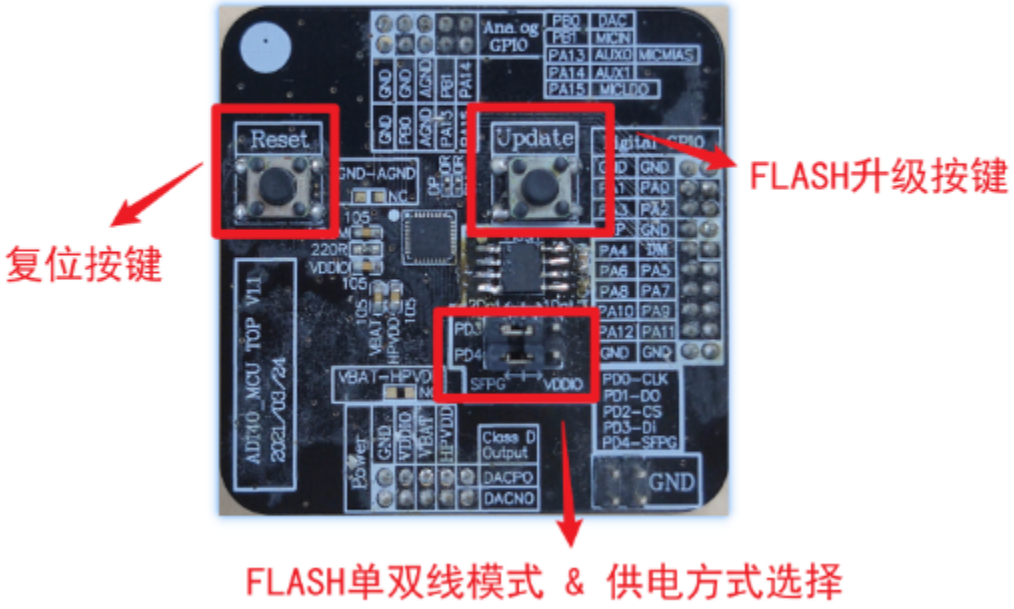
AD14N，AC104N 使用的硬件环境开发板由底板和顶板两部分组成。
底板包含的模块如下图所示：



其中模块：

模块	模块详情
MIC 模块	MIC 输入引脚固定为 PB01，偏置电压可由 MICLDO（PA15）或 MICBIAS（PA13）提供，也可以使用外部偏置，不占用芯片 I/O 口；
LINEIN 模块	LINEIN 输入引脚可选 AUX0（PA13）或 AUX1（PA14），需将对应引脚与 AUX_OUT 短接；
Audio PA 功放模块	单端 DAC 输出引脚为 PB0，需将 PA_IN 与 PB0 短接；
Class-D APA 模块	差分 DAC 输出引脚为 DACN 和 DACP；
LED 模块	该模块的 3 个 LED 灯都是高电平点亮；
红外模块	该模块用于接收红外信号，需将 IRVDD 与 VDDIO 短接；
按键模块	该模块有 4 个 AD 按键以及 4 个 I/O 按键，其中 AD 按键外接 10k 上拉电阻；
电源模块	供电可通过正极接“VUSB”，负极接“GND”实现，且需要将 VBAT 和 VUSB 短接、HPVDD 和 VBAT 短接；
USB 模块	该模块可以用来通信 USB 设备，或给开发板下载程序以及供电；
TF 卡模块	该模块支持驱动一线模式的 TF 卡；
外挂 FLASH 模块	该模块可挂载一颗存放资源的 FLASH；

顶板包含的模块如下图所示：



顶板的“FLASH 工作模式以及供电方式选择区”可以选择 FLASH 是跑 1 线模式还是 2 线模式，并且可以选择 POWER 引脚供电使用 VDDIO 还是 PD4 引脚。

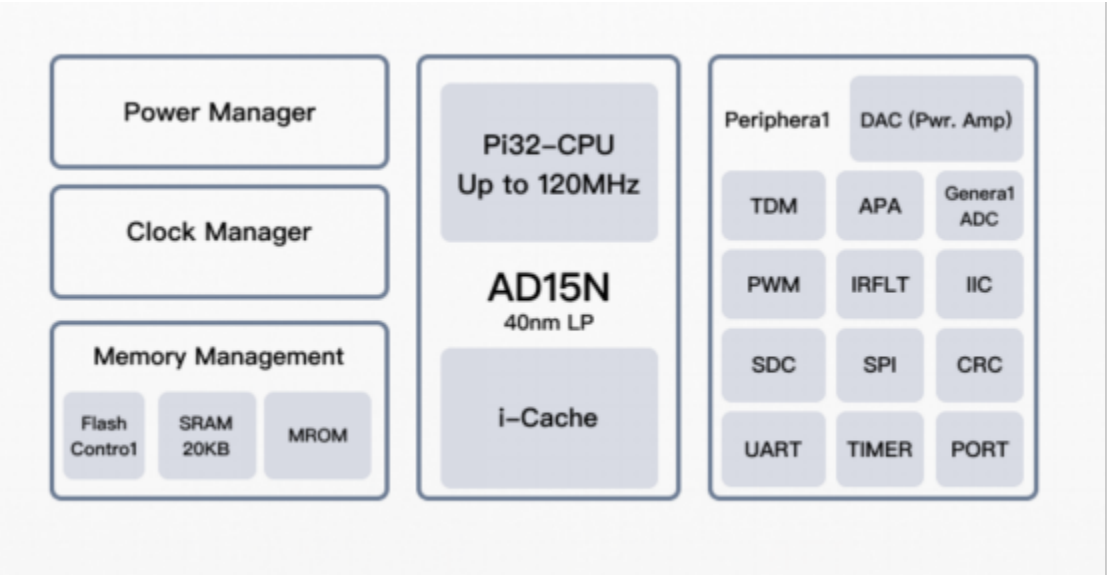
系统 flash 单双线配置及 flash 电源配置：

使用 USB 升级 FLASH 的时候，插入 USB 线连电脑和开发板，按住“FLASH 短路按键”不松手

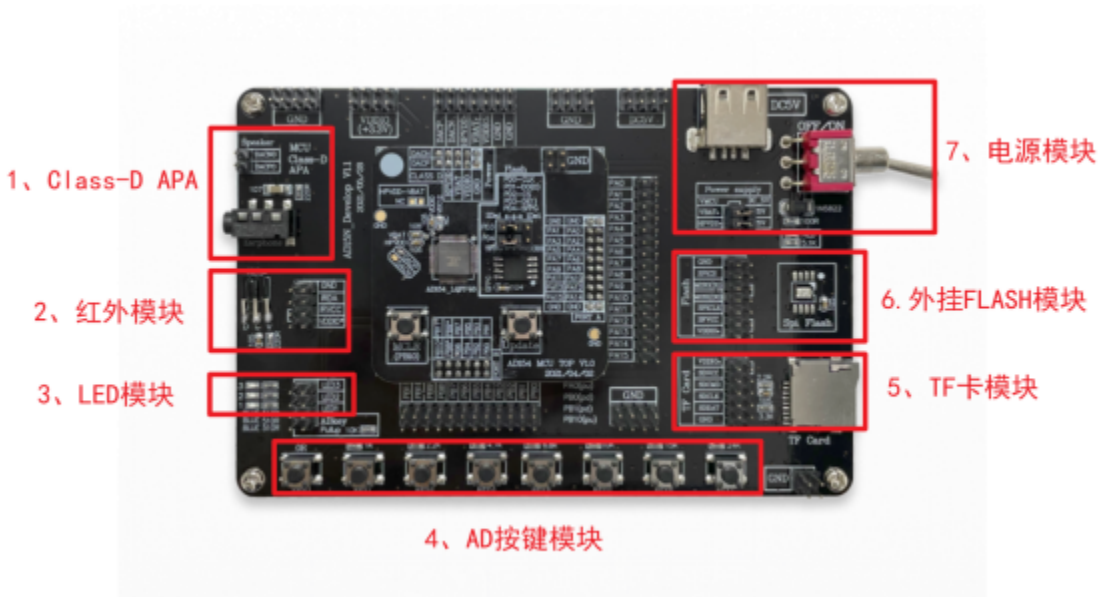
之后再短按“复位按键”复位系统，之后松开“FLASH 短路按键”就可以进入升级，此时电脑可以正常出现设备盘符即是操作成功了。

3.1.2 AD15N 硬件开发环境

AD15N 芯片框图如下图所示：



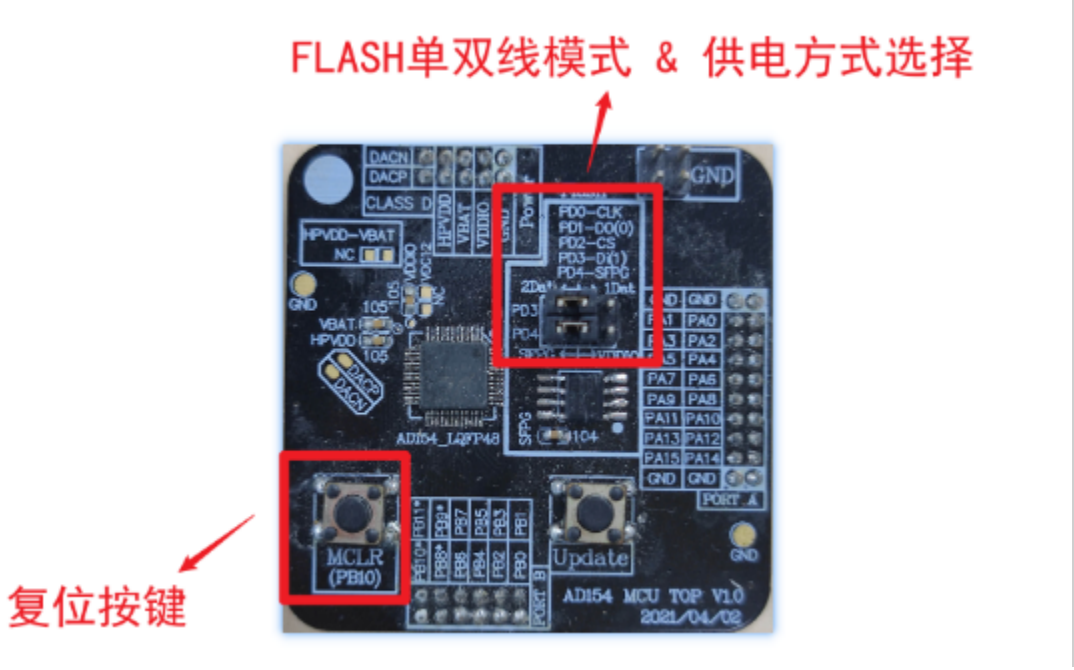
AD15N 使用的硬件环境开发板由底板和顶板两部分组成。
底板包含的模块如下图所示：



其中模块：

模块	模块详情
Class-D APA 模块	差分 DAC 输出引脚为 DACN 和 DACP;
红外模块	该模块用于接收红外信号，需将 IRVDD 与 VDDIO 短接;
LED 模块	该模块的 3 个 LED 灯都是高电平点亮;
按键模块	该模块有 8 个 AD 按键，外接 10k 上拉电阻;
TF 卡模块	该模块支持驱动一线模式的 TF 卡;
外挂 FLASH 模块	该模块可挂载一颗存放资源的 FLASH;
电源模块	AD15N USB 口只用来供电，芯片自身并没有 USB 模块。特别地，AD150 开发烧写使用串口升级方式，供电使用 USB UPDATER 插入 USB 口供电，并且 USB UPDATER TX 接到开发板的 PB9。

顶板包含的模块如下图所示:

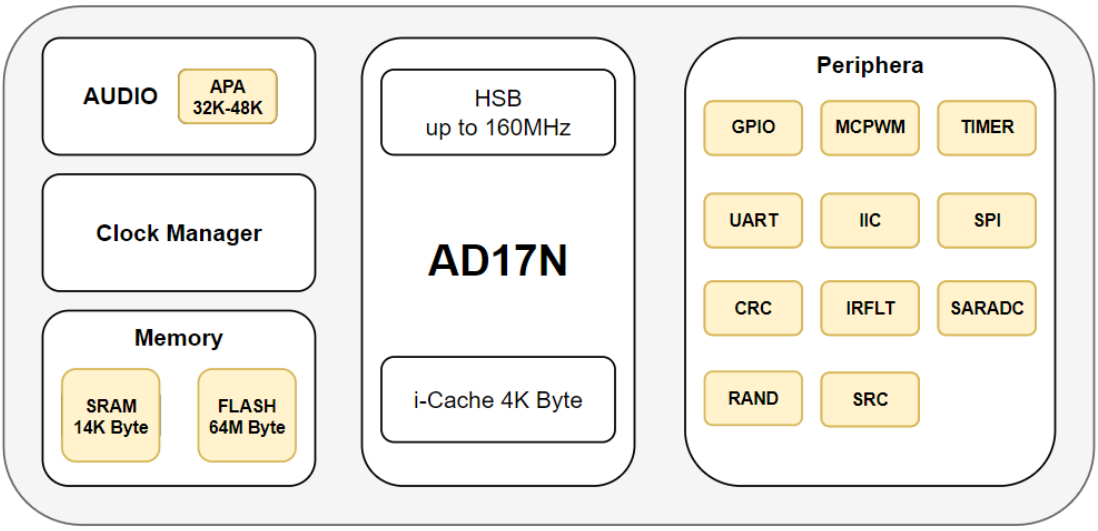


AD15N 使用串口方式升级，升级时候并不需要按着 Update，只需要点击运行升级脚本，USB UPDATER 就会让开发板自动进入升级模式了。

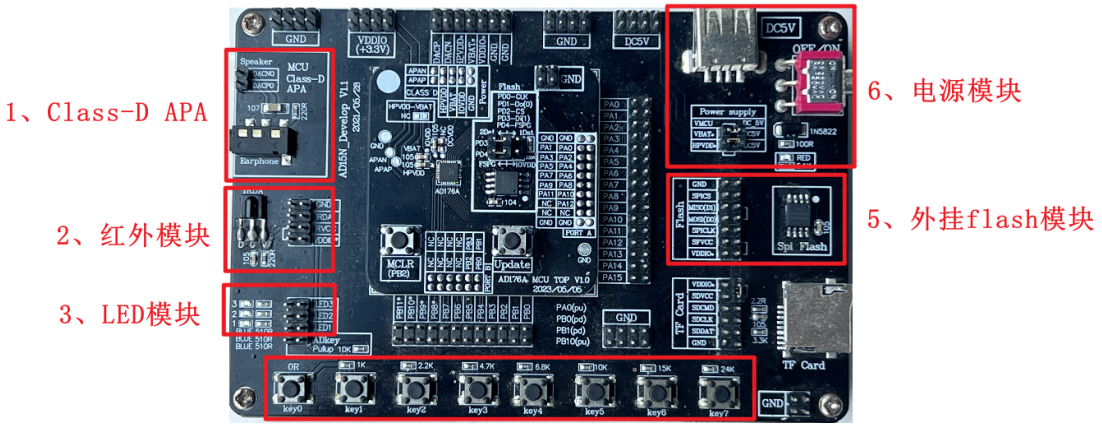
顶板的“FLASH 工作模式以及供电方式选择区”可以选择 FLASH 是跑 1 线模式还是 2 线模式，并且可以选择 POWER 引脚供电使用 VDDIO 还是 PD4 引脚。

3.1.3 AD17N 硬件开发环境

AD17N 芯片框图如下图所示：



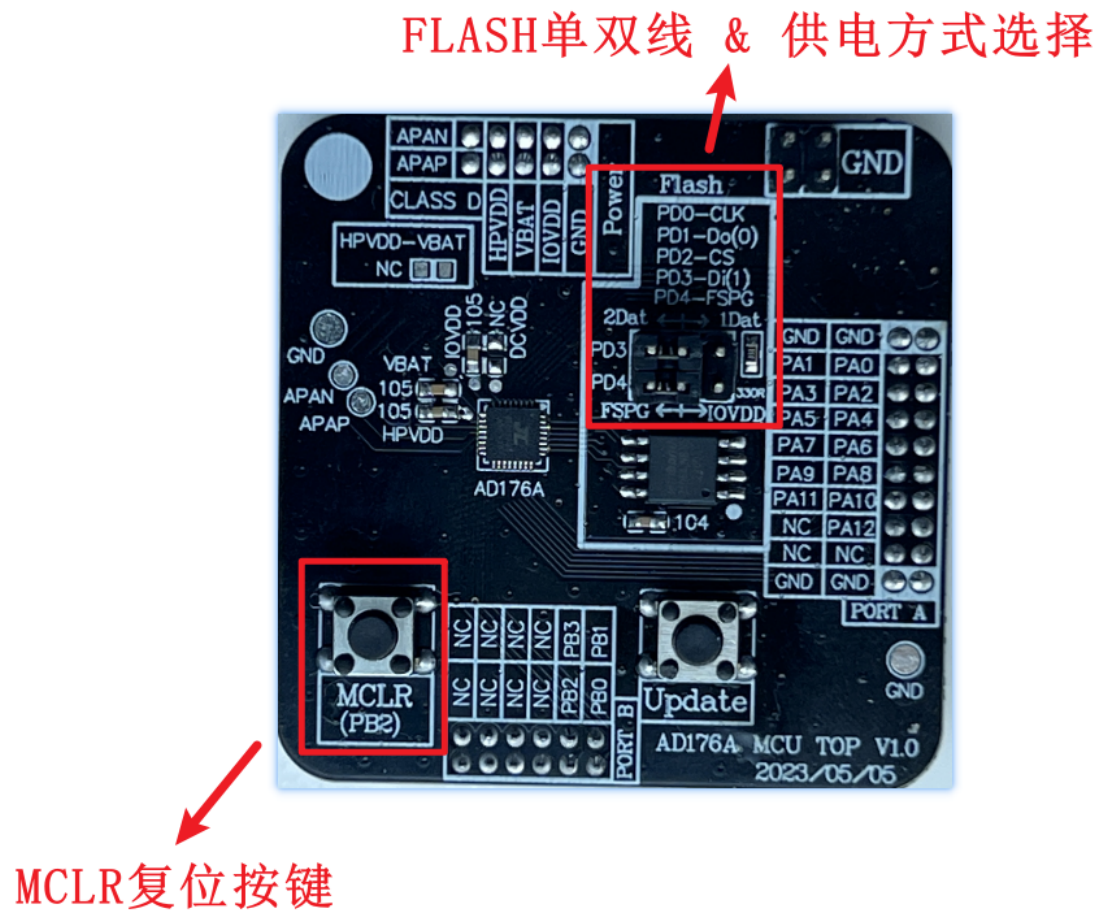
AD17N 使用的硬件环境开发板由底板和顶板两部分组成。
底板包含的模块如下图所示：



其中模块：

模块	模块详情
Class-D APA 模块	差分输出引脚为 APAN 和 APAP；
红外模块	该模块用于接收红外信号，需将 IRVDD 与 VDDIO 短接；
LED 模块	该模块的 3 个 LED 灯都是高电平点亮；
按键模块	该模块有 8 个 AD 按键，外接 10k 上拉电阻；
外挂 FLASH 模块	该模块可挂载一颗存放资源的 FLASH；
电源模块	AD17N USB 口只用来供电，芯片自身并没有 USB 模块。特别地，AD17N 开发烧写使用串口升级方式，供电使用 USB 强制升级工具插入 USB 口供电，并且 USB 强制升级工具 TX 接到开发板的 PB1。

顶板包含的模块如下图所示：



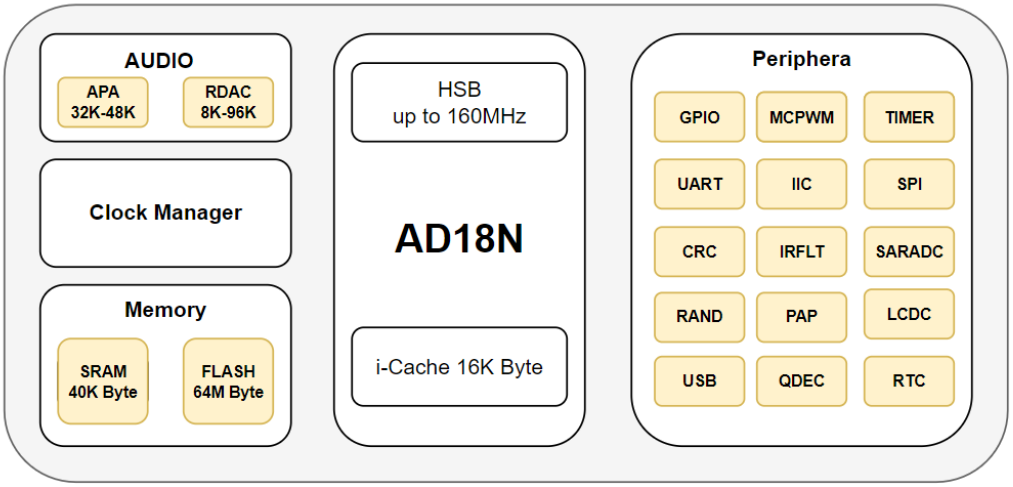
AD17N 使用串口方式升级，升级时候并不需要按着 Update，只需要点击运行升级脚本，USB 强制下载工具就会复位开发板让其进入升级模式。

顶板的“FLASH 工作模式以及供电方式选择区”可以选择 FLASH 是跑 1bit 模式还是 2bit 模式，

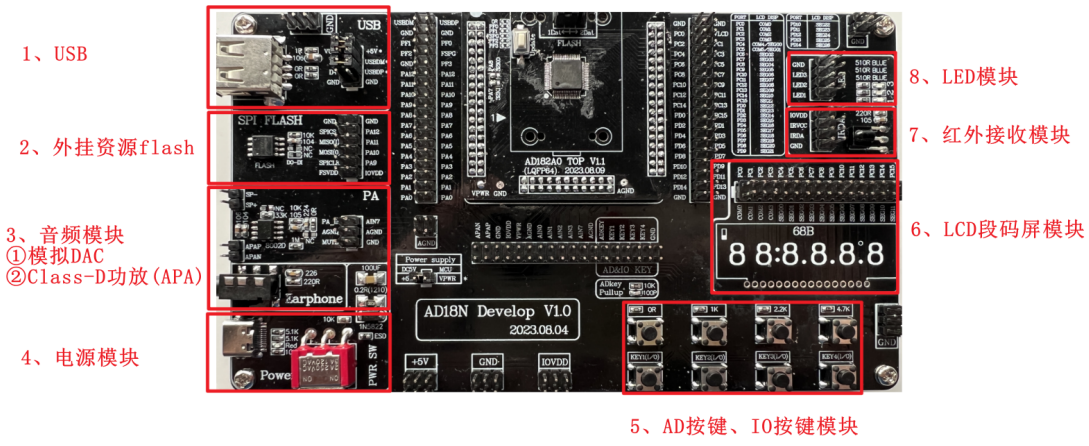
并且可以选择 POWER 引脚供电使用 VDDIO 还是芯片 PD4(SPFG) 引脚。

3.1.4 AD18N 硬件开发环境

AD18N 芯片框图如下图所示：



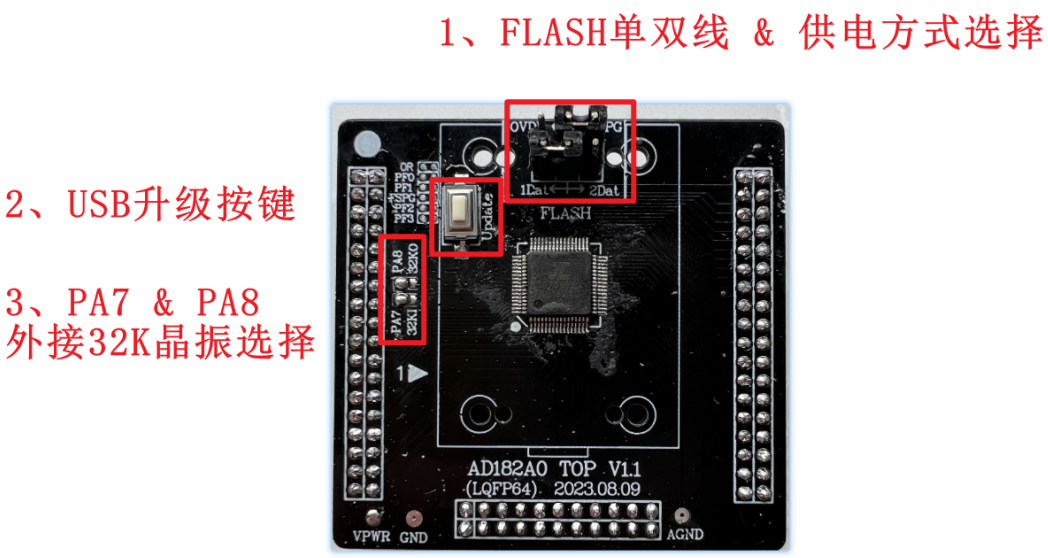
AD18N 使用的硬件环境开发板由底板和顶板两部分组成。
底板包含的模块如下图所示：



其中模块：

模块	模块详情
USB 模块	该模块可以用来通信 USB 设备，或给开发板下载程序以及供电；
外 挂 FLASH 模块	该模块可挂载一颗存放资源的 FLASH；
Audio PA 功放模块	单端模拟 DAC 输出引脚为 AIN7，需将 PA_IN 与 AIN7 短接；
Class-D APA 模块	差分输出引脚为 APAN 和 APAP；（耳机座子接 APA）
电源模块	供电可通过接 USB 线或者 Type-C 线实现，且需要将 +5V 和 VPWR 短接、HPVDD 和 VPWR 短接；
按键模块	该模块有 4 个 AD 按键，外接 10k 上拉电阻与不同阻值的下拉电阻；同时还有 4 个独立 IO 按键；
LCD 断码屏模块	该模块支持最多 4com * 12seg 的段码屏；
红外模块	该模块用于接收红外信号，需将 IRVDD 与 IOVDD 短接；
LED 模块	该模块的 3 个 LED 灯都是高电平点亮；

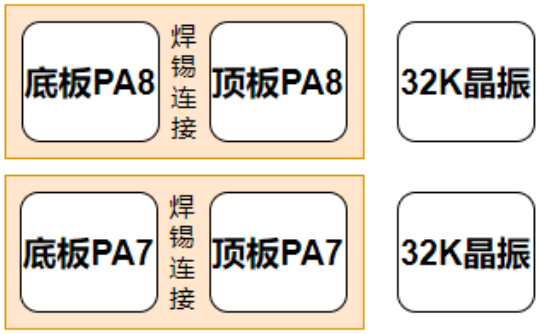
顶板包含的模块如下图所示：



AD18N 使用 USB 升级，升级前需接入 USB 强制下载工具并进入下载模式。（下载器红灯常量即进入下载模式），再使用 Codeblock 编译下载即可；

顶板的“FLASH 工作模式以及供电方式选择区”可以选择 FLASH 是跑 1bit 模式还是 2bit 模式，并且可以选择 POWER 引脚供电使用 VDDIO 还是芯片 PD4(FSPG) 引脚。

顶板的“PA7 & PA8 外接 32K 晶振选择”可以选择顶板的 PA7、PA8 直接接到底板的 PA7、PA8，还是直接接到外挂 32K 晶振引脚；



PA7 PA8 用作普通I/O



PA7 PA8 外接32K晶振

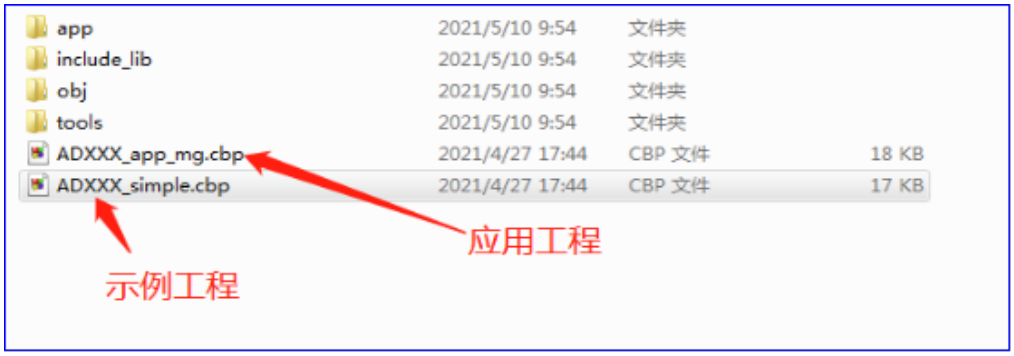
3.2 SDK 工程与配置

目前 AD14N / AD15N / AC104N / AD17N 支持的工程如下：

芯片/工程	app_mg	simple	mbox_mg	mcu	voice_toy
AD14N / AC104N	√	√	√	√	√
AD15N	√	√		√	√
AD17N				√	√
AD18N				√	√

- **app_mg 应用工程：**
  该工程有完整玩具功能，能实现解码、编码录音、MIDI、LINEIN 等功能；同时将 VM、按键、休眠应用在工程中。
- **simple 示例工程：**
  该工程可实现解码、扩音功能等功能；未将 VM、按键、休眠应用在工程中，但是相关代码能在工程中直接调用使用。

芯片/工程	支持的 SDK 版本
AD14N	ad140-release_v1.0.0 ~ v1.4.0、开源 sdk_v1.0.0 ~ v1.6.0
AD15N	ad150-release_v1.0.0 ~ v1.4.0、开源 sdk_v1.0.0 ~ v1.6.0



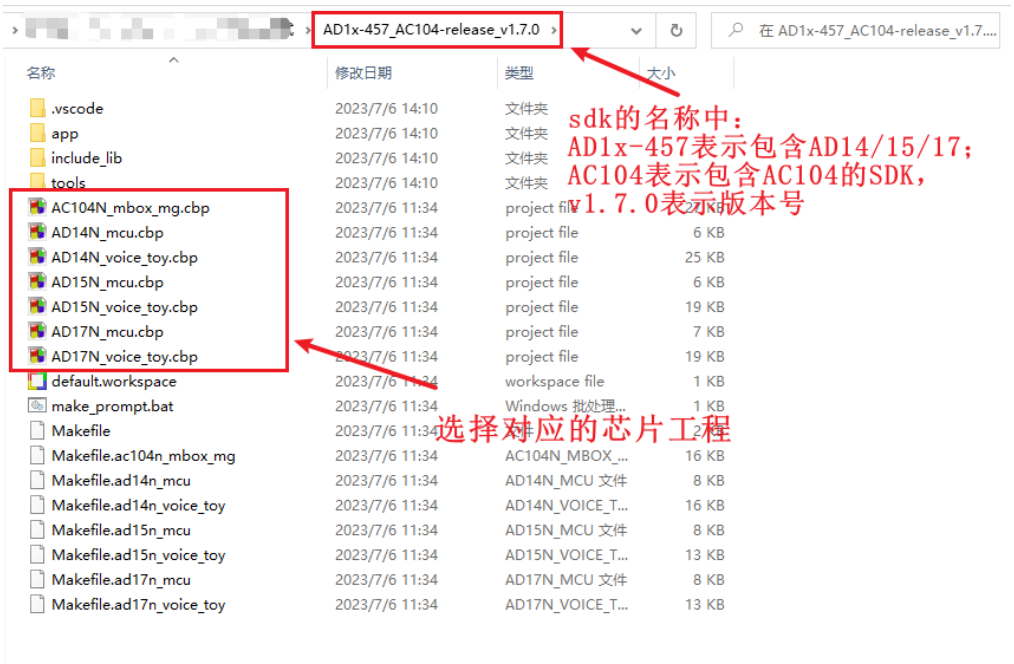
- **mbox_mg 应用工程：**
  该工程有完整插卡音箱功能，能实现标准 MP3、WAV 解码、FM、USB 从机、录音、扩音功能等功能；同时将 VM、升级、按键、休眠应用在工程中。

芯片/工程	支持的 SDK 版本
AC104N	ac104-release_v1.0.0 ~ v1.4.0、开源 SDK_v1.6.0 ~ 至今最新版本



- **voice_toy 应用工程：**
  该工程有完整发声玩具功能，在 simple 工程基础上集成了 app_mg 功能；能实现解码、MIDI 功能；同时将 VM、按键、休眠应用在工程中。

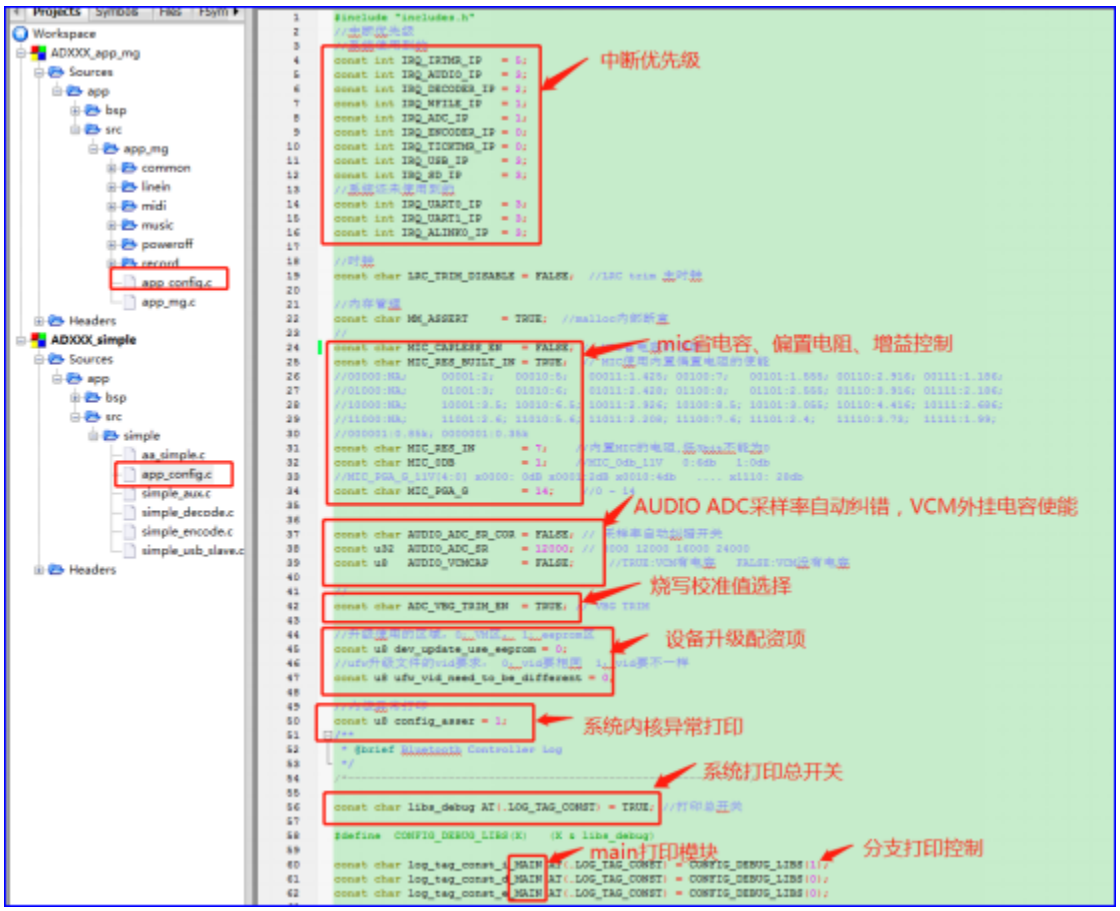
芯片/工程	支持的 SDK 版本
AD14/15/17/18/AC104	AD1x-457_AC104-release_v1.7.0 ~ 至今最新版本



- **mcu 应用工程：**
  该工程为 MCU 工程，仅初始化串口打印、时钟、电源、异常等模块，不包含其他功能。

3.2.1 应用配置文件《app_config.c》介绍

应用配置文件《app_config.c》能配置与系统相关的配置：



1、中断优先级；

注：控制各个模块的中断优先级！

2、LRC_TRIM_DISABLE：LRC 校准时钟，LRC 必须经过了烧写器校准后才能使用

① FALSE：使用 LRC 校准时钟偏差，生产必须选择 FALSE；
② TRUE：不使用 LRC 校准时钟偏差；
注：默认开启 *LRC* 校准时钟，时钟不准的芯片一定要过烧写器校准！

3、MM_ASSERT：MALLOC 内部断言使能

① FALSE：关闭 malloc 内部断言；
② TRUE：打开 malloc 内部断言，默认打开；

4、MIC_CAPLESS_EN：MIC 省电容使能

① FALSE：不省电容；
② TRUE：省电容；
注：省电容方案底噪较大，不建议使用！

5、MIC_RES_BUILT_IN：MIC 电路偏置方式

① 0：使用 MICLDO（PA15）提供偏置电压，需短接 V_MIC 和 MICLDO；

② 1：使用 MICBIAS（PA13）提供偏置电压，需短接 MIC 和 MICBIAS；
③ 2：不使用内部偏置（不占用 I/O 口）；

6、MIC_LDO_V：MICLDO 增益 “MIC_0DB”

注：此选项只在 *MIC_RES_BUILT_IN == 0* 时起作用，以下为千欧姆
① 0 : 1.8V；
② 1 : 2.1V；
③ 2：2.4V；
④ 3：2.7V；

7、MIC_RES_IN：MICBIAS 电路内置偏置电阻挡位选择

注：此选项只在 *MIC_RES_BUILT_IN == 1* 时起作用，以下为千欧姆

value	kkilohm	value	kkilohm
0b00100	7;	0b00101	1.555;
0b00110	2.916;	0b00111	1.186;
0b01000	NA;	0b01001	3;
0b01010	6;	0b01011	2.428;
0b00000	NA;	0b00001	2;
0b00010	5;	0b00011	1.425;
0b01100	8;	0b01101	2.555;
0b01110	3.916;	0b01111	2.186;
0b10000	NA;	0b10001	3.5;
0b10010	6.5;	0b10011	2.926;
0b10100	8.5;	0b10101	3.055;
0b10110	4.416;	0b10111	2.686;
0b11000	NA;	0b11001	2.6;
0b11010	5.6;	0b11011	2.208;
0b11100	7.6;	0b11101	2.4;
0b11110	3.73;	0b11111	1.99;
0b000001	0.85;	0b0000001	0.35

8、MIC_0DB：MIC 前级增益

value	db
0	6db
1	0db

9、MIC_PGA_G：MIC 后级增益

value	db
0b0000	0db
0b0001	2db
0b0010	4db
…	…
0b1110	28db

10、AUDIO_ADC_SR_COR：录音采样率纠错使能

① FALSE：关闭自动纠错，当录音设置的采样率不在 AUDIO ADC 工作的范围内时，返回错误，启动 AUDIO ADC 失败；
② TRUE：开启纠错，当录音设置的采样率不在 AUDIO ADC 工作的范围内时，将采样率设置在 AUDIO_ADC_SR 对应的值！如果 AUDIO_ADC_SR 正确那么启动成功，否则失败；

11、AUDIO_ADC_SR：录音采样率纠错值

① 采样率：8000
② 采样率：12000
③ 采样率：16000
④ 采样率：24000
⑤ 其他：非法值

12、AUDIO_VCMCAP：VCM 是否挂电容，挂了电容后音质会有较大提升：

① FALSE：没有挂电容；
② TRUE：有挂电容；
注：只有部分封装有将 *VCMCAP* 引脚引出，详情请看具体芯片封装原理图！

13、ADC_VBG_TRIM_EN：内部 VBG 校准使能

① FALSE：使用默认值校准；
② TRUE：使用烧写值校准，生产必须选择 TRUE；

14、GPIO_CRITICAL_OPT：GPIO 中断临界保护使能

① FALSE：关闭 GPIO 中断临界保护；
② TRUE：打开 GPIO 中断临界保护；

15、MAINTRACK_USE_CHN：MIDI 乐谱使用主轨道选择

① 0：用 track 号来区分；
② 1：用 channel 号来区分；

16、MAX_DEC_PLAYER_CNT：MIDI 乐谱解码最大同时发声 key 数

① 范围：1~31，会影响到 midi 乐谱解码运算 buff 大小；

17、MAX_CTR_PLAYER_CNT：MIDI 琴最大同时发声 key 数

① 范围：1~31，会影响到 midi 琴解码运算 buff 大小；

18、dev_update_use_eeprom：设备升级占用区域，占用的区域数据是失效

① 0：占用 VM 区域（一般用于 AD14N 的内置 flash 录音）；
② 1：占用 EEPROM 区域（一般用于 VM 记忆）；

19、dev_update_keep_io_status：设备升级 I/O 保持使能

① 0：不保持 I/O，升级时复位 I/O；

② 1：保持 I/O，升级过程中 I/O 保持升级前状态；

20、ufw_vid_need_to_be_different：设备升级对 ufw 升级文件的 vid 要求

① 0：升级文件 vid 与当前设备 vid 相同才可升级；
② 1：升级文件 vid 与当前设备 vid 不同才可升级；
③ 2：升级文件 vid > 当前设备 vid 才可升级；
④ 3：升级文件 vid < 当前设备 vid 才可升级；

注：*vid 在 isd_config.ini 文件中修改！*

21、config_asser：内核异常打印“config_asser”。

① 0：关闭内核异常打印；
② 1：开启内核异常打印；

注：*生产的工程要关闭内核异常打印，内核异常打印会关闭看门狗并且 while(1) 停留！*

22、libs_debug：打印总开关

① FALSE：关闭打印；
② TRUE：开启打印；

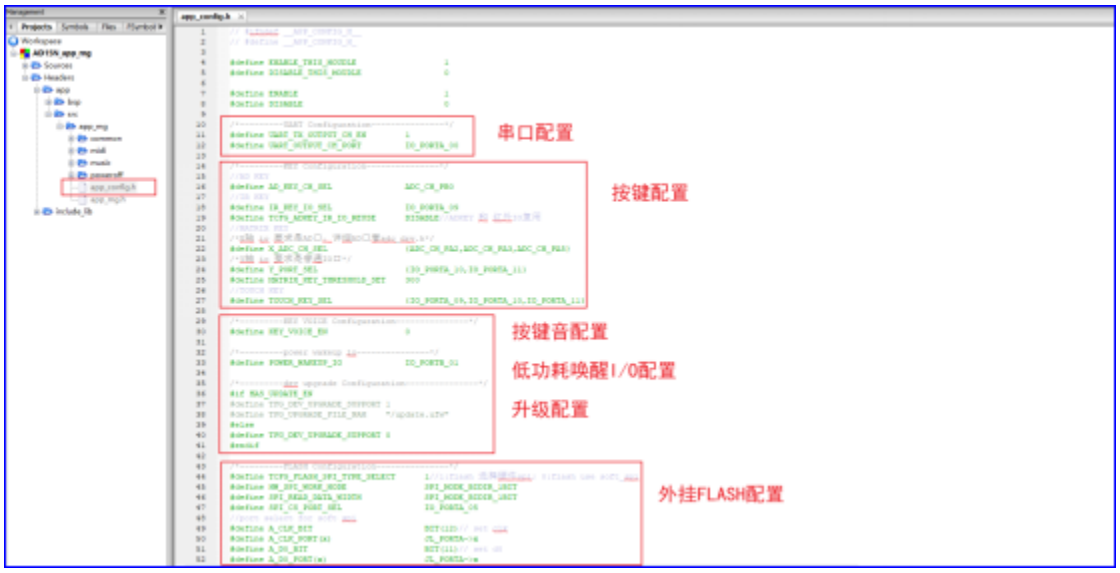
23、log_tag_const_x_yyyy：分支打印开关，仅仅在打印总开关打开的时候有效

① x=i：控制打印函数（log_info / log_info_hexdump）的打印开关；
② x=d：控制打印函数（log_debug / log_debug_hexdump）的打印开关；
③ x=e：控制打印函数（log_error / log_error_hexdump）的打印开关；
④ x=c：控制打印函数（log_char）的打印开关；
⑤ yyyy：控制相应的打印块；

注：*若要打开相应打印，将 CONFIG_DEBUG_LIBS(0) 改为 CONFIG_DEBUG_LIBS(1) 即可！*

3.2.2 应用配置文件《app_config.h》介绍

应用配置文件《app_config.h》能配置与系统相关的配置：



串口配置

1、UART_TX_OUTPUT_CH_EN：串口打印输出任意引脚使能

① DISABLE：使用固定引脚输出，默认串口引脚 AD14 / AC104：PA4，AD15N：PA6;
② ENABLE：使能串口任意引脚输出，输出到;UART_OUTPUT_CH_PORT 定义的引脚;
注：使能串口输出任意引脚会占用一路 *output channal* 通路!

2、UART_OUTPUT_CH_PORT：串口打印输出引脚选择，不可选择芯片没有的 I/O 口;

按键配置

注：按键部分功能需要在 *key.h* 中配置!

配置项目		含义
AD_KEY_CH_SEL		AD 按键采集引脚选择，可查看 adc_drv.h 中支持的引脚通道
IR_KEY_IO_SEL		红外按键捕获引脚选择，会占用一路 input channal 通路
IR_KEY_IO_SEL		红外按键捕获引脚选择，会占用一路 input channal 通路
X_ADC_CH_SEL &	X_ADC_CH_SI	X 轴要求选择支持 ADC 的 I/O;
Y_PORT_SEL	Y_PORT_SEL	Y 轴选择普通 I/O
TOUCH_KEY_SEL		触摸按键 I/O 口选择
KEY_VOICE_EN		触摸按键 I/O 口选择

低功耗配置

1、POWER_WAKEUP_IO：低功耗唤醒 I/O 选择，请查看 power_api.c 中的配置

SD 模块配置

1、TFG_SD_EN：SD 卡模块使能（新版本 SDK 在 app_moduels.h 中定义该宏）

2、SPI_SD_IO_REUSE：外挂 FLASH 与 SD 卡模块 I/O 复用使能

USB 模块配置

1、TCFG_PC_ENABLE：PC 模块使能

2、TCFG_USB_EXFLASH_UDISK_ENABLE：外🄵 FLASH 模拟 U 盘使能

3、TCFG_UDISK_ENABLE：U 盘模块使能

外挂 FLASH 配置（FLASH Configuration）

AD17N / AD18N 支持任意 I/O 映射，外挂 flash I/O 需要在 device_list.c 中指定；

1、TCFG_FLASH_SPI_TYPE_SELECT：spi 选择

① 1：选择硬件 spi，固定 I/O 口，可查看 device_list.c 或者芯片原理图的 spi 模块引脚；
② 0：选择软件 spi，可指定任意 I/O 口；

2、TFG_SPI_UNIDIR_MODE_EN：外挂 flash 运行 1bit 模式选择

① 1：使能外挂 flash 1bit 模式；
② 0：不使能外挂 flash 1bit 模式；

3、HW_SPI_WORK_MODE & SPI_READ_DATA_WIDTH：spi 模式

① SPI_MODE_BIDIR_1BIT：全双工，di 接收，do 发送；
② SPI_MODE_UNIDIR_1BIT：半双工，do 分时接收/发送；
③ SPI_MODE_UNIDIR_2BIT：半双工，di & do 共 2bit 分时接收/发送；

4、SPI_CS_PORT_SEL：spi cs 引脚选择

5、软件 spi 配置：

① A_CLK_BIT & A_CLK_PORT(x)：软件 spi CLK 引脚选择；
② A_DO_BIT & A_DO_PORT(x)：软件 spi DO 引脚选择；
③ A_DI_BIT & A_DI_PORT(x)：软件 spi DI 引脚选择；
④ SOFT_SPI_WORK_MODE：软件 spi 模式，可选择 1bit 或者 2bit 模式；

Mbox_mg 工程独有的配置

宏定义	含义
LED_5X7	LED 数码管使能，会占用部分 I/O，请查看 mbox_main.h 中的引脚分布
BREAK_POINT_AUTO_SAVE	实时断点保存使能
POWER-DOWN_UDISK_MODE_EN	U 盘进入 powerdown 使能
FM_SD_IO_REUSE	FM 与 SD 卡模块 I/O 复用使能
LINEIN_DET_MULT_SD	SD 卡模块与 LINEIN 检测 I/O 复用使能

3.2.3 应用 APP 配置文件《app_modules.h》介绍

各个工程下有对应 cpu 的应用配置文件《app_modules.h》，其中的宏定义能配置支持的编解码格式、以及 APP 模式相关的配置；每一个配置项都可独立开关，关闭没有用到的宏定义，可以 * 减少生成的代码量，节省程序空间和运行空间 * 。

该文件中出现的宏，即该工程拥有的功能；对于默认为 0 的宏，使能为 1 即可正常使用；

解码模块

宏定义	含义
DECODER_UMP3_EN	UMP3 解码使能；
DECODER_MP3_ST_EN	标准 MP3 解码使能（mbox 独有）；
DECODER_WAV_EN	WAV 解码使能（mbox 独有）；
DECODER_F1A_EN	F1A 解码使能；
DECODER_MIDI_EN	MIDI 解码使能；
DECODER_MIDI_KEYBOARD_EN	MIDI 琴解码使能；
DECODER_A_EN	A 格式解码使能；
HAS_MIO_EN	MIO 功能使能；

音频算法模块

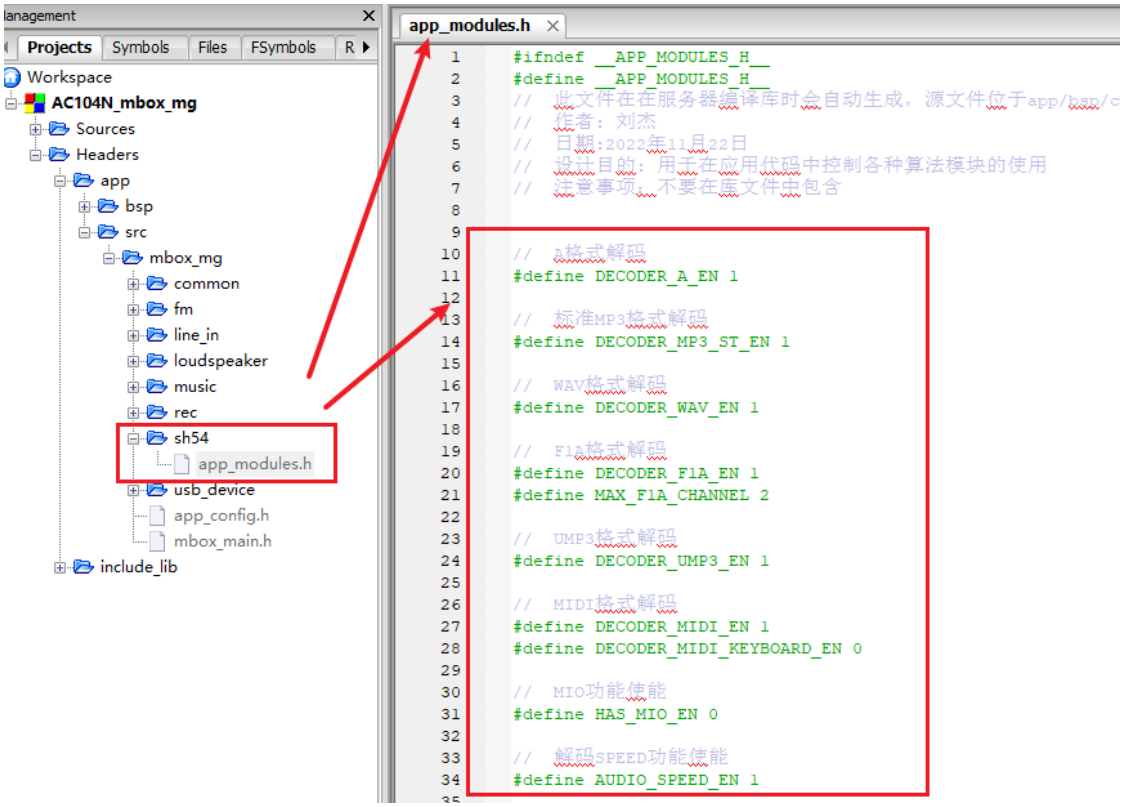
宏定义	含义
AUDIO_SPEED_EN	变速变调算法使能
SPEAKER_EN	扩音使能；
VO_PITCH_EN	变音算法使能；
PCM_EQ_EN	EQ 算法使能；
ECHO_EN	ECHO 算法模块使能；

驱动模块功能

宏定义	含义
EXT_FLASH_EN	外挂 FLASH 使能，使用外挂 FLASH 需要打开该宏；
TFG_SD_EN	SD 卡模块使能，使用 SD 卡需要打开该宏；
UART_UPDATE_EN	带电池串口升级使能；

芯片应用功能

宏定义	含义
RECORD_MODE_EN	录音模式使能；
LINEIN_MODE_EN	LINEIN（AUX）模式使能；
SIMPLE_DEC_EN	多路解码模式使能；
LOUDSPEAKER_EN	扩音模式使能；
FM_EN	FM 模块使能；



app_modules.h 配置

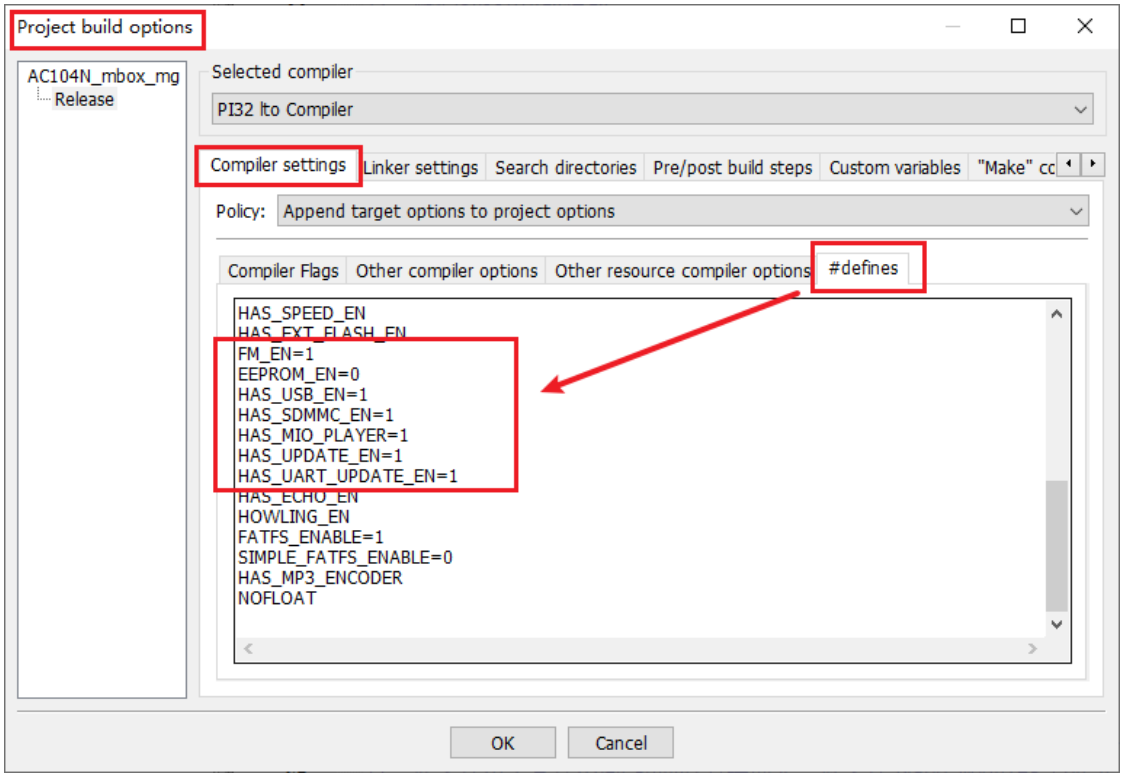
3.2.4 CodeBlocks 工程宏定义介绍

除了 app_config.c、app_config.h 和 app_modules.h 中的配置外，部分模块总开关放在 CodeBlocks 工程的宏定义中，不使用某些模块时，将宏关闭可以节约代码量；

在没有 app_modules.h 的旧版本 SDK 中，模块宏需要在 CodeBlocks 工程的宏定义中关闭。

芯片 Codeblocks 宏定义功能

宏定义	含义
AUDIO_ADC_EN	AUDIO ADC 使能；
ENCODER_EN	编码使能，有该配置即支持编码功能；
EEPROM_EN	EEPROM 使能，使用 EEPROM 需要打开该宏，并在 SDK 工程中添加相应的 c 文件以及 h 文件；



3.3 新版本 SDK 工程裁切（2023 年 7 月发布的 SDK 起始）

从 2023 年 7 月发布的 AD1x-457_AC104-release_v1.7.0 起始，SDK 添加了 app_modules.h 模块管理机制；

裁切工程只需将 app_modules.h 中需要关闭的宏定义配置为 0 即可；



3.4 旧版本 SDK 工程裁切

3.4.1 如何将解码格式从工程中移除

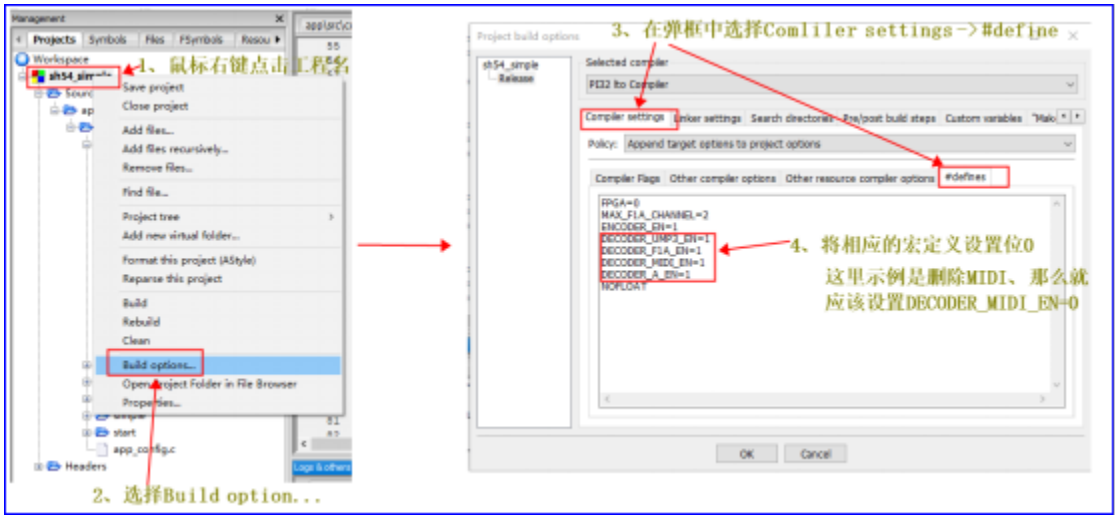
旧版本发布的 SDK 里面支持 F1A、UMP3、A、midi 四种格式，AD15N 支持 F1A、A、MIDI 三种格式，在开发过程中如果不需要支持某些格式，那么可以将相关的代码删除。这样可以极大的节省生成的代码大小；

下面以移除 MIDI 解码格式为例介绍，主要分四步：

① 将相关的宏定义清 0；
② 移除相关的库文件；
③ 移除相关的源文件；
④ 删除其资源定义；

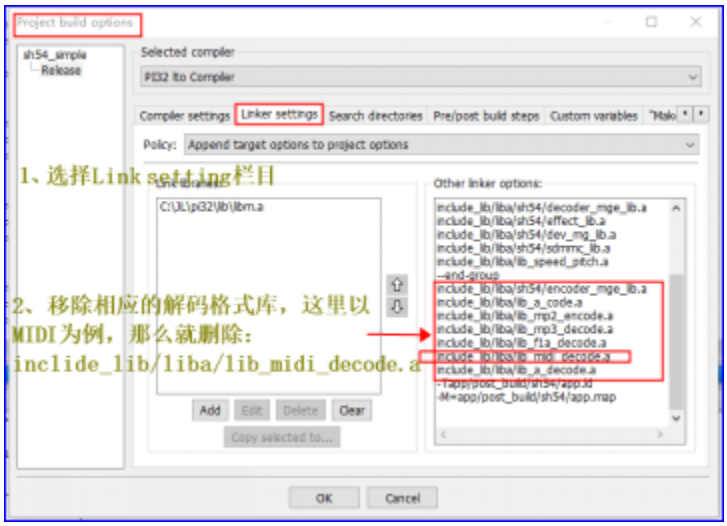
裁切解码第一步

打开 <Project build option> 窗口，将需要 MIDI 格式相应的宏定义设置为 0:



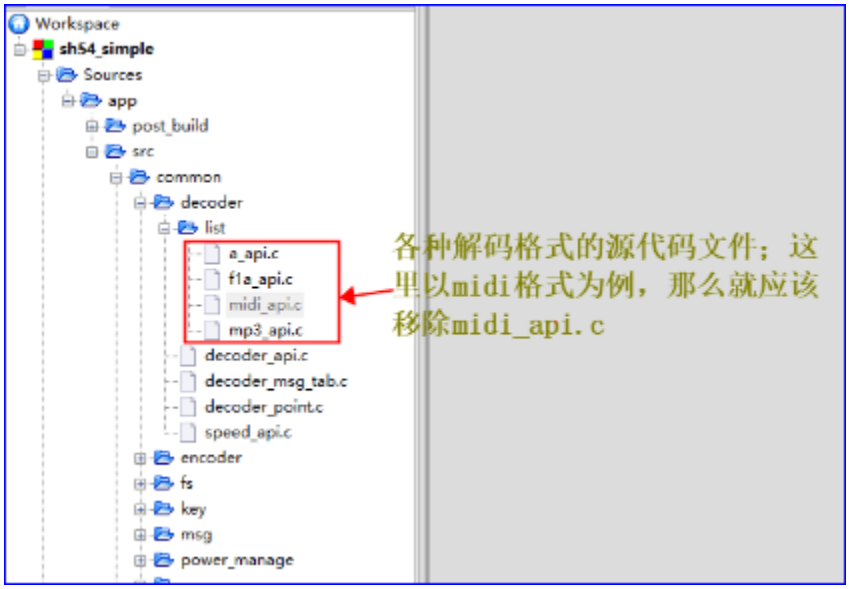
裁切解码第二步

上一步打开的 <Project build option> 窗口中删除 MIDI 的库包含:



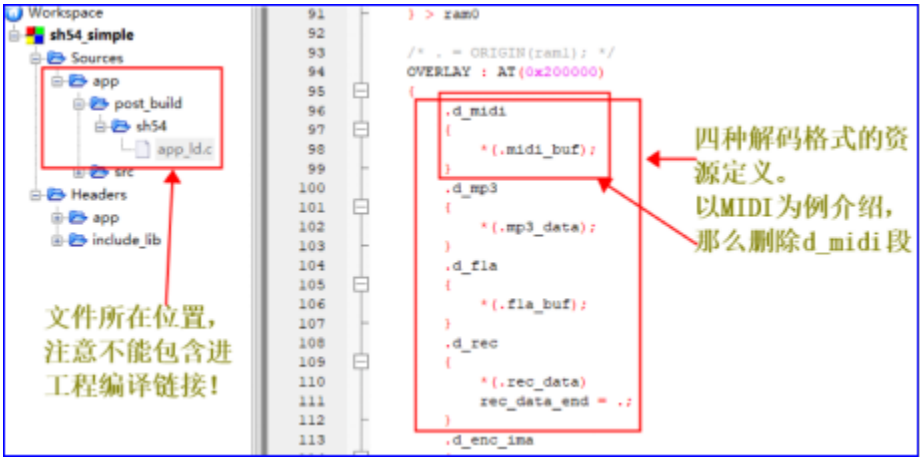
裁切解码第三步

移除工程目录 app\src\common\decoder\list\下的解码源文件;



裁切解码第四步

在文件 app\post_build\sh54\app_ld.c 中删除相关格式用到的资源定义，切记此文件在不可放在工程中编译，如果添加进了工程，一定要将其编译和链接包含取消掉.



3.4.2 如何将压缩格式从工程中移除

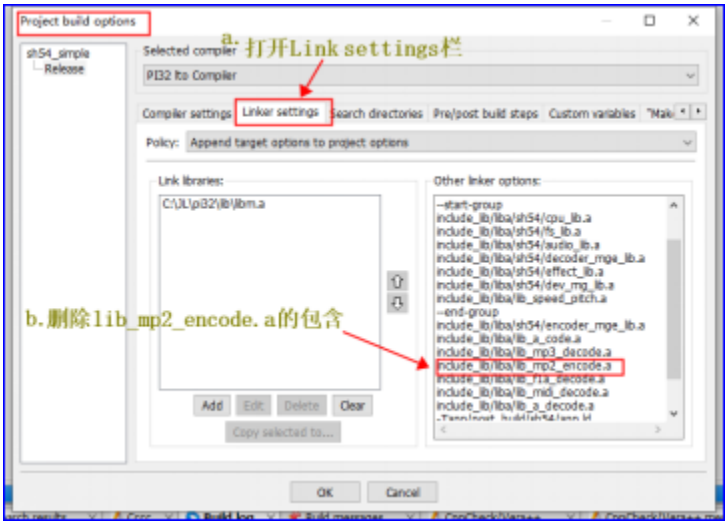
目前发布的 AD14N SDK 里面支持 UMP3、A 两种格式的压缩，AD15N 不支持录音压缩，在开发过程中如果不需要支持某些格式，那么可以将相关的代码删除。这样可以极大的节省生成的代码大小；

下面以移除 UMP3 压缩格式为例介绍，主要分三步：

① 移除相关的库文件；② 移除相关的源文件；③ 删除源码使用到的地方定义；

裁切压缩第一步

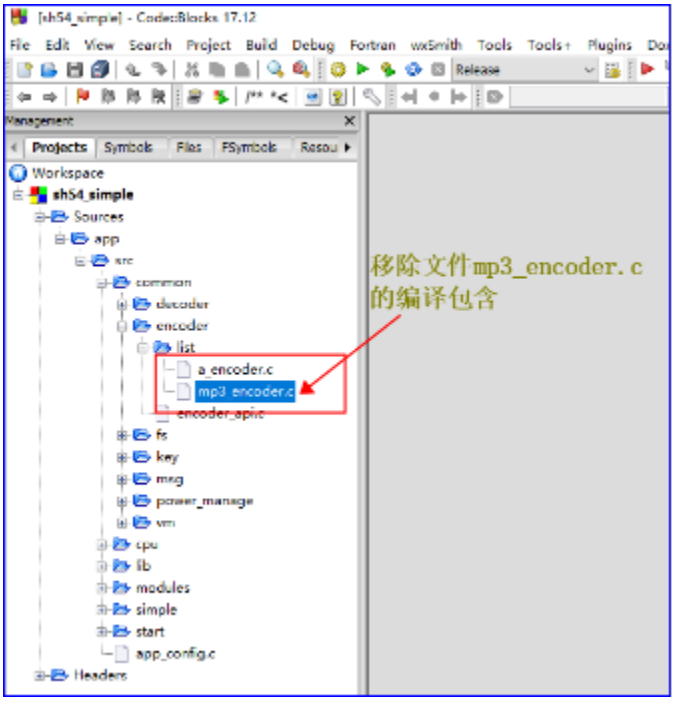
打开 <Project build option> 窗口，删除 mp3 编码库的库包含：



注：部分宏配置需要在《app_modules.h》中关闭！详情请见上文《裁切解码第一步》

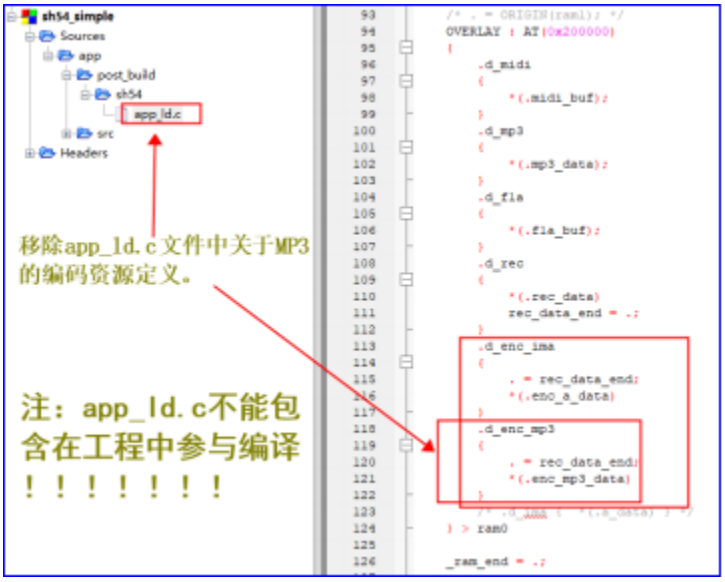
裁切压缩第二步

移除工程目录 app\src\common\decoder\list\下的压缩源文件：



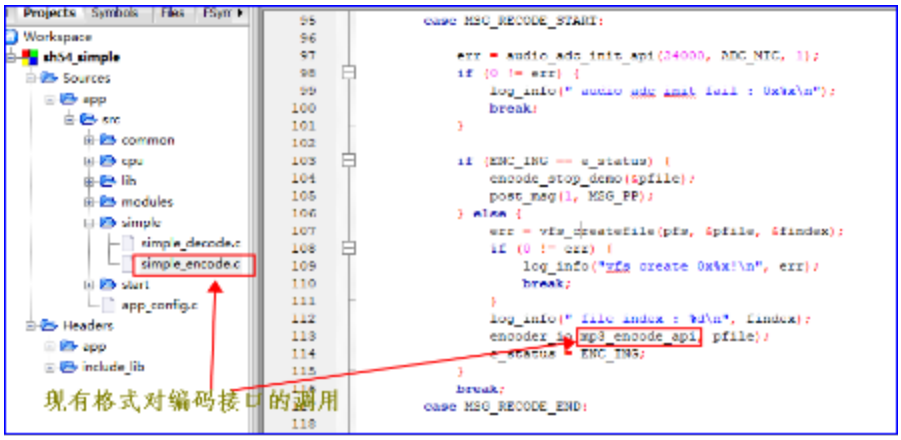
裁切压缩第三步

在文件 app\post_build\sh54\app_ld.c 中删除相关格式用到的资源定义，切记此文件在不可放在工程中编译，如果添加进了工程，一定要将其编译和链接包含取消掉。



裁切压缩第四步

修改一些编译错误，这里主要指在在程序中对编码接口的调用:



CHAPTER 4

下载目录 & 工具使用环境介绍

本章节主要介绍下载目录和工具使用环境。AD14N / AD15N / AC104N / AD17N 编译完成后，会在下载目录生成代码文件，并与资源文件整合形成烧录文件，通过 CodeBlocks 调用下载工具烧写到芯片内置 FLASH 中；同时我司有许多工具支持压缩转换和打包音频，将资源文件下载到内置或外挂 FLASH 中。

dowload_bat.c 文件介绍:

- 指定 *app* 大小 *-app*
- 添加资源文件 *-res*
- 擦除 *flash* 部分区域或所有区域 *-format*
- 添加 *key* 文件 *-key*
- 外挂资源 *flash* 下载

isd_config.ini 文件介绍:

- 系统 *flash SPI* 配置 *-SPI*
- *Cache* 空间配置 *-CACHE_WAY*
- *UBOOT* 启动烧 *key* 配置 *-BOOT_TYPE*
- 程序偏移配置 *-EOFFSET & EOFFSET2UP*
- *LRC* 时钟源校准配置 *-LRC*
- 外挂资源 *flash* 下载引脚配置 *-EX_FLASH & EX_FLASH_IO*
- 开辟自定义空间配置

外挂资源 flash 下载介绍:

- 外挂资源 *flash* 下载
- 外挂资源 *flash* 下载引脚配置 *-EX_FLASH & EX_FLASH_IO*

4.1 下载目录介绍

AD14N / AD15N / AC104N / AD17N / AD18N 下载目录位于 app\post_build\sh5x\voice_toy(工程名称)，一共分为三种文件：

（一）生产相关文件

- app.bin：编译生成的代码文件
- jl_isd.bin：FLASH 烧录文件，包含代码文件和资源文件
- jl_isd.fw：用于我司一拖二/八工具批量生产的 FLASH 烧录文件

（二）资源文件

- dir 文件：通过打包工具生成的打包文件，通常用于存放音频资源

（三）下载工具

- download_bat.c：下载批处理文件，可指定内置和外挂 FLASH 烧写内容
- isd_config.ini：工程配置文件
- 下载目录工具更新.jlxproj：该工具可在线更新下载目录的 exe 工具

本小节着重介绍下载工具的 download_bat.c 以及 isd_config.ini 文件；

4.1.1 dowload_bat.c 文件介绍

download_bat.c 文件支持把代码和资源文件以及部分配置烧写到系统 FLASH，也支持将指定资源文件烧写到外挂 FLASH，详细的功能说明图《dowload_bat.c 介绍》所示，具体参数如下：

- -app app.bin 0x20000:

  该指令指定了烧写代码文件 app.bin，并且指定了代码存放的系统 FLASH 区域大小固定为 0x20000，即无论 app.bin 大小为多少，代码区域都会暂用指定的大小，预留出空间，方便后续代码修改后升级使用；用户也可以去掉 0x20000，不指定代码存放区域的大小，系统会根据 app.bin 大小自动分配，但代码改动可能导致资源位置变化，需要重新下载所有资源。

- -res xxx:

  该指令会把指定的资源文件烧写进入系统 FLASH，不同资源文件以空格隔开。

- -format vm:

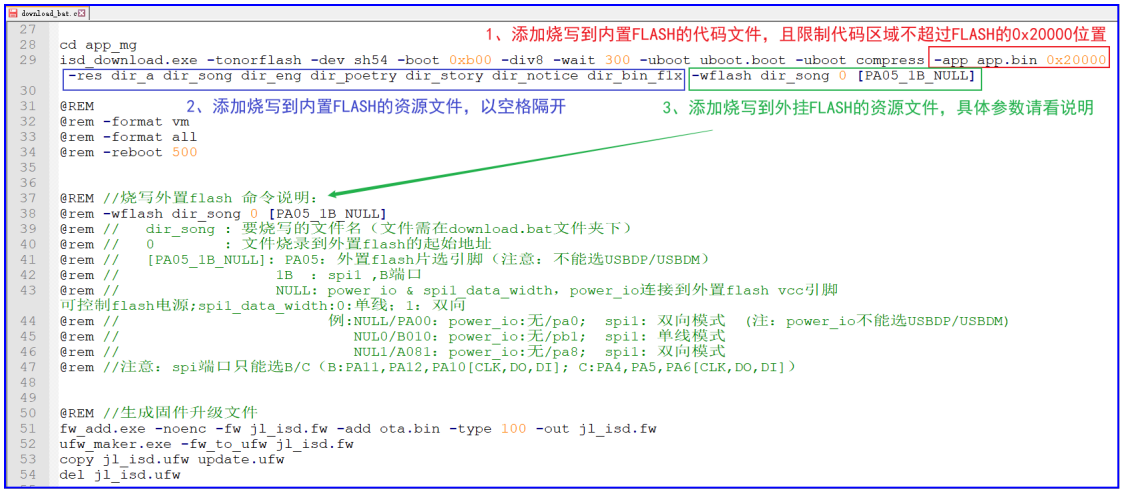
  该指令会擦除 isd_config.ini 文件中指定的“VM”区域。注：需要注意区分”VM”区域以及 VM 掉电存储功能。前者为 isd_config_ini 文件中指定的一块名为“VM”的 FLASH 空间区域，一般用于存放 AD14N 内置 FLASH 的录音数据；后者为代码中的 VM 断电保存功能，其实际存放在 isd_config.ini 文件中指定的一块名为“EEPROM”的 FLASH 空间区域中！

- -format all:

  该指令会擦除整颗系统 FLASH 数据。

- -key *.key:

  该指令会加密烧写文件。参数 *.key 为申请的 key 文件，不同系列不可混用 key 文件。




```
cd app_mg
isd_download.exe -tonorflash -dev sh54 -boot 0xb00 -div8 -wait 300 -uboot uboot.boot -uboot compress -app app.bin 0x20000
 -res dir_a dir_song dir_eng dir_poetry dir_story dir_notice dir_bin_flx -wflash dir_song 0 [PA05_1B_NULL]

@REM
@rem -format vm
@rem -format all
@rem -reboot 500


@REM //烧写外置flash 命令说明:
@rem -wflash dir_song 0 [PA05_1B_NULL]
@rem //   dir_song : 要烧写的文件名（文件需在download.bat文件夹下）
@rem //   0        : 文件烧录到外置flash的起始地址
@rem //   [PA05_1B_NULL]: PA05: 外置flash片选引脚（注意：不能选USBDP/USBDM）
@rem //                   1B  : spi1 ,B端口
@rem //                   NULL: power_io & spi1_data_width，power_io连接到外置flash vcc引脚
可控制flash电源;spi1_data_width:0:单线；1：双向
@rem //                         例:NULL/PA00: power_io:无/pa0;  spi1: 双向模式  (注：power_io不能选USBDP/USBDM)
@rem //                            NUL0/B010: power_io:无/pb1;  spi1: 单线模式
@rem //                            NUL1/A081: power_io:无/pa8;  spi1: 双向模式
@rem //注意：spi端口只能选B/C（B:PA11,PA12,PA10[CLK,DO,DI]; C:PA4,PA5,PA6[CLK,DO,DI]）


@REM //生成固件升级文件
fw_add.exe -noenc -fw jl_isd.fw -add ota.bin -type 100 -out jl_isd.fw
ufw_maker.exe -fw_to_ufw jl_isd.fw
copy jl_isd.ufw update.ufw
del jl_isd.ufw
```

dowload_bat.c 介绍

外挂资源 flash 下载

download_bat.c 支持将资源文件下载到外挂资源 flash 中，不同芯片配置参数不同，其中：

一、**AD14N / AD15N / AC104N 下载外挂 Flash 命令**

- -wflash dir_xxx 0 [PA05_1B_NULL]:

  1. dir_xxx：要烧写的文件名（文件需在 download_bat.c 所在的目录下）；
  2. 0： 文件烧录到外挂资源 flash 的起始地址；
  3. [PA05]： 外挂资源 flash 的 CS 片选引脚（注意，不能选 USBDP、USBDM）；
  4. [1B]： 选择芯片的 spi1，B 端口引脚，端口只能选择 B/C；
  5. [NULL]： 选择 “NULL” 时为双线，DO、DI 分开连接，选择 “NUL0” 时为单线，DO&DI 短接；

二、**AD17N / AD18N 下载外挂 Flash 命令**

- -ex_flash dir_song_ext：

该指令会根据 isd_config.ini 中定义的外挂 flash 引脚（详情请看 EX_FLASH 与 EX_FLASH_IO 说明），将指定的资源文件下载到外挂 flash 中，外挂 flash 引脚配置将于后文 isd_config.ini 文件介绍中说明。

> 注意：准备下载到外挂 *flash* 的 *dir* 文件，需要经过 *packres.exe* 指定下载地址再次打包生成新的 *dir* 文件，才可使用 *-ex_flash* 命令。例如将 *dir_song* 文件下载到外挂 *flash*，需要通过 *packres* 脚本添加 “*packres.exe -n res -o dir_song_ext dir_song 0 -normal*” 命令，指定 *0* 地址将 *dir_song* 文件打包成 *dir_song_ext* 文件，在 *download_bat.c* 中添加 *-ex_flash dir_song_ext* 指令即可将 *dir_song* 文件完整烧写至外挂 *flash* 中。

4.1.2 isd_config.ini 文件介绍

isd_config.ini 是一个工程配置文件，通过它可以配置程序中与烧写、启动的相关参数。

（一）**EXTRG_CFG_PARAM**

- PID: 工程标识，不可超过 16 个字节；
- VID: 工程版本号，用于升级判断；
- NEED_RESERVED_4K: 4K 保留区配置，详情请看《RESERVED_AREA 区域说明》；

  NEED_RESERVED_4K = YES：保留 4K 配置区；

  NEED_RESERVED_4K = NO：不保留 4K 配置区；
- SPECIAL_OPT：选择是否实时根据 FLASH 支持的对齐方式组织文件；

  SPECIAL_OPT = 0：根据下载时所连样机 FLASH 的对齐方式组织文件，只生成 1 份 4K 或者 256 对齐的烧录文件；

  SPECIAL_OPT 不存在：生成多份不同对齐方式的烧录文件；

  注：烧录文件在组织过程中支持按 4K 或者 256 字节对齐；若用 256 字节对齐的烧录文件，去生产最小擦除单位为 4K 字节的芯片内置 FLASH，会出现开不了机的问题!!! 注释该配置会关闭该功能，默认生成多份不同对齐方式的文件，由烧写器判断烧写哪一份。
- FORCE_4K_ALIGN：选择是否强制只生成 1 份 4K 字节对齐的 FLASH 烧录文件

  注：如果方案需要在线升级，建议使用强制 4K 对齐，SDK 默认只生成 1 份 4K 对齐的烧录文件！
- FLASH_SIZE：指定 FLASH 大小，用于生成指定大小的烧录文件

  ① 0x800000：按照 8M 字节的 FLASH 的空间生成烧录文件，如果在线下载时发现实际的 FLASH 与指定的大小不匹配，则会报错；

注：在没有直接指定 FLASH 大小的情况下，最后生成的下载文件会以市面 FLASH 能支持的最小文件的形式组织；例如：

1、当所有文件小于 256K 字节时，系统会以总大小为 256K 为标准为录音分配剩余空间；

2、当所有文件大小等于 300K 时，系统会以 512K 为标准为录音分配剩余空间；

（二）**SYS_CFG_PARAM**

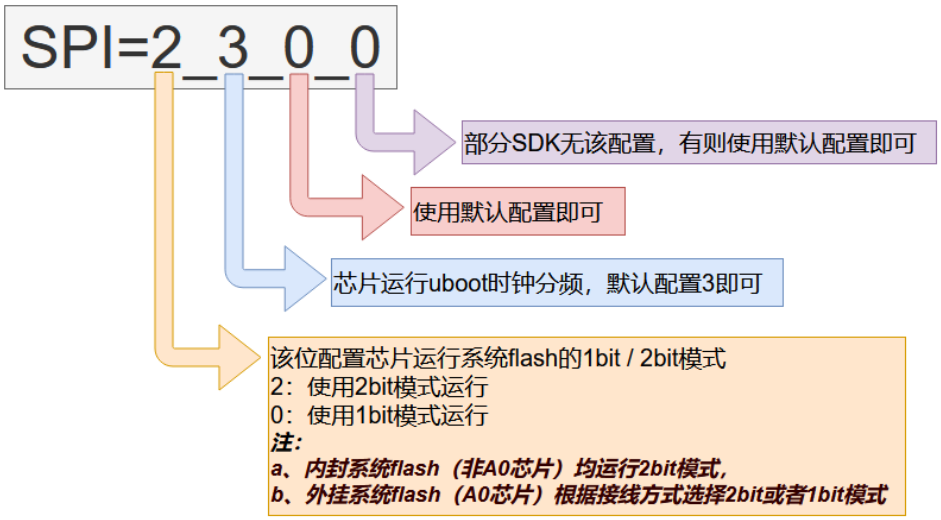
- SPI：芯片内置 FLASH 的 SPI 配置，配置说明如下图所示；

  注意：对于主频速度要求高的方案，尽量使用两线模式，有利于系统稳定性和功耗控制。

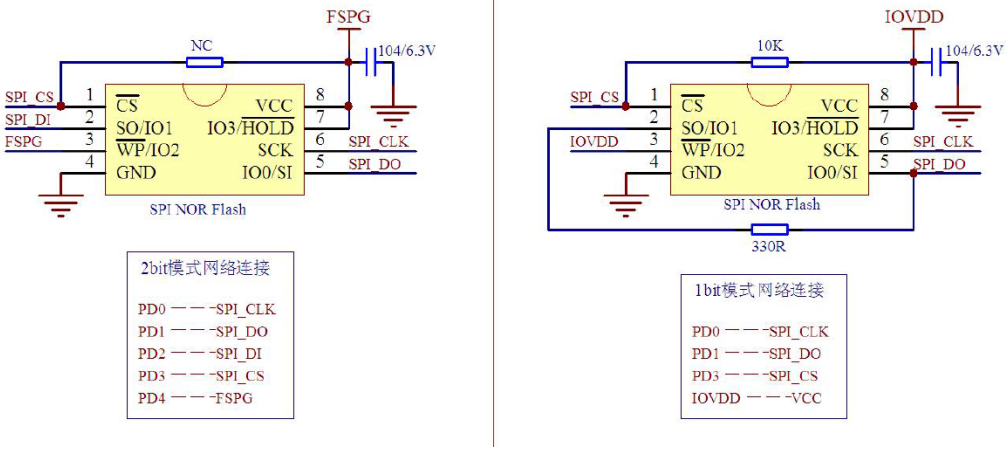
  ① 2_3_0：内置 FLASH 使用双线数据模式；

  ② 0_3_0：内置 FLASH 使用单线数据模式；

芯片	AD14N / AC104N	AD15N	AD17N	AD18N
是否支持该功能	支持	支持	支持	支持



SPI 配置参数说明



系统 flash 2 线/1 线接法

- CACHE_WAY：芯片使用 cache 的 WAY 数（1 WAY = 4K 字节）；多出来的 cache 可以作为普通 ram 使用；

  注意：*Cache 会优先使用地址靠前的空间；例如 AC104N 有 4 条 WAY 共 16K 字节，若 CACHE_WAY 配置为 2，系统会占用 16K 字节的前 8K 用于 cache 缓存，后 8K 字节可用作普通 RAM，具体请参考 AC104N SDK 的 ld 文件；*

芯片	AD14N / AC104N	AD15N	AD17N	AD18N
是否支持该功能	支持，可选 1~4	支持，可选 1~2	**不支持**	**不支持**

- BOOT_TYPE：AD14N / AD15N 在 2021 年 5 月 18 日之后发布的 SDK 已经使用了新的工具生成芯片密钥。新的密钥系统支持芯片用户派生密钥（leaf key）为空的芯片；原厂密钥（root key）加上客户派生密钥 (leaf key) 组成完整的芯片密钥，方便客户备货；

  ① BOOT_TYPE = 0：芯片启动时发现 KEY 不一致，不启动工程；

  ② BOOT_TYPE = 非零：芯片启动时发现 KEY 不一致，新系统启动烧写芯片密钥的功能，

之再尝试启动系统；如果芯片已有的 KEY 和现有的要烧进去的 KEY 冲突，有烧坏芯片密钥的可能!!!

芯片	AD14N / AC104N	AD15N	AD17N	AD18N
是否支持该功能	支持	支持	**不支持**	**不支持**

- EOFFSET & EOFFSET2UP：代码偏移配置，部分芯片 FLASH 需要配置偏移参数；

芯片	AD14N / AC104N	AD15N	AD17N	AD18N
是否支持该功能	支持	支持	**不支持**	**不支持**

- LRC：LRC=-1 时，烧写器烧写时校准芯片 LRC 时钟，默认开启；

芯片	AD14N / AC104N	AD15N	AD17N	AD18N
是否支持该功能	支持	支持	支持	支持

- EX_FLASH：外挂 flash 烧写配置

  ① PA05_1A_NULL：配置顺序为“CS-SPI 端口-电源口”；

  ② 用 VDDIO 供电电源口参数为 NULL，使用时配置 CS 脚 I/O 即可；

芯片	AD14N / AC104N	AD15N	AD17N	AD18N
是否支持该功能	**不支持**	**不支持**	支持	支持

- EX_FLASH_IO：外挂 flash 烧写引脚配置

  ① 2_PA01_PA02_PA03_NULL_NULL：配置顺序为“数据宽度-CLK-DO-DI-D2-D3”；

  ② 双线模式时数据宽度为 2，D2、D3 口为 NULL，即 2_PA01_PA02_PA03_NULL_NULL；

  ③ 单线模式时数据宽度为 0，D2、D3 口为 NULL，即 0_PA01_PA02_PA03_NULL_NULL；

  ④ 四线模式时数据宽度为 4，需要配置 D2、D3 口；

芯片	AD14N / AC104N	AD15N	AD17N	AD18N
是否支持该功能	**不支持**	**不支持**	支持	支持

（三）RESERVED_CONFIG

SDK 默认开辟了以下区域：

- VM 区域：用于 AD14N 内置 flash 录音
- EEPROM 区域：用于 VM 掉电存储功能

需要分清楚 VM 区域与 VM 掉电存储功能！VM 区域是 flash 一片空间的名称，VM 掉电存储是一种功能

- XXXX_ADR：区域起始地址
  - ① 0：VM 区域起始位置；
  - ② AUTO：由工具自动分配起始地址；

- XXXX_LEN：区域大小
- XXXX_OPT：区域操作属性
  - ① 0：下载代码时擦除指定区域；
  - ② 1：下载代码时不操作指定区域；
  - ③ 2：下载代码时给指定区域加上保护；

注：VM 区域的起始地址为 0，且实际大小为系统剩余的空间，扩展录音空间可查看（一）中 FLASH_SIZE 配置的说明；EEPROM 区域一般用于代码中的 VM 断电记忆存储，区域大小不少于 8K，否则 VM 初始化会失败；用户可自行定义其他空间区域；

4.2 开发工具介绍

AD14N / AD15N / AC104N 支持的开发工具如下：

资源打包工具：能将多个文件打包成一个文件；

音频文件转换工具：能将音频文件转换为系统能支持的格式；

MIDI 转换工具：能将 midi 文件转换为系统能支持的格式以及制作 midi 音色库；

MIO 工具：能制作波形随音频律动的 MIO 文件；

SYD 系统制作工具：能格式化 SD 卡/U 盘并挂载我司 SYD 文件系统的工具；

以上工具可在 AD14N_AD15N_gitee 开源网站上或询问我司深圳办人员获取，部分工具内含说明文档，本节将介绍文件打包工具和音频转换工具；

4.2.1 资源打包工具

下载到 FLASH 的资源文件需要经过资源打包工具处理之后才能使用，打包工具包括：new_packres.bat、packres.exe 和 pRFiles.exe；其中 new_packres.bat 支持的命令详情请看同目录下的 readme.md 文件。

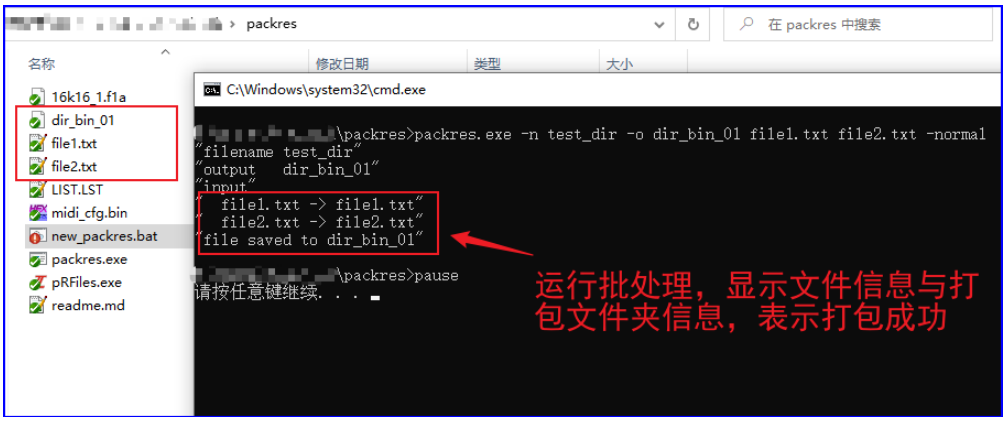
（一）少量文件直接打包

1、将需要打包的文件放在工具根目录下，打开 new_packres.bat 文件，指定需要打包的文件以及输出的文件夹名；



new_packres.bat 资源打包批处理说明

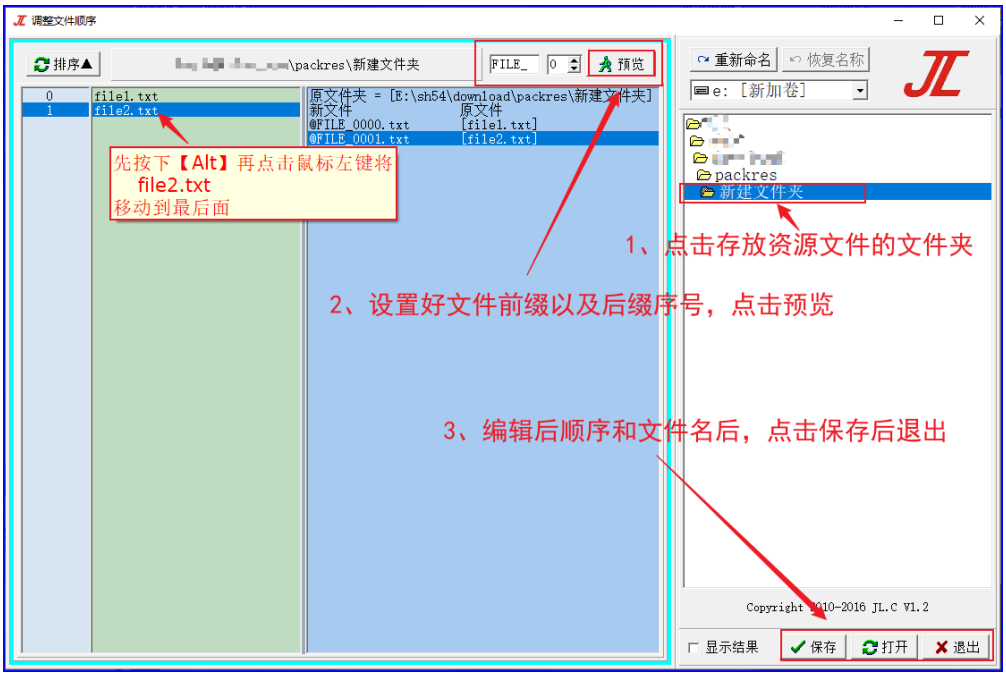
2、保存 new_packres.bat 文件，并双击该批处理，工具根目录下生成打包文件夹表示打包成功；



new_packres.bat 运行结果图

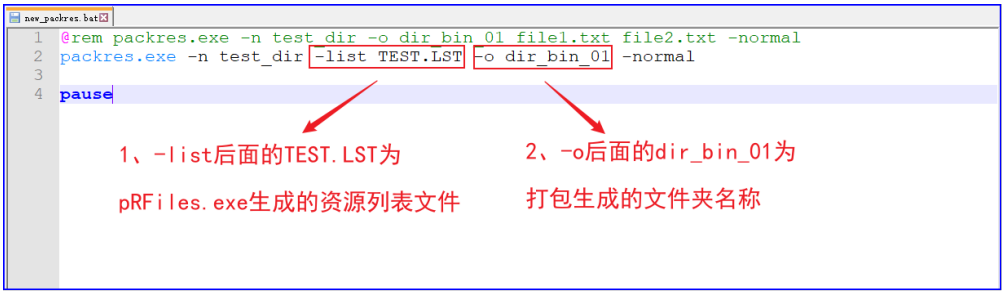
（二）大量文件通过 pRFiles.exe 批量打包

1、将大量文件放在文件夹中，打开 pRFiles.exe，如下图所示进行操作，保存为 LST 文件，以保存为 TEST.LST 为例；



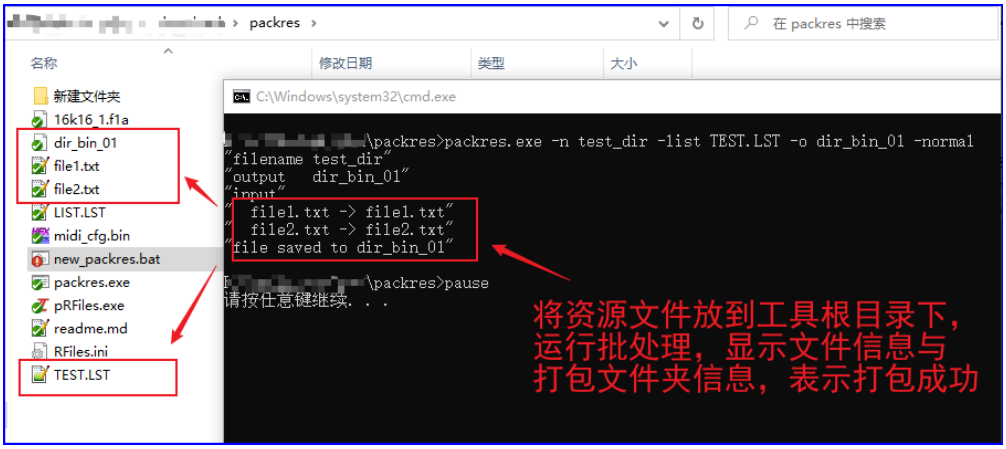
pRFiles.exe 打包工具操作图

2、将大量文件放在文件夹中，打开 pRFiles.exe，如下图所示进行操作，保存为 LST 文件，以保存为 TEST.LST 为例；



new_packres.bat 资源打包批处理说明

3、保存 new_packres.bat 文件，把资源文件全部放到工具根目录下，并双击该批处理，工具根目录下生成打包文件夹表示打包成功；



new_packres.bat 运行结果图

4.2.2 音频转换工具

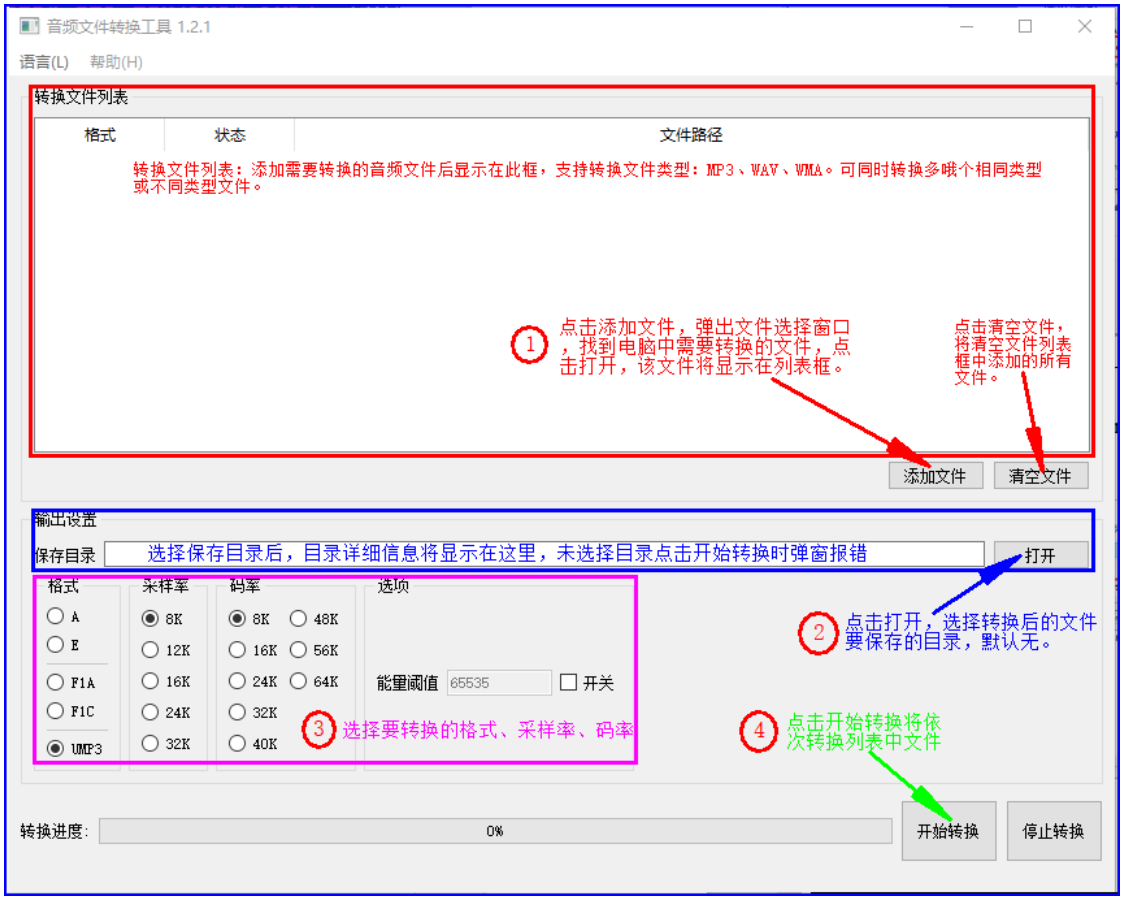
音频文件转换工具 _1.2.17.exe 是我司用于压缩转换 WAV、MP3 和 WMA 音频的工具，可将音频转换成我司的 A（B、E）格式、F1A（F1B、F1C）格式、F1X 格式以及 UMP3 格式；**（一）音频转换步骤**

1、添加需要转换的音频文件；

2、选择输出音频的保存目录；

3、选择需要转换的格式、采样率、码率以及其他选项；

4、点击开始转换；



音频文件转换步骤

（二）目标格式选择

转换工具可生成三种类型文件，第一类 *.a、.*b*、.e；第二类 *.f1a、.*f1b*、**f1c*；第三类.ump3。每种类型都有采样率或码率供自由选择，一般采样率高了对应选择的码率也会高些，采样率跟码率都影响音质。

类型格式选择：选哪个格式，一般考虑音质、设备资源以及系统运行速度。例如需要音质高而 flash 大小够用，系统资源够跑，就选用高压缩比的类型。一般要低码率的话，可以先选 f1a/f1b/f1c，然后系统需要速度快，就选 a。如果 f1a/f1c 没法满足要求，如压音乐之类的，可以选 ump3。如需要重复播报的音频段或者音频间无缝衔接的，可以选择 f1x。

- a、b、e 格式：

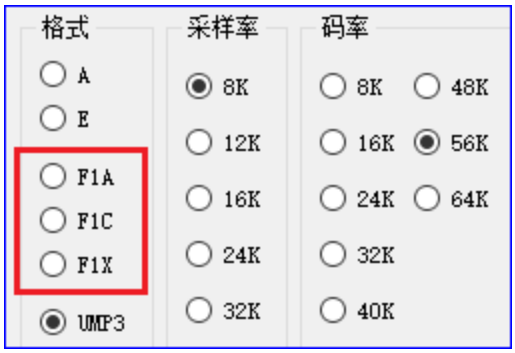
该格式转换压缩比比较低，速度快，资源小。可用于人声或者背景音乐。该格式特点是音质较差，但占用内存小。



a、b、e 格式配置图

- f1a、f1b、f1c、f1x 格式：

该格式转换压缩比比较高，比较适用人声和音乐，音质一般，其中 f1a、f1b、f1c 可用于无缝衔接，f1x 格式可用于音频拼接。



f1a、f1b、f1c、f1x 格式配置图

- ump3 格式：

该格式有较多的采样率和码率可以选择，可以按情况来压缩音乐或者人声，该格式特点是占用内存大，音质较好。



ump3 格式配置图

（三）其他参数说明

1、f1a / f1b / f1c 无缝衔接转换和提高小文件压缩比

如果勾选是否无缝循环，则可以首尾无缝对接循环重复播放同一个音频。其能量转换阈值范围 0 到 65536，代表 0 到 1，默认参数配成 10，即砍掉接近完全静音的部分。运算过程中的音频也是归一化到 0 到 1 来计算的。所以实际应用中，调整的阈值范围一般是选取 0 到 0.1，约 0 到 6000.。可以设成 2048。试下循环效果。

如果勾选循环附加预处理，会对 f1a/f1b/f1c 文件再做多一次处理，分析计算文件前后的帧叠位置，计算前后的砍帧个数，然后对文件前后做一次前后自适应砍帧的动作。

如果勾选提高小文件压缩比，会使压缩后的目标文件码率较为接近目标码率。f1a 是 cbr 的编码。需要长时间来平均来达到接近目标码率。如果文件都很小的时候，压出来的目标文件码率可能会距离目标码率差距较大。勾上提高小文件压缩比，会接近些。注：建议无缝循环的时候，只勾选是否无缝循环选项，听得出卡顿的文件，再勾选循环附加预处理，因为这个步骤的砍帧可能会出现有些文件前后会被砍比较多的情况！

2、f1x 制作说明

制作 f1x 音频时，需要选择音频头字节数和头尾去除帧数。

音频头字节数用于文件寻址，与原始转换文件大小有关。头字节数为 3 时，生成的 f1x 文件超过 16M 字节会报错，文件较多时可节约空间；头字节数为 4 时，则无限制。

头尾去除帧数用于选择去除的 f1x 内各个音频文件头尾的音频数据包数，可以调整衔接效果。但不可完全依赖该功能，对于衔接效果不理想的音频，需要从音源文件去改善。

3、ump3 的能量阈值

ump3 格式有能量阀值选项，勾选开关便可以设置阀值，音频前后端会根据这个值砍掉一些低能量的音频，使播放开始和结束时更柔和。阀值范围：0-2147483648，默认参数配成 65535. 即砍掉接近完全静音的部分，值越大能量剪切越多。鼠标悬停在能量阀值输入框无任何操作两秒以上会显示阀值范围。

4.3 多算法授权简单介绍

使用该功能前请咨询杰理助理；

多算法授权，指的是：在一个 SDK 中，使用了算法提供商的库函数，而算法提供商可以控制芯片的烧录次数。

（一）使用场景

1. 算法提供商想要控制生产商使用算法库的次数（即：算法提供商按生产的样机收取费用）。

具体详细说明可以点击下方链接：
多算法授权使用说明

CHAPTER 5

时钟驱动

AD14N / AD15N / AC104N / AD17N 支持动态设置系统 HSB、LSB 时钟，支持多种分频组合；

时钟驱动中限制着内置系统 flash 的 spi 时钟速度，SDK 为兼容大多数 flash，默认的 spi 时钟不超过 60MHz；

最高系统时钟如下：

最高系统时钟 / CPU	AD14N / AC104N	AD15N	AD17N	AD18
HSB（系统时钟）	192MHz（超频）	160MHz	160MHz	160MHz
LSB	96MHz（超频）	80MHz	80MHz	96MHz

AD14/15/17 以及 AC104 时钟相关函数：

```
void pll_sel(u32 pll_clock, _PLL_DIV pll_div, _PLL_B_DIV pll_b_div);
int clk_get(const char *name);
```

AD18 时钟相关函数：：

```
int clk_set(const char *name, int clk);
int clk_get(const char *name);
```

5.1 AD14N / AC104N 系统时钟超频说明

AD14N / AC104N 支持最高 192MHz 的系统时钟，在运行大于 160MHz 系统时钟前需要提高系统内核电压，否则可能会导致运行异常；

支持 192MHz 时钟的内核电压，需要调用以下接口修改，需要在系统时钟初始化前调用：

VDC13_VOL_SEL(VDC13_VOL_SEL_140V);

SYSVDD_VOL_SEL(SYSVDD_VOL_SEL_129V);

系统时钟不超过	内核电压挡位	内置系统 Flash	A0 芯片外挂系统 Flash
160mhz	默认值	支持	支持
192Mhz	1.29v	支持	支持

表系统时钟支持情况

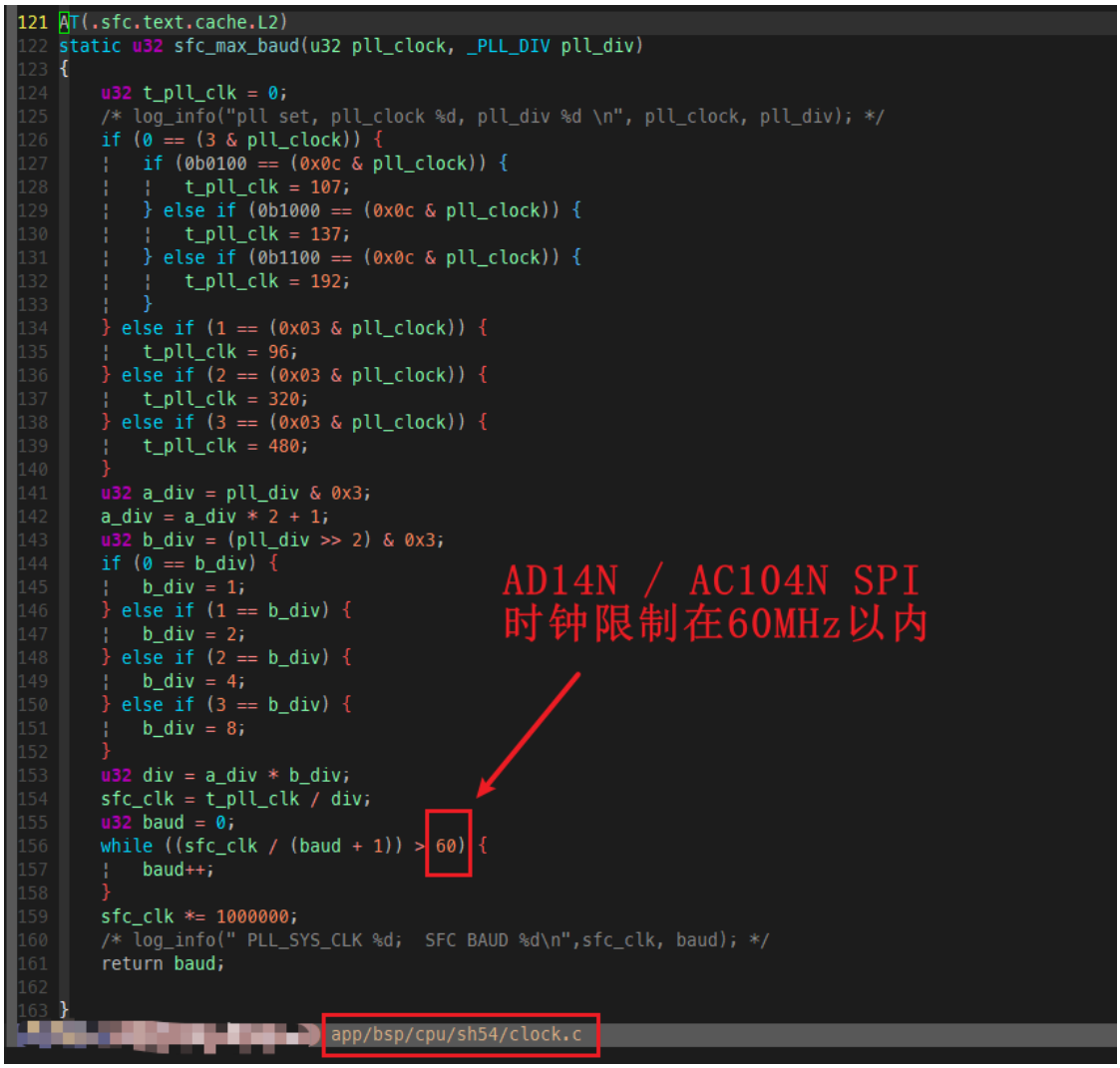
5.2 内置系统 flash spi 时钟说明

当使用 A0 芯片时，系统 Flash 需要外挂，由于 Flash 的型号以及性能，加上 PCB 布线等情况的影响，不能预测系统 Flash 波特率运行在超过 50Mhz、甚至是到 96MHZ 的具体情况；在此种情况下，客户如果需要系统 Flash 波特率运行在超过 50Mhz，需要自行评估。

系统 Flash 波特率	内置系统 Flash	A0 芯片外挂系统 Flash
“<=60Mhz”	支持	支持
“>=60mhz” & “<=96Mhz”	支持	不能直接支持
“>96MHZ”	不支持	不支持

表系统 Flash 波特率支持情况

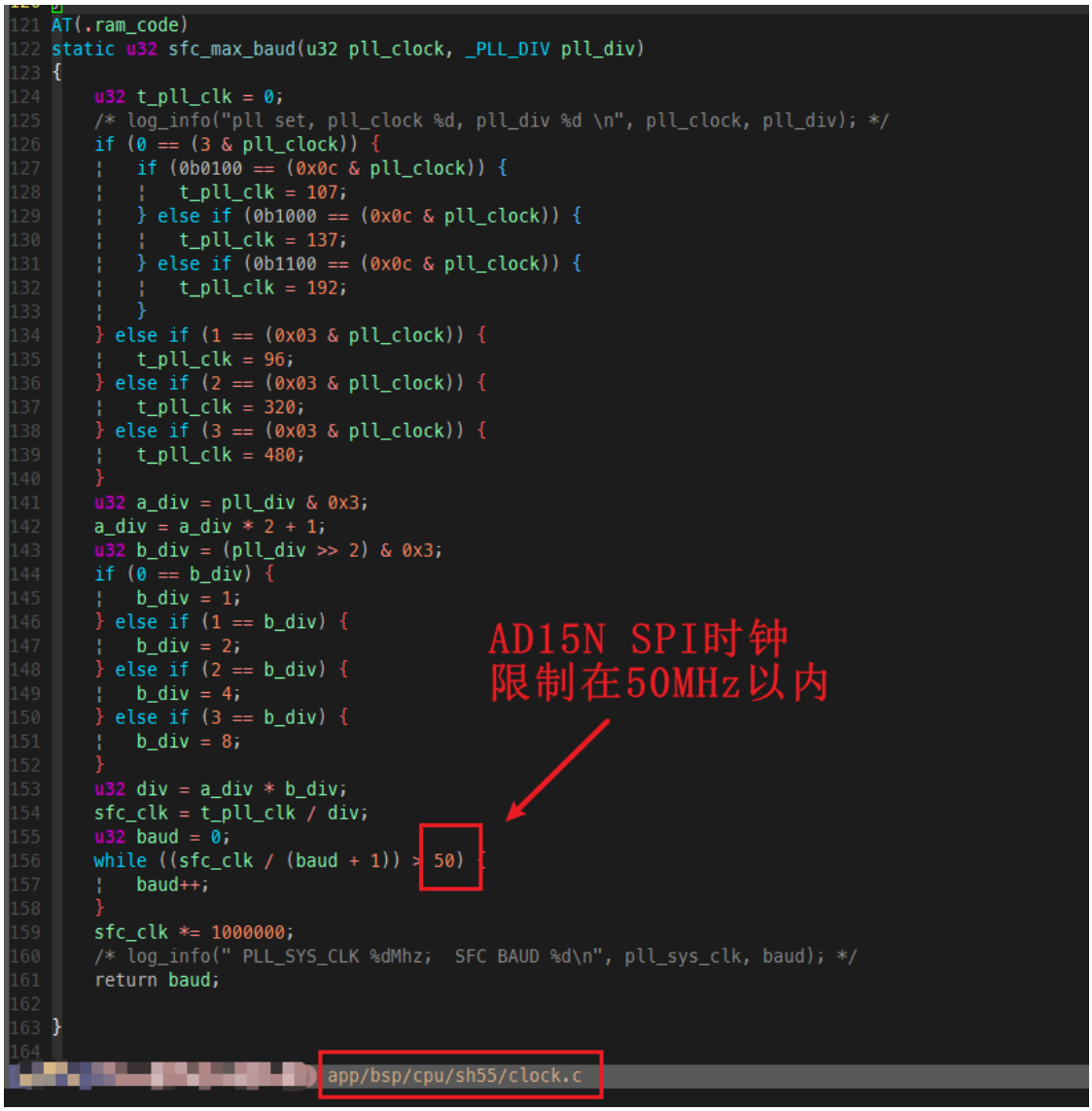
5.2.1 AD14N / AC104N SPI 时钟限制




```
121 AT(.sfc.text.cache.L2)
122 static u32 sfc_max_baud(u32 pll_clock, _PLL_DIV pll_div)
123 {
124     u32 t_pll_clk = 0;
125     /* log_info("pll set, pll_clock %d, pll_div %d \n", pll_clock, pll_div); */
126     if (0 == (3 & pll_clock)) {
127     ¦   if (0b0100 == (0x0c & pll_clock)) {
128     ¦   ¦   t_pll_clk = 107;
129     ¦   } else if (0b1000 == (0x0c & pll_clock)) {
130     ¦   ¦   t_pll_clk = 137;
131     ¦   } else if (0b1100 == (0x0c & pll_clock)) {
132     ¦   ¦   t_pll_clk = 192;
133     ¦   }
134     } else if (1 == (0x03 & pll_clock)) {
135     ¦   t_pll_clk = 96;
136     } else if (2 == (0x03 & pll_clock)) {
137     ¦   t_pll_clk = 320;
138     } else if (3 == (0x03 & pll_clock)) {
139     ¦   t_pll_clk = 480;
140     }
141     u32 a_div = pll_div & 0x3;
142     a_div = a_div * 2 + 1;
143     u32 b_div = (pll_div >> 2) & 0x3;
144     if (0 == b_div) {
145     ¦   b_div = 1;
146     } else if (1 == b_div) {
147     ¦   b_div = 2;
148     } else if (2 == b_div) {
149     ¦   b_div = 4;
150     } else if (3 == b_div) {
151     ¦   b_div = 8;
152     }
153     u32 div = a_div * b_div;
154     sfc_clk = t_pll_clk / div;
155     u32 baud = 0;
156     while ((sfc_clk / (baud + 1)) > 60) {
157     ¦   baud++;
158     }
159     sfc_clk *= 1000000;
160     /* log_info(" PLL_SYS_CLK %d;  SFC BAUD %d\n",sfc_clk, baud); */
161     return baud;
162
163 }
```

AD14N / AC104N SPI 时钟限制

5.2.2 AD15N SPI 时钟限制




```
AT(.ram_code)
static u32 sfc_max_baud(u32 pll_clock, _PLL_DIV pll_div)
{
    u32 t_pll_clk = 0;
    /* log_info("pll set, pll_clock %d, pll_div %d \n", pll_clock, pll_div); */
    if (0 == (3 & pll_clock)) {
        if (0b0100 == (0x0c & pll_clock)) {
            t_pll_clk = 107;
        } else if (0b1000 == (0x0c & pll_clock)) {
            t_pll_clk = 137;
        } else if (0b1100 == (0x0c & pll_clock)) {
            t_pll_clk = 192;
        }
    } else if (1 == (0x03 & pll_clock)) {
        t_pll_clk = 96;
    } else if (2 == (0x03 & pll_clock)) {
        t_pll_clk = 320;
    } else if (3 == (0x03 & pll_clock)) {
        t_pll_clk = 480;
    }
    u32 a_div = pll_div & 0x3;
    a_div = a_div * 2 + 1;
    u32 b_div = (pll_div >> 2) & 0x3;
    if (0 == b_div) {
        b_div = 1;
    } else if (1 == b_div) {
        b_div = 2;
    } else if (2 == b_div) {
        b_div = 4;
    } else if (3 == b_div) {
        b_div = 8;
    }
    u32 div = a_div * b_div;
    sfc_clk = t_pll_clk / div;
    u32 baud = 0;
    while ((sfc_clk / (baud + 1)) > 50) {
        baud++;
    }
    sfc_clk *= 1000000;
    /* log_info(" PLL_SYS_CLK %dMhz;  SFC BAUD %d\n", pll_sys_clk, baud); */
    return baud;

}
```

AD15N SPI 时钟限制

5.2.3 AD17N SPI 时钟限制



AD17N SPI 时钟限制

5.3 时钟相关函数

5.3.1 函数 void pll_sel(u32 pll_clock, _PLL_DIV pll_div, _PLL_B_DIV pll_b_div)

此函数实现动态配置系统时钟源和分频，系统时钟配置关系如下：

sys_clk = pll_clock / pll_div / pll_b_div;

注：选择系统时钟时，优先调整二级分频 *pll_b_div*，无法满足所需要的时钟下再调整一级分频 *pll_div*！

AD17N 的二级分频参数命名为 hsb_div，枚举类型为 _HSB_CLK_DIV，效果与其他芯片一致！

其中参数：

```
1. pll_clock：PLL时钟源
    ①、PLL_96M;
    ①、PLL_107M;
    ①、PLL_137M;
    ①、PLL_192M;
    ①、PLL_320M;
    ①、PLL_480M;
2. pll_div：PLL时钟一级分频
    ①、可选PLL_DIV1~PLL_DIV56，具体请参考_PLL_DIV枚举；
```

(续下页)

(接上页)

```
3. pll_b_div: PLL时钟二级分频
    ①、可选PLL_B_DIV1~PLL_B_DIV8，具体请参考_pll_b_div枚举；
```

5.3.2 函数 int clk_get(const char *name);

此函数实现获取各时钟域的频率，具体参数请查看函数内部。

5.3.3 函数 int clk_set(const char *name, int clk)

此函数用于 AD18，实现动态配置系统时钟源和分频

其中参数:

```
1. name: 时钟名，可配置以下时钟
    ①、sys: 系统时钟，即hsb时钟；
    ①、lsb: 低速时钟；
    ①、sfc: sfc时钟；
2. clk: 时钟设置目标频率
```

CHAPTER 6

内置系统 flash 写保护

本说明适用于开源 SDK，主要介绍如何将内置 flash 的代码与资源区域设置为写保护状态，包括以下几点：

- 对于 A0 芯片，介绍用户如何根据 flash 手册添加写保护参数；
- 对于内封 flash 的芯片，如何使用烧写器自动获取对应 flash 型号的写保护参数；
- 介绍程序中如何打开与关闭内置 flash 写保护，介绍写保护相关接口；

支持的 SDK 版本

芯片	支持的 SDK 版本
AD14N	ad140-release_v1.4.0 开源 sdk_v1.6.0 以及之后的 SDK
AC104N	ac104-release_v1.4.0 开源 sdk_v1.6.0 以及之后的 SDK
AD15N	ad150-release_v1.4.0 开源 sdk_v1.6.0 以及之后的 SDK
AD17N	开源 sdk_v1.7.0 以及之后的 SDK
AD18N	开源 sdk_v1.8.0 以及之后的 SDK

6.1 A0 芯片如何添加写保护参数

6.1.1 flash 写保护工具说明

SDK 的下载目录（路径：app\post_build\sh5x\voice_toy(工程)\）中存放有 “flash_write_protect” 文件夹，文件夹中包含以下文件：

- flash_wp_info.csv：该文件以文本形式记录各种 flash 的写保护参数
- csv2dir.bat：该脚本实现将上述 csv 文件转成小机可识别的 dir_sys_info 文件

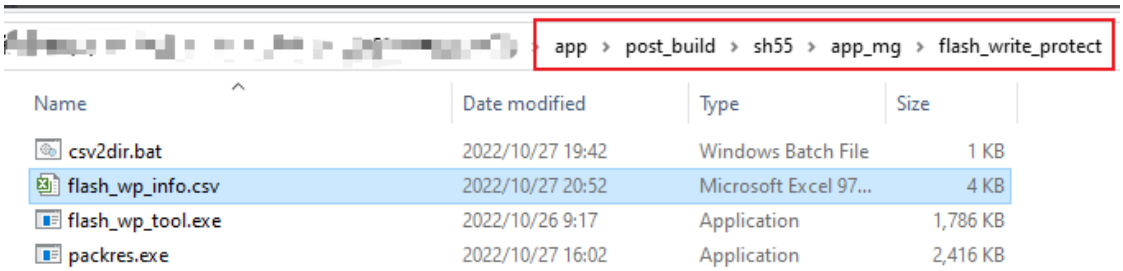


图 2.1 flash 写保护工具

其中，flash_wp_info.csv 文件中涵盖了内封 flash 的绝大部分型号。对于 A0 芯片，用户可自行根据使用的 flash 型号 datasheet 添加相应的写保护参数以及指令。

添加完成后，点击 csv2dir.bat 脚本，即可生成 dir_sys_info 文件，将该文件放在下载目录中，添加到 download.bat 的资源文件中下载到小机中，小机即可识别并添加写保护。

运行 csv2dir.bat 脚本时，需要注意是否有执行成功。

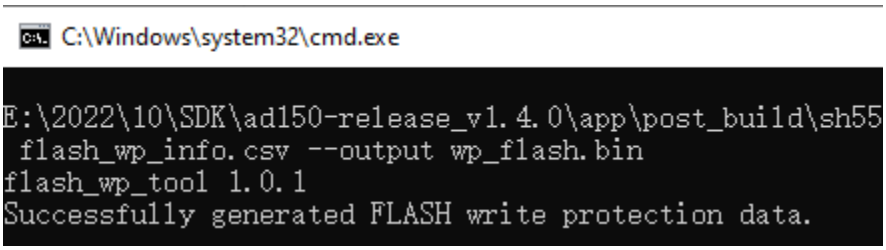


图 2.2 csv2dir.bat 脚本执行成功显示信息

没有执行成功时，需要关注错误信息，重新修改 flash_wp_info.csv 文件；下图的错误信息显示同时存在两组相同的 flash 写保护参数。

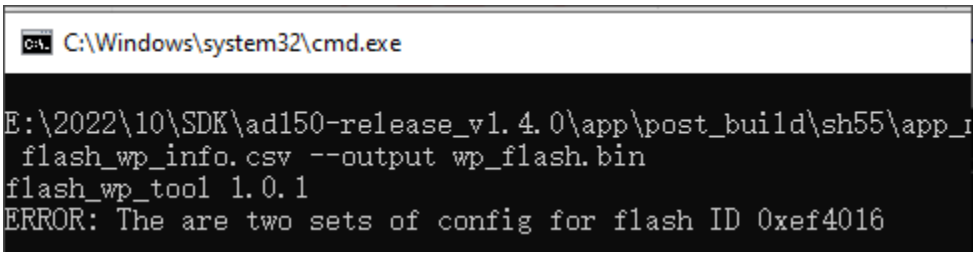


图 2.3 csv2dir.bat 脚本执行失败显示信息

6.1.2 新增 flash 写保护参数与指令

如下对于 A0 的芯片，用户可自行根据需要使用的 flash 的 datasheet，在 flash_wp_info.csv 文件中添加写保护参数以及相关指令，下文以华邦 W25Q32 FLASH 为例，介绍根据手册添加参数。

Flash SR1、SR2 寄存器和写保护范围

在 flash datasheet 中，可通过查找关键字“BP1”快速找到 SR1 寄存器，查找关键字“LB1”快速找到 SR2 寄存器；

检查 SR1 寄存器中是否有 SEC、TB、BP2、BP1、BP0 位，检查 SR2 寄存器是否有 CMP 位；

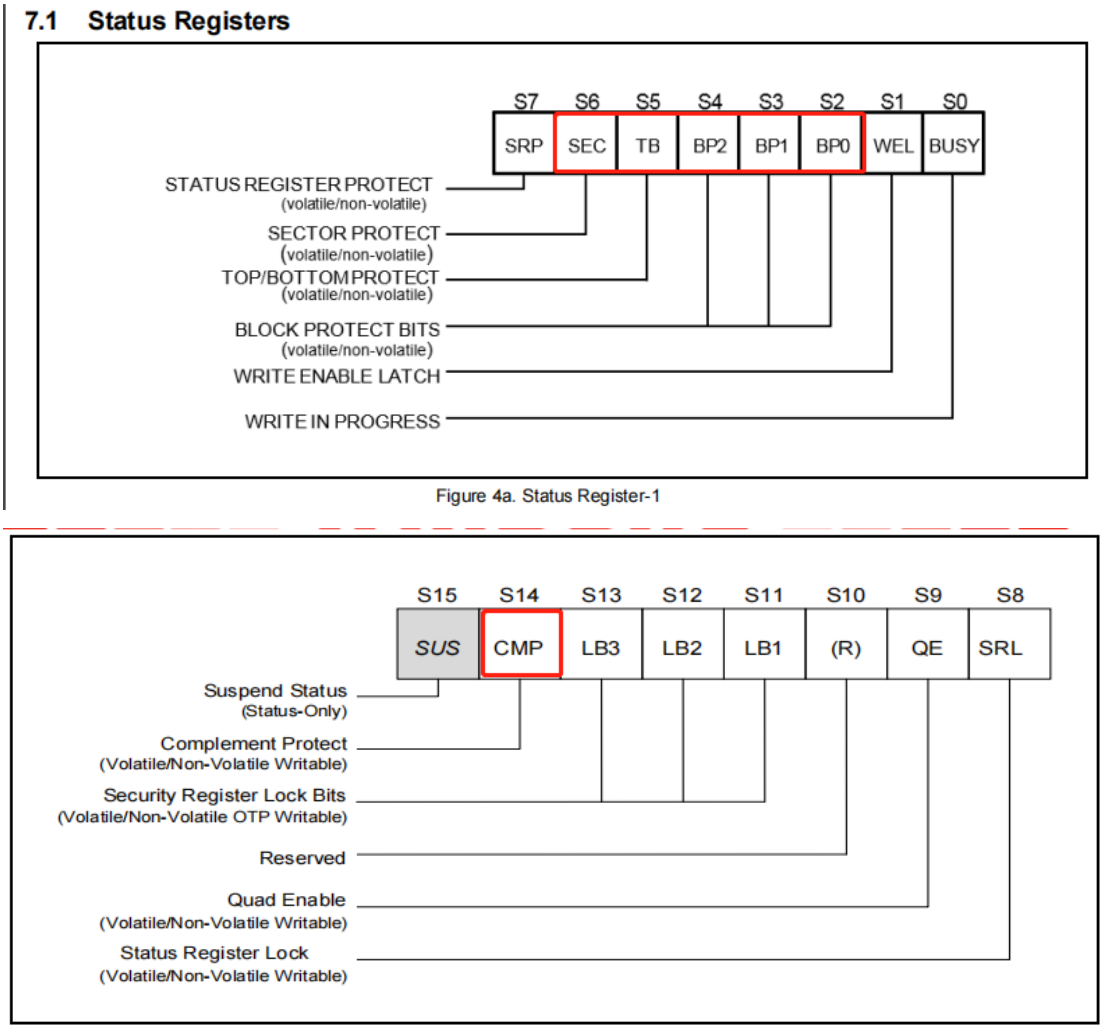


图 2.4 flash SR1 与 SR2 寄存器

通过查找关键字“Density”快速搜索写保护列表，并查看 flash 支持的写保护范围。

7.1.8 Status Register Memory Protection (WPS = 0, CMP = 0)

STATUS REGISTER[1]					W25Q32JV (32M-BIT) MEMORY PROTECTION[3]			
SEC	TB	BP2	BP1	BP0	PROTECTED BLOCK(S)	PROTECTED ADDRESSES	PROTECTED DENSITY	PROTECTED PORTION[2]
X	X	0	0	0	NONE	NONE	NONE	NONE
0	0	0	0	1	63	3F0000h – 3FFFFFh	64KB	Upper 1/64
0	0	0	1	0	62 and 63	3E0000h – 3FFFFFh	128KB	Upper 1/32
0	0	0	1	1	60 thru 63	3C0000h – 3FFFFFh	256KB	Upper 1/16
0	0	1	0	0	56 thru 63	380000h – 3FFFFFh	512KB	Upper 1/8
0	0	1	0	1	48 thru 63	300000h – 3FFFFFh	1MB	Upper 1/4
0	0	1	1	0	32 thru 63	200000h – 3FFFFFh	2MB	Upper 1/2

7.1.9 Status Register Memory Protection (WPS = 0, CMP = 1)

STATUS REGISTER[1]					W25Q32JV (32M-BIT) MEMORY PROTECTION[3]			
SEC	TB	BP2	BP1	BP0	PROTECTED BLOCK(S)	PROTECTED ADDRESSES	PROTECTED DENSITY	PROTECTED PORTION[2]
X	X	0	0	0	0 thru 63	000000h – 3FFFFFh	4MB	ALL
0	0	0	0	1	0 thru 62	000000h – 3EFFFFh	4,032KB	Lower 63/64
0	0	0	1	0	0 and 61	000000h – 3DFFFFh	3,968KB	Lower 31/32
0	0	0	1	1	0 thru 59	000000h – 3BFFFFh	3,840KB	Lower 15/16
0	0	1	0	0	0 thru 55	000000h – 37FFFFh	3,584KB	Lower 7/8
0	0	1	0	1	0 thru 47	000000h – 2FFFFFh	3MB	Lower 3/4
0	0	1	1	0	0 thru 31	000000h – 1FFFFFh	2MB	Lower 1/2

图 2.5 flash 写保护范围

如何添加 flash 写保护参数

打开 flash_wp_info.csv 文件并根据 3.1.1 中的信息填写参数，我司配置 flash 写保护范围从 0 地址开始，向后覆盖。

表格找到上一组 flash 数据空两至三行后，在第一行填写 flash 标号、flash id 以及写使能（Write Enable）指令。

	A	B	C	D	E	F	G	H
1	addr	cmp	BP4	BP3	BP2	BP1	BP0	
222	flash标号	id	写使能命令					
223	W25Q32	0xef4016	06H					
224	128	0	0					
225	256	0	0					
226	512	0	0					
227	1024	0	0					
228	2048	0	0					
229	3072	1	0	0	1	0	1	
230	3584	1	0	0	1	0	0	
231	3968	1	0	0	0	1	0	
232	4032	1	0	0	0	0	1	
233	RDSR1	05H						
234	RDSR2	35H						
235	WRSR	01H	sr1	sr2				
236								
237								

8.2 Instruction Descriptions
8.2.1 Write Enable (06h)
8.2.2 Write Enable for Volatile Status Register (50h)
8.2.3 Write Disable (04h)
8.2.4 Read Status Register-1 (05h), Status Register-2 (35
8.2.5 Write Status Register-1 (01h), Status Register-2 (31
8.2.6 Read Data (03h)

flash 标号、ID 以及写使能指令信息

第二行开始填写 flash 写保护参数，最大支持 9 组 flash 写保护参数。

通过保护范围的大小，设置 SEC、TB、BP0、BP1、BP2（不同的 flash SEC 和 TB 标识可能是其它，如 BP4、BP3）和 CMP 的值。

例如：当 CMP = 0 时，SEC TB BP2 BP1 BP0 为 0b01101，可保护 000000h - 0fffffh 范围内 1MB 的数据。

	A	B	C	D	E	F	G
1	addr	cmp	BP4	BP3	BP2	BP1	BP0
222							
223	W25Q32	0xef4016	06H				
224	128	0	0	1	0	1	0
225	256	0	0	1	0	1	1
226	512	0	0	1	1	0	0
227	1024	0	0	1	1	0	1
228	2048	0	0	1	1	1	0
229	3072	1	0	0	1	0	1
230	3584	1	0	0	1	0	0
231	396						
232	403						
233	RDSR1						
234	RDSR2						
235	WRSR						
236							
237							
238							
239							
240							
241							
242							
243							

7.1.8 Status Register Memory Protection (WPS = 0, CMP = 0)

STATUS REGISTER(1)					W25Q32JV (32M-BIT) MEMORY PROTECTION(3)			
SEC	TB	BP2	BP1	BP0	PROTECTED BLOCK(S)	PROTECTED ADDRESSES	PROTECTED DENSITY	PROTECTED PORTION(2)
0	1	0	0	1	0	000000h – 00FFFFh	64KB	Lower 1/64
0	1	0	1	0	0 and 1	000000h – 01FFFFh	128KB	Lower 1/32
0	1	0	1	1	0 thru 3	000000h – 03FFFFh	256KB	Lower 1/16
0	1	1	0	0	0 thru 7	000000h – 07FFFFh	512KB	Lower 1/8
0	1	1	0	1	0 thru 15	000000h – 0FFFFFh	1MB	Lower 1/4
0	1	1	1	0	0 thru 31	000000h – 1FFFFFh	2MB	Lower 1/2
X	X	1	1	1	0 thru 63	000000h – 3FFFFFh	4MB	ALL

图 2.7 flash 写保护范围填写说明 1

当 CMP = 1 时，SEC TB BP2 BP1 BP0 为 0b00001，可保护 000000h - 3effffh 范围内 4032KB 的数据。

	A	B	C	D	E	F	G
1	addr	cmp	BP4	BP3	BP2	BP1	BP0
222							
223	W25Q32	0xef4016	06H				
224	128	0	0	1	0	1	0
225	256	0	0	1	0	1	1
226	512	0	0	1	1	0	0
227	1024	0	0	1	1	0	1
228	2048	0	0	1	1	1	0
229	3072	1	0	0	1	0	1
230	3584	1	0	0	1	0	0
231	3968	1	0	0	0	1	0
232	4032	1	0	0	0	0	1
233	RDSR1	05H					
234	RDSR2	35H					
235	WRSR	01H					
236							
237							
238							
239							
240							
241							
242							
243							
244							

7.1.9 Status Register Memory Protection (WPS = 0, CMP = 1)

STATUS REGISTER(1)					W25Q32JV (32M-BIT) MEMORY PROTECTION(3)			
SEC	TB	BP2	BP1	BP0	PROTECTED BLOCK(S)	PROTECTED ADDRESSES	PROTECTED DENSITY	PROTECTED PORTION(2)
X	X	0	0	0	0 thru 63	000000h – 3FFFFFh	4MB	ALL
0	0	0	0	1	0 thru 62	000000h – 3EFFFFh	4,032KB	Lower 63/64
0	0	0	1	0	0 and 61	000000h – 3DFFFFh	3,968KB	Lower 31/32
0	0	0	1	1	0 thru 59	000000h – 3BFFFFh	3,840KB	Lower 15/16
0	0	1	0	0	0 thru 55	000000h – 37FFFFh	3,584KB	Lower 7/8
0	0	1	0	1	0 thru 47	000000h – 2FFFFFh	3MB	Lower 3/4
0	0	1	1	0	0 thru 31	000000h – 1FFFFFh	2MB	Lower 1/2

图 2.8 flash 写保护范围填写说明 2

flash 读写 SR1、SR2 指令

Flash 配置写保护时，需要读写 SR1、SR2 寄存器。查找手册找到读写 SR1、SR2 的指令，并填入表格中。

如果 datasheet 上有 31H 命令，优先使用分开写 SR1、SR2 模式；

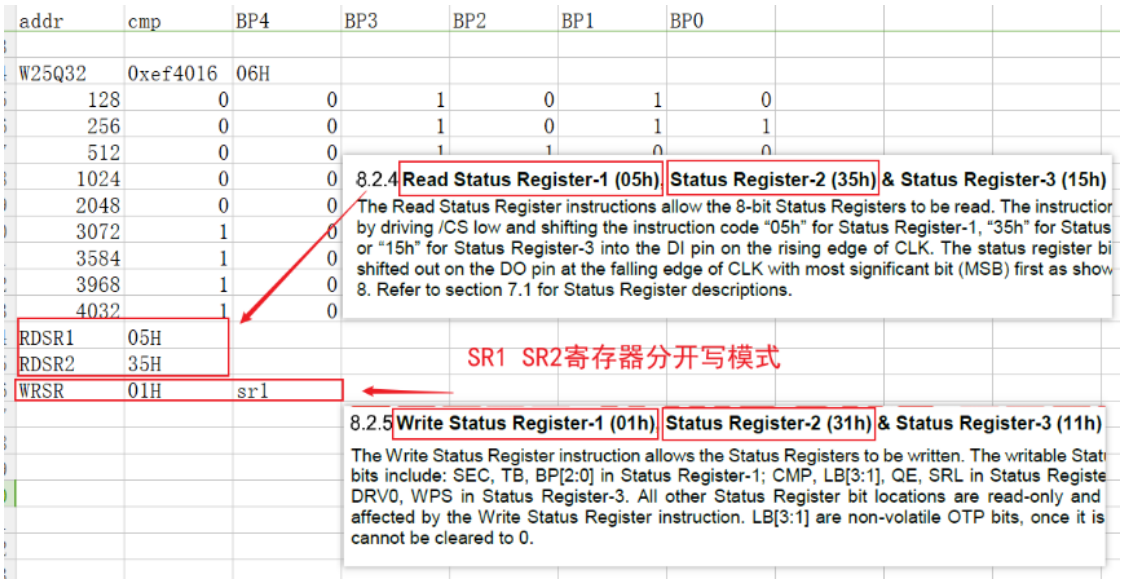


图 2.9 flash 分开写 SR1 和 SR2 寄存器指令

若上述分开写 SR1、SR2 模式在芯片运行中添加写保护不成功，则使用 01H 指令连续写 SR1、SR2 模式；

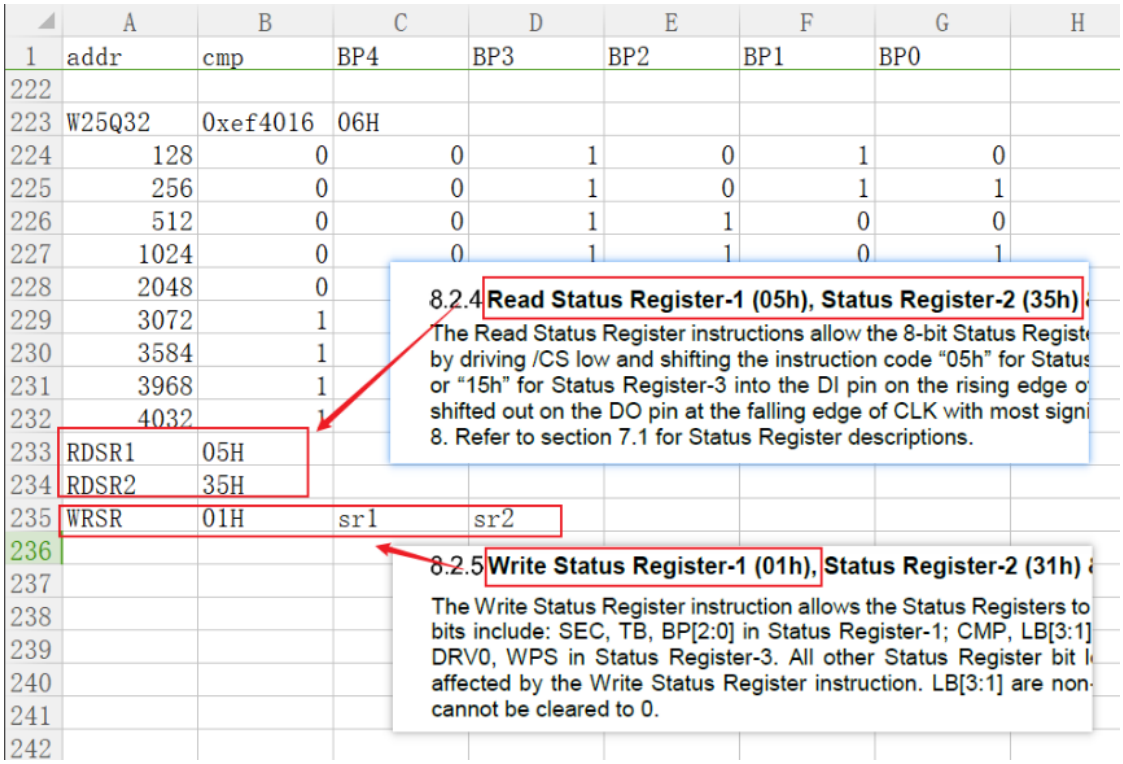


图 2.10 flash 连续写 SR1 和 SR2 寄存器指令

部分 Flash 不存在 SR2 寄存器，则无需写 SR2 相关指令。



	A	B	C	D	E	F	G
1	addr	cmp	BP4	BP3	BP2	BP1	BP0
106							
107	4_ZE ZB25	0x5E3213	06H				
108	256	0	0	0	1	1	0
109	384	0	0	0	1	0	1
110	448	0	0	0	1	0	0
111	480	0	0	0	0	1	1
112	496	0	0	0	0	1	0
113	504	0	0	0	0	0	1
114	RDSR1	05H					
115	WRSR	01H	sr1				

图 2.11 flash 只读写 SR1 寄存器指令

6.1.3 写保护参数下载说明

添加好写保护参数后，根据 2.1 的说明执行脚本生成 dir_sys_info 文件，并将其添加到 SDK 资源列表中，小机通过路径搜索找到该文件并进行解析。

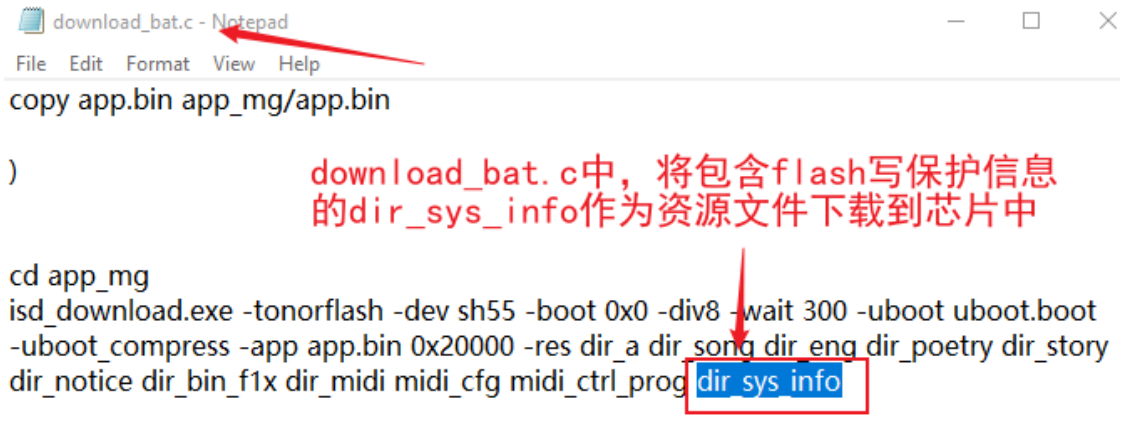


图 2.12 download_bat.c 中将 dir_sys_info 添加到资源列表中

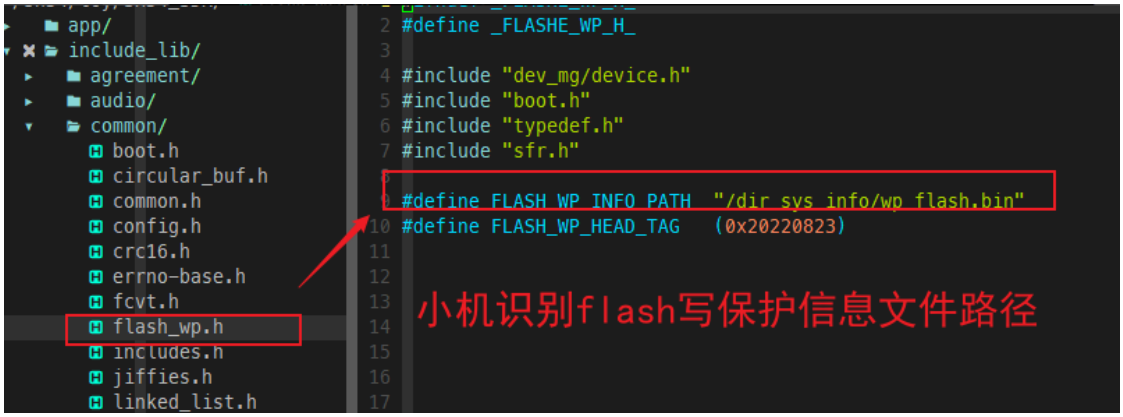


图 2.13 小机识别 flash 写保护信息文件路径

6.1.4 相关函数说明

函数 struct flash_wp_arg *find_flash_wp_file_arg(void)

该函数实现解析 dir_sys_info 中的 wp_flash.bin 文件，逐个查找与正在运行的 flash 匹配的写保护参数，其中：

1. **返回值：**
   - 成功，返回匹配成功的写保护结构体 flash_wp_arg 指针；
   - 失败，返回 NULL；

6.2 内封 flash 芯片如何添加写保护参数（旧）

上文通过表格的方式保存 flash 写保护参数，存在更新不及时的问题。对于杰理非 A0 的芯片，内封的 flash 型号可能会发生变动，不适合用表格形式记录。

我司烧写器可联网更新 flash 写保护参数，烧写器可在烧写阶段将内封 flash 对应的写保护参数记录在 flash 最尾部，由程序去查找该文件并配置写保护。

本文将介绍如何配置 ini 文件实现该功能；

支持旧的内封 flash 写保护策略的芯片：

支持写保护策略的芯片	AD14N / AC104N	AD15N	AD17N

6.2.1 配置烧写器获取内封 flash 写保护参数

在 SDK 的下载目录（路径：app\post_build\sh5x\app_mg（或其他工程）\）中打开 isd_config.ini 文件，并在其中添加烧写器 flash 写保护配置项。

。[BURNER_PASSTHROUGH_CFG] FLASH_WRITE_PROTECT=YES; 。

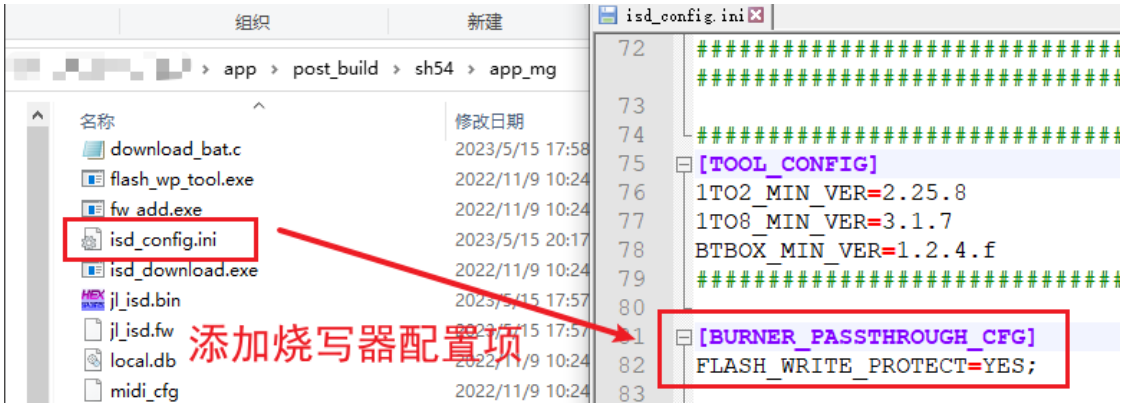


图 3.1 isd_config.ini 文件添加烧写器写保护配置项

6.2.2 配置强制下载工具获取内封 flash 写保护参数

isd_config.ini 文件添加了 FLASH_WRITE_PROTECT=YES 配置后，使用强制升级工具更新程序会报如下错误：

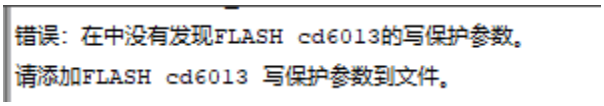


图 3.2 强制升级工具报错

解决办法是将 FLASH_WRITE_PROTECT 注释掉或者配置为 NO。

6.2.3 相关函数说明

函数 int norflash_set_write_protect(u8 enable_write_protect)

该函数实现解析烧写器保存在 flash 末尾的写保护参数，其中：

1. 1. 返回值：- 成功，返回匹配成功的写保护结构体 flash_wp_arg 指针；- 失败，返回 NULL；

6.3 内封 flash 芯片如何添加写保护参数（新）

部分芯片支持新的内封 flash 写保护策略，固件 API 接口没有改变，isd_config.ini、download.bat 配置发生变化，具体请参考文档：OTP_CFG(VOTP) 区域配置说明

支持新的内封 flash 写保护策略的芯片：

支持写保护策略的芯片	AD18N

6.4 程序中开启与关闭写保护功能

SDK 中默认会在内置 flash 初始化的最后，配置 flash 写保护状态。

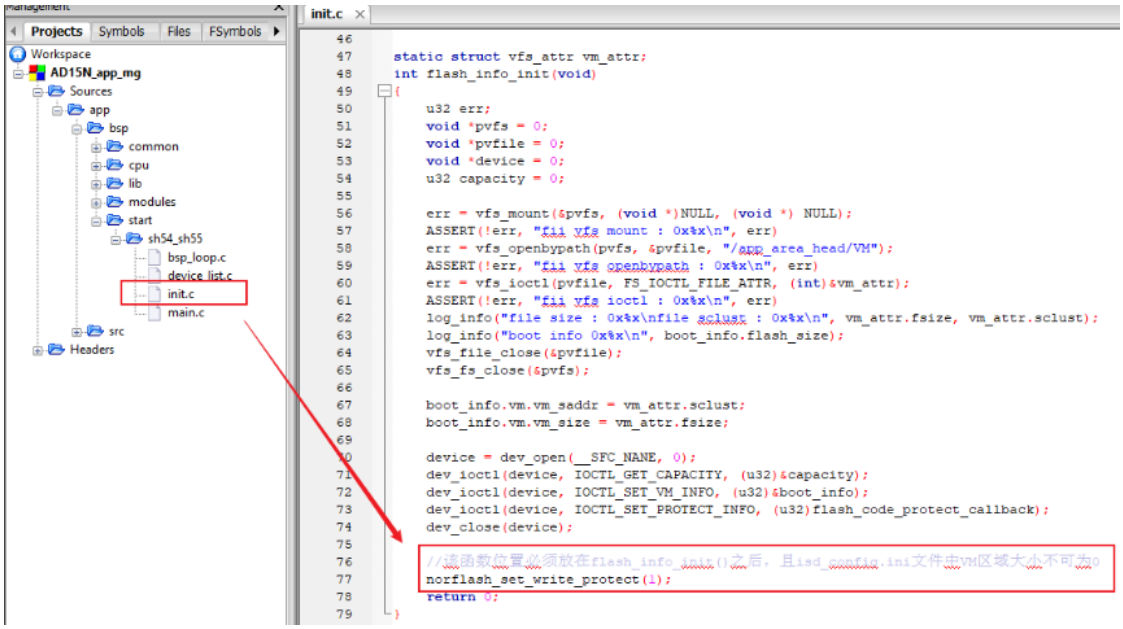


图 4.1 SDK 中添加写保护位置

6.4.1 相关函数说明

函数 int norflash_set_write_protect(u8 enable_write_protect)

该函数实现开启或关闭内置 flash 写保护功能，其中参数：

1. **Enable_write_protect**
   - 1：开启写保护
   - 0：关闭写保护
2. **返回值**
   - 开启写保护成功：写保护最大地址；
   - 关闭写保护成功/开启写保护失败：0

函数 int norflash_write_protect_config(struct device *device, u32 addr, struct flash_wp_arg *p)

该函数实现配置 flash 写保护范围，其中参数：

1. device: 设备句柄，传 NULL 即可；
2. addr：写保护截止范围，传代码资源区后的地址；
3. p：flash 写保护参数；
4. **返回值：**
   - 开启写保护成功：写保护最大地址；
   - 关闭写保护成功/开启写保护失败：0

函数 u16 norflash_read_sr1_sr2(void)

该函数实现获取内置 flash 的 Status Register 1 和 Status Register 2 寄存器，其返回值高 8 位为 SR2，低 8 位为 SR1；

函数 u32 flash_code_protect_callback(u32 offset, u32 len)

该函数实现软件上限制驱动操作内置 flash（存放代码的 flash）.

内置 flash 进行写或擦除操作前会回调该函数，判断会操作到代码与资源区域，则不进行相应操作;

可减低程序跑飞导致程序意外擦写 flash 的概率。其中参数：

1. offset：设备进行写或擦除操作的地址
2. len：设备进行写或擦除操作的长度
3. **返回值：**
   - 开启写保护成功：写保护最大地址；
   - 关闭写保护成功/开启写保护失败：0

CHAPTER 7

中断系统

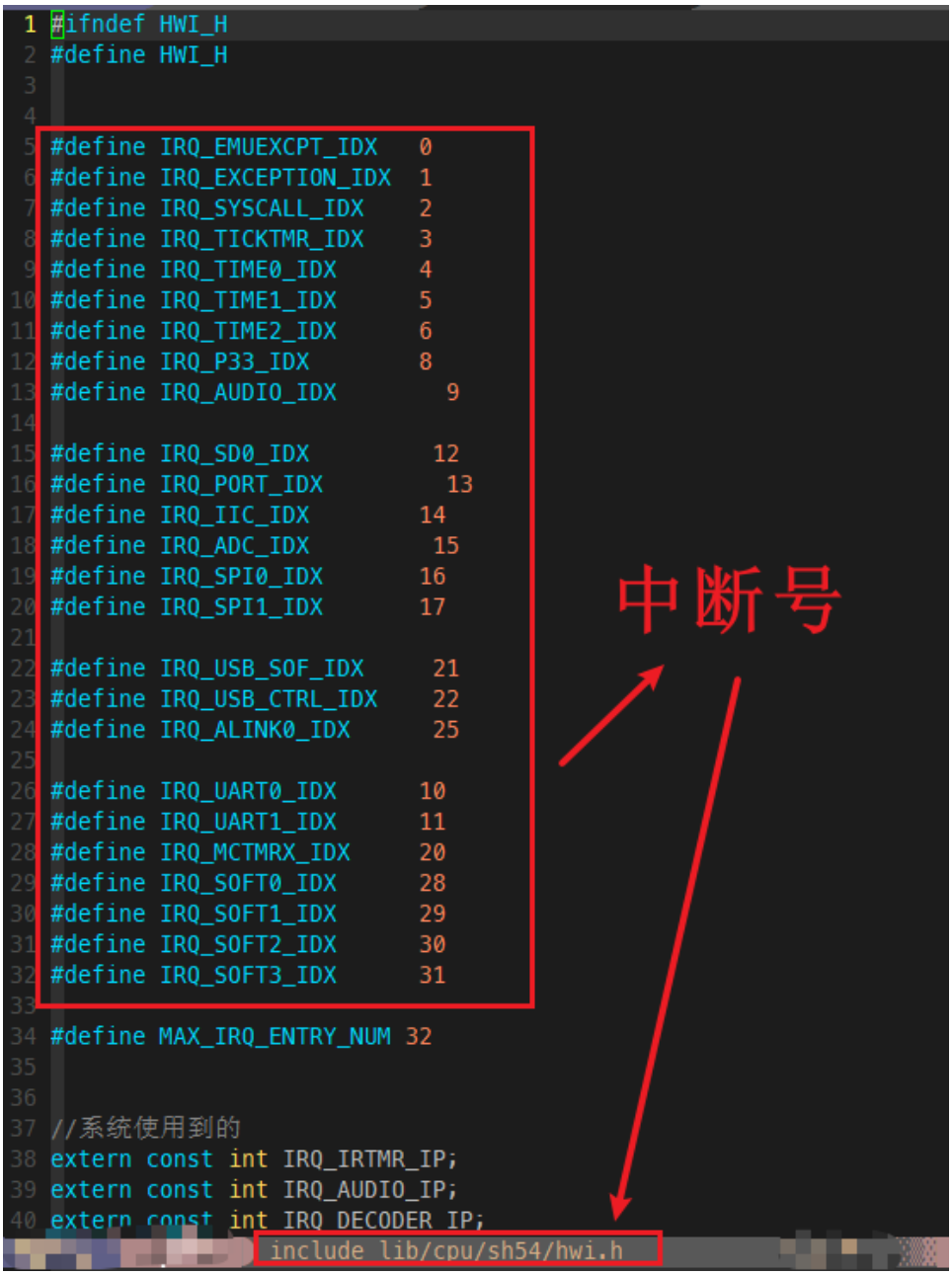
AD 系列支持众多硬件中断，拥有 8 级中断优先级，支持中断嵌套；除了硬件中断，还支持软件中断；中断数量如下表所示：

CPU	中断数量	中断优先级
AD14N / AC104N	32 个（包含 4 个软中断）	0~7，共 8 级
AD15N	32 个（包含 4 个软中断）	0~7，共 8 级
AD17N	32 个（包含 4 个软中断）	0~7，共 8 级
AD18N	64 个（包含 4 个软中断）	0~7，共 8 级

7.1 中断的注册与注销

AD 系列注册一个中断，需要中断号、中断响应函数以及中断优先级；

- 中断号：每一个中断会分配一个唯一的中断号，可在 hwi.h 文件中查看；



AD14N 中断号示意图

- 中断响应函数：中断影响函数类型必须为无返回值的 void 类型，且中断函数定义时需添加中断标志；



中断响应函数添加中断标志示意图

- 中断优先级：中断响应优先级，共 8 级；

7.1.1 函数 void HWI_Install(unsigned char index, unsigned int isr, unsigned char priority)

该函数实现注册一个中断，其中参数:

```
1、index：需要注册的中断号；
2、isr：中断响应函数；
3、priority：中断优先级；
```

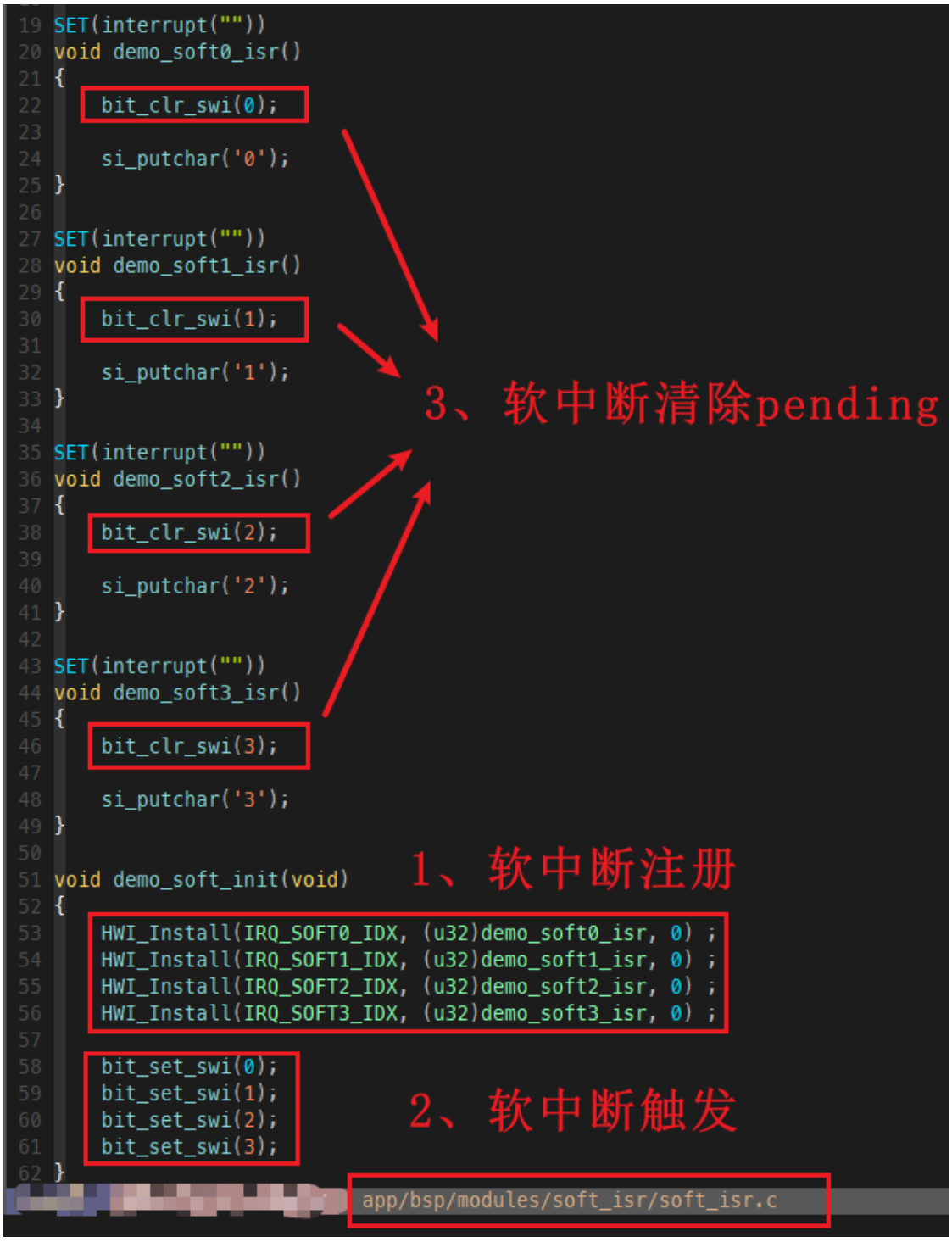
7.1.2 函数 void HWI_Uninstall(unsigned char index)

该函数实现注销一个中断，其中参数:

```
1、index：需要注销的中断号；
```

7.2 软中断说明

AD 系列支持软件中断；软件中断的注册、注销方式与硬件中断一致；SDK 中可在 soft_isr.c 文件中查看示例：



软件中断使用示例

7.2.1 函数 void bit_set_swi(unsigned char index)

该函数实现触发一个软件中断，其中参数:

```
1、index：需要触发的软件中断序号（软中断0~软中断3分别对应0~3）；
```

注：需要注意此处 *index* 与中断号不同；软中断 *2* 的中断号在 *AD14N* 上是 *30*，需要触发该中断时，该接口传参的 *index* 传 *2* 即可！

7.2.2 函数 void bit_clr_swi(unsigned char index)

该函数实现清除一个软中断的 Pending，其中参数:

```
1、index：需要清除pending的软件中断序号（软中断0~软中断3分别对应0~3）；
```

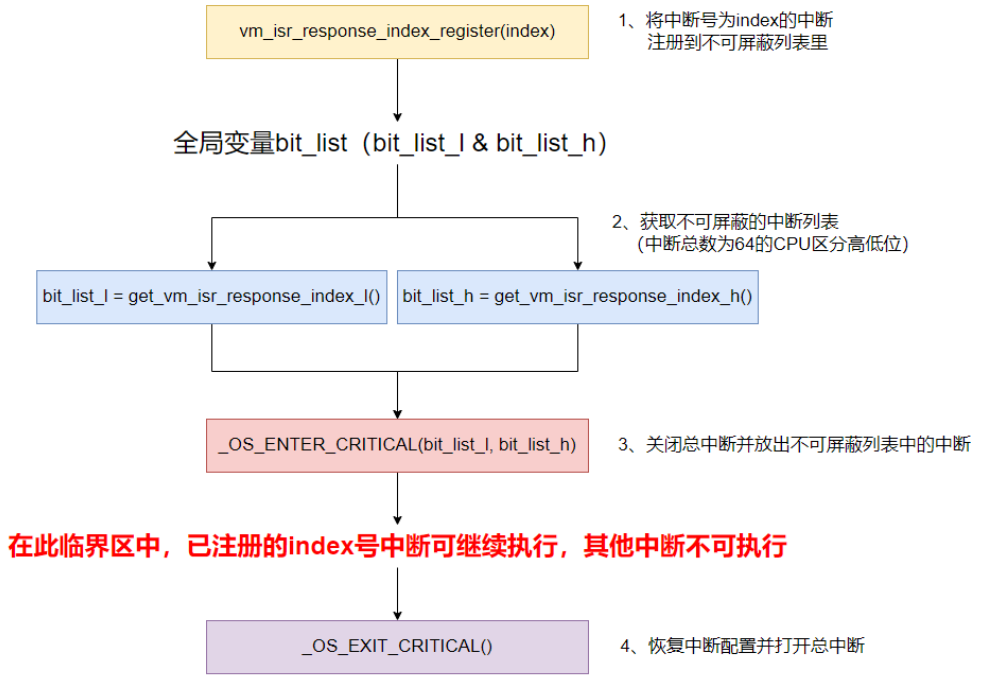
注：需要注意此处 *index* 与中断号不同；软中断 *2* 的中断号在 *AD14N* 上是 *30*，需要清除该中断 *pending* 时，该接口传参的 *index* 传 *2* 即可！

7.3 软件不可屏蔽中断

在进行 VM 擦写时系统会关闭总中断并操作系统 flash，该过程耗时较长，可能会导致播歌卡顿或其他外设中断无法及时响应；

SDK 提供了软件不可屏蔽中断，在关闭总中断的同时允许放出部分中断继续执行（例如放出 audio 中断优化操作 vm 卡音问题），该部分中断函数以及依赖的变量都需要放在 ram 里；

本小节介绍软件不可屏蔽中断相关的接口与使用，流程如下:



软件不可屏蔽中断运行流程图

7.3.1 函数 void _OS_ENTER_CRITICAL(u32 bit_list)

该函数适用于中断总数为 32 的芯片（AD14N / AD15N / AD17N 等），实现关闭总中断的同时放出 bit_list 参数中置 1 的中断，其中参数:

```
1、bit_list：需要单独放出的中断list，bit0~bit31分别对应0号~31号中断；
```

例如 AD14N 需要同时放出 audio（中断号为 9）和 ticktimer 中断（中断号为 3）；那么 bit_list 传入 0x208 即可；

7.3.2 函数 void _OS_ENTER_CRITICAL(u32 bit_list_l, u32 bit_list_h)

该函数适用于中断总数为 64 的芯片（AD16N / AD18N 等），实现关闭总中断的同时放出 bit_list_l 和 bit_list_h 参数中置 1 的中断，其中参数:

```
1、bit_list_l：需要单独放出的中断list的低32位，bit0~bit31分别对应0号~
↪31号中断；
2、bit_list_h：需要单独放出的中断list的高32位，bit0~bit31分别对应32号~
↪63号中断；
```

例如 AD18N 需要同时放出音频 APA 中断 (中断号为 37) 和 ticktimer(中断号为 3)，那么 bit_list_h 传入 0x20，bit_list_l 传入 0x8 即可；

7.3.3 函数 void _OS_EXIT_CRITICAL(void)

该函数适用于中断总数为 32 和 64 的芯片，实现恢复中断配置并退出关中断状态；

7.3.4 函数 void vm_isr_response_index_register(u8 index)

SDK 中存在一个全局变量用于存储需要放出来的中断 list，该函数实现往该变量中添加中断 bit，其中参数:

```
1、index：需要添加的不屏蔽中断号；例如AD14N的audio中断号为9，则传入9即可；
```

7.3.5 函数 void vm_isr_response_index_unregister(u8 index)

该函数适用于中断总数为 32 和 64 的芯片，实现恢复中断配置并退出关中断状态，其中参数:

```
1、index:删除已添加的不屏蔽中断号；
```

7.3.6 函数 u32 get_vm_isr_response_index_l(void)

该函数实现获取全局的不可屏蔽中断 list 的低 32 位，其中:

```
1、对于中断总数为32的芯片，获取到完整的bit_list;
2、对于中断总数为64的芯片，获取到低32位中断的bit_list;
```

7.3.7 函数 u32 get_vm_isr_response_index_h(void)

该函数实现获取全局的不可屏蔽中断 list 的高 32 位，仅中断总数为 64 的芯片有该接口;

7.4 硬件不可屏蔽中断

硬件不可屏蔽中断允许中断在关闭总中断 (即调用了函数 local_irq_disable()) 的情况下继续运行;
目前支持硬件不可屏蔽中断的芯片有：AD17N、AD18N;

注：若需要中断完全不被打断，需要软硬件不可屏蔽中断功能搭配使用;

7.4.1 函数 void irq_unmask_set(u8 index)

该函数实现将传入的 index 中断的优先级设置为最高，并加入硬件不可屏蔽中断列表；其中参数:

```
1、index:需要添加的不屏蔽中断号;
```

7.4.2 函数 void irq_unmask_disable(u8 index)

该函数实现关闭硬件不可屏蔽中断功能;

CHAPTER 8

GPIO 接口说明

AC104/AD14/15/17/18 SDK 提供了操作 gpio 相关功能的函数接口；
其头文件位于 include_lib/cpu/sh5x/gpio.h；
下面分为 4 个部分进行 GPIO 接口函数介绍：

(1)" *DACP，DACN 口设置（AC104/AD14/15/17）* "
(2)" *普通 IO 口 (AC104/AD14/15/17/18)*"
(3)" *模块 IO 映射*"
(4)" *常见问题和注意事项*"

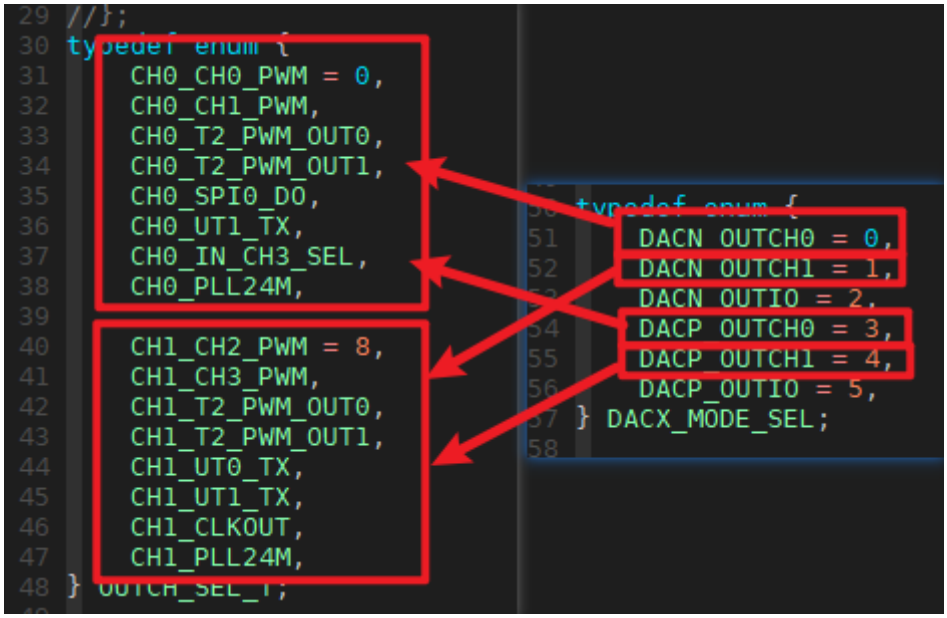
8.1 DACP，DACN 口设置（AC104/AD14/15/17）

（1）void gpio_dacnp_out_mode_init(DACX_MODE_SEL mode, OUTCH_SEL_T outchsel, u8 out)

该函数实现将 DACNO,DACNP 作为普通 IO 输出，或者作为 outputch 输出，其中参数：

```
mode     : DACP引脚选择做普通IO引脚 (DACP_OUTIO) 或者做输出通道(DACP_OUTCHx);
outchsel : 选择output源，该参数只在mode参数选择做输出方式时有效;
out      : 该参数只在mode选择做普通IO引脚输出时有效;
           0: 输出0;
           1: 输出1;
```

注意：
当 **mode** 选择输出通道时，**mode** 选择的通道需要与 **outchsel** 通道一样；
如：**mode** 选择了 DACN_OUTCH0, 则 **outchsel** 必须选择通道 0 的 output 源（即 0~7）；（对应下图）



DACNP_out_mode_init

（2）void gpio_dacnp_out_set(DACX_MODE_SEL mode, u8 out)

该函数实现将 DACNO,DACNP 作为普通 IO 输出，输出 0 还是 1

```
mode : DACN_OUTIO或者DACP_OUTIO;
out  : 0: 输出0;
       1: 输出1;
```

注意：

使用 DACP,DACN 脚前需注意先关闭 APA 模块
详细步骤可点击下方链接：
FAQ:DACN/DACP 引脚如何设置为普通 I/O 输出或高阻态?

8.2 普通 IO 口 (AC104/AD14/15/17/18)

寄存器设置示例:

```
示例PA1 IO口常见操作：
JL_PORTA->DIR |= BIT(1);      //PA1口设为输入
JL_PORTA->DIE |= BIT(1);      //PA1口使能数字输入
JL_PORTA->PU |= BIT(1);       //PA1口打开上拉
JL_PORTA->PD |= BIT(1);       //PA1口打开下拉
JL_PORTA->OUT |= BIT(1);      //PA1口输出高电平
JL_PORTA->OUT &= ~BIT(1);     //PA1口输出低电平
```

函数使用示例：

```
gpio_set_direction(IO_PORTA_01,1);      //PA1口设为输入
gpio_set_die(IO_PORTA_01,1);            //PA1口使能数字输入
gpio_set_pull_up(IO_PORTA_01,1);        //PA1口打开上拉
gpio_set_pull_down(IO_PORTA_01,1);      //PA1口打开下拉
gpio_write(IO_PORTA_01,1);              //PA1口输出高电平
gpio_write(IO_PORTA_01,0);              //PA1口输出低电平

gpio_set_direction(IO_PORT_DP,1);       //USB_DP口设为输入
gpio_set_die(IO_PORTA_01,1);            //USB_DP口使能数字输入
gpio_set_pull_up(IO_PORTA_01,1);        //USB_DP口打开上拉
gpio_set_pull_down(IO_PORTA_01,1);      //USB_DP口打开下拉
gpio_write(IO_PORTA_01,1);              //USB_DP口输出高电平
gpio_write(IO_PORTA_01,0);              //USB_DP口输出低电平
```

下面介绍相关函数

（1）int gpio_set_direction(u32 gpio, u32 dir)
该函数实现设置 io 口方向，其中参数:

```
gpio  : 参考宏IO_PORTx_xx, 如: IO_PORTA_00;
dir   : 0: 输出;
        1: 输入;
返回值 : 0: 设置成功;
        其他值: 设置失败;
```

（2）int gpio_set_pull_up(u32 gpio, int value)
该函数实现设置 gpio 口上拉状态, 其中参数:

```
gpio  : 参考宏IO_PORTx_xx, 如: IO_PORTA_00;
value : 0: 不上拉;
        1: 上拉;
```

注意:

```
该函数只有GPIO设为输入时有效。
```

（3）int gpio_set_pull_down(u32 gpio, int value)
该函数实现设置 gpio 口下拉状态，其中参数

```
gpio  : 参考宏IO_PORTx_xx, 如: IO_PORTA_00;
value : 0: 不下拉;
        1: 下拉;
```

注意:

```
该函数只有GPIO设为输入时有效。
AD18部分IO下拉使用寄存器配置无效（即JL_PORTx->
↪PD），需要用p33接口函数，该函数内部已做处理。
```

详细说明可参考链接*AD18 部分 IO 下拉使用寄存器设置无效*

（4）int gpio_set_die(u32 gpio, int value);
该函数实现设置 io 口数字输入状态，其中参数:

```
gpio  : 参考宏IO_PORTx_xx, 如: IO_PORTA_00
value : 0: IO模拟输入;
        1: IO数字输入;
```

(续下页)

(接上页)

```
返回值 : 0: 设置成功;
         其他值: 设置失败;
```

(5) u32 gpio_set_dieh(u32 gpio, u32 value)
该函数实现设置 io 口输入状态，其中参数:

```
gpio  : 参考宏IO_PORTx_xx, 如: IO_PORTA_00;
value : 0: IO模拟输入;
        1: IO普通输入;
返回值 : 0, 设置成功;
         其他值, 设置失败;
```

(6) int gpio_read(u32 gpio)
该函数实现读取 io 逻辑输入电平，io 口需要设置成数字输入，其中参数:

```
gpio  : 参考宏IO_PORTx_xx, 如: IO_PORTA_00;
返回值 : 0: 逻辑0;
         1: 逻辑1;
```

(7) u32 gpio_write(u32 gpio, u32 value)
该函数实现设置 io 输出值，其中参数

```
gpio  : 参考宏IO_PORTx_xx, 如: IO_PORTA_00;
value : 0: 输出0;
        1: 输出1;
```

8.3 模块 IO 映射

注意：不同芯片映射函数有所区分。

下面分 3 个部分进行介绍:

(1)" *AC104/AD14/AD15*"
(2)" *AD17*"
(3)" *AD18*"

8.3.1 AC104/AD14/AD15

(1) u32 gpio_output_channle(u32 gpio, u32 clk)
该函数实现将 outputchannel 输出到 io 口；其中参数:

```
gpio  : 参考宏IO_PORTx_xx, 如: IO_PORTA_00;
clk   : 参考gpio.h文件中的枚举, 如: CH0_CH0_PWM;
返回值 : 0: 设置成功;
         其他值: 设置失败;
```

注意:

（1）mcpwm 映射时前两路（即 mcpwm0/1）用通道 0，后两路（即 mcpwm2/3）用通道 1；
（2）如果串口使用了固定引脚，则默认使用了 CH1_UT0_TX;

8.3.2 AD17

（1）int gpio_och_sel_output_signal(u32 gpio, enum OUTPUT_CH_SIGNAL signal)
该函数实现部分模块 IO 输出映射到任意 IO

```
gpio  ：参考宏IO_PORTx_xx，如：IO_PORTA_00；
signal：参考gpio.h文件中OUTPUT_CH_SIGNAL的枚举，如：OUTPUT_CH_SIGNAL_TIMER1_PWM；
返回值 ：无效；

可参考定时器配置（timer_pwm.c）
```

（2）int gpio_och_disable_output_signal(u32 gpio, enum OUTPUT_CH_SIGNAL signal)
该函数实现关闭对应模块 IO 输出映射

```
gpio  ：参考宏IO_PORTx_xx，如：IO_PORTA_00；
signal：参考gpio.h文件中OUTPUT_CH_SIGNAL的枚举，如：OUTPUT_CH_SIGNAL_TIMER1_PWM；
返回值 ：无效；
```

（3）int gpio_ich_sel_input_signal(u32 gpio, enum INPUT_CH_SIGNAL signal, enum INPUT_CH_TYPE type)
该函数实现部分模块 IO 输入映射到任意 IO

```
gpio  ：参考宏IO_PORTx_xx，如：IO_PORTA_00；
signal：参考gpio.h文件中INPUT_CH_SIGNAL的枚举，如：INPUT_CH_SIGNAL_TIMER1_CIN；
type  ：参考gpio.h文件中的INPUT_CH_TYPE枚举，如：INPUT_CH_TYPE_GP_ICH；
返回值 ：通道号；

可参考红外按键设置（irflt.c）
```

（4）int gpio_ich_disable_input_signal(u32 gpio, enum INPUT_CH_SIGNAL signal, enum INPUT_CH_TYPE type)
该函数实现关闭对应模块 IO 输入映射

```
gpio  ：参考宏IO_PORTx_xx，如：IO_PORTA_00；
signal：参考gpio.h文件中INPUT_CH_SIGNAL的枚举，如：INPUT_CH_SIGNAL_TIMER1_CIN；
type  ：参考gpio.h文件中的INPUT_CH_TYPE枚举，如：INPUT_CH_TYPE_GP_ICH；
返回值 ：通道号；
```

8.3.3 AD18

（1）u32 gpio_mux_out(u32 gpio, enum GPIO_OUTPUT_FUN fun)
该函数实现部分模块输出映射到任意 io 口；其中参数:

```
gpio  ：参考宏IO_PORTx_xx，如：IO_PORTA_00；
fun   ：参考gpio.h文件中GPIO_OUTPUT_FUN的枚举，如：GPIO_OUT_COMP0_MC_PWM0_H；
```

可参考定时器模块配置（timer_drv.c）

（2）u32 gpio_mux_out_close(u32 gpio, u32 fd)
该函数实现关闭对应模块输出映射；其中参数:

```
gpio  ：参考宏IO_PORTx_xx，如：IO_PORTA_00；
fd    ：参考gpio.h文件中GPIO_OUTPUT_FUN的枚举，如：GPIO_OUT_COMP0_MC_PWM0_H；
```

（3）int gpio_mux_in(u32 gpio, enum GPIO_INPUT_FUN fun)
该函数实现部分模块输入映射到任意 io 口；其中参数:

```
gpio  ：参考宏IO_PORTx_xx，如：IO_PORTA_00；
fun   ：参考gpio.h文件中GPIO_INPUT_FUN的枚举，如：GPIO_INPUT_ICH0_TIMER0_CLK；
```

可参考红外按键模块配置（irflt.c）

（4）u32 gpio_mux_in_close(u32 fd)
该函数实现关闭对应模块的输入映射；其中参数:

```
fd    ：参考gpio.h文件中GPIO_INPUT_FUN的枚举，如：GPIO_INPUT_ICH0_TIMER0_CLK；
```

8.4 常见问题和注意事项

8.4.1 AC104/AD14/AD15/AD17 生产不良率高

[20230626] SDK 同步补丁：修改 GPIO HD 档位默认配置

8.4.2 AD18 部分 IO 下拉使用寄存器设置无效

AD18 PC 口和 PD 口使用寄存器设置下拉无效，建议使用 gpio_set_pull_down 函数来设置 IO 口下拉状态。

CHAPTER 9

设备管理

各芯片支持的设备类型如下：

芯片支持的设备	内置 flash	外置 flash	SDMMC	usb
AD14N / AC104N	√	√	√	√
AD15N	√	√	√	
AD17N	√	√		
AD18N	√	√		√

设备管理库的目录位于 include_lib/liba/dev_mg_lib.a。其中主要的函数有：

```
int  devices_init();
bool dev_online(const char* name);
void* dev_open(const char* name, void* arg);
int  dev_ioctl(void* device, int cmd, u32 arg);
int  dev_close(void* device);
int  dev_bulk_read(void* _device, void* buf, u32 offset, u32 len);
int  dev_bulk_write(void* _device, void* buf, u32 offset, u32 len);
int  dev_byte_read(void* _device, void* buf, u32 offset, u32 len);
int  dev_byte_write(void* _device, void* buf, u32 offset, u32 len);
```

一般设备使用流程：

1. 定义并赋值设备表；
2. 开机调用 devices_init() 完成对所有设备的初始化；
3. 调用 dev_open 打开参数中名字为 name 的设备；
4. 对设备进行读写（dev_bulk_read/dev_bulk_write/dev_byte_read/dev_byte_write）等操作；

9.1 设备注册表

设备注册表 device_tab 中注册了全部需要使用到的设备，其中成员如下:

1. name：设备名称的字符串（自定义名称）
2. ops：设备内部接口函数集合（不可更改）
3. priv_data：设备使用的底层硬件资源及相关模式配置（可根据需要自行配置）

如果要添加设备，直接在下图注册表中接着添加。开机时初始化程序会自动执行初始化。

注：内置 FLASH 只读时不需要挂设备驱动，因为内部硬件已经实现了支持，在使用时向文件系统传递 NULL 的设备句柄就能实现读取内部的 sydf 文件系统!

设备注册表定义及初始化如图 3.1 所示。

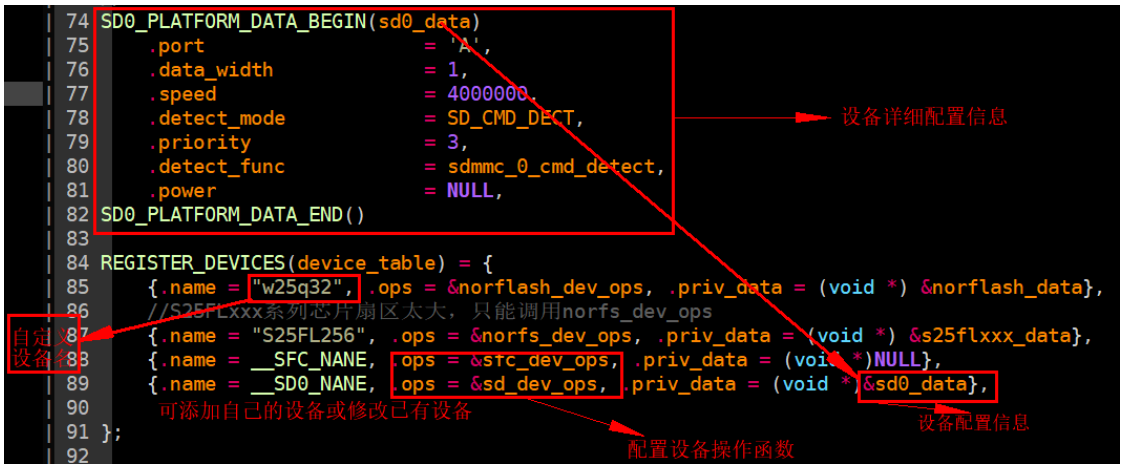


图 1 设备注册表

注意 flash 注册时有两组接口：norflash_dev_ops、norfs_dev_ops（位置见图 2/图 3）。

注册 norflash_dev_ops 接口时，起始地址及长度以 512 字节为单位，比如读取 (dev_bulk_read) 时带入地址 1 实际是从 1*512=512 开始读取，且写函数内部自带擦除操作，可以直接写入。

注册 norfs_dev_ops 接口时，起始地址及长度以 1 字节为单位，写函数内部没有擦除，调写函数前需确定要写入的内存地址中没有存入数据，如果有数据要自己写擦除操作。

（注意：读写 flash 设备操作不是调用图 2 图 3 中函数，而是调用 dev_bulk_read/dev_bulk_write 执行读写）

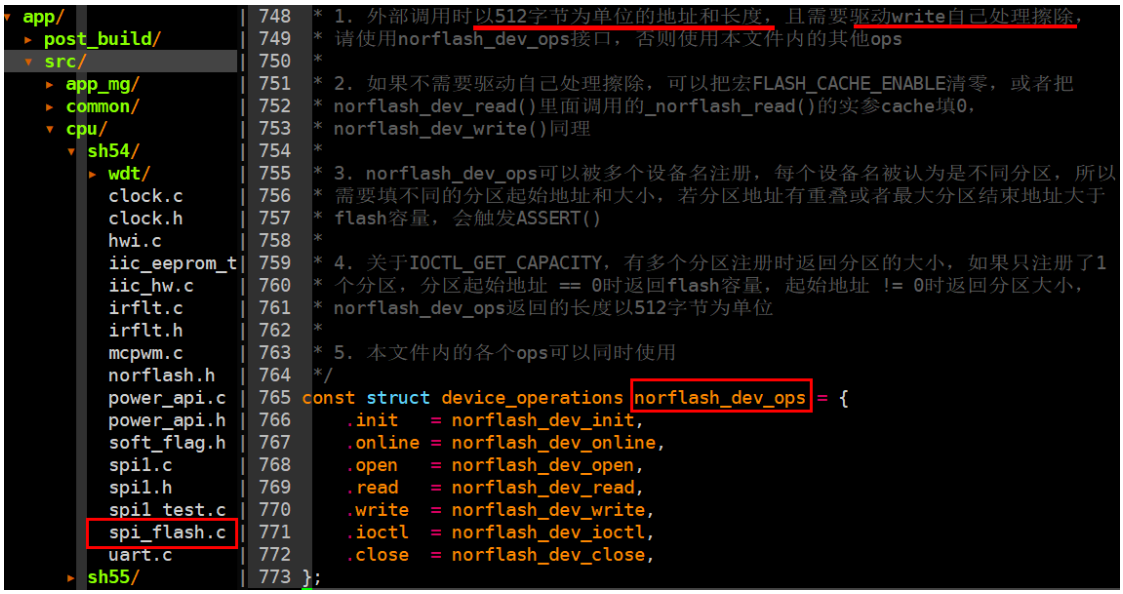


图 2 flash 设备 norflash_dev_ops 接口

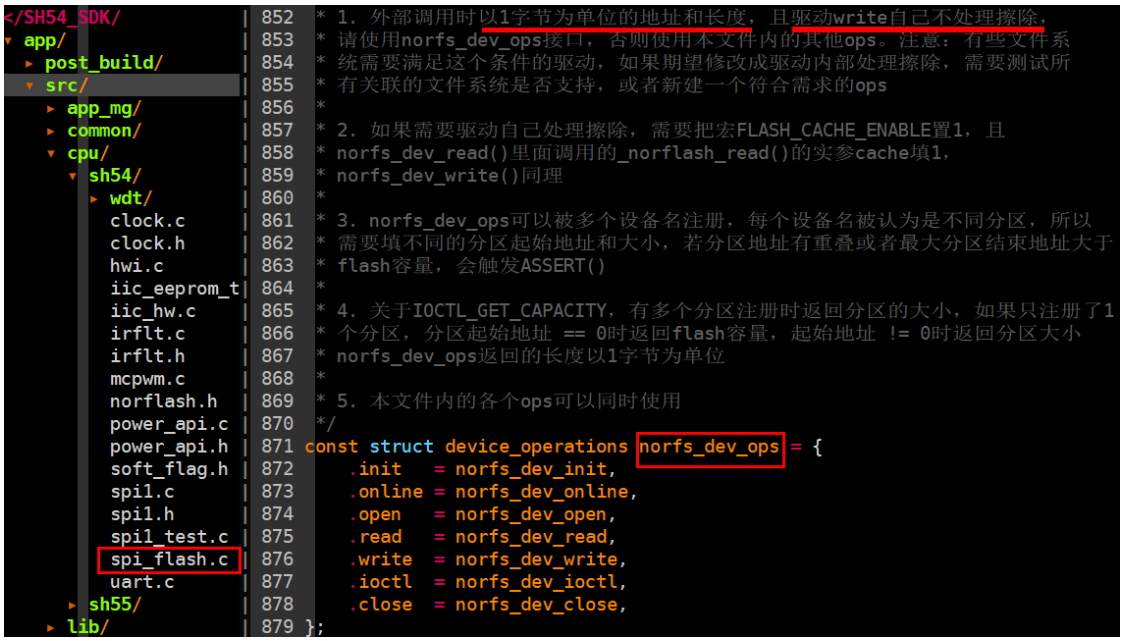


图 3 flasflash 设备 norfs_dev_ops 接口 h 配置信息

图 1 中设备详细数据配置信息介绍见下图，SD 接口配置信息见图 4，flash 配置信息图 5。

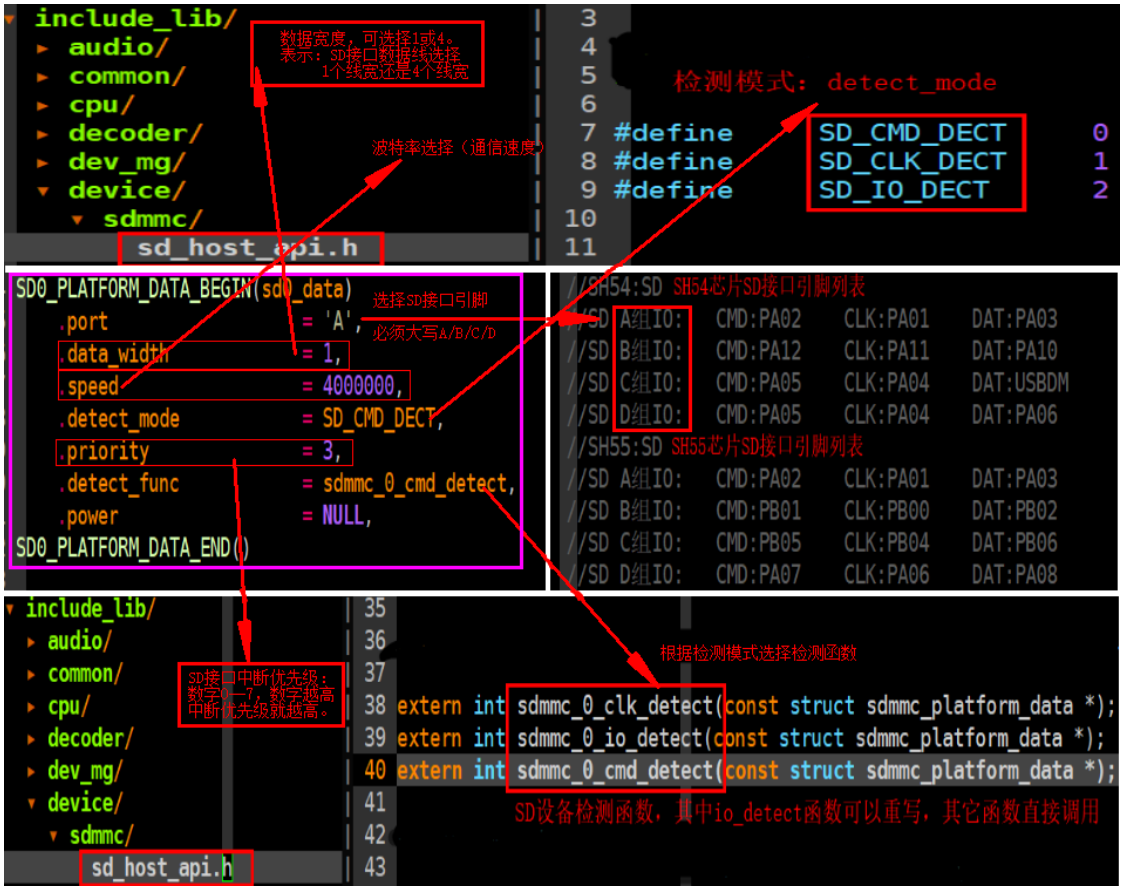


图 4 SD 接口配置信息

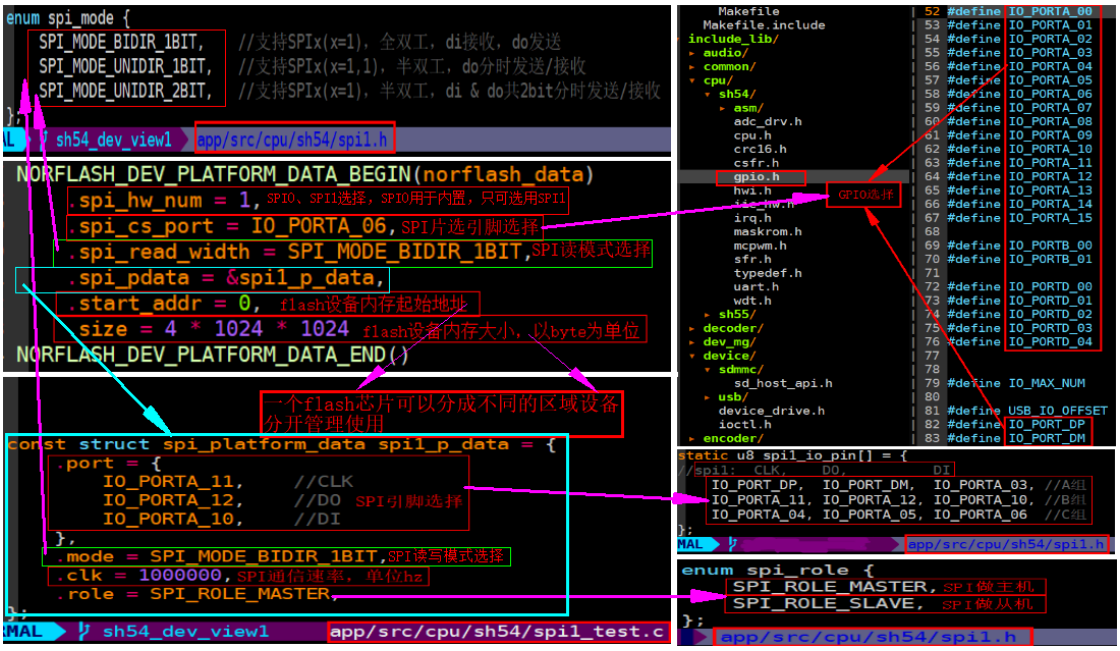


图 5 flash 配置信息

9.2 设备函数

9.2.1 函数 int devices_init()

此函数会将 device_tab 中注册的设备全部初始化，这一步一般在开机时进行。返回值未使用。

9.2.2 函数 bool dev_online(const char* name)

此函数会根据传入的设备名称返回设备在线情况。
返回值: 1 表示设备在线;0 表示不在线;

9.2.3 函数 void* dev_open(const char* name, void* arg)

此函数可以打开设备，函数会返回设备句柄供读写等函数调用。只有打开设备，才能使用读写等函数。函数调用前定义好设备句柄变量:

1. name: 需要打开的设备名称;
2. arg: 设备参数（多数时候为 0，不使用）;
3. 返回值：设备句柄，用于读写等操作函数;

设备名称可自定义，名称信息见下图 6。

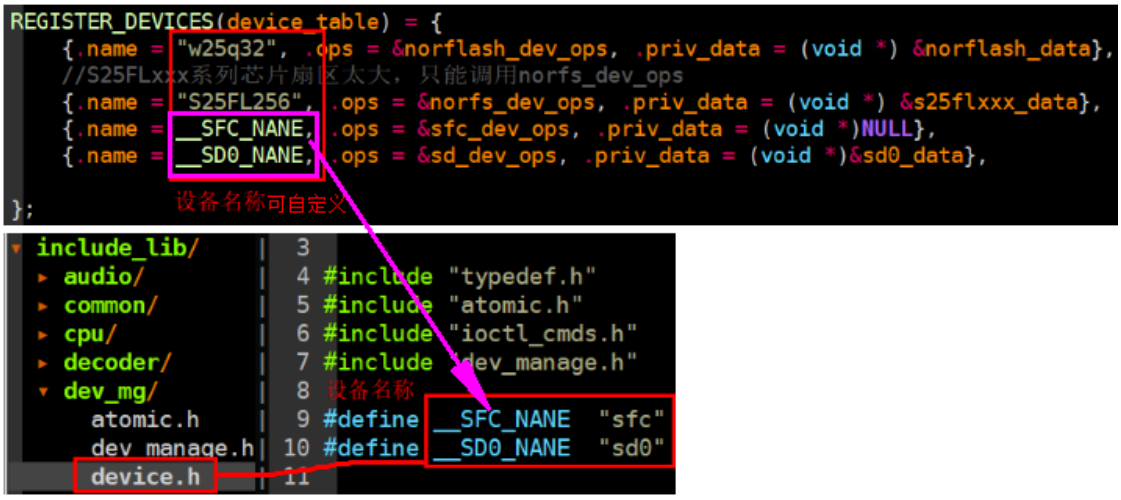


图 6 注册表名称信息

9.2.4 函数 int dev_bulk_read(void* _device, void* buf, u32 offset, u32 len)

本函数实现从设备某个地址 offset 读取长度为 len * 512 字节的数据到 buf 的功能:

1. _device：dev_open 函数返回的设备句柄；
2. buf：缓存，保存读出的数据，需自定义；
3. offset：读取起始设备地址；
4. len：需要读取的块数，通常为 len*512 字节长度；
5. 返回值：正常情况返回长度即形参 Len，错误时返回 0

注意：形参 offset、len 范围要在图 7 定义的起始地址和大小范围内才能正常操作设备。

9.2.5 函数 int dev_bulk_write(void* _device, void* buf, u32 offset, u32 len)

本函数实现把长度为 len * 512 字节的 buf 数据写入设备的功能，写入起始地址为 offset:

1. _device: dev_open 函数返回的设备句柄；
2. Buf ： 缓存，保存着需写入的数据；
3. Offset ： 写入起始设备地址；
4. Len ： 需要写入的块数，通常为 len*512 字节长度。
5. 返回值: 正常情况返回长度即形参 Len，错误时返回 0

注意：形参 offset、len 范围要在图 7 定义的起始地址和大小范围内才能正常操作设备。

9.2.6 函数 int dev_byte_read(void* _device, void* buf, u32 offset, u32 len)

本函数实现从设备某个地址 offset 读取长度为 len 字节的数据到 buf 的功能:

1. _device：dev_open 函数返回的设备句柄；
2. buf：缓存，保存读出的数据，需自定义；
3. offset：读取起始设备地址；
4. len：需要读取的字节长度；
5. 返回值：正常情况返回长度即形参 Len，错误时返回 0

注意：形参 offset、len 范围要在图 7 定义的起始地址和大小范围内才能正常操作设备。

9.2.7 函数 int dev_byte_write(void* _device, void* buf, u32 offset, u32 len)

本函数实现把长度为 len 字节的 buf 数据写入设备的功能，写入起始地址为 offset:

1. _device: dev_open 函数返回的设备句柄；
2. Buf： 缓存，保存着需写入的数据；
3. Offset： 写入起始设备地址；
4. Len： 需要写入的字节长度。
5. 返回值: 正常情况返回长度即形参 Len，错误时返回 0

注意：形参 offset、len 范围要在图 7 定义的起始地址和大小范围内才能正常操作设备。

9.2.8 函数 int dev_ioctl(void* device, int cmd, u32 arg)

本函数实现获取设备信息或设置设备信息，擦除设备等功能:

1. device：设备句柄；
2. cmd：命令，include_lib/dev_mg/ioctl_cmds.h 内能查到定义；详细命令表参见图 8。常用命令有：（注：不同设备支持命令会有差别，并不是支持所以命令）
   a. IOCTL_CMD_SUSPEND, 设备挂起
   b. IOCTL_CMD_RESUME，设备解挂
   c. IOCTL_GET_ID，获取设备 ID
   d. IOCTL_GET_SECTOR_SIZE，获取设备扇区大小
   e. IOCTL_GET_BLOCK_SIZE，获取设备块大小
   f. IOCTL_GET_CAPACITY，获取设备容量
   g. IOCTL_GET_STATUS，获取设备状态
   h. IOCTL_ERASE_SECTOR，擦除设备扇区
   i. IOCTL_ERASE_BLOCK，擦除设备块
   j. IOCTL_ERASE_CHIP，对设备整片擦除
3. arg：获取信息时：用作保存返回的设备信息如容量、ID 等; 擦出时：用作带入要擦除的地址。
4. 返回值：反应函数运行状态。0：表示正常其他值：出错

include_lib/dev_mg/ioctl_cmds.h

```
#define IOCTL_SET_IRQ_NUM                1
#define IOCTL_SET_PRIORITY               2
#define IOCTL_SET_DATA_WIDTH             3
#define IOCTL_SET_SPEED                  4
#define IOCTL_SET_DETECT_MODE            5
#define IOCTL_SET_DETECT_FUNC            6
#define IOCTL_SET_DETECT_TIME_INTERVAL   7
#define IOCTL_SET_PORT                   8
#define IOCTL_SET_PORT_FUNC              9
#define IOCTL_SET_CS_PORT_FUNC           10
#define IOCTL_SET_READ_MODE              11
#define IOCTL_SET_WRITE_MODE             12
#define IOCTL_SET_WRITE_PROTECT          13
#define IOCTL_SET_START_BIT              14
#define IOCTL_SET_STOP_BIT               15
#define IOCTL_FLUSH                      16
#define IOCTL_REGISTER_IRQ_HANDLER       17
#define IOCTL_UNREGISTER_IRQ_HANDLER     18
#define IOCTL_GET_SYS_TIME               19
#define IOCTL_SET_SYS_TIME               20
#define IOCTL_GET_ALARM                  21
#define IOCTL_SET_ALARM                  22
#define IOCTL_SET_CAP_LOWSPEED_CARD      23
#define IOCTL_SET_VDD50_EN               30
#define IOCTL_GET_WEEKDAY                32
#define IOCTL_CLR_READ_MODE              33
#define IOCTL_SET_READ_CRC               34
#define IOCTL_GET_READ_CRC               35
#define IOCTL_GET_VOLUME                 36
#define IOCTL_SET_VOLUME                 37
#define IOCTL_SET_ALARM_ENABLE           38
#define IOCTL_CMD_RESUME                 39
#define IOCTL_CMD_SUSPEND                40
#define IOCTL_SET_BASE_ADDR              41
#define IOCTL_SET_ASYNC_MODE             42

#define IOCTL_GET_ID                     100
#define IOCTL_GET_SECTOR_SIZE            101
#define IOCTL_GET_BLOCK_SIZE             102
#define IOCTL_GET_CAPACITY               103
#define IOCTL_GET_WIDTH                  104
#define IOCTL_GET_HEIGHT                 105
#define IOCTL_GET_BLOCK_NUMBER           106
#define IOCTL_CHECK_WRITE_PROTECT        107
#define IOCTL_GET_STATUS                 108
#define IOCTL_GET_TYPE                   109
#define IOCTL_GET_MAX_LUN                110
#define IOCTL_GET_CUR_LUN                111
#define IOCTL_SET_CUR_LUN                112
#define IOCTL_SET_FORCE_RESET            113
#define IOCTL_SET_CAPACITY               114
#define IOCTL_GET_NORFS_INFO             115

#define IOCTL_ERASE_SECTOR               200
#define IOCTL_ERASE_BLOCK                201
#define IOCTL_ERASE_CHIP                 202
#define IOCTL_SET_ENC_END                203
#define IOCTL_ERASE_PAGE                 204

#define IOCTL_SET_DATA_CALLBACK          301
```

图 8 cmd 可选择的命令

9.2.9 函数 int dev_close(void* device)

本函数实现关闭设备的功能，对于一些设备调用此函数能省电:

1. device：dev_open 返回的设备句柄；
2. 返回值：暂无意义

CHAPTER 10

文件系统管理

支持的文件系统列表：

文件系统	详情
sydfs	只读文件系统，用于存放资源文件到 norflash；
norfs	循环覆盖文件系统，可读写; 针对 norflash 设计；
simple_fat	简单 fat 文件系统驱动, 占用资源少，不支持搜索和多文件同时打开；支持 FAT12/16/32, EXFAT;
fatfs	完整的 fat 文件系统驱动, 占用资源多，支持搜索；支持 FAT12/16/32, EXFAT;

各芯片 SDK 支持的文件系统类型如下：

芯片支持的设备	sydfs	norfs	simple_fat	fatfs
AD14N	√	√	√	
AC104N	√	√		√
AD15N	√			
AD17N	√			
AD18N	√	√		√

SDK 中通过 vfs 来集中管理各类文件系统，涉及的文件目录包括：

- app\bsp\common\fs 目录下的所有 c 文件；
- include_lib\fs 目录下的所有 h 文件；
- errno-base.h 文件；

备注：

vfs 有以下主要函数：

1. void vfs_init(void);

2. u32 vfs_mount(void **ppvfs, void *device, void *type);
3. u32 vfs_openbypath(void *pvfs, void **ppvfile, const char *path);
4. u32 vfs_openbyindex(void *pvfs, void **ppvfile, u32 index);
5. u32 vfs_createfile(void *pvfs, void **ppvfile, u32 *pindex);
6. u32 vfs_read(void *pvfile, void *buf, u32 len);
7. u32 vfs_write(void *pvfile, void *buf, u32 len);
8. u32 vfs_seek(void *pvfile, u32 offset, u32 mode);
9. u32 vfs_file_close(void **ppvfile);
10. u32 vfs_fs_close(void **ppvfs);
11. u32 vfs_file_name(void *pvfile, void *name, u32 len);
12. int vfs_get_attrs(void *pvfile, void *pvfs_attr);
13. int vfs_ioctl(void *pvfile, int cmd, int arg);

备注：其中，fatfs 中独有的函数：

int vfs_ftell(void *pvfile, void *parm);

int vfs_get_fsize(void *pvfile, void *parm);

struct vfscan *vfs_fscan_new(void *pvfs, const char *path, const char *arg, u8 max_deepth, int (*callback)(void), struct vfscan *fsn, struct vfscan_reset_info *info);

int vfs_fscan_release(void *pvfs, struct vfscan *fs);

int vfs_select(void *pvfs, void **ppvfile, struct vfscan *fs, int sel_mode, int arg);

int vfs_get_folderinfo(void **pvfile, struct vfscan *fs, int *start_num, int *end_num);

int vfs_mk_dir(void *pvfs, char *folder, u8 mode);

int vfs_get_encfolder_info(void *pvfs, char *folder, char *ext, u32 *last_num, u32 *total_num);

int vfs_file_delete(void *pvfile);

10.1 文件系统介绍

Sdk 中默认支持的文件系统有：sydfs、norfs、fatfs（AC104，AD17,AD18）、simple_fatfs（AD14N）；

可以在 app\bsp\common\fs 目录下查看各文件系统的文件以及支持的接口；

本小节将介绍各个文件系统以及支持的 ioctl 对应的 cmd 命令:

1. “*sydfs 文件系统*”
2. “*norfs 文件系统*”
3. “*fat/simple_fat 文件系统*”

10.1.1 sydfs 文件系统

Sydfs 是系统数据运行的 **只读文件系统**；
Sydfs 可以通过 sdk 的 download 工具来下载存放资源文件，并于系统运行时读取；
该文件系统能使用的接口图所示：

```
177 const struct vfs_operations sydf_vfs_ops sec_used(.vfs_operations) = {
178     .fs_type = "sydfile",
179     .init        = sydf_init_api,
180     /* .init        = NULL, */
181     .mount       = sydf_monut_api,
182     .openbypath  = sydf_open_api,
183     .openbyindex = sydf_openbyindex_api,
184     .openbyfile = sydf_openbyfile_api,
185     .createfile  = NULL,
186     .read        = sydf_read_api,
187     .write       = NULL,
188     .seek        = sydf_seek_api,
189     .close_fs    = sydf_fs_close_api,
190     .close_file  = sydf_file_close_api,
191     .fget_attr   = sydf_attrs_api,
192     .name        = sydf_file_name_api,
193     .ioctl       = sydf_ioctl_api,
194 };
```

其中 ioctl 支持的 cmd 命令有：

1. FS_IOCTL_FILE_TOTAL:

```
示例：
    int file_total;
    vfs_ioctl(pfile, FS_IOCTL_FILE_TOTAL, (int)&file_total);
    file_total即为pfile文件所在文件系统的总文件数；
```

2. FS_IOCTL_FS_TOTAL:

```
示例：
    int file_total;
    vfs_ioctl(pfs, FS_IOCTL_FILE_TOTAL, (int)&file_total);
    file_total即为pfs文件系统的总文件数；
```

3. FS_IOCTL_FILE_ATTR:

```
示例：
    struct vfs_attr attr;
    vfs_ioctl(pfile, FS_IOCTL_FILE_ATTR, (int)&attr);
    attr即可获取pfile文件的位置和大小；
```

备注： 通过 ioctl 获取的文件位置仅为该 pfile 文件在其上层目录下的相对偏移；若要获取该文件相对于 flash 的偏移地址，需要调用 vfs_get_fsize()；

10.1.2 norfs 文件系统

Norfs 是一个循环管理的可读写文件系统；
Norfs 内部空间环形管理，空间不足时新写入的文件会覆盖之前的文件；
norfs 文件系统只通过文件号进行管理，创建新文件后文件号会累加；
该文件系统一般在录音时使用；
该文件系统能使用的接口如图所示：

```
104 const struct vfs_operations norfs_vfs_ops sec_used(.vfs_operations) = {
105     .fs_type = "norfs",
106     /* .init        = norfs_init_api, */
107     .init        = NULL,
108     .mount       = norfs_monut_api,
109     .openbypath  = NULL,
110     .openbyindex = norfs_openbyindex_api,
111     .createfile  = norfs_createfile_api,
112     .read        = norfs_read_api,
113     .write       = norfs_write_api,
114     .seek        = norfs_seek_api,
115     .close_fs    = NULL,
116     .close_file  = norfs_file_close_api,
117     .fget_attr   = NULL,
118     .name        = norfs_name_api,
119     .ioctl       = norfs_ioctl_api,
120 };
```

其中 ioctl 支持的 cmd 命令有：

1. FS_IOCTL_FILE_TOTAL，同上
2. FS_IOCTL_FS_TOTAL，同上
3. FS_IOCTL_FILE_ATTR，同上
4. FS_IOCTL_FILE_SYNC:

```
示例：
    u32 flen = 0;
    vfs_ioctl(pfile, FS_IOCTL_FILE_SYNC, (int)&flen);
␣
↪该命令一般在录音结束时调用，用于更新norfs录音文件头，flen传入时为需要舍弃的录音末尾字节
```

5. FS_IOCTL_FILE_INDEX:

```
示例：
    int file_index;
    vfs_ioctl(pfile, FS_IOCTL_FILE_INDEX, (int)&file_index);
    file_index即为pfile文件所在norfs文件系统的最新文件序号；
```

6. FS_IOCTL_FS_INDEX:

```
示例：
    int file_index;
    vfs_ioctl(pfs, FS_IOCTL_FS_INDEX, (int)&file_index);
    file_index即为pfs所指的norfs文件系统的最新文件序号；
```

10.1.3 fat/simple_fat 文件系统

FatFs 是一个通用的文件系统;
在本机中，FAT 不支持对 FAT12、exFAT 和 sector 大小大于 512byte 的 FAT 文件系统进行写操作，读操作正常;
Simple_FAT 为 FAT 裁切版，只能通过路径打开文件和读文件接口;
FAT 文件系统能使用的接口如下图所示:

```
const struct vfs_operations fat_vfs_ops sec_used(.vfs_operations) = {
    .fs_type = "fat",
    /* .init        = fat_init_api, */
    .mount       = fat_monut_api,
    .openbypath  = fat_openR_api,
    .createfile  = fat_openW_api,
    .read        = fat_read_api,
    .write       = fat_write_api,
    .seek        = fat_seek_api,
    .close_fs    = fat_fs_close_api,
    .close_file  = fat_file_close_api,
    .fget_attr   = fat_attrs_api,
    .name        = fat_file_name_api,
    .flen        = fat_get_fsize_api,
    .ftell       = fat_get_pos_api,
    .fdelete     = fat_delete_file_api,
    /* .openbyindex = fat_openbyindex_api, */
    /* .openbyclust = fat_openbyclust_api, */
    .openbyfile  = fat_openbyfile_api,
    .fscan_interrupt  = fat_fscan_interrupt_api,
    .fscan_release  = fat_fscan_release_api,
    .fsel        = fat_sel_api,
    .ioctl       = fat_ioctl_api,
};
```

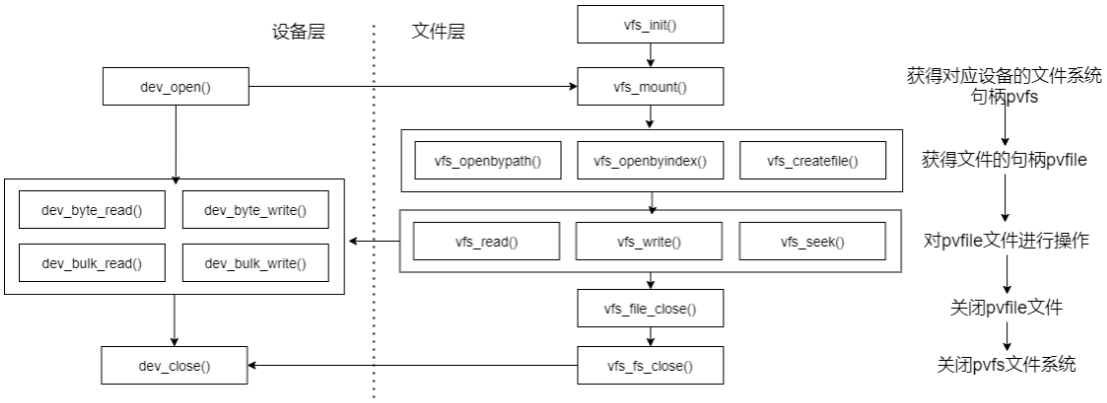
其中 ioctl 支持的 cmd 命令有:

1. FS_IOCTL_FILE_INDEX:

```
示例:
    int file_index;
    vfs_ioctl(pfile, FS_IOCTL_FILE_INDEX, (int)&file_index);
    file_index即为pfile文件所在fat文件系统的最新文件序号;
```

10.2 文件系统使用一般流程

如图，vfs 运行大致流程如下;



小技巧: 可以通过调用接口得到的返回值查看是否调用成功，以及失败的原因；

失败原因可在 Headers/include_lib/common/errno-base.h 中查看。

10.3 通用文件系统管理函数

通用文件系统管理函数将介绍 sydfs、norfs、fatfs 通用的接口，各文件系统具体能使用哪些接口请查看上面的几张图片，下一小节将介绍 fatfs 独有的相关文件系统接口；

10.3.1 函数 void vfs_init(void)

此函数实现对所有文件系统驱动的初始化；在开机第一次调用即可。

10.3.2 函数 u32 vfs_mount(void **ppvfs, void *device, void *type)

此函数实现对指定设备挂在指定类型的文件系统，所挂载的系统必须设备自身支持；其中参数:

```
1. ppvfs: 文件系统句柄指针，若句柄指向为NULL，函数会自行申请空间；
2. device: 设备句柄；
    ① 需要打开sydfs时，若句柄为空，则会使用内置flash；
    ② 需要打开norfs等其他文件系统时，句柄不能为空；
3. type: 文件系统类型的字符串；
    ① 可选择”sydfile”、”norfs”、”fat”或0，若该值为0，默认使用sydfile；
    ② 注：norfs文件系统仅为录音到内置FLASH的应用使用！！
4. 返回值：
    ① 成功 0；
    ② 失败 错误请查看errno-base.h。
```

示例 1

```
在内置flash挂载syd文件系统
void *device = dev_open(__SFC_NANE, 0);//获取内置flash设备句柄
void *pfs = NULL;
u32 err = vfs_mount(&pfs, device, “sydfile”); //内置flash挂载syd文件系统
if (pfs != NULL && err == 0) {
```

(续下页)

(接上页)

```
    //内置flash挂载sydfs成功
}
```

示例 2

```
在外挂flash上挂载norfs文件系统
void *device = dev_open(__EXT_FLASH_NANE, 0);//获取外挂flash设备句柄
void *pfs = NULL;
u32 err = vfs_mount(&pfs, device, "norfs"); //外置flash挂载norfs文件系统
if (pfs != NULL && err == 0) {
    //外置flash挂载norfs成功
}
```

示例 3

```
在SD卡上挂载fat文件系统
void *device = dev_open(__SD0_NANE, 0);//获取sd卡设备句柄
void *pfs = NULL;
u32 err = vfs_mount(&pfs, device, "fat"); //sd卡挂载fat文件系统
if (pfs != NULL && err == 0) {
    //sd卡挂载fat成功
}
```

10.3.3 函数 u32 vfs_openbypath(void *pvfs, void **ppvfile, const char *path)

此函数实现打开指定路径的文件，norfs 文件系统无法使用此功能。其中参数：

```
1. pvfs：已挂载的文件系统句柄；
2. ppvfile：文件句柄指针，若句柄指向为NULL，函数会自行申请空间；
3. path：路径字符串；
   ␣
↪①  如果字符串只是包含文件夹名，那么会打开文件夹，打开的文件夹仅可用于校验CRC16；
    ②  如果字符串里面包含了完成的文件名，那么会同时包含文件名！
    ③  如果字符串为空，则返回错误；
4. 返回值：
    ①成功 0；
    ②失败 错误请查看errno-base.h。
```

示例：

```
在挂载（vfs_mount）成功的前提下，打开dir_song里的so001.f1a歌曲
void *pfile = NULL;
u32 err = vfs_openbypath(pfs, &pfile, "dir_song/so001.f1a");
if (pfile != NULL && err == 0) {
    //成功打开dir_song文件夹里的so001.f1a文件
}
```

10.3.4 函数 u32 vfs_openbyindex(void *pvfs, void **ppvfile, u32 index)

此函数实现打开指定文件号的文件，其中参数：

```
1. pvfs：已挂载的文件系统句柄；
2. ppvfile：文件句柄指针，若句柄指向为NULL，函数会自行申请空间；
3. index：文件序号；
4. 返回值：
      ① 成功 0；
      ② 失败 错误请查看errno-base.h。
```

10.3.5 函数 u32 vfs_createfile(void *pvfs, void **ppvfile, u32 *pindex)

此函数实现 norfs、fatfs 创建一个新的文件，syd_fs 无法使用此功能。文件创建和操作结束后，需要调用 vfs_file_close() 关闭文件，其中参数：

```
1. pvfs：已挂载的文件系统句柄；
2. ppvfile：文件句柄指针，若句柄指向为NULL，函数会自行申请空间；
3. pindex：文件序号；
    ① norfs：传入的pindex会被赋值为最新的文件序号；
        注意：norfs文件创建和操作结束后，需要先调用vfs_ioctl()接口传入FS_
↪IOCTL_FILE_SYNC命令更新文件信息后再关闭文件；
    ② fatfs：传入文件存放位置，如”/test.txt”即在根目录创建test.txt文件；；
        注意：路径有文件夹时文件夹必须存在，否则创建失败。可先调用vfs_mk_
↪dir()创建文件夹；创建文件时支持长文件名，长文件名路径需传入Unicode码进去并开启FOPEN_
↪LONG宏定义和相应的库(详情见demo代码)；
    ③ sydfs：不支持此功能；
4. 返回值：
    ① 0；
    ② 失败 错误请查看errno-base.h。
```

示例 1：

```
挂载norfs文件系统成功前提下，创建一个新的文件
void *pfile = NULL;
int index;
u32 err = vfs_createfile(pfs, &pfile, (u32)&index);
if (pfile != NULL && err == 0) {
    //norfs文件系统下文件创建成功，index会被赋值为所创建文件的序号
}
```

示例 2：

```
挂载fat文件系统成功前提下，创建一个新的文件
void *pfile = NULL;
char *path = “/abcd.txt”;
u32 err = vfs_createfile(pfs, &pfile, (u32)path);
if (pfile != NULL && err == 0) {
    //fat文件系统下文件abcd.txt创建成功
}
```

10.3.6 函数 u32 vfs_read(void *pvfile, void *buf, u32 len)

此函数实现对指定文件的读操作。其中参数：

```
1. pvfile：已打开的文件句柄；
2. buf：数据buff；
3. len：需要读取的字节长度；
4. 返回值：
    ①  成功 读取的字节数；
    ②  失败 0。
```

示例：

```
打开文件成功前提下，读取文件数据
u8 read_buf[32];
u32 read_len = vfs_read(pfile, (void \*)read_buf, 32);
if (read_len != 0) {
    //成功读取pfile中read_len长度的数据到read_buf里
}
```

10.3.7 函数 u32 vfs_write(void *pvfile, void *buf, u32 len)

此函数实现对指定文件的写操作，syd_fs 无法使用此功能，其中参数：

```
1. pvfile：已打开的文件句柄；
2. buf：数据buf；
3. len：需要写入的字节长度；
4. 返回值：
    成功 写入的字节数；
    失败 错误请查看errno-base.h。
```

示例：

```
打开文件成功前提下，往文件写入数据
u8 write_buf[32] = {0, 1, 2, 3 ... 31};
u32 write_len = vfs_write(pfile, (void *)write_buf, 32);
if (write_len != 0) {
    //成功将write_buf中write_len长度的数据写入到pfile
}
```

10.3.8 函数 u32 vfs_seek(void *pvfile, u32 offset, u32 mode)

此函数实现改变指定文件中读写指针的偏移，其中参数：

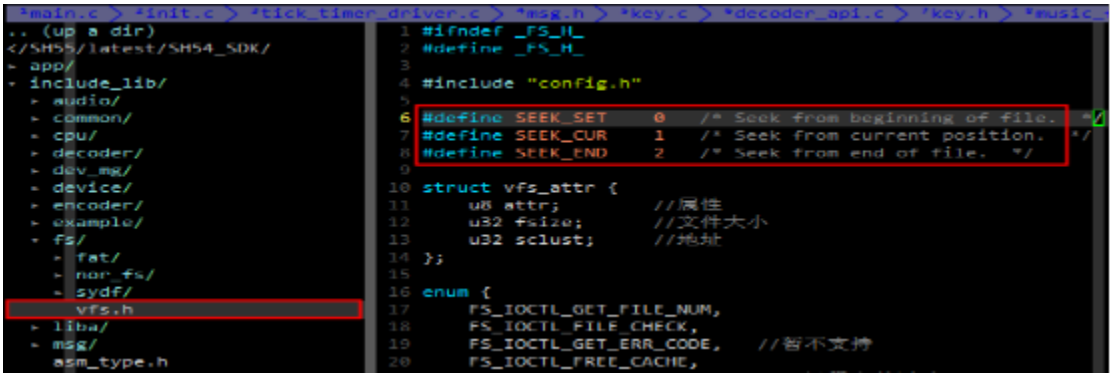
```
1. pvfile：已打开的文件句柄；
2. offset：偏移量；
3. mode：seek模式选择。如图4.2，seek模式有3种，分别为：
    ①  SEEK_
↪SET：从文件起始位置开始向后偏移，首先会判断偏移量是否大于文件长度，若小于则使文件读写指针rw_
↪p = offset；
    ②  SEEK_CIR：从文件读写指针位置开始向后偏移，使rw_p = rw_p + offset；
    ③  SEEK_
↪END：从文件末尾位置向前偏移，首先会判断偏移量是否大于文件长度，若小于则使rw_
```

(续下页)

(接上页)

```
↪p = 文件长度 - offset;
4. 返回值：0。
```

下图为 seek 模式的选择：



10.3.9 函数 u32 vfs_file_close(void **ppvfile)

此函数实现关闭文件并释放句柄，其中参数：

```
1. ppvfile：已打开的文件句柄指针；
2. 返回值：0。
```

示例：

```
vfs_file_close(&pfile);
if (pfile == NULL) {
    //成功关闭pfile文件
}
```

10.3.10 函数 u32 vfs_fs_close(void **ppvfs)

此函数实现关闭文件系统并释放相关句柄，nor_fs 无法使用此功能，其中参数：

```
1. ppvfs：已挂载的文件系统句柄指针；
2. 返回值：NULL。
```

示例：

```
vfs_fs_close(&pfs);
if (pfs == NULL) {
    //成功关闭pfs文件系统
}
```

10.3.11 函数 u32 vfs_file_name(void *pvfile，void *name, u32 len)

此函数实现获取指定文件句柄的文件名，其中参数：

```
1. pvfile：已打开的文件句柄；
2. name：存放文件名的buf；
3. len：获取文件名的长度；
4. 返回值：成功 返回获取文件名的长度；
```

10.3.12 函数 int vfs_get_attrs(void *pvfile, void *pvfs_attr)

此函数实现记录文件的属性，包括文件大小和地址，nor_fs 无法使用此功能，其中参数：::

1. pvfile：已打开的文件句柄；
2. pvfs_attr：文件系统属性，需要传入 vfs_attr 结构体类型的指针；
3. 返回值：0。

示例：

```
struct vfs_attr file_attr;
vfs_get_attrs(pfile, &file_attr);
//文件地址和大小分别保存在file_attr.sclust和file_attr.fsize中
```

10.3.13 函数 int vfs_ioctl(void *pvfile, int cmd, int arg)

此函数实现对指定文件的相关操作，各文件系统具体的 cmd 命令请查看《文件系统介绍》小节，其中参数：

```
1. pvfile：已打开的文件句柄；
2. cmd：命令；
3. arg：参数；
4. 返回值：
    成功 0；
    失败 错误请查看errno-base.h。
```

cmd：命令主要有 7 种：

vfs_ioctl cmd 命令	命令含义
FS_IOCTL_D	获取文件所在文件系统下的文件数。该命令会将 arg 的值作为地址，并把文件句柄所在文件系统下的所有文件数赋给 arg 指向的值；若 arg 参数为 0，返回错误 E_PAGE_NULL；
FS_IOCTL_F	同上；
FS_IOCTL_F	获取文件系统下的文件数。该命令会将 arg 的值作为地址，并将特定情况下传入的文件系统句柄下的所有文件数赋值给 arg 指向的值；若 arg 参数为 0，返回错误 E_PAGE_NULL；
FS_IOCTL_F	获取文件地址参数。该命令会将 arg 的值作为地址并指向 vfs 地址参数，并把文件长度以及地址赋值给 vfs 地址参数；若 arg 参数为 0 或文件句柄为 NULL，返回错误 E_PAGE_NULL 或 E_PFILE_NULL；
FS_IOCTL_F	同步文件。该命令会将录音文件同步并将 arg 的值作为末尾裁切的字节长度；若 arg 参数为 0 则不进行裁切；
FS_IOCTL_F	通过文件指针获取文件索引号。该命令会将 arg 的值作为地址，并把文件的索引号赋值给 arg 指向的值。
FS_IOCTL_F	通过文件系统指针获取文件索引号。该命令会将 arg 的值作为地址，并把文件的索引号赋值给 arg 指向的值。

10.3.14 函数 int vfs_file_crc(void *pvfile)

此函数实现计算文件的 CRC16 校验值，目前仅 SYD 可使用此功能，其中参数：

```
1. pvfile: 已打开的文件句柄；
2. 返回值: CRC校验值，计算错误则返回0；
注意: V1.4.
↪0以及之后的SDK，可以通过openbypath方式打开资源文件进行校验，也可打开dir文件用于校验；打开dir文
```

示例：

```
vfs_openbypath(pfs, &pfile, "/dir_song");        //校验打包后的dir文件
u16 crc16 = vfs_file_crc(pfile);
```

示例 2：

```
vfs_openbypath(pfs, &pfile, "/dir_song/so001.f1a");//校验资源文件
u16 crc16 = vfs_file_crc(pfile);
```

10.4 FAT 文件系统管理函数

下面的函数为仅 FAT 使用的接口，在 app\bsp\common\fs\fat\fat_resource.c 中的 fat_demo() 接口，可以找到下列函数的使用示例；

10.4.1 函数 int vfs_ftell(void *pvfile, void *parm)

此函数实现获取文件读写指针，仅 FAT 可使用该功能。其中参数：

```
1. pvfile：已打开的文件句柄；
2. param：用于保存文件系统读写指针；
3. 返回值：
    成功 0；
    失败 错误请查看errno-base.h。
```

10.4.2 函数 int vfs_get_fsize(void *pvfile, void *parm)

此函数实现获取文件大小，仅 FAT 可使用该功能。其中参数：

```
1. pvfile：已打开的文件句柄；
2. param：用于保存文件大小的指针；
3. 返回值：
    成功 0；
    失败 错误请查看errno-base.h。
```

10.4.3 函数 struct vfscan *vfs_fscan_new(void *pvfs, const char *path, const char *arg, u8 max_deepth, int (*callback)(void), strcut vfscan *fsn, struct vfscan_reset_info *info)

此函数实现 FAT 文件系统的文件扫描配置，会返回文件扫描句柄供其他扫描接口使用，与 vfs_fscan_release() 搭配调用，可在 ac104n sdk 中的 play_file.c 中查看该函数具体使用示例。其中参数：

```
1、pvfs：已挂载的文件系统句柄；
2、path：文件扫描的起始路径；
3、arg：文件扫描配置参数；
    例：static const char scan_parm_test[] =  "-t"
                                              "MP1MP2MP3WAVTXT"
                                              " -sn -r"
                                             ;
    ①  首行" -
↪t"表示需要扫描的文件类型，后面"MP1MP2MP3WAVTXT"即为扫描的后缀名，三个字符为一种类型，最多不
    ②  第三行"  -sn"表示扫描文件的顺序是按照文件号排序，另一种组合"  -
↪st"表示按照时间排序；
    ③  第三行"  -r"表示搜索会包含文件夹子目录；
    ④  各种扫描参数配置间，需要以空格隔开，其他配置有：
        a)扫描是否将文件夹算入总数，可配置"  -d"；
        b)扫描只读文件配置"  -ar"，扫描非读文件配置"  -a/
↪r"，不配置默认扫描所有文件；
4、max_deep：扫描深度，最大为8；
5、callback：扫描回调；
6、fsn：文件扫描句柄；
7、info：扫描结果信息，包含文件总数和文件夹总数等信息；
8、返回值：
    成功 返回文件扫描句柄；
    失败 错误请查看errno-base.h。
```

小技巧： 扫描文件后缀名若超过 3 个字符，则只识别前面 3 个字符

后缀名若少于 3 个字符，则不足部分用？代替，如（A 格式：A??）

10.4.4 函数 void vfs_fscan_release(void *pvfs, struct vfscan *fs)

此函数实现释放文件扫描句柄，仅 FAT 可使用该功能，与 vfs_fscan() 搭配调用。其中参数：

```
1、pvfs：已挂载的文件系统句柄；
2、fs：文件扫描句柄；
3、返回值：无。
```

10.4.5 函数 int vfs_get_folderinfo(void **pvfile, struct vfscan *fs, int *start_num, int *end_num)

此函数实现文件夹扫描，需使用 vfs_fscan() 返回的文件扫描句柄，仅 FAT 可使用该功能。其中参数：

```
1、pvfile：需要扫描的设备里的文件；
2、fs：文件扫描句柄；
3、Start_num：用于保存第一个文件夹的第一个文件号；
4、End_num：用于保存最后一个文件夹的最后一个文件号；
5、返回值：
    成功 0；
    失败 错误请查看errno-base.h。
```

10.4.6 函数 int vfs_mk_dir(void *pvfs, char *folder, u8 mode)

此函数实现获取创建文件夹，仅 FAT 可使用该功能。其中参数：

```
1、pvfs：已挂载的文件系统句柄；
2、folder：文件夹路径，如”/JL_REC”；
3、mode：目录属性；
    ①  1：设置为隐藏属性；
    ②  0：不设置；
4、返回值：
    成功 0；
    失败 错误请查看errno-base.h。
```

10.4.7 函数 int vfs_get_encfolder_info(void *pvfs, char *folder, char *ext, u32 *last_num, u32 *total_num)

此函数实现获取录音文件夹信息，包括录音文件总数，仅 FAT 可使用该功能。其中参数：

```
1、pvfs：已挂载的文件系统句柄；
2、folder：文件夹路径，如”/JL_REC”；
3、ext：录音文件的后缀名，如”MP3”；
4、last_num：第一个可以使用的录音文件的文件序号；
5、total_num：总录音文件数；
6、返回值：
```

(续下页)

(接上页)

```
    成功 0;
    失败 错误请查看errno-base.h。
```

10.4.8 函数 u32 vfs_file_delete(void *pvfile)

此函数实现文件的删除，仅 FAT 可使用该功能。其中参数：

```
1、pvfile：已打开的文件句柄；
2、返回值：
    成功 0;
    失败 错误请查看errno-base.h。
```

10.4.9 函数 int vfs_select(void *pvfs, void **ppvfile, struct vfscan *fs, int sel_mode, int arg)

此函数实现 FAT 文件系统的文件查找，支持多种查找方式，具体示例可查看 fat_demo(void) 接口。其中参数：

```
1、pvfs：已挂载的文件系统句柄；
2、ppvfile：文件句柄指针，若句柄指向为NULL，函数会自行申请空间；
3、fs：文件扫描句柄；
4、sel_mode：文件查找方式：
    ①  FSEL_BY_NUMBER：按文件号查找；
    ②  FSEL_BY_SCLUST：按簇号查找；
    ③  FSEL_BY_PATH：按路径查找；
5、arg：文件查找参数；
    ①  按文件号查找时传入文件号；
    ②  按簇号查找时传入簇号；
    ③  按路径查找时传入相应路径，支持通配符查找；
6、返回值：
    成功 0;
    失败 错误请查看errno-base.h。
```

CHAPTER 11

解码接口

目前 DAC 一直工作在 32K 的采样率，所有解码通路经过重采样之后输出到 DAC。
各种芯片与其支持音频格式如下所示：

格式	后缀名	AD14N	AD15N	AC104N	AD17N	AD18N
f1a	.f1a .f1b .f1c .f1x	支持	支持	支持	支持	支持
ump3	.ump3	支持	不支持	支持	不支持	支持
midi	.mid .mfa .mda	支持	支持	支持	支持	支持
a	.a .b .e	支持	支持	支持	支持	支持
MP3	.mp3 .mp2 .mp1	不支持	不支持	支持	不支持	支持
WAV	.wav	不支持	不支持	支持	不支持	支持

其中，midi 文件后缀名详情：

工具版本	乐谱后缀名	音色库后缀名
旧版无压缩	.mid	.mid
旧版有压缩	.mid	.mda
新版	.mfa	.mda

目前支持最多支持的解码通路如下：

CPU	两路解码	三路解码
AD14N	f1a / ump3 / midi + a 解码	f1a1 + f1a2 + a 解码
AC104N	f1a / ump3 / midi / 标准 mp3 / wav + a 解码	f1a1 + f1a2 + a 解码
AD15N	f1a / midi + a 解码	不支持
AD17N	f1a / midi + a 解码	不支持
AD18N	f1a / ump3 / midi + a 解码	f1a1 + f1a2 + a 解码

解码管理的接口函数:

```
void decoder_init(void);
dec_obj *decoder_io(void *pfile, u32 dec_ctl, dp_buff * dbuff,u8 loop);
void irq_decoder_ret(dec_obj *obj,u32 ret);
```

解码功能控制函数:

```
void decoder_stop(dec_obj *obj, DEC_STOP_WAIT wait);
void decoder_pause(dec_obj *obj);
```

各个格式的解码接口函数:

```
u32 a_decode_api(void *p_file,void **p_dec,void *p_dp_buf)
u32 f1a_decode_api_1(void *p_file,void **p_pdec,void *t_dp_buf)
u32 f1a_decode_api_2(void *p_file,void **p_pdec,void *t_dp_buf)
u32 midi_decode_api(void *p_file,void **ppdec,void *p_dp_buf)
u32 ump3_decode_api(void *p_file, void **p_dec,void *p_dp_buf)
u32 mp3_st_decode_api(void *p_file, void **p_dec, void *p_dp_buf)
u32 mp3_st_decode_api(void *p_file, void **p_dec, void *p_dp_buf)
```

11.1 解码使用一般流程

在开机时需要调用函数 void decoder_init(void)，此函数实现对解码管理器进行初始化，会初始化解码控制器里面的一些变量。

在解码之前要确保已经完成了需要解码的设备，以及设备中的文件进行了初始化。获取到了文件句柄，初始化 DAC 之后便可以开始正常的解码流程:

1、调用 decoder_io 实现对解码器的控制，此函数的传参可以控制:

```
a.对文件在指定的几种解码格式中解析，自动选取正确的格式进行解码;
b.控制解码输出的PCM数据结果是否经过音效处理，目前阶段主要是变速变调;
c.输入一断点buff，有了此buff才能实现续播功能;
d.控制续播的次数;
```

2、经过上面的一步已经完成了对解码的启动，此时可以调用 decoder_pause、decoder_stop 实现暂停解码与停止解码的功能;

11.2 数据流获取

所有解码格式初始化时，会注册数据输入、输出接口给解码器进行回调。用户可根据需求自行修改数据输入、输出接口。

涉及文件 apps/app/bsp/common/decoder/mp_io.c，涉及的结构体如下:

```
struct if_decoder_io {
                void *priv,
                int(*input)(void *priv, u32 addr, void *buf, int len);
                int(*output)(void *priv, void *data, int len);
};
```

结构体中注册的函数原型：

```
int mp_input(void *priv, u32 addr, void *buf, int len)
{
        dec_obj *obj = priv;
        vfs_seek(obj->p_file, addr, SEEK_SET);
        int rlen = vfs_read(obj->p_file, buf, len);
        return rlen;
}
/*␣
↪解码开始时会调用input接口读取音频文件数据，其中参数addr为文件内的偏移地址，len为读取长度，返回
↪*/

int mp_output(void *priv, void *data, int len)
{
        dec_obj *obj = priv;
        return sound_output(&obj->sound, data, len);
}
/*␣
↪数据解码完成后解码器会调用output接口将解码后的pcm数据写入dac的buff，其中参数data为数据输出的起
↪*/
```

```
const struct if_decoder_io ump3_dec_io0 = {
    &dec_ump3_hld,      //input跟output函数的第一个参数，解码器不做处理，直接回传，可以为NULL
    mp_input,
    mp_output,
};

u32 ump3_decode_api(void *p_file, void **p_dec, void *p_dp_buf)
{
    u32 buff_len, i;
    /* void *name; */
    /* char name[VFS_FILE_NAME_LEN] = {0}; */
    decoder_ops_t *ops;
    log_info("ump3_decode_api\n");
NORMAL  +0 ~0 -0  dev ✘  app/bsp/common/decoder/list/ump3_api.c
```


数据流输入输出结构

11.3 解码管理接口函数

11.3.1 函数 dec_obj *decoder_io(void *pfile, u32 dec_ctl, dp_buff * dbuff,u8 loop)

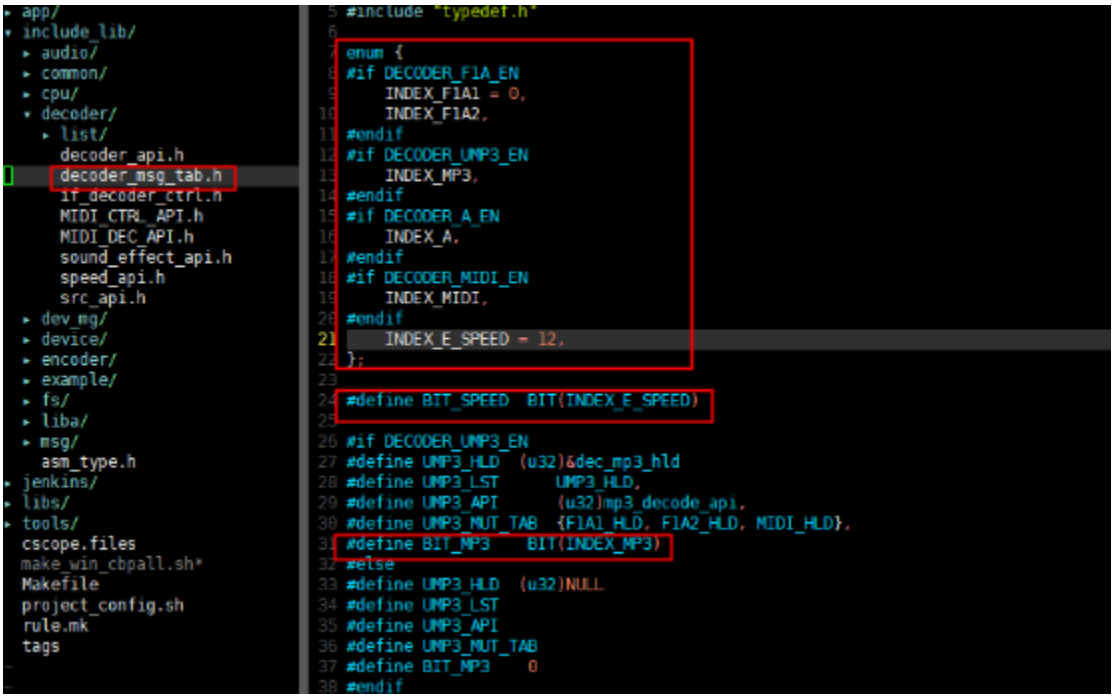
此函数实现对以及获取到的文件进行解码，其中参数：

```
1、void *pfile，已经获取到的文件句柄；
2、u32 dec_
↪ctl，解码控制参数，通过对不BIT的置一操作实现对相应功能的控制：
    ①  BIT_UMP3，置上此位为可以将UMP3解码加入到解码轮询中；
    ②  BIT_F1A1/，置上此位可以将第一路f1a/b/c解码加入到解码轮询中；
    ③  BIT_F1A2，置上此位为可以将第二路f1a/b/c解码加入到解码轮询中；
    ④  BIT_MIDI，置上此位为可以将midi解码加入到解码轮询中；
    ⑤  BIT_A，置上此位可以将a/b/e解码加入到解码轮询中；
```

(续下页)

(接上页)

```
    ⑥  BIT_MP3_ST，置上此位可以将标准MP3解码加入到解码轮询中；
    ⑦  BIT_WAV，置上此位可以将标准WAV解码加入到解码轮询中；
    ⑧  BIT_
↪SPEED，置上此位之后，当前这一路解码输出的PCM数据会经过变数变调之后再输出给DAC。
3、dp_buff * dbuff，断点缓存，用于续播功能：
    ①  NULL，不能实现续播功能，
    ②  非NULL，会使用给定的buff实现续播功能
4、u8 loop，续播次数控制(0~255)：
    ①  0，播放完成后不续播；
    ②  1~244，续播次数；
    ③  255，一直持续续播；
5、返回值，解码句柄，利用解码句柄可以实现对当前解码通道进行控制（主要是暂停和播放）。
```



11.3.2 函数 void irq_decoder_ret(dec_obj *obj,u32 ret)

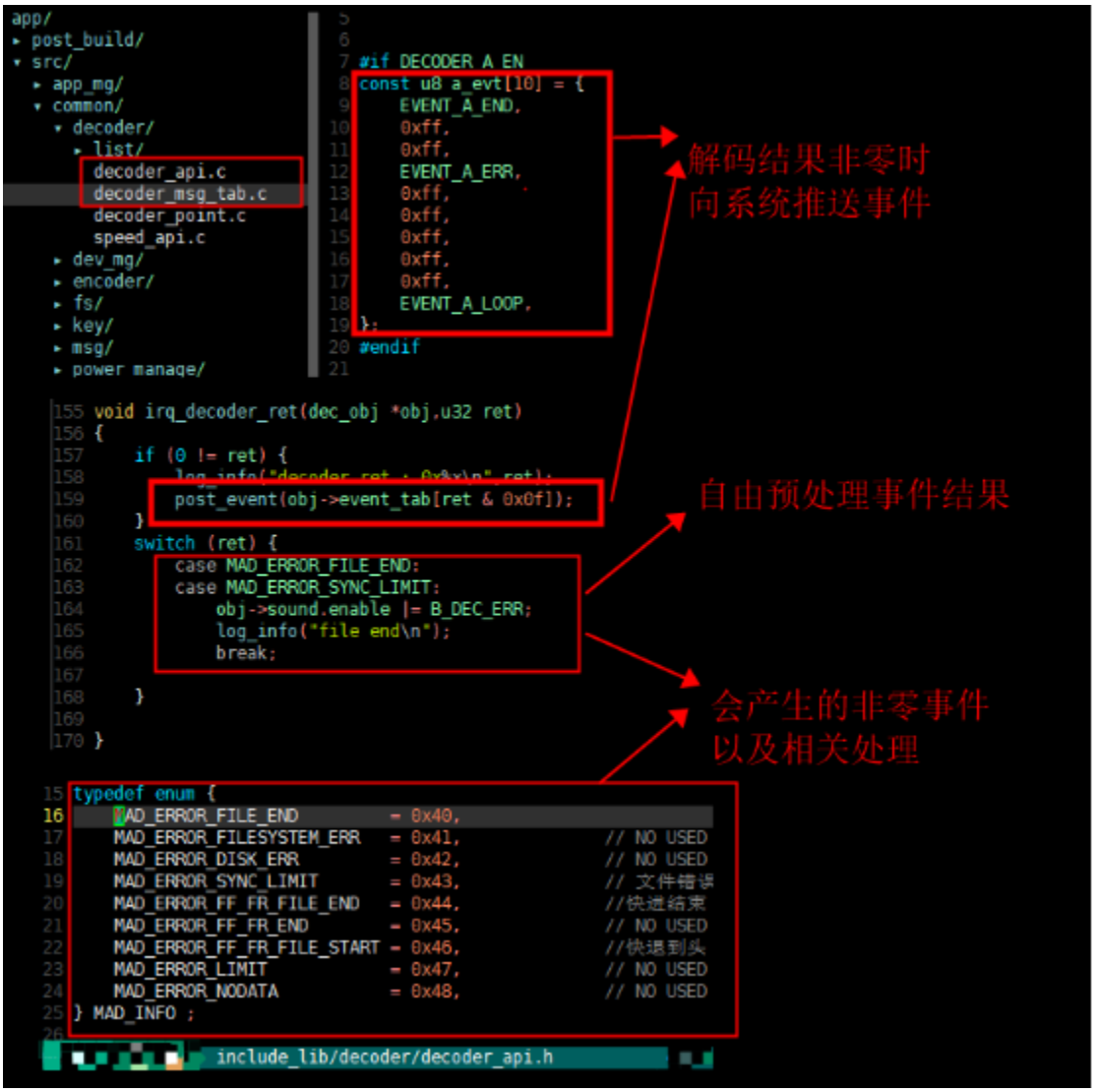
此函数是解码器在解码过程中的回调函数，不能再这里做长时间的操作，因为这样会卡慢解码的速度，其中参数：

```
1、dec_obj *obj，解码句柄
2、ret，解码运行的结果，
    0为成功，
    其他如图
        ①  MAD_ERROR_FILE_END，解码文件结束
        ②  MAD_ERROR_FILESYSTEM_ERR，未使用
        ③  MAD_ERROR_DISK_ERR，未使用
        ④  MAD_ERROR_SYNC_LIMIT，文件错误
        ⑤  MAD_ERROR_FF_FR_FILE_END，快进到头
        ⑥  MAD_ERROR_FF_FR_END，未使用
        ⑦  MAD_ERROR_FF_FR_FILE_START，快退到头
        ⑧  MAD_ERROR_LIMIT，未使用
```

(续下页)

(接上页)

```
        ◎ MAD_ERROR_NODATA，未使用
3、返回值，无。
```



11.4 解码功能控制函数

11.4.1 函数 void decoder_pause(dec_obj *obj)

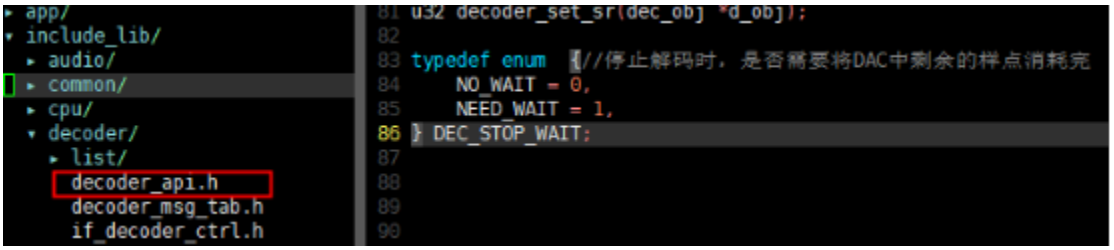
此函数实现在解码过程中对当前解码进行暂停播放与继续播放的控制，其中参数：

```
1、dec_obj *obj，解码句柄，启动解码时从函数decoder_io的返回值获取得到。
```

11.4.2 函数 void decoder_stop(dec_obj *obj, DEC_STOP_WAIT wait)

此函数实现停止当前解码的功能，其中参数：

```
1、dec_obj *obj，解码句柄，启动解码时从函数decoder_io的返回值获取得到。
2、DEC_STOP_WAIT wait，是否等待DAC消耗完解码输出的PCM数据后再停止解码
    ①  NO_WAIT，直接停止解码
    ②  NO_WAIT，等待DAC消耗完全部的解码输出数据后再停止解码
```



11.5 各个格式的解码函数

各种格式的解码启动函数统一被 decoder_io 所调用，其一般名称为：XXX_decode_api。XXX 为其格式名称。

```
u32 a_decode_api(void *p_file,void **p_dec,void *p_dp_buf)
u32 f1a_decode_api_1(void *p_file,void **p_pdec,void *t_dp_buf)
u32 f1a_decode_api_2(void *p_file,void **p_pdec,void *t_dp_buf)
u32 midi_decode_api(void *p_file,void **ppdec,void *p_dp_buf)
u32 mp3_decode_api(void *p_file, void **p_dec,void *p_dp_buf)
```

11.5.1 统一介绍 decode_api

函数 u32 XXX_decode_api(void *p_file, void **p_dec,void *p_dp_buf):

```
①  void *p_file，文件句柄
②  void **p_dec，存放解码句柄指针的指针
③  void *p_dp_buf，断点buff，提供给续播使用，注意MIDI不支持
```

CHAPTER 12

F1X 格式解码流程说明

12.1 简介

f1x 音频，隶属于 fla 系列格式，能够做到自定义拼接多段音频达到重复播放以及无缝衔接的效果。

其中，使用“音频转换工具”可以调整参数改变音频间衔接的效果；代码中可以指定音频的播放循序。

SDK 中默认按照 f1X 内的文件顺序选择文件，再根据 g_loop_tab[] 数组的值指定各段音频的循环次数。

注：本文需要使用到“音频文件转换工具 *_1.2.12*”及以上版本，需要 *AD14N / AD15N v1.0.7* 和 *AC104N v1.0.5* 及其之后的 *sdk* 支持。

12.2 结构体 / 函数说明

12.2.1 结构体 f1x_ctl

该结构体有以下参数：

a. Index：当前解码的音频在 f1x 文件的序号，与工具制作 f1x 时输入的文件顺序有关；

b. Cnt：当前音频的循环次数；

c. loop_tab[F1X_MAX_UNIT]：用户自定义的音频循环次数，默认为 u8 g_loop_tab[10]；

d. buff[F1X_MAX_UNIT]：各个音频段的偏移地址；

12.2.2 函数 void *f1x_play_api(void *pfile, u8 *loop_tab, u32 size, u8 index, u32 addr)

该函数是 f1x 音频的初始化函数，通过该函数可以获得 f1x 文件各段音频的偏移地址；如有有断点，还会找到断点对应的音频序号；其中参数：

a. Pfile：文件句柄；

b. Loop_tab：用户传入的音频循环表的指针，sdk 中传入的是 g_loop_tab[] 数组指针；

c. **Size：用户传入的音频循环表的大小；**
注意：g_loop_tab 默认是 10 个成员，与 F1X_MAX_UNIT 对应；若用户音频数量较多，需要同时修改 g_loop_tab[] 数组大小以及 F1X_MAX_UNIT 的值；

d. Index：f1x 解码通道，类似 f1a 可以同时解 2 路，由解码框架自行分配；

e. Addr：有断点时，传入的断点信息；

f. 返回值：成功 f1x 管理句柄指针 &f1x_io；失败 0

12.2.3 函数 int f1x_parsing(void *pfile, u32 *buff, u32 sum)

该函数用于解析 f1x 音频文件，获取各段音频的偏移地址；用户可以根据各段音频的偏移地址在函数 set_play_file() 中自定义播放；其中参数：

a. Pfile：文件句柄；

b. Buff：用于保存各段音频偏移地址的 buff；

c. Sum：f1x 文件中的音频总数；

d. 返回值：成功 0，失败错误请查看 errno-base.h

12.2.4 函数 u32 set_play_file(void *priv, u32 *startpos, u32 *endpos)

f1x 经过函数 f1x_play_api() 初始化并播放完第一段音频后，解码器会调用本函数选择下一段要播放的音频；sdk 中是按照 f1x 文件中默认的音频顺序进行播放的，用户可修改该函数自定义播放顺序；其中参数：

a. Priv：该私有参数传入的是正在解码的 f1x 文件的管理结构体 f1x_ctl；

b. Startpos：下一段准备解码的音频起始偏移地址，由 f1x_ctl 的 buff 成员提供；

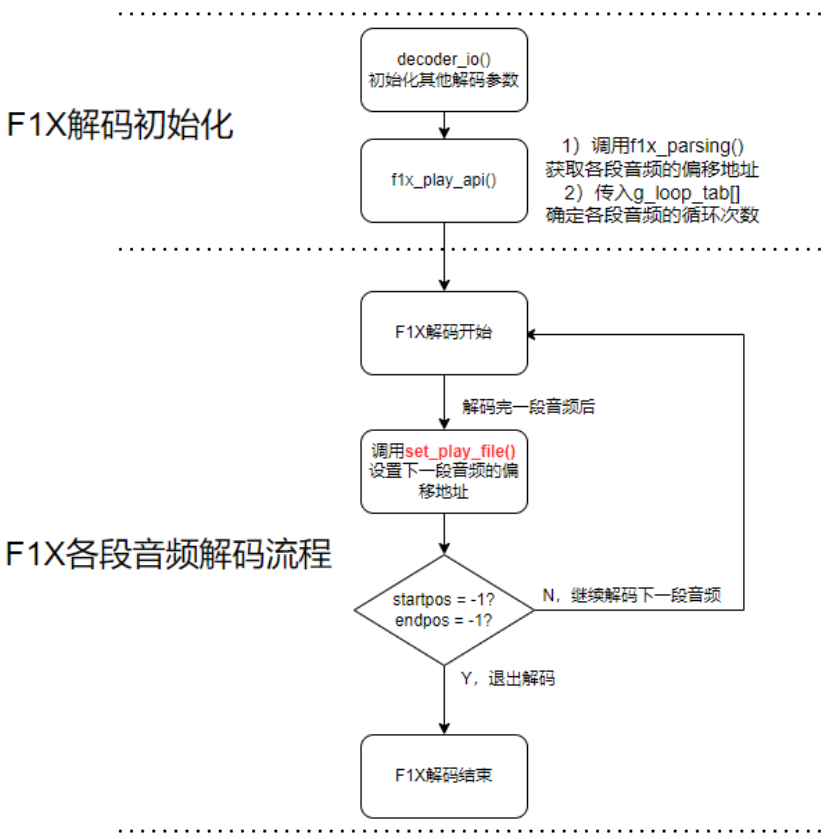
c. Endpos：下一段准备解码的音频结束偏移地址；

d. 返回值：播放下一段音频 0, 退出播放 -1

12.3 解码流程

如下图，函数 f1x_play_api() 完成相应初始化操作，包括获取各段音频在 f1x 文件的偏移地址，以及确定各段音频的循环次数。

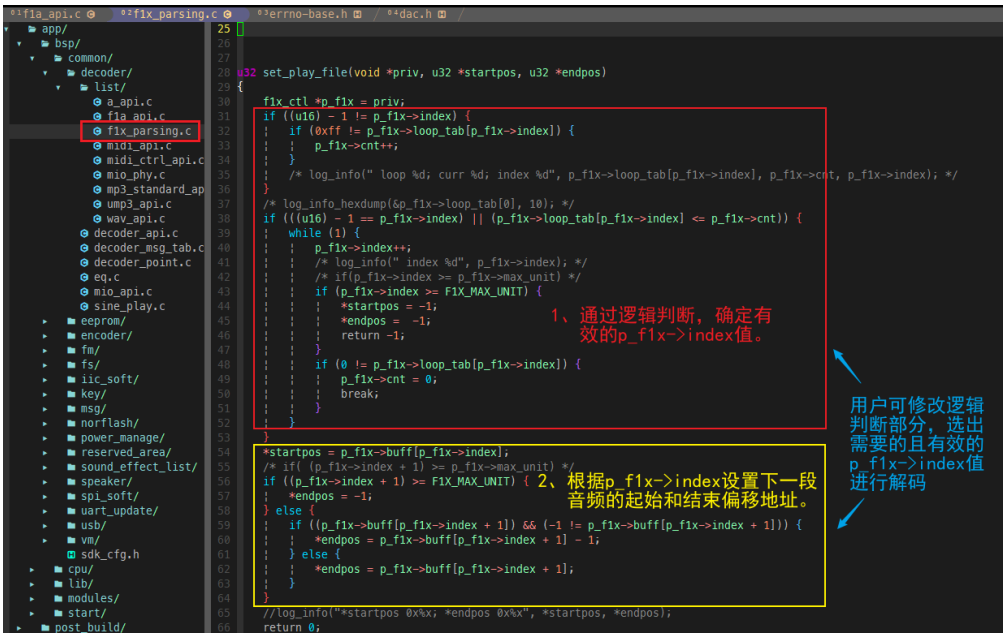
初始化结束开始播放第一段音频，第一段音频解码结束后会调用函数 set_play_file() 设置接下来要解码的音频偏移地址，以选择要播放的音频。



函数 set_play_file() 是 f1x 功能的核心。

SDK 中该函数默认流程是按照制作 f1x 文件时的文件顺序，根据 g_loop_tab[] 中定义的循环次数进行循环播放，不支持乱序播放。

如下图，用户可以修改函数 set_play_file() 中的逻辑判断部分，自定义播放顺序。



CHAPTER 13

编码接口

各芯片支持的编码情况如下：

格式	AD14N	AC104N	AD15N	AD17N	AD18N
支持最高 ADC 采样率	24000	24000	不支持 ADC	不支持 ADC	不支持 ADC
标准 MP2 编码	不支持	支持			
ump2 编码	支持	支持			
a 编码	支持	支持			

编码前需要配置编码采样率以及码率，各编码格式支持的采样率以及码率如下：

标准 MP2 编码码率（BR）/ 采样率（SR）对应关系 采样率（sr）	8000	12000	16000	22050	24000	32000	44100	48000
码率（BR）	+8kbps					32kbps		
	16kbps					48kbps		
	24kbps					56kbps		
	32kbps					64kbps		
	40kbps					80kbps		
	48kbps					96kbps		
	56kbps					112kbps		
	64kbps					128kbps		
	80kbps					160kbps		
	88kbps					160kbps		
	96kbps					224kbps		
	112kbps					256kbps		
	128kbps					320kbps		
	160kbps					384kbps		

ump2 编码采样率（sr）/ 压缩比（cr）对应关系 采 样 率 (SR)	8000	12000	16000	22050	24000	32000	44100	48000
压缩比（CR）	3							
	4							
	5							
	6							
	7							
	8							
注：ump2 码率 (br) 的范围为：采样率 (sr) * 16 / 压缩比（CR）, 其中压缩比的范围为 3 ~ 8；例如 SR = 24K, 那么 BR 的范围为:（24 * 16 / 8）~ (24 * 16 / 3)								

A 格式编码码率 (BR) / 采样率 (SR) 对应关系 采样率 (SR)	8000	12000	16000	24000	32000
码率 (BR)	32kbps	48kbps	64kbps	96kbps	128kbps

所有编码被统一管理。ADC 能工作在 8K、12K、16K、24K 这几种采样率。

编码用到的接口函数:

```
void encoder_io(u32(* fun)(void* ), void* pfile);
void stop_encode(void* pfile, u32 dlen);
u32 a_encode_api(void* p_file);
u32 mp3_encode_api(void* p_file);
```

13.1 编码应用流程

在启动录音前需要先调用函数 rec_phy_init，此函数实现对编码管理器进行初始化，会初始化编码控制器里面的一些变量资源。

在编码之前要确保已经完成了设备以及文件系统的初始化，以及在设备中创建好了录音文件，并获取到了相应的文件句柄，对于 NOR_FS 而言创建创建文件时能直接返回文件号:

1. 调用 encoder_io 实现对编码的启动，此函数会自动适应当前 ADC 工作的频率;
2. 当录音到一定时间后，需要关闭录音可以调用 stop_encode 实现停止录音;

注：录音只是支持一路录音，所以不需要获取管理编码句柄，这里会和解码有区别!

13.2 编码功能控制接口函数

13.2.1 void encoder_io(u32(* fun)(void*), void* pfile)

此函数可以启动 f1a/b/c 或 a/b/e 格式的编码:

1. u32(* fun)(void*)，传递不同的编码函数只恨可以实现编写不同格式的音频文件:
    a. a_encode_api，a/b/e 格式的编码启动接口函数;
    b. mp3_encode_api，f1a/b/c 格式的编码启动接口函数;
2. void* p_file，创建文件时，获取到的文件句柄。

13.2.2 void stop_encode(void* pfile, u32 dlen)

此函数可以启动停止编码:

1. void* p_file，创建文件时，获取到的文件句柄;
2. u32 dlen，停止解码时，抛弃文件结尾部分数据的长度;

CHAPTER 14

JL_MIO 功能

JL_MIO 用于在音频文件中添加一些波形信息，我司芯片环境中可以使用这些波形信息进行相应的控制。mio 文件制作详情见 JLmio 使用说明文档，本章介绍 mio 在 SDK 中的应用。

14.1 JLmio 一般流程

14.1.1 使用 JLmio.exe 创建 mio 文件

详情见 JLmio 使用说明。生成的 mio 文件会在工具目录的 Data 文件夹下；

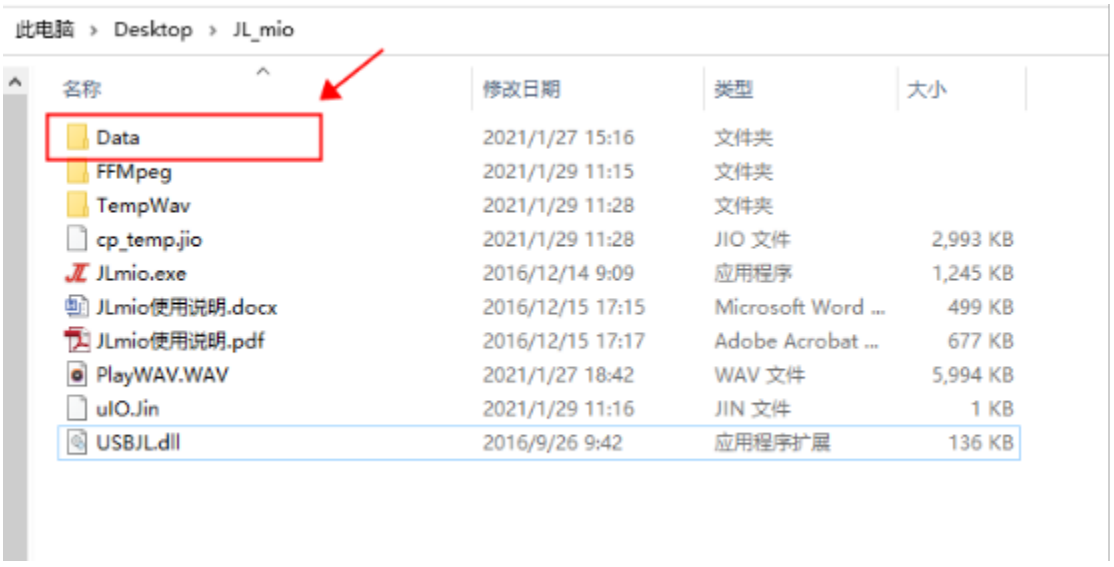


图 1 mio 文件生成位置

14.1.2 使用打包工具将音频文件与所制成的 mio 文件放于同一目录下

打包工具请参考 2.3 打包工具使用，注意：音频文件需要与生成的 mio 文件放于同一目录下；生成目录文件时，不能将 mio 放在目录的第一个文件位置处，否则音乐播放可能会失败！

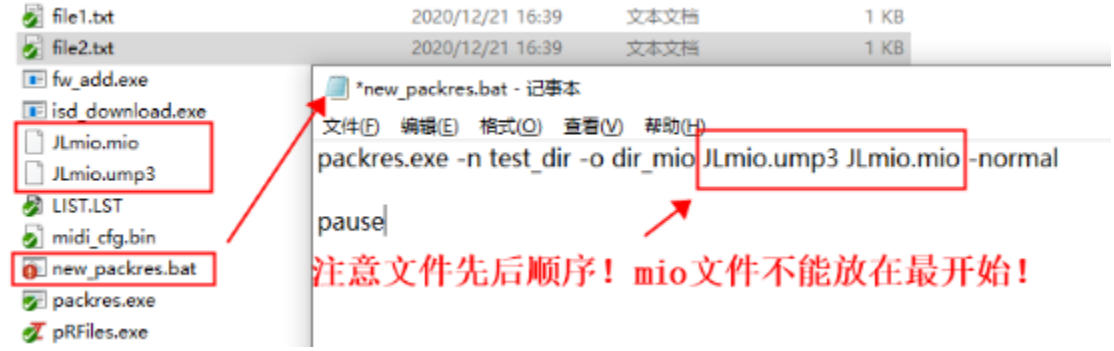


图 2 mio 文件打包位置顺序注意事项

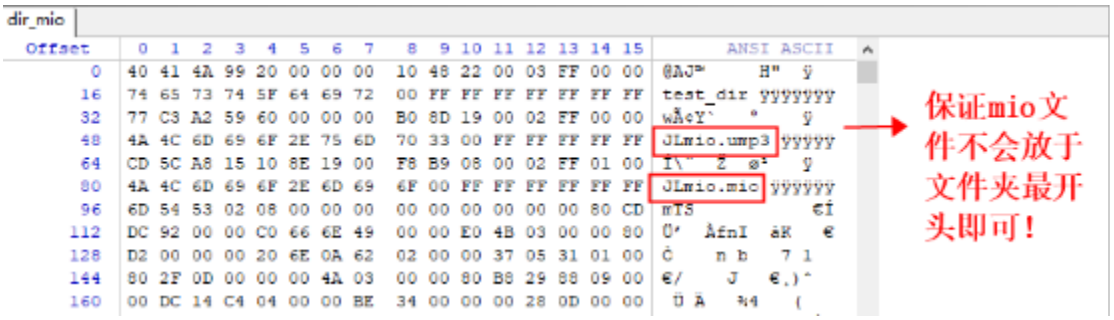


图 3 mio 文件正确打包位置顺序示意图

14.1.3 在 download.bat 中添加相应 mio 文件夹

在 download.bat 中添加所打包的包含音频文件与 mio 文件的文件夹目录 dir_mio，可放在 dir_a 文件夹后的任意位置；



图 4 将 dir_mio 文件夹添加进下载目录中

14.1.4 在 app_music.c 中添加相应 mio 文件夹

在 app_music 中添加所打包的包含音频文件与 mio 文件的文件夹目录，可放在任意位置并用按键进行文件夹切换；

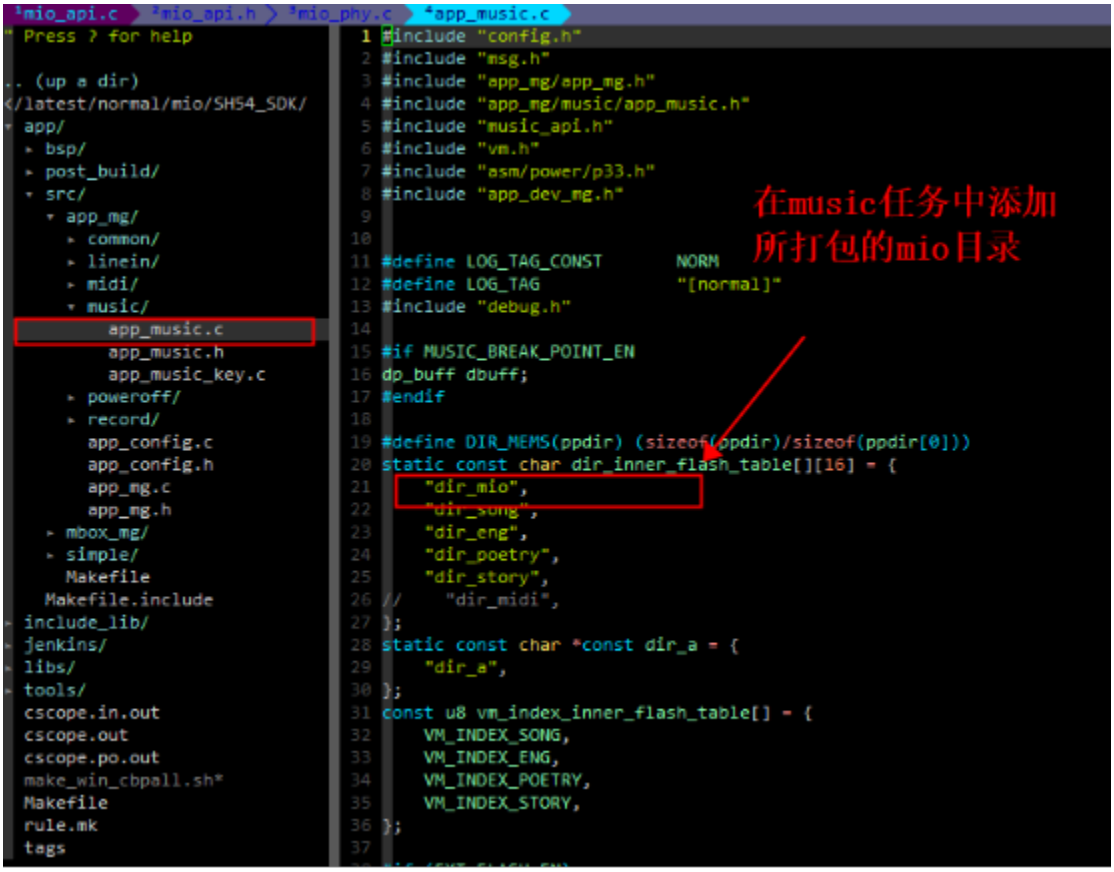


图 5 在 app_music 中添加 mio 文件夹

14.1.5 在 mio_api.c 中将相应宏开关打开

在 mio_api 中将 MIO_EN 宏定义打开即可，SDK 默认使用 1 路 PWM（PA15）以及 2 路 IO 口（PA1 与 PA2）作为示例，用户可自行添加并完善功能；注意：mio 功能会占用相应引脚，若不用 mio 功能需关闭改宏开关。

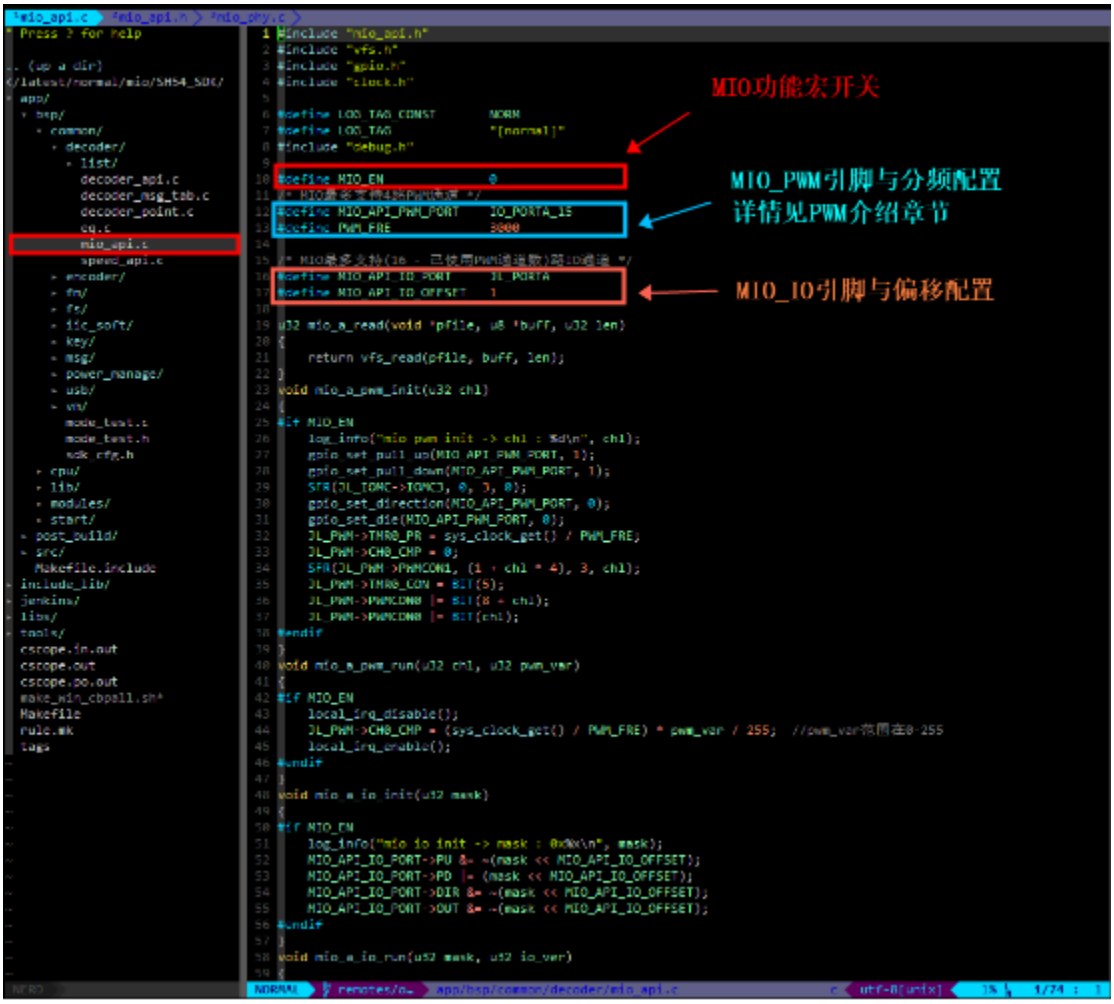


图 6 mio_pwm 与 mio_io 宏定义配置

14.2 JLmio 接口介绍

14.2.1 void mio_a_pwm_init(u32 chl)

此函数实现 mio_pwm 通道输出的初始化，其中参数：

1、chl：需要进行初始化的pwm通道编号（mio最多支持4路pwm通道）；

14.2.2 void mio_a_pwm_run(u32 chl，u32 pwm_var)

此函数实现 mio_pwm 通道输出，其中参数：

```
1、chl：正在输出的pwm通道编号；
2、pwm_var：mio文件中读取到的pwm数值，可用于配置pwm输出占空比；
```

14.2.3 void mio_a_io_init(u32 mask)

此函数实现 mio_io 通道输出的初始化，其中参数：

```
1、mask：需要进行初始化的io通道（mio最多支持（16-pwm通道）路io通道）；
    ①  mio文件中设置的每一个io通道依次对应mask中每个位；
    ②  如单io通道mask会传入0x01，双io通道mask会传入0x3，以此类推；
```

14.2.4 void mio_a_io_run(u32 mask，u32 io_ver)

此函数实现 mio_io 通道输出，其中参数：

```
1、mask：需要进行初始化的io通道；
2、io_ver：mio文件中读取到的io数值，可用于配置IO口的电平；
```

CHAPTER 15

玩具应用篇（新工程 - voice_toy）

15.1 简介

2023 年 7 月 11 日发布的 AD1x-457_AC104-release_v1.7.0 SDK，新增了 voice_toy 工程；

该工程在旧版玩具工程 app_mg 与示例工程 simple 基础上进行优化，使应用示例更加简单、直观，便于用户上手开发；

各芯片支持的应用模式如下：

应用模式	AD14N	AD15N	AD17N	AD18N
toy_music	√	√	√	√
toy_midi	√	√	√	√
toy_midi_keyboard	√	√	√	√
toy_record	√			
toy_speaker	√			
toy_linein	√			
toy_softoff	√	√	√	√
toy_idle(demo)	√	√	√	√

不同的模式间会有资源和按键互斥的关系。应用上面做了一个简单的模式切换管理来管理不同模式的资源和按键。

模式管理的整体管理思路是，进入该模式的时候初始化模式需要使用的资源和按键处理函数，退出该模式的时候关闭该模式需要使用的资源和释放按键处理函数。

模式初始化以及模式切换函数：

1. void app(void);
2. void app_next_mode(void)

模式管理接口的文件摆布：

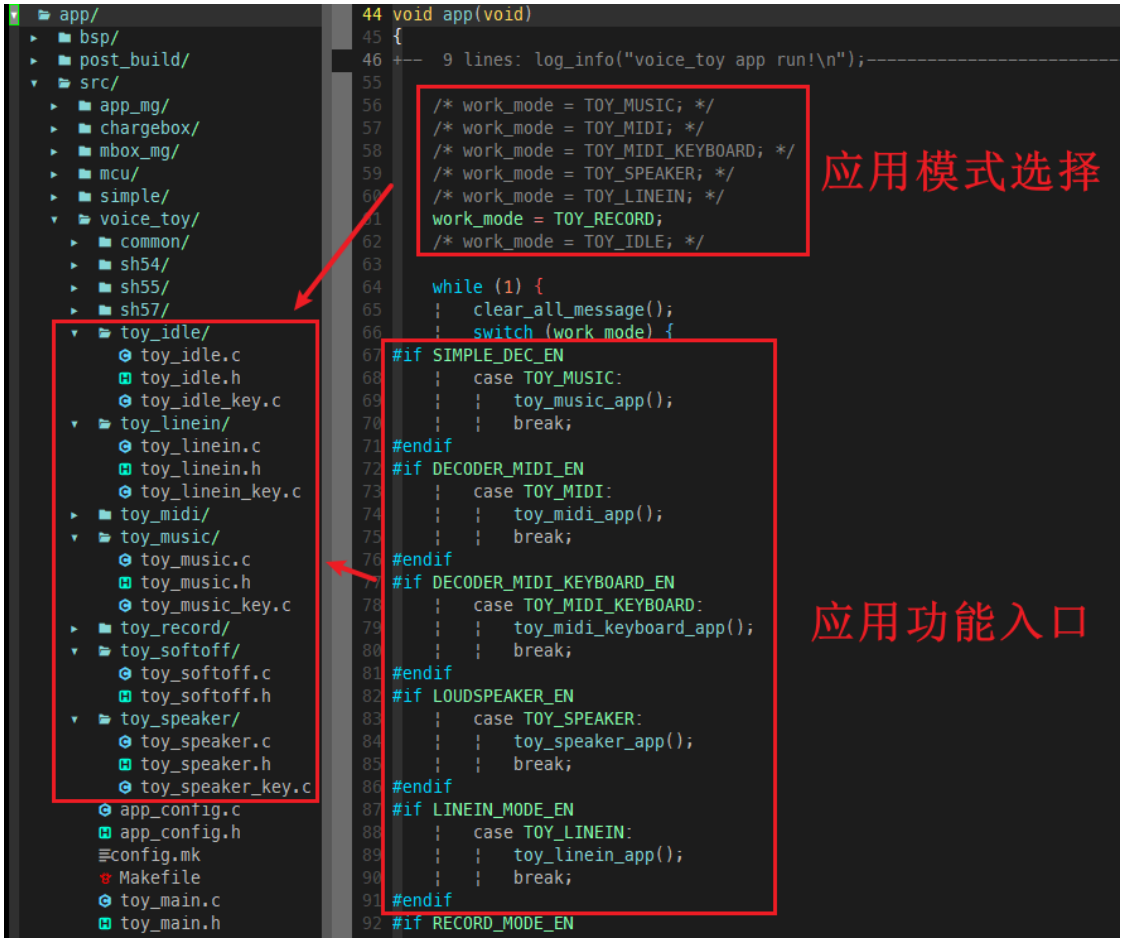


图 1 voice_toy 应用模式管理

15.1.1 模式相关函数

函数 void app_next_mode(void)

此函数切换下一个应用模式，上一个应用切换前需要释放申请的资源:

15.2 toy_music 多路解码模式

15.2.1 模式说明

- 多路解码模式使用了 SYD 文件系统，能够播放打包成 dir 文件的 F1A/B/C/X、UMP3，并叠加 A 文件解码，同时支持外挂 FLASH 解码。
- 该模式还具备 VM 记忆，可记录解码文件序号与断点。
- 多路解码使用 play_control 结构体，其成员如下：：

```
typedef struct __play_control {
void *pfs;           //文件系统句柄
void *pfile;         //文件句柄
dec_obj *p_dec_obj; //解码器句柄
void *pdp;           //断点指针
u32  findex;         //当前播放的文件序号
u32  ftotal;         //当前文件夹中的文件总数
u16  dec_type;       //解码器支持的解码类型，如BIT_F1A1
u8   dev_index;      //当前挂载设备的序号，如INNER_FLASH_RO
u8   play_mode;      //播放模式(Music模式使用)
void *device;        //当前挂载设备的句柄
void *pdir;          //文件夹指针,用于切换文件夹
u8   *p_vm_tab;      //存放vm项指针,用于记忆不同文件夹的掉电数据
u8   dir_total;      //文件夹总数
u8   dir_index;      //当前播放的文件夹序号
u8   loop;           //播放循环次数
// u8   flag;           //标志位(Music模式使用)
} play_control;
```

- 涉及的文件目录：app\src\voice_toy\toy_music，涉及的应用函数如下：

15.2.2 解码播放函数

函数 u32 simple_dev_fs_mount(play_control *ppctl)

此函数是多路解码挂载设备和 SYD 文件系统，其中参数：

```
1. ppctl: 音乐播放器句柄指针;
2. 返回值:
    成功 返回0;
    失败 错误值可查看errno-base.h;
```

函数 u32 simple_dev_fs_close(play_control *ppctl)

此函数实现多路解码释放文件、文件系统句柄，并关闭设备；释放的句柄分别是结构体 play_control 中的成员 pfile，pfs 以及 device，其中参数：

```
1. ppctl: 音乐播放器句柄指针;
2. 返回值: 0;
```

函数 u32 get_decoder_status(dec_obj *obj)

此函数实现获取多路解码当前的状态，分别有播放停止（MUSIC_STOP）、播放暂停（MUSIC_PAUSE）以及播放中（MUSIC_PLAY）三种状态，其中参数：

```
1. obj: 解码句柄;
2. 返回值: 返回当前解码器播放状态;
```

函数 u32 simple_play_file_bypath(play_control *ppctl, const char *path)

此函数实现通过传入的路径播放文件，其中参数：

```
1. ppctl：音乐播放器句柄指针；
2. path：音频文件路径（对于外挂flash，忽略第一级目录）；
3. 返回值：
    成功 返回0；
    失败 错误值可查看errno-base.h；
```

函数 u32 simple_play_file_byindex(play_control *ppctl, u32 index)

此函数实现通过传入的序号播放文件，ppctl 中的 pdir 为需要播放的文件夹路径，其中参数：

```
1. ppctl：音乐播放器句柄指针；
2. index：音频文件序号；
3. 返回值：
    成功 返回0；
    失败 错误值可查看errno-base.h；
```

函数 u32 play_one_file(play_control *ppctl)

此函数实现通过 ppctl 中的 findex 参数播放文件，播放成功后记录 VM 数据，该接口由应用层调用，其中参数

```
1. ppctl：音乐播放器句柄指针；
2. 返回值：
    成功 返回0；
    失败 错误值可查看errno-base.h；
```

函数 u32 play_prev_file(play_control *ppctl)

此函数实现播放上一首音频，播放成功后记录 VM 数据，该接口由应用层调用，其中参数

```
1. ppctl：音乐播放器句柄指针；
2. 返回值：
    成功 返回0；
    失败 错误值可查看errno-base.h；
```

函数 u32 play_next_file(play_control *ppctl)

此函数实现播放下一首音频，播放成功后记录 VM 数据，该接口由应用层调用，其中参数

```
1. ppctl：音乐播放器句柄指针；
2. 返回值：
    成功 返回0；
    失败 错误值可查看errno-base.h；
```

函数 u32 simple_switch_device(play_control *ppctl)

此函数实现切换播放设备（内置-外挂 flash 互相切换），该接口由应用层调用，其中参数

```
1. ppctl: 音乐播放器句柄指针;
2. 返回值:
    成功 返回0;
    失败 错误值可查看errno-base.h;
```

函数 u32 simple_next_dir(play_control *ppctl)

此函数实现切换下一个文件夹，该接口由应用层调用，其中参数

```
1. ppctl: 音乐播放器句柄指针;
2. 返回值:
    成功 返回0;
    失败 错误值可查看errno-base.h;
```

函数 int simple_play_speed_set(play_control *ppctl, SPEED_PITCH_PARA_STRUCT *sp_parm)

此函数实现在播放过程中动态调整变速变调参数，该接口需要在链接变速变调音效后才可在应用层使用，其中参数:

```
1. ppctl: 音乐播放器句柄指针;
2. 变速变调参数: 其结构体成员具体可查看音效变速变调章节;
3. 返回值:
    成功 返回0;
    失败 返回-1;
```

15.3 toy_midi midi 解码模式

15.3.1 模式说明

- midi 解码模式使用了 SYD 文件系统，能够播放打包生成 dir 的 midi 文件，并叠加 A 文件解码。
- midi 解码同样使用了 play_control 结构体作为播放句柄。

15.3.2 相关函数

函数 void midi_decode_init(void)

该函数实现查找 midi 解码的音色库并记录音色库信息，在系统初始化中调用;

函数 void midi_dec_config(dec_obj *p_dec_pbj, u32 cmd, void *parm)

此函数实现查找 midi 解码的功能参数，cmd 的具体介绍请看 midi 文件第 2 节 MIDI 乐谱解码介绍，其中参数码的音色库并记录音色库信息，在系统初始化中调用；

函数 void midi_mode_set(dec_obj *p_dec_pbj, u8 mode)

该函数实现配置 midi 解码的模式，其中参数：：

```
1、p_dec_obj：midi解码句柄；
2、mode：midi解码模式，可用mode如下：
    ①  CMD_MIDI_CTRL_MODE_0，正常解码模式；
    ②  CMD_MIDI_CTRL_MODE_1，one key one note模式；
    ③  CMD_MIDI_CTRL_MODE_2，只推消息不出声模式；
    ④  CMD_MIDI_CTRL_MODE_W2S，外部音源模式；
```

函数 void midi_okon_goon(dec_obj *p_dec_pbj)

该函数实现 midi 解码 okon 模式启动下一个音符，其中参数：

1、p_dec_obj：midi 解码句柄

函数 void midi_init_info(MIDI_INIT_STRCUT *init_info)

该函数实现 midi 解码配置初始化，会给 midi 解码器主动调用，其中参数：：

```
1、init_info：midi解码初始化配置结构体，具体请参照代码中的配置信息
```

15.4 toy_midi_keyboard midi 琴模式

15.4.1 模式说明

- MIDI 琴功能支持通过函数驱动 midi 解码。该应用依赖我司自定义的 mid 音色库，用户可通过我司 mid 工具进行转换，详情请看《JLmidi 工具使用说明》。

15.4.2 相关函数

函数 void midi_ctrl_decode_init(void)

该函数实现查找 midi 琴的音色库并记录音色库信息；

函数 u32 toy_midi_ctrl_set_prog(dec_obj *obj, u8 prog, u8 chn)

该函数实现查找设置 midi 琴的乐器，0 其中参数：：

```
1.obj：midi琴解码句柄；
2.prog：乐器号，与midi音色库制作工具中的乐器号一致。详情请看《杰理AD14 AD15N
↪MIDI应用说明文档》说明；
3.chn：发声通道，该值范围在0~15；
4.返回值：成功 0；
    失败 其他值；
```

函数 u32 toy_midi_ctrl_pitch_bend(dec_obj *obj, u16 pitch_val, u8 chn)

该函数实现配置 midi 琴弯音轮参数，其中参数：：

```
1.obj：midi琴解码句柄；
2.pitch_val：弯音值，默认值是256，可根据该值调整；
3.chn：发声通道，该值范围在0~15；
4.返回值：成功 0；
    失败 其他值；
```

函数 u32 toy_midi_ctrl_dec_config_api(dec_obj *obj, u32 cmd, void *parm)

该函数实现配置 midi 琴的其他功能，其功能参数和 midi 解码中的一致，其中参数：：

```
1.obj：midi琴解码句柄；
2.cmd：可用命令如下：
    CMD_MIDI_CTRL_TEMPO；
    CMD_MIDI_SET_EX_VOL；
    CMD_INIT_CONFIGS；
    CMD_MIDI_SET_SEMITONE；
    CMD_MIDI_SET_SWITCH；
3.parm：命令参数，可查看midi解码中的介绍；
4.返回值：成功 0；
        失败 其他值
```

函数 u32 toy_midi_ctrl_note_on(dec_obj *obj, u8 nkey, u8 nvel, u8 chn)

该函数实现弹奏指定通道的音符，其中参数：：

```
1.obj：midi琴解码句柄；
2.nkey：音符，与midi音色库制作工具中的音符号一致，范围在0~127；
3.nvel：琴键按下力度，范围在0~
↪127；部分乐器（如钢琴）有效，不同力度会使按下后的余音长短不同；
4.chn：发声通道，该值范围在0~15；
5.返回值：成功 0；
        失败 其他值；
```

函数 u32 toy_midi_ctrl_note_off(dec_obj *obj, u8 nkey, u8 chn, u16 decay_time)

该函数实现查找设置 midi 琴的乐器。对于声音不会衰弱的乐器（如笛子），调用 Note_on 函数后需要调用该函数以结束音符，对于声音会衰弱的乐器（如钢琴）可不用。其中参数：：

```
1.obj：midi琴解码句柄；
2.nkey：音符，与midi音色库制作工具中的音符号一致，范围在0~127；
3.chn：发声通道，该值范围在0~15；
4.decay_time：衰减时间，单位ms，若为0则使用音色自带的衰减；
5.返回值：成功 0；
失败 其他值；
```

函数 u32 toy_midi_ctrl_vel_vibrate(dec_obj *obj, u8 nkey, u8 vel_step, u8 vel_rate, u8 chn)

该函数实现配置音符的颤音效果，其中参数：：

```
1.obj：midi琴解码句柄；
2.nkey：音符，与midi音色库制作工具中的音符号一致，范围在0~127；
3.vel_step：抖动步长，该值范围在0~7；
4.vel_rate：抖动幅度，该值范围在0~15；
5.chn：发声通道，该值范围在0~15；
6.返回值：成功 0；
    失败 其他值；
```

函数 u32 midi_ctrl_melody_stop_trigger(void *priv, u8 key)

该函数实现停止 midi 琴解码，在所有音符停止后会被调用，其中参数：：

```
1.obj：midi琴解码句柄；
2.返回值：无；
```

函数 void midi_ctrl_callback_init(dec_obj *obj, u32(*melody_callback)(void *, u8, u8), u32(*melody_stop_callback)(void *, u8))

该函数实现配置音符开始与结束的回调函数，其中参数：：

```
1.obj：midi琴解码句柄；
2.melody_callback：音符开始回调函数，回调函数的原型参照《杰理AD14 AD15N␣
↪MIDI应用说明文档》1.2.6.1主旋律的音符回调功能中的说明；
1.melody_stop_callback：音符结束回调函数，回调函数的原型参照《杰理AD14 AD15N␣
↪MIDI应用说明文档》1.2.6.2主旋律的音符结束回调功能中的说明；
2.返回值：无；
```

函数 void toy_midi_ctrl_get_play_key(dec_obj *obj, u8 chn)

该函数实现获取指定通道正在播放的所有 key 值，其中参数：：

```
1.obj: midi琴解码句柄;
2.chn: 需要查询的通道;
返回值: 无;
```

15.5 toy_speaker 扩音模式

15.5.1 模式说明

- 扩音模式实现将 mic 采集到的数据直通或者串联算法输出到 dac。
- 扩音模式算法支持的算法详细见音效算法章节。

15.5.2 相关函数

函数 void audio_adc_speaker_start(void)

该函数实现扩音模式初始化 adc 和 dac，并且串联算法的整体流程，初始化扩音模式时使用；

函数 audio_adc_speaker_reless(void)

该函数实现扩音模式断开 adc 和 dac 数据流链接流程，并且释放 src 变采样申请使用的资源，在关闭扩音模式时使用；

15.6 toy_linein linein 模式

15.6.1 模式说明

- 该模式实现数字 linein，该模式下需要将 AUDIO_ADC 和 AUDIO_DAC 配置到相同的采样率，可调节音量

15.7 toy_record 录音模式

15.7.1 模式说明

- 录音模式实现将 mic 采集到的数据进行编码，并通过 norfs 文件系统方式保存至内置 flash(放置代码的 flash)
- 支持 A 格式和 ump3 格式编码

- 录音模式使用 Encode_Control 结构体，其成员如下：：

```
typedef struct _Encode_Control {
char fs_name[8];              //编码文件系统名
void *device;                 //编码设备句柄
void *pfs;                    //编码文件系统句柄
void *pfile;                  //编码文件句柄
union {
    u32 rec_file_index;       //nor_fs最新文件号
};
u8 dev_index;                 //编码设备序号
ENC_STATUS enc_status;        //编码器工作状态
} Encode_Control;
```

15.7.2 相关函数

函数 static int encode_start(Encode_Control *obj)

该函数实现录音功能启动。内部会开启 ADC 模块和整个录音的设备和文件流程；

函数 int norfs_enc_file_create(Encode_Control *obj)

该函数实现录音模式创建录音文件，内部包含开设备、挂载 norfs 文件系统和创建文件流程，其参数：

```
1、obj：录音句柄；
2、返回值：
    成功 返回0；
    失败 其他值；
```

函数 void encoder_stop(Encode_Control *obj)

该函数实现录音模式停止编码，内部会关闭编码、写文件流程和 ADC 模块；

函数 dec_obj *norfs_enc_file_decode(Encode_Control *obj，u16 dec_type)

该函数实现录音模式播放录音文件，内部包含开设备、挂载 norfs 文件系统和创建文件流程，其参数：

```
1、obj：录音句柄；
2、dec_type：解码格式；
3、返回值：
    成功 返回0；
    失败 其他值；
```

CHAPTER 16

玩具应用篇（旧工程 - app_mg）

16.1 简介

该工程在 2023 年 7 月 11 日发布的 AD1x-457_AC104-release_v1.7.0 SDK 后停止维护，后续仅维护 voice_toy 工程；

AD140，AD150 的应用模式有 MIDI 模式、音乐模式、录音模式和关机模式，不同的模式间会有资源和按键互斥的关系。应用上面做了一个简单的模式切换管理来管理不同模式的资源和按键。

模式管理的整体管理思路是，进入该模式的时候初始化模式需要使用的资源和按键处理函数，退出该模式的时候关闭该模式需要使用的资源和释放按键处理函数。

模式初始化函数：

1. int app_mg_init(APP_T app,void* param);
2. void app(void);

模式切换函数：

1. int app_switch(APP_T app,void* param);
2. int app_switch_next(void);
3. int app_switch_priv(void);
4. u8 app_get(void);

模式管理接口的文件摆布：

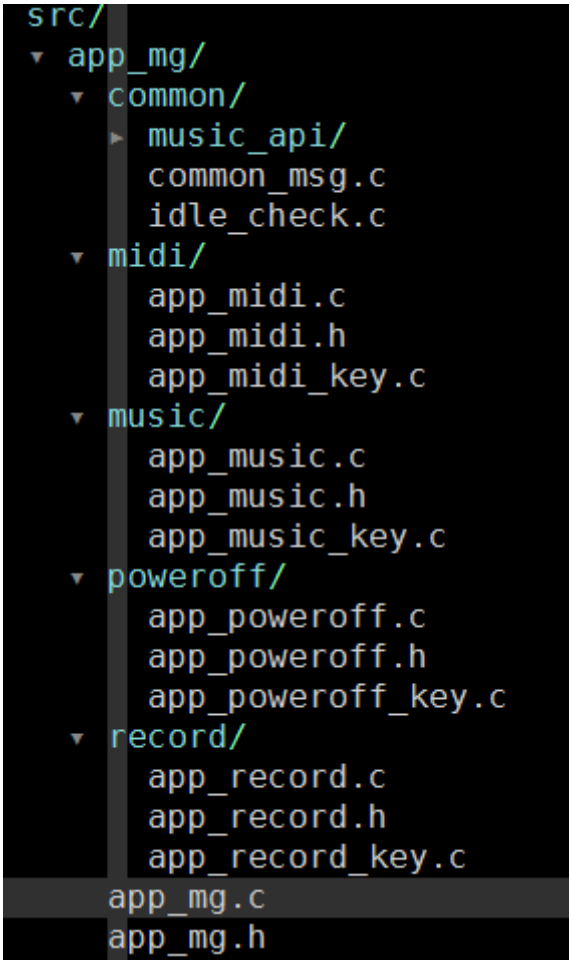


图 1 模式管理文件摆放

16.2 模式初始化函数

16.2.1 函数 int app_mg_init(APP_T app,void* param)

此函数初始化了第一个模式: 1. app： 初始化的模式; 2. param: 初始化的模式参数;

16.2.2 函数 void app(void)

此函数为模式管理的入口函数，并且模式间的切换由该函数完成。

16.3 模式切换函数

16.3.1 函数 int app_switch(APP_T app,void* param)

此函数可以切换到指定的模式:

1. app : 指定的 app 模式;
2. param: 切换模式参数;

16.3.2 函数 int app_switch_next(void)

此函数用来自动切换 app 到下一个模式，模式列表看 APP_T。

16.3.3 函数 int app_switch_priv(void)

此函数用来自动切换 app 到上一个模式，模式列表看 APP_T。

16.3.4 函数 u8 app_get(void)

此函数用来获取当前的 app 是在哪个模式。

16.4 常用工程配置文件

16.4.1 “app_config.c”文件

“app_config.c”可以配置中断优先级，AUDIO 参数，打印等。

详细请看章节“1.2.1 应用配置文件《app_config.c》介绍”。

16.4.2 函数 u8 app_get(void)

“app_config.h”配置了串口，按键选项，按键音，SD 卡使能，USB 模块配置项。

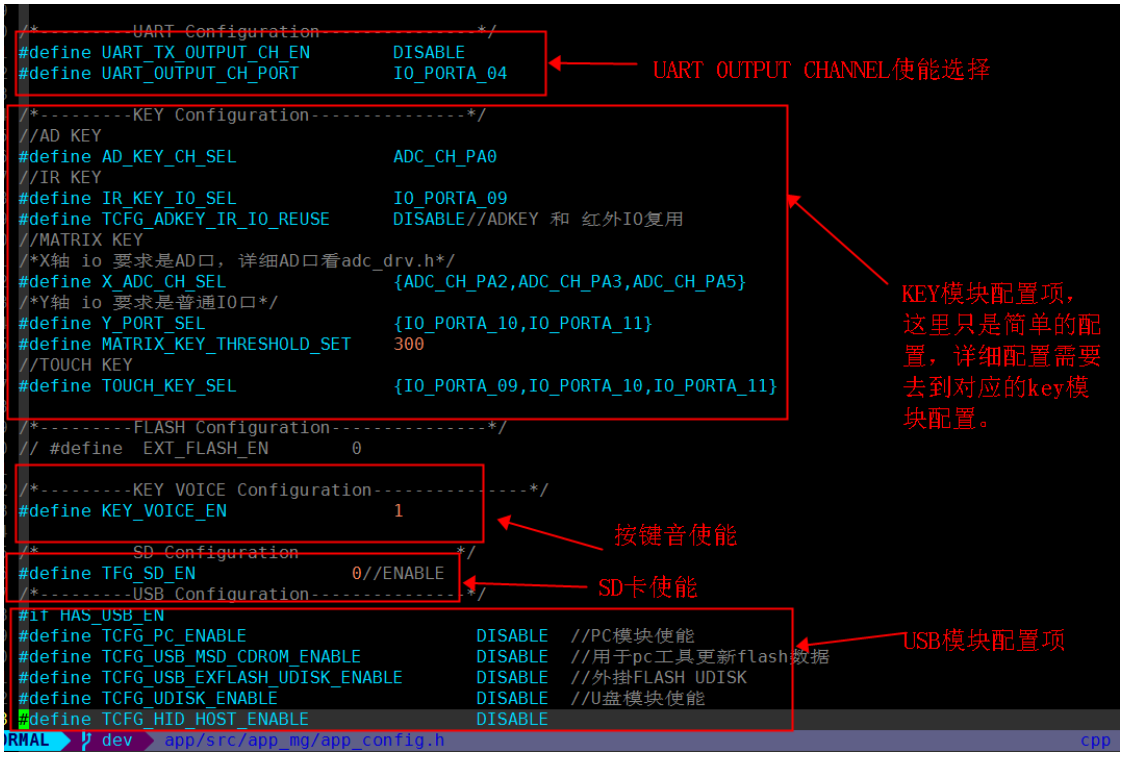


图 2 “app_config.h”文件介绍

16.4.3 code block 工程配置

code block 工程配置配置了一些模块使能，用户可以在这里选择开启和关闭相关的模块。

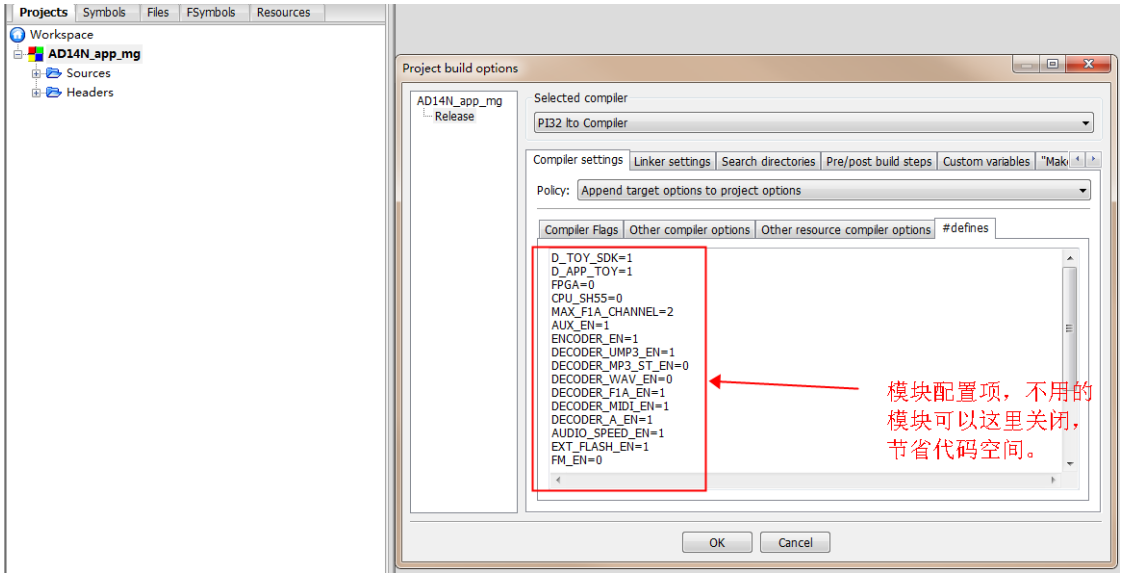


图 3 code block 工程配置

CHAPTER 17

小音箱应用

AC104N 的应用模式有 MIDI 模式、音乐模式、录音模式和关机模式，不同的模式间会有资源和按键互斥的关系。应用上面做了一个简单的模式切换管理来管理不同模式的资源和按键。

模式管理的整体管理思路是，进入该模式的时候初始化模式需要使用的资源和按键处理函数，退出该模式的时候关闭该模式需要使用的资源和释放按键处理函数。

模式初始化函数:

```
1. int app_mg_init(APP_T app,void* param);
2. void app(void);
```

模式切换函数:

```
1. int app_switch(APP_T app,void* param);
2. int app_switch_next(void);
3. int app_switch_priv(void);
4. u8 app_get(void);
```

模式管理接口的文件摆布 (见下图):

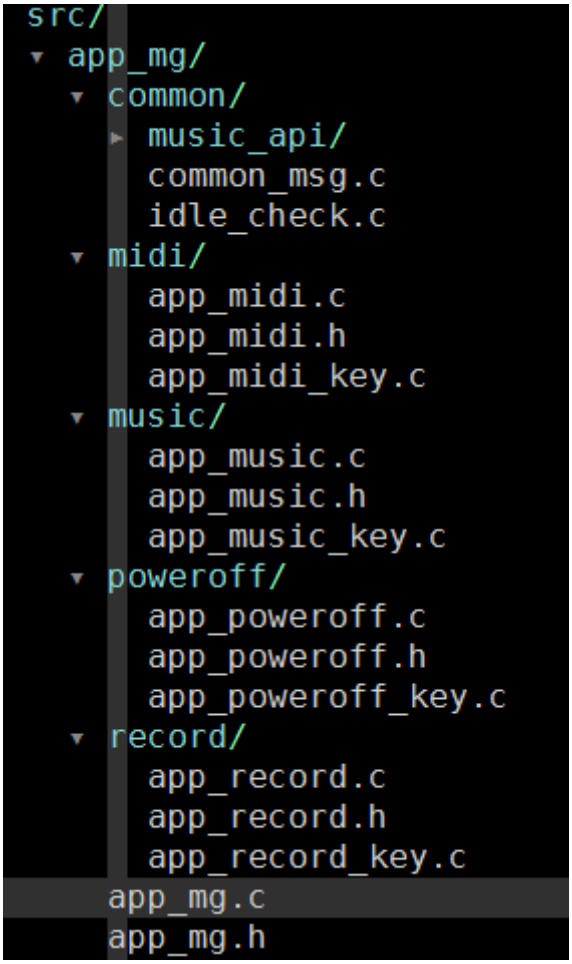

src/
app_mg/
common/
music_api/
common_msg.c
idle_check.c
midi/
app_midi.c
app_midi.h
app_midi_key.c
music/
app_music.c
app_music.h
app_music_key.c
poweroff/
app_poweroff.c
app_poweroff.h
app_poweroff_key.c
record/
app_record.c
app_record.h
app_record_key.c
app_mg.c
app_mg.h

图 4 模式管理文件摆放

CHAPTER 18

AUDIO DAC

AD14N / AD15N / AC104N / AD17N / AD18N 内置应用于语音场合的 DAC，此 DAC 支持 16bit 精度，单声道。

各芯片音频性能对比如下：

芯片 APA 差分性能对比	AD14N(PWM	AD15N(PWM	AD17N(MODE2	AD18N(MODE2)
Output Swing(空载)	2.3Vrms	2.3Vrms	2.2Vrms	2.2Vrms
THD	-33dB	-33dB	-70+dB	-66+dB

芯片 RDAC 性能对比	AC104N	AD16N	AD18N
Output Swing(空载)	430mVrms	760mVrms	807mVrms
SNR	83dB	97dB	84dB
DR	82dB	96dB	84dB
THD	-70dB	-82dB	-75dB
Noise Floor	30uVrms	11uVrms	50uVrms

同时 AD14N / AC104N 支持最多 3 路 DAC 解码输出，AD15N / AD17N 最多支持 2 路 DAC 解码输出。

各芯片支持的 AUDIO DAC 采样率如下：

DAC 采样率	AD14N/AC104N	AD15N	AD17N	AD18N(APA)	AD18N(RDAC)
8000	√	√	不支持	不支持	√
12000	√	√	不支持	不支持	√
16000	√	√	不支持	不支持	√
24000	√	√	不支持	不支持	√
32000	√	√	√	√	√
44100	不支持	不支持	√	√	√
48000	不支持	不支持	√	√	√

本章介绍 DAC 相关结构体以及接口。Audio DAC 涉及相关文件，分别是 app/lib/dac_api.c 和 include_lib/audio/dac_api.h 以及 include_lib/audio/dac.h。

Audio DAC 有以下结构体：

```
typedef struct _DAC_MANAGE {
sound_out_obj *sound[DAC_CHANNEL_NUMBER];
void (*kick[DAC_CHANNEL_NUMBER])(void *);    //kick函数
u8 ch;                                       //解码通道数
u16 vol_phy;                                 //模拟音量
u8 vol;                                      //数字音量
u8 flag;                                     //标志位
} DAC_MANAGE;
```

Audio DAC 主要有以下函数接口：

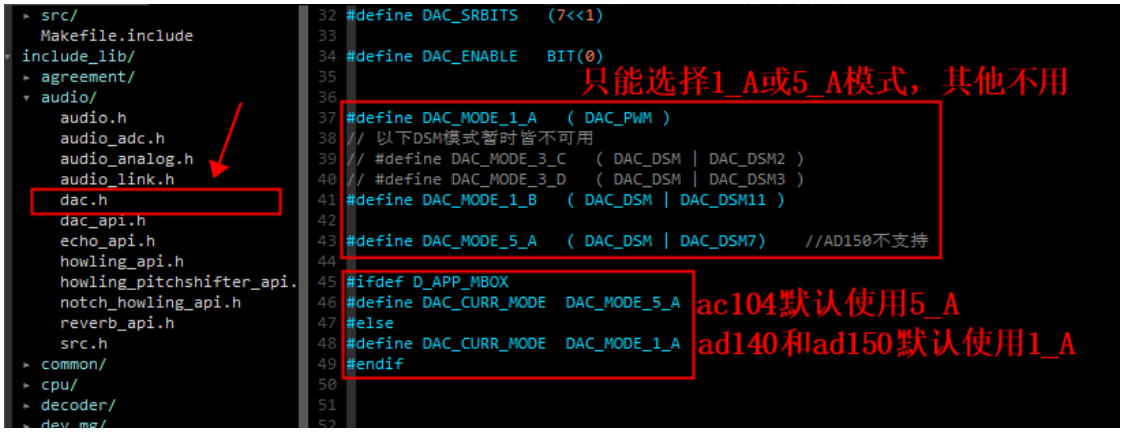
```
void dac_mode_init(void);
void dac_init_api(u32 sr);
void dac_power_on(u32 sr, bool delay_flag);
void dac_off_api(void);
bool dac_mute(bool mute);
bool regist_dac_channel(void *psound, void *kick);
bool unregist_dac_channel(void *psound);
u8 dac_vol(char set, u8 vol);
void fill_dac_fill(u8 *buf, u32 len);
```

18.1 AUDIO DAC 配置参数

18.1.1 AD14N / AD15N / AC104N DAC 驱动模式介绍

AD14N / AC104N 支持的 DAC 驱动模式模式在 dac_cpu.h 文件中可以进行选择，其中：

芯片 DAC 工作模式 工作模式特点	DAC_PWM 差分推，功耗较低	DAC_DSM2 差分/单端推，音质好，功耗较高	DAC_DSM3 & DAC_DSM11 差分推，音质好、功耗高	DAC_DSM7 输出模拟信号，音质好
AD14N / AC104N	支持	仅 C 版芯片支持	仅 C 版芯片支持	支持单端推
AD15N	支持	支持	仅 C 版芯片支持	不支持

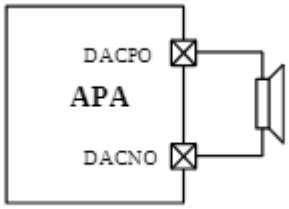


dac 工作模式设置示意图

DAC 的驱动方式介绍：

1）Class-D 直接驱动喇叭 (DAC_PWM、DAC_DSM2/3/11 模式)

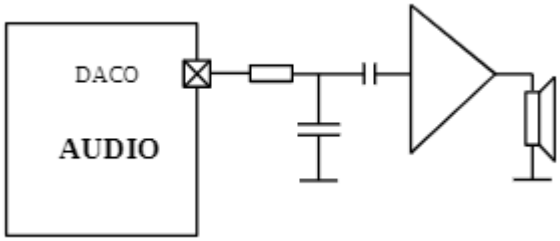
该驱动方式通过 DACP 和 DACN 引脚输出，内置功放。



ClassD 直接驱动喇叭示意图

2）单端输出驱动单端功放 (DAC_DSM7 模式)

该驱动方式在 AD14N / AC104N 通过 PB0 输出，需要外接功放芯片。(AD15N 不支持该驱动方式)



dac 单端输出模式示意图

18.1.2 AD17N APA 驱动模式介绍

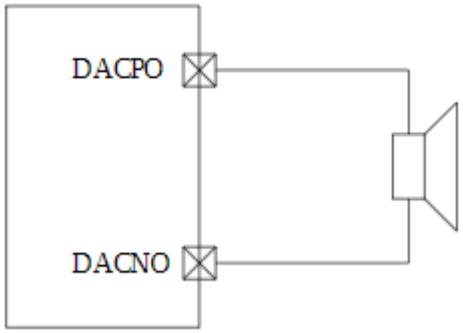
AD17N 支持的 APA 工作模式在 dac_cpu.h 文件中可以进行选择，其中：

AD17N APA 工作模式	工作模式特点
MODE1	支持单端，单端输出性能好
MODE2	支持差分，差分输出性能好
MODE3	暂不支持

APA 的驱动方式有：

1. **差分直推喇叭（推荐使用 MODE2）**

   该驱动方式对应 MODE1、MODE2 模式，通过 APAP 和 APAN 引脚输出，内置功放。



差分直推喇叭示意图




```
#define APA_MODE_BTL          (0 << 11)
#define APA_MODE_PBTL         (1 << 11)

#define APA_MODE1_NOR          (0 << 9)
#define APA_MODE2_SNR          (1 << 9)
#define APA_MODE3_THD          (2 << 9)
#define APA_MODE_BITS          (3 << 9)

#define APA_MUTE              (1 << 8)

#define APA_PND               (1 << 7)
#define APA_CPND              (1 << 6)
#define APA_IE_EN             (1 << 5)
#define APA_PWM_EN            (1 << 4)
#define APA_DSM_EN            (1 << 3)

//apa 输出采样率
#define APA_SR48K     (0<<0)
#define APA_SR44K1    (1<<0)
#define APA_SR32K     (2<<0)
#define APA_SRBITS    (7<<0)
#define DAC_SRBITS    APA_SRBITS


#define APA_CLK_DEFAULT       APA_PLL240M
#define APA_MODE_DEFAULT      APA_MODE2_SNR
#define DAC_DEFAULT           (APA_IE_EN | APA_MODE_DEFAULT | APA_MODE_BTL)


#define audio_clk_init()  SFR(JL_CLOCK->CLK_CON2,  0,  2,  APA_CLK_DEFAULT)

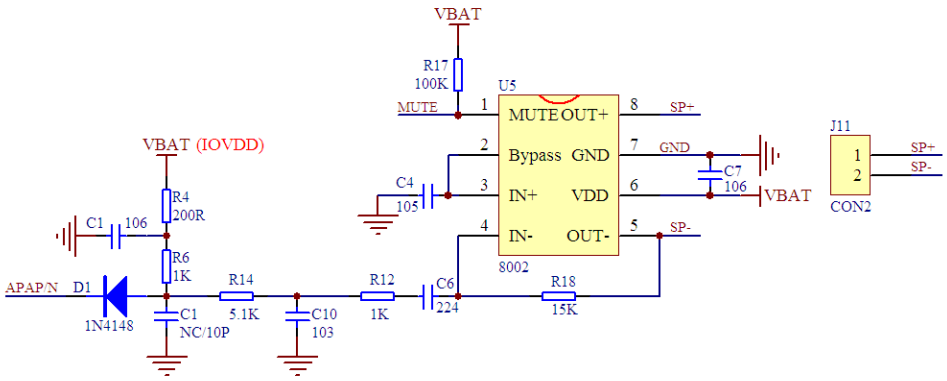
#define SR_DEFAULT  32000


#endif
```

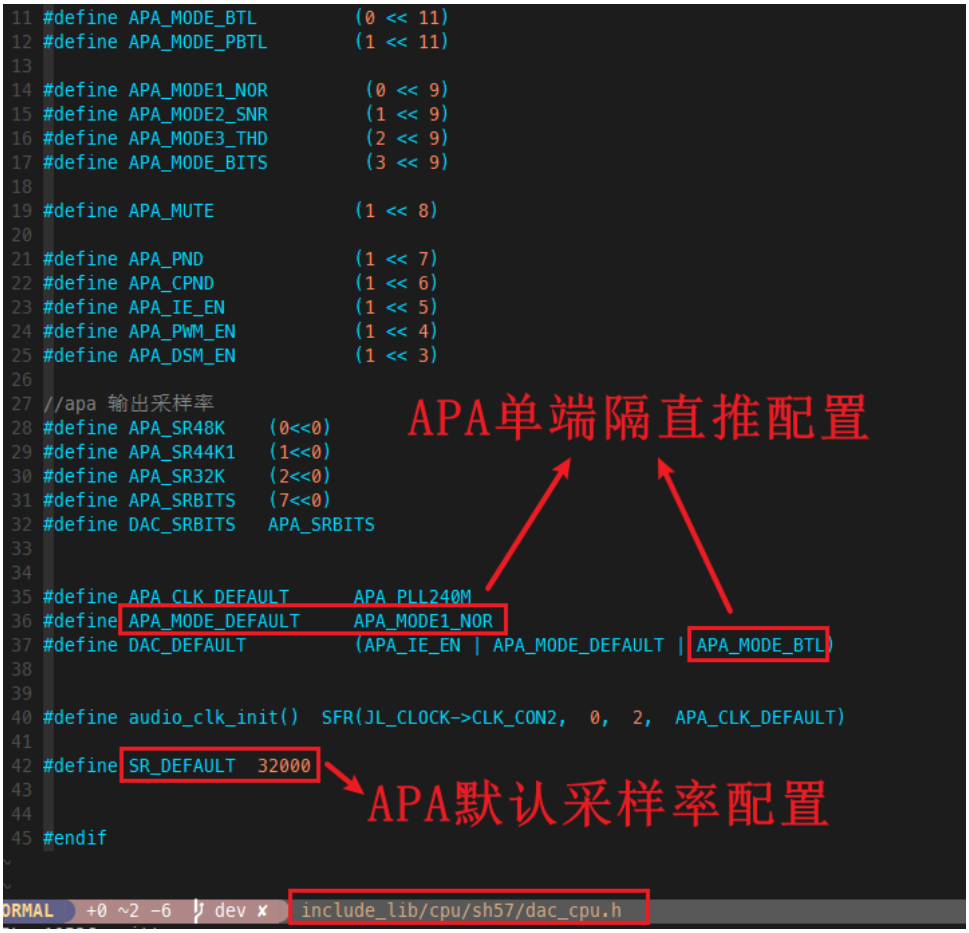
差分直推模式配置

2. **单端隔直推功放（推荐使用 MODE1）**

该驱动方式对应 MODE1 模式，通过 APAP、APAN 引脚输出推功放（不支持立体声）。



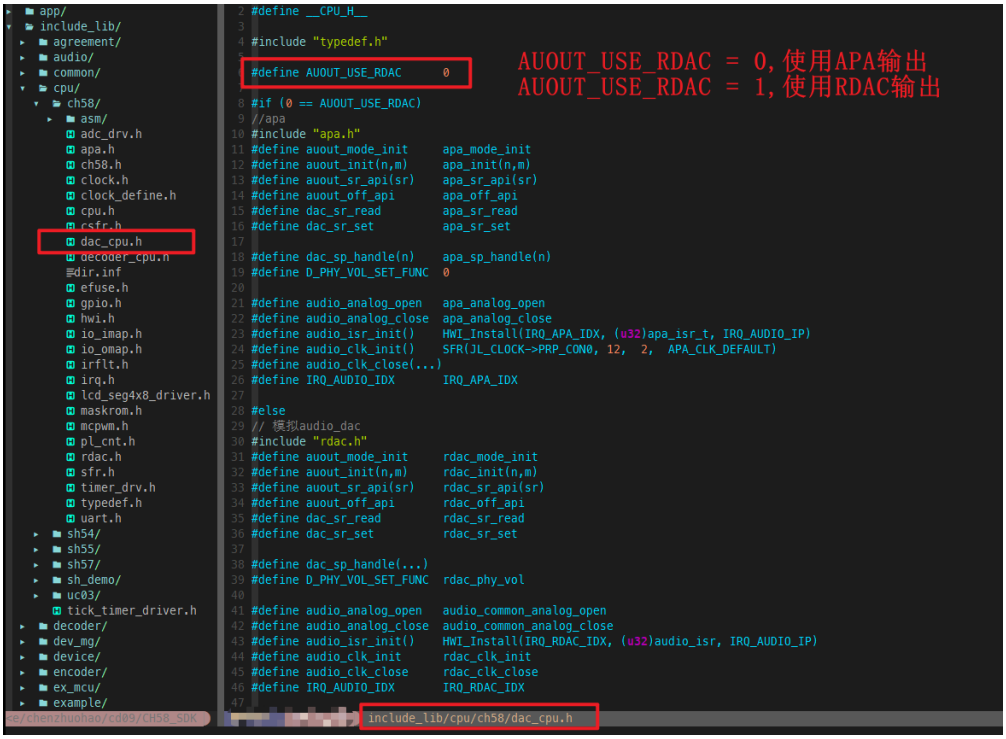
单端隔直推功放原理图



单端隔直推模式配置

18.1.3 AD18N AUDIO 驱动模式介绍

AD18N AUDIO 支持 Class-D 功放的 APA 工作模式，以及 16bit 单声道模拟 DAC 输出的 RDAC 模式；可在 dac_cpu.h 文件中可以进行选择，其中：



AD18N 音频输出方式选择

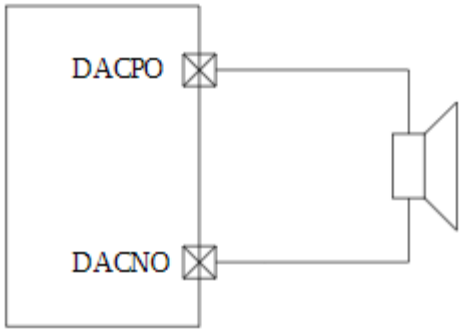
其中 APA 配置如下：

AD18N APA 工作模式	工作模式特点
MODE1	支持单端，单端输出性能好
MODE2	支持差分，差分输出性能好
MODE3	暂不支持

APA 的驱动方式有：

1. **差分直推喇叭（推荐使用 MODE2）**

   该驱动方式对应 MODE1、MODE2 模式，通过 APAP 和 APAN 引脚输出，内置功放。



差分直推喇叭示意图

```
//APA 模式
#define APA_PLL320M          1
#define APA_PLL240M_LOW      2
#define APA_PLL240M_HIGH     3
// #define APA_HSBCLK           3

#define APA_MODE_BTL         (0 << 11)
#define APA_MODE_PBTL        (1 << 11)

#define APA_MODE1_NOR         (0 << 9)
#define APA_MODE2_SNR         (1 << 9)
#define APA_MODE3_THD         (2 << 9)//暂不支持
#define APA_MODE_BITS         (3 << 9)

#define APA_MUTE             (1 << 8)

#define APA_PND              (1 << 7)
#define APA_CPND             (1 << 6)
#define APA_IE_EN            (1 << 5)
#define APA_PWM_EN           (1 << 4)
#define APA_DSM_EN           (1 << 3)

//apa 输出采样率
#define APA_SR48K    (0<<0)
#define APA_SR44K1   (1<<0)
#define APA_SR32K    (2<<0)
#define APA_SRBITS   (7<<0)


#define APA_CLK_DEFAULT      APA_PLL240M_LOW
#define APA_MODE_DEFAULT     APA_MODE2_SNR
#define APA_DEFAULT          (APA_IE_EN | APA_MODE_DEFAULT | APA_MODE_BTL)

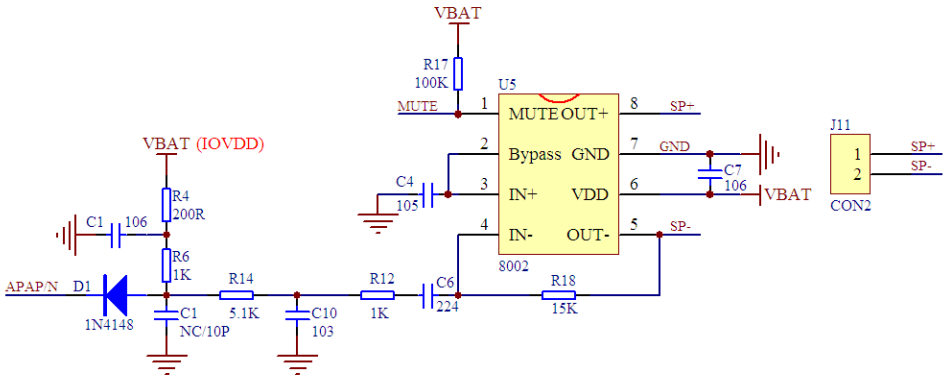
#define apa_sp_handle(n)  do {                                  \
    static signed char ii = 1;                  \
    if (ii > 0) {                               \
        ii = -1;                                \
    } else {                                    \
        ii = 1;                                 \
    }                                           \
    (n) += ii;                                  \
} while(0);
include_lib/cpu/ch58/apa.h
```

APA差分输出配置

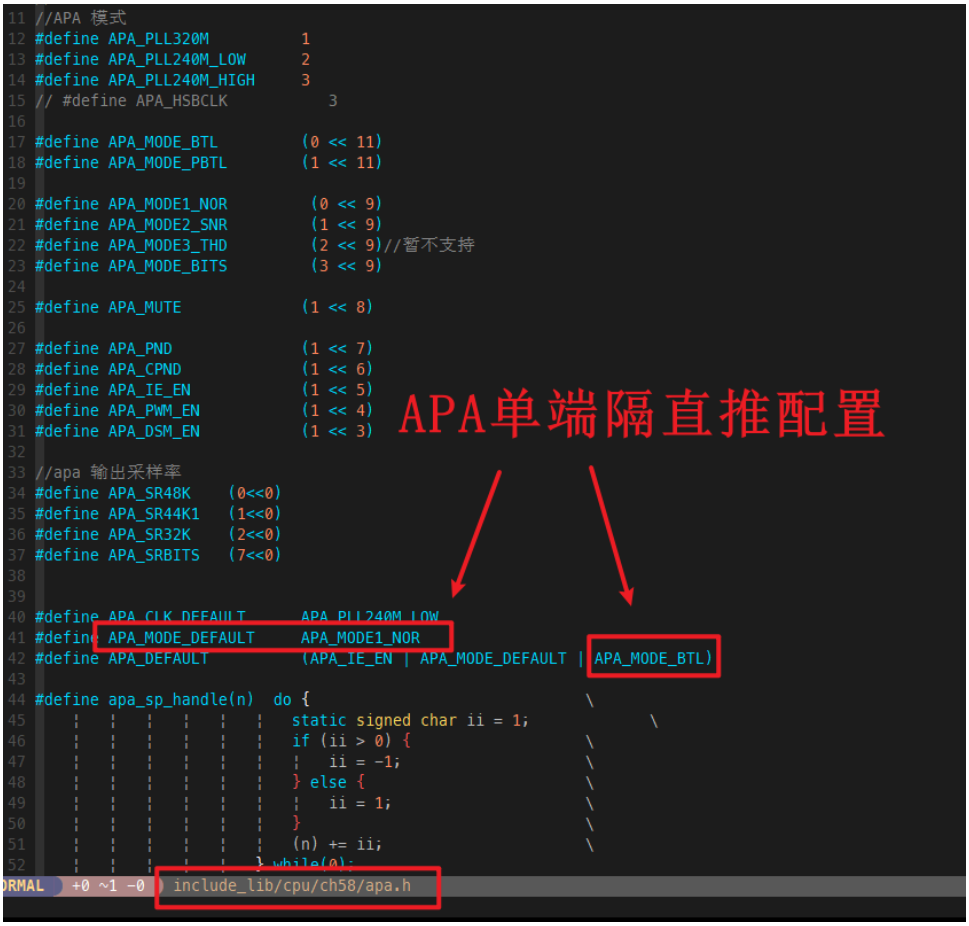
差分直推模式配置

2. **单端隔直推功放（推荐使用 MODE1）**

该驱动方式对应 MODE1 模式，通过 APAP、APAN 引脚输出推功放（不支持立体声）。



单端隔直推功放原理图

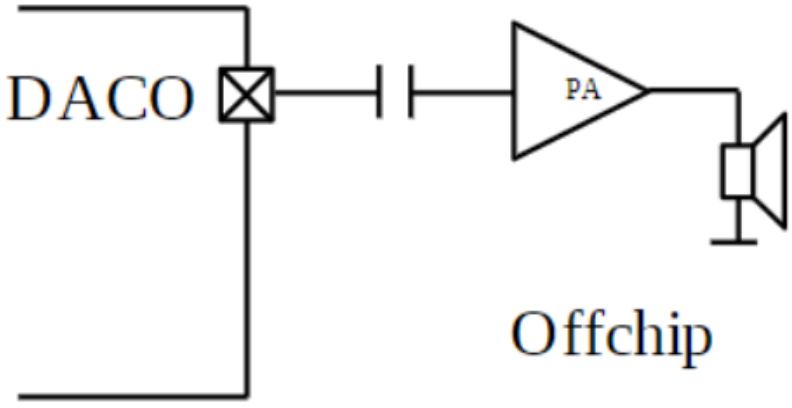


单端隔直推模式配置

RDAC 的驱动方式有：

1. **单端隔直推功放**

   该驱动方式在 AD18N 通过 DAC 脚输出，需要外接功放芯片。注：部分封装没有引出 *DAC 引脚，不可使用 RDAC 功能！*



dac 单端隔直推功放原理图

18.2 AUDIO DAC 接口介绍

18.2.1 函数 void dac_mode_init(void)

此函数实现对 DAC 管理结构体和相关解码参数进行初始化，开机初始化调用一次即可。

18.2.2 函数 void dac_init_api(u32 sr)

此函数实现对 DAC 硬件资源和相关寄存器进行初始化，此处会根据 dac.h 文件中宏定义 DAC_CURR_MODE 的值来选择开启 DAC 工作模式，开机初始化调用一次即可，其中参数：

```
1、sr：采样率，默认选择为32000；
    ①  8000；
    ②  12000；
    ③  16000；
    ④  24000；
    ⑤  32000；
    ⑥  44100；
    ⑦  48000；
```

18.2.3 函数 void dac_power_on(u32 sr, bool delay_flag)

此函数实现所有 DAC 模块的初始化，其中参数：

```
1、sr：采样率，默认选择为32000；
    ①  8000；
    ②  12000；
    ③  16000；
    ④  24000；
    ⑤  32000；
    ⑤  44100；
    ⑤  48000；
2、delay_flag：延迟标志，上电时初始化dac需要延迟至少1.2ms，运行后不需要；
    ①  0，不延迟；
    ②  1，延迟；
```

18.2.4 函数 void dac_off_api(void)

此函数实现关闭所有 DAC 模块。

18.2.5 函数 bool dac_mute(bool mute)

此函数实现对 DAC 的软件 mute，其中参数：

```
1、mute：mute值；
①  1，设置mute状态；
②  0，解除mute状态；
```

18.2.6 函数 u8 dac_vol(char set, u8 vol)

此函数实现设置 DAC 的数字音量，其中参数：

```
1、set：
    ①  '+'，       增大一格数字音量；
    ②  '-'，       减小一个数字音量；
    ③  'r'，       返回当前数字音量；
    ④  0，         传入0，直接将数字音量设置另一参数vol的值；
2、vol：0-31，当set为0时，直接设置为此音量，最大值为31；
3、返回值：当前设置的数字音量值；
```

18.2.7 函数 bool regist_dac_channel(void *psound, void *kick)

此函数实现将 sound_out_obj 注册到 DAC 中，AD140 中一共有 3 路 DAC，其中参数：

```
1、psound：sound_out_obj指向的sound结构体；
2、kick：kick函数；
3、返回值：
    成功 true；
    失败 false；
```

18.2.8 函数 bool unregist_dac_channel(void *psound)

此函数实现将 DAC 注册列表中指定的 sound_out_obj 删除，其中参数：

```
1、      psound：sound_out_obj指向的sound结构体；
2、返回值：
    成功 true；
    失败 false；
```

18.2.9 函数 void fill_dac_fill(u8 *buf, u32 len)

此函数实现将解码后的音频样点数据经过处理后推给 DAC 的缓存 buf 使 DAC 进行转换，其中参数：

```
1、buf：DAC即将要进行转换的解码样点数据缓存buf；
2、len：该buf的大小，默认为1k个byte，存有512个样点；
```

该接口实现将解码后的样点数据（存放于 dac_mge.sound[i]->p_obuf 中）经过处理后输出至 DAC。DAC 支持 16bit 精度，一个样点为 2 个 byte。其具体操作如下：

```
①  DAC转换一次后，会在转换结束后调用该接口将新的数据样点传给DAC；
②  该接口首先会找到正在解码播放的通道，并读取该解码器解码出来的样点buf；
③  rptr[i]是解码读出的一个原始样点数据，可以读取该数组获得数据；
④  sp_buf[sp_cnt]是经过音量处理后的样点数据，可以读取该数组获得数据；
⑤  sp_buf[sp_cnt]中的数据为最终DAC进行转换的样点数据；
⑥  待512个样点数据全部处理完后，启动kick继续进行下一轮DAC转换；
```

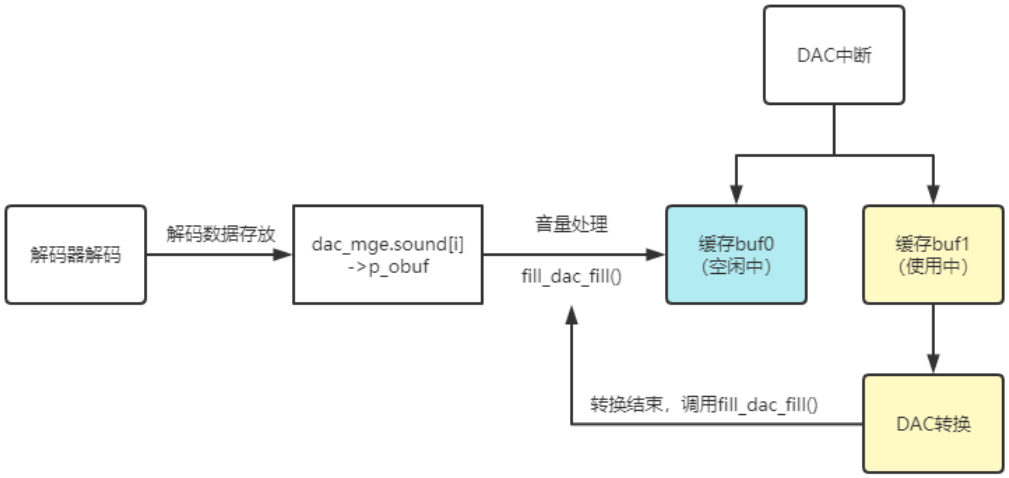


图 4 fill_dac_fill() 调用示意图

18.2.10 函数 void single_apa_startup_mute_cb(void)

此函数在 audio 初始化时由 audio 库回调，使用 APA 引脚 MUTE 住功放防止 po 声。**仅 AD15N/AD17N 单端输出时使用**。

18.2.11 函数 void single_apa_mute(u8 mute)

此函数控制 APA 作功放 MUTE 的 IO 引脚。**仅 AD15N/AD17N 单端输出时使用**，其中参数：

```
1、mute：
    1：mute住功放；
    0：解mute；
```

CHAPTER 19

AUDIO ADC（AD14N / AC104N）

AD14N / AC104N 内置应用于语音场合的 ADC，外接驻极体 MIC，用于录制外界声音。此 ADC 支持 16bit 精度，单声道，支持 8k/12k/16k/24k 四种采样率；同时支持最多 2 路 ADC 编码输入。本章介绍 ADC 相关结构体以及接口。Audio ADC 涉及两个文件，分别是 app/lib/adc_api.c 和 include_lib/audio/audio_adc.h。

Audio ADC 有以下结构体：

```
typedef struct _AUDIO_ADC_MANAGE {
sound_out_obj *sound[ADC_CHANNEL_NUMBER];
void (*kick[ADC_CHANNEL_NUMBER])(void *);    //kick函数
u8 flag;                                     //标志位
} AUDIO_ADC_MANAGE;
```

Audio ADC 主要有以下函数接口：

```
void audio_adc_mode_init(void);
u32 audio_adc_init_api(u32 sr, ADC_MODE mode, u32 ch);
void audio_adc_off_api(void);
bool regist_audio_adc_channel(void *psound, void *kick);
bool unregist_audio_adc_channel(void *psound);
```

19.1 1 AUDIO ADC 接口介绍

19.1.1 1.1 函数 void audio_adc_mode_init(void)

此函数实现对 ADC 管理结构体和相关编码参数进行初始化，开机初始化调用一次即可。

19.1.2 1.2 函数 u32 audio_adc_init_api(u32 sr, ADC_MODE mode, u32 ch)

此函数实现对 ADC 硬件资源和相关寄存器进行初始化，此处会根据 mode 的值来选择 ADC 模式，其中参数：

```
1、sr: 采样率，默认选择为24000;
    ① 8000;
    ② 12000;
    ③ 16000;
    ④ 24000;
2、mode: ADC模式，默认选择为ADC_MIC;
    ① ADC_MIC;
    ② ADC_LINE_IN;
3、ch: 选择的ADC通道;
    该参数仅对LINE_IN模式有效，MIC模式下传入0即可:
4、返回值: 成功 0;
```

19.1.3 1.3 函数 void audio_adc_off_api(void)

此函数实现关闭所有 ADC 模块。

19.1.4 1.4 函数 bool regist_audio_adc_channel(void *psound, void *kick)

此函数实现将 sound_out_obj 注册到 ADC 中，AD140 中一共有 2 路 ADC，其中参数：

```
1、psound: sound_out_obj指向的sound结构体;
2、kick: kick函数;
3、返回值:
    成功 true;
    失败 false;
```

19.1.5 1.5 函数 bool unregist_audio_adc_channel(void *psound)

此函数实现将 ADC 注册列表中指定的 sound_out_obj 移除，其中参数：

```
1、psound: sound_out_obj指向的sound结构体;
2、返回值:
    成功 true;
    失败 false;
```

19.1.6 1.6 函数 void fill_audio_adc_fill(u8 *buf, u32 len)

此函数在 ADC 中断中被调用，用于将 ADC 转换出来放在 buf 里的缓存数据推到指定的空间保存，其中参数：

```
1、buf: ADC转换后样点存放的缓存buf;
2、len: 该buf的大小，默认为64个byte，存有32个样点;
```

该接口实现将 ADC 转换完成的数据样点写到相应的空间（存放于 audio_adc_mge.sound[i]->p_obuf）中，其具体操作如下：

```
① ADC转换一次后，会在中断结束后调用该接口将转换的数据样点取走存放于相应的空间;
② 该接口首先会选择正在编码播放的通道，并找到存放样点数据的buf;
③ 一次性将buf中的数据写入注册了该ADC通道的audio_adc_mge.sound[i]->p_obuf里;
④ 启动kick继续下一轮编码;
```

CHAPTER 20

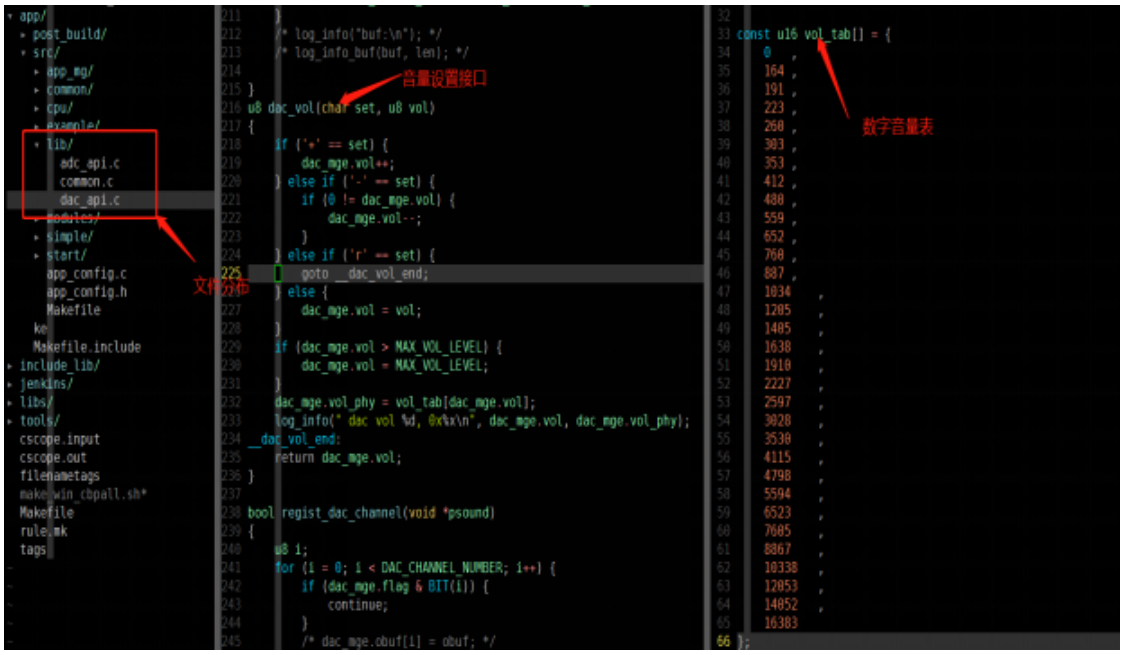
音量

本章节主要介绍音量控制。

20.1 底层音量接口

AD14/15/17/18 提供了数字音量设置接口，可以实现音量加 1，音量减 1，获取音量和设置音量到特定值操作。

音量设置接口的文件分布如下所示：



20.1.1 函数 void dac_mode_init(u16 vol, void *phy_vol_set_func)

此函数实现了音量初始化，其中参数：

```
1、vol：初始化设置的音量值。
2、phy_vol_set_
↪func：设置模拟音量的函数指针，用于AD18的RDAC输出模式配置模拟音量，设置后通过传入的回调函数调节
```

20.1.2 函数 u8 dac_vol(char set,u8 vol)

此函数实现了播放音频的音量设置，其中参数：

```
1、set：'+'为音量加1，'-
↪'为音量减1，'r'为获取音量，'s'为设置音量为vol的值。
2、vol：set参数为's'时候需要设置的音量值。
```

20.2 音量应用介绍

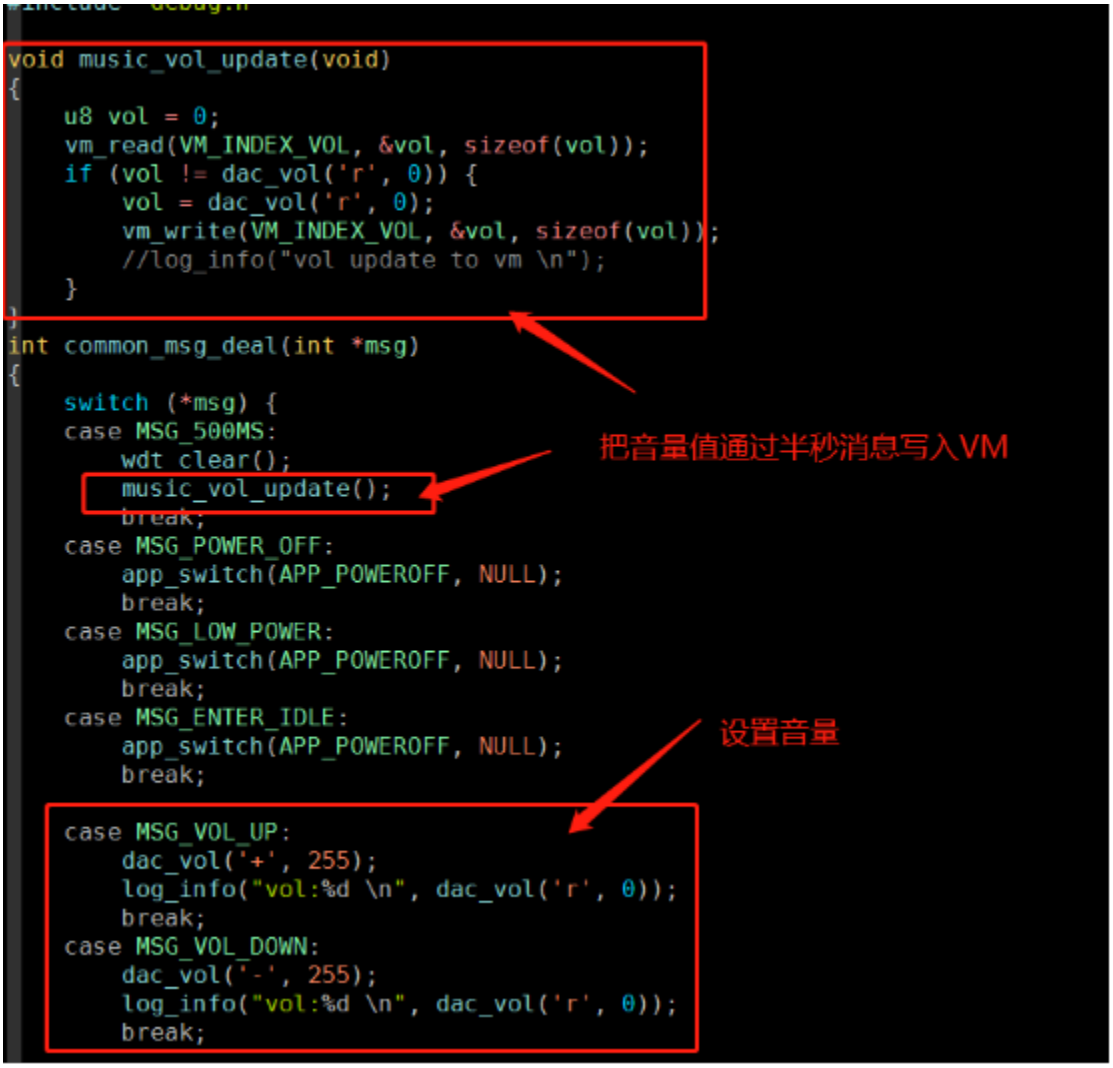
上电时候默认设置音量为固定值 16：

```
175     d_key_voice_init();
176
177     audio_init();
178     dac_mode_init(16, D_PHY_VOL_SET_FUNC);
179     dac_init_api(SR_DEFAULT, 1);
180
181     d_mio_module_init();
```

关机时候默认设置音量为固定值 16：

```
static int app_poweroff_init(void *param)
{
    decoder_init();
    dac_vol(0, 16);
    key_table_sel((void *)poweroff_msg_filter);
```

设置音量值的时候，会把音量写入 vm，可以上电时候读取 vm 值来做音量记忆。音量值写入 vm 动作放在半秒消息做，防止写 vm 频率过高导致播放卡音：




```
void music_vol_update(void)
{
    u8 vol = 0;
    vm_read(VM_INDEX_VOL, &vol, sizeof(vol));
    if (vol != dac_vol('r', 0)) {
        vol = dac_vol('r', 0);
        vm_write(VM_INDEX_VOL, &vol, sizeof(vol));
        //log_info("vol update to vm \n");
    }
}
int common_msg_deal(int *msg)
{
    switch (*msg) {
    case MSG_500MS:
        wdt_clear();
        music_vol_update();
        break;
    case MSG_POWER_OFF:
        app_switch(APP_POWEROFF, NULL);
        break;
    case MSG_LOW_POWER:
        app_switch(APP_POWEROFF, NULL);
        break;
    case MSG_ENTER_IDLE:
        app_switch(APP_POWEROFF, NULL);
        break;

    case MSG_VOL_UP:
        dac_vol('+', 255);
        log_info("vol:%d \n", dac_vol('r', 0));
        break;
    case MSG_VOL_DOWN:
        dac_vol('-', 255);
        log_info("vol:%d \n", dac_vol('r', 0));
        break;
```

CHAPTER 21

混响（echo）

混响（echo）音效一般用于普通扩音，可配置干湿音、回声等效果；一般需要与啸叫抑制算法一起使用。

21.1 1 配置参数说明

- **DOWN_S_FLAG：下采参数**
  1 代表下采，0 代表不下采；下采申请的 buff 会较小，但是回声部分高频会丢失；
- **echo_work_buf_len：echo 运算需要的 buff 大小**
  根据是否下采以及输入音频采样率决定，可通过 ops->need_buf() 接口获取大小；

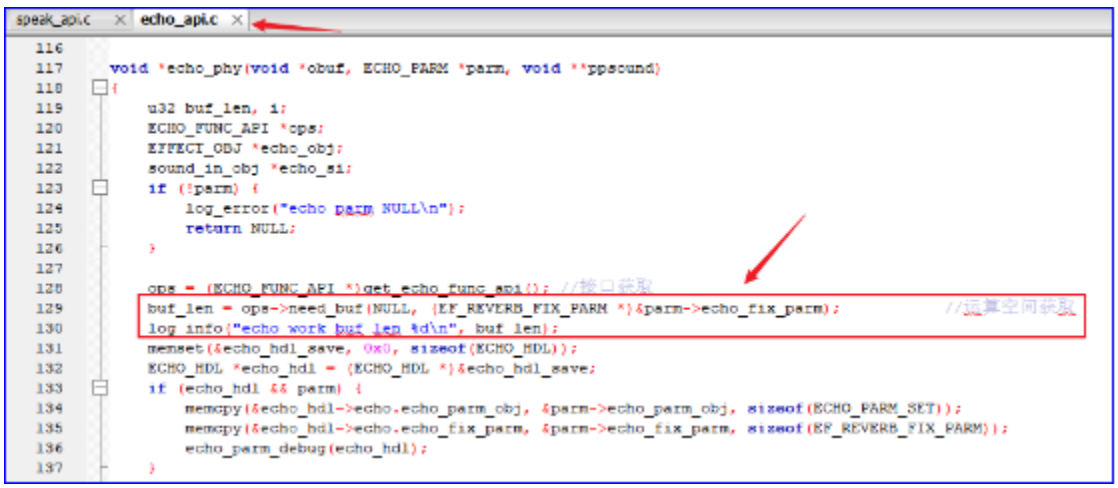


图 1 echo buff 大小获取

21.2 2 相关函数

21.2.1 2.1 函数 void *link_echo_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr)

该函数为串联音频链路的标准接口，实现响应算法的启动，并添加到音频链路上，其中参数：

```
1、p_sound_out：当前算法的前级音频通道句柄；
2、② p_dac_cbuf：输出的音频buff；
3、pp_effect：存放当前音效算法控制句柄指针的指针；
4、sr：当前输入音频的采样率；
5、返回值：启动之后最新的音频通道句柄；
```

21.2.2 2.2 函数 void *echo_api(void *obuf, u32 sr, void **ppsound)

该函数用于初始化混响（echo）的相关参数，由函数 link_echo_sound() 调用即可，需要关注的其内部参数的初始化：

```
1、decayval：回升衰减比，范围：0~70；
2、delay：回声延时，范围：0~max_ms，超过max_ms会当场max_ms；
3、energy_vad_threshold：mute阈值，默认值为512；低于该能量值的声音输出静音；␣
↪扩音方案中可以用该方式降低底噪；
4、direct_sound_enable：干声叠加使能位；
5、max_ms：最大回声延时时间，最大值为100；
6、sr：输入音频的采样率，支持16K / 24K，会影响混响算法运算buff的大小；
7、wetgain：湿声增益，默认为3000；
8、drygain：干声增益，默认为4096，干声叠加使能为0时不起作用；
```

CHAPTER 22

移频啸叫抑制（pitchshifter_howling）

如果在一个环路系统的 AD->···->DA 中存在某个频点明显的响应峰值，该位置容易形成啸叫。因此，AD->···-> 移频->···->DA, 在一个 AD 到 DA 的环路中，移频模块将频率不断移动，加入非线性运算，来改变整个环路的频响，防止同一个频率被反复放大。
移频防啸叫模块，可以配合 EQ 模块，自适应陷波模块一起处理。具体根据实际样机的资源效果做选择。

22.1 1 配置参数说明

- **FRESHIFT_SPEED_MODE_QUALITY：滤波器音阶数**
  滤波器音阶数配置范围 [2, 8]，音阶数越大，运算量越大；
- **howling_work_buf：移频啸叫抑制运算需要的 buff**
  根据移频方式和滤波器音阶数决定 buff 大小，可通过 ops->need_buf() 接口获取大小；

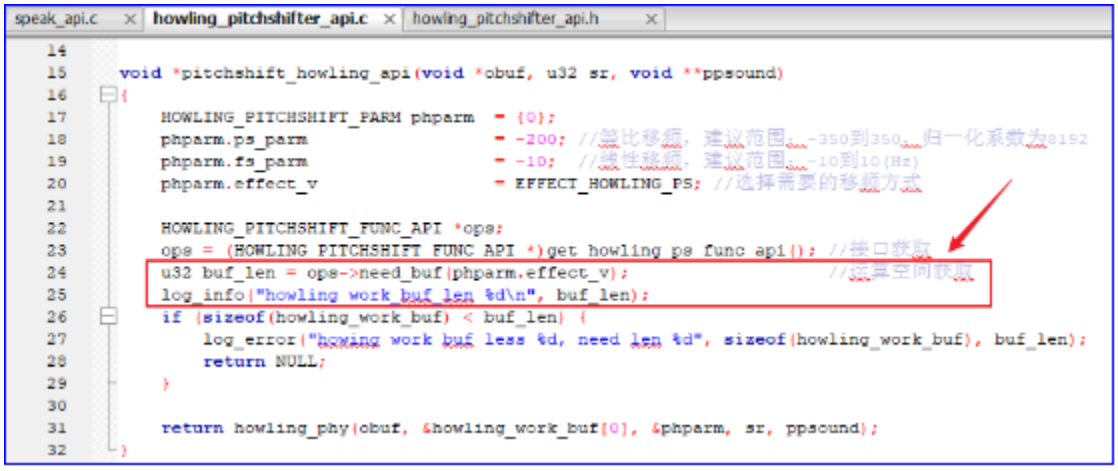


图 1 pitch_shifting buff 大小获取

22.2 2 相关函数

22.2.1 2.1 函数 void *link_pitchshift_howling_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr)

该函数为串联音频链路的标准接口，实现响应算法的启动，并添加到音频链路上，其中参数：

```
1、p_sound_out：当前算法的前级音频通道句柄；
2、②  p_dac_cbuf：输出的音频buff；
3、pp_effect：存放当前音效算法控制句柄指针的指针；
4、sr：当前输入音频的采样率；
5、返回值：启动之后最新的音频通道句柄；
```

22.2.2 2.2 函数 void *pitchshift_howling_api(void *obuf, u32 sr, void **ppsound)

该函数用于初始化移频抑制啸叫（pitchshifter_howling）的相关参数，由函数 link_pitchshift_howling_sound() 调用即可，需要关注的其内部参数的初始化：

```
1、effect_v：移频方式，可选EFFECT_HOWLING_PS或EFFECT_HOWLING_FS；
2、ps_parm：等比移频，建议范围[-350, 350]，归一化系数为8192，effect_
↪v置上EFFECT_HOWLING_PS有效；
3、fs_parm：线性移频，建议范围[-10, 10]（Hz），effect_v置上EFFECT_HOWLING_
↪FS有效；
```

CHAPTER 23

自适应陷波啸叫抑制（notch_howling）

使用扩音时，当麦克风到喇叭整体环路反馈大于 1 的时候，就会发生啸叫。只要改变整个环路的反馈，就可以抑制啸叫。自适应陷波抑制啸叫是其中一种算法，可以抑制改变环路反馈声音的频率，环路反馈一直改变，起到了平衡磨损的作用，防止某个频率一直放大。

23.1 1 配置参数说明

- **notch_howling_work_buf：自适应陷波啸叫抑制运算需要的 buff**
  根据配置参数决定 buff 大小，可通过 ops->need_buf() 接口获取大小；

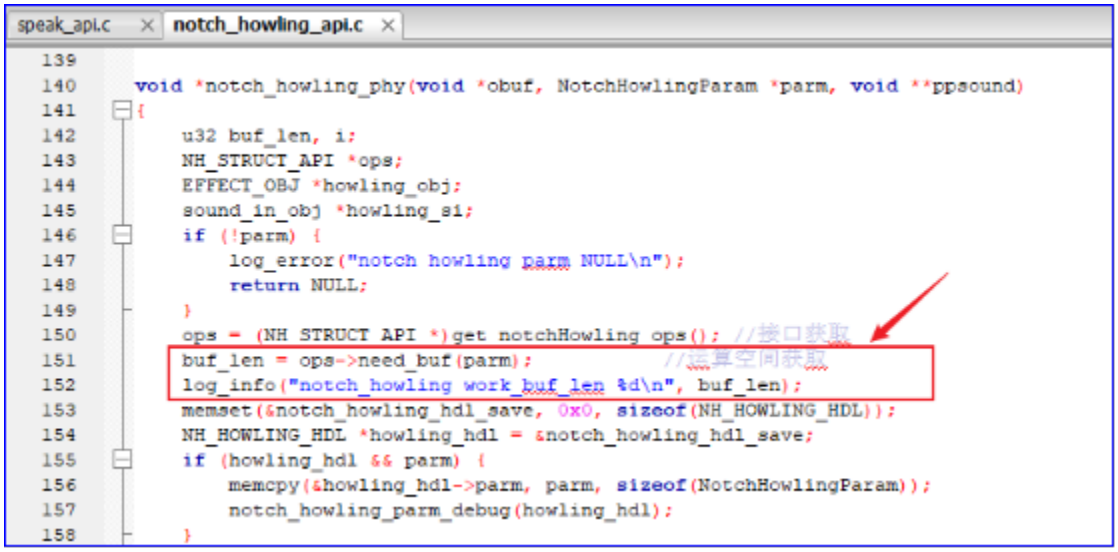


图 1 notch_howling buff 大小获取

23.2 2 相关函数

23.2.1 2.1 函数 void *link_notch_howling_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr)

该函数为串联音频链路的标准接口，实现响应算法的启动，并添加到音频链路上，其中参数：

```
1、p_sound_out：当前算法的前级音频通道句柄；
2、② p_dac_cbuf：输出的音频buff；
3、pp_effect：存放当前音效算法控制句柄指针的指针；
4、sr：当前输入音频的采样率；
5、返回值：启动之后最新的音频通道句柄；
```

23.2.2 2.2 函数 void *notch_howling_api(void *obuf, u32 sr, void **ppsound)

该函数用于初始化自适应陷波抑制啸叫（notch_howling）的相关参数，由函数 link_notch_howling_sound() 调用即可，需要关注的其内部参数的初始化：

```
1、gain：陷波器压制程度，该值越大防啸叫效果越好，但发声啸叫频点误检时音质会更差；
2、Q：陷波器带宽，该值越小防啸叫效果越好，但发声啸叫频点误检时音质会更差；
3、fade_
↪time：陷波器启动与释放时间，该值越小启动与释放越快，但可能导致杂音出现和音质变差；
4、threshold：频点啸叫判定阈值，该值越小防啸叫效果越好，但越容易误检；
5、SampleRate：输入音频的采样率；
```

CHAPTER 24

均衡器（PCM_EQ）

AD14N / AC104N 可以使用杰理 EQ 工具进行离线 EQ 调试，将需要的频段增益抬高或者压低，使音质感受更佳。

24.1 1 配置参数说明

- **eq_filt_32000：PCM_EQ 处理 32K 采样率音频的运算数据**
  该数据由 EQ 工具生成，最多可以有 10 段 EQ 数据（seg0~seg9），详情请见 EQ 工具使用说明章节，不同采样率拥有各自的 eq_filt 表；
- **g_work_buf：PCM_EQ 运算需要的 buff**
  根据 EQ 使用的频点数（段数）决定 buff 大小，可通过 pcm_eq_buf_len() 接口获取大小；

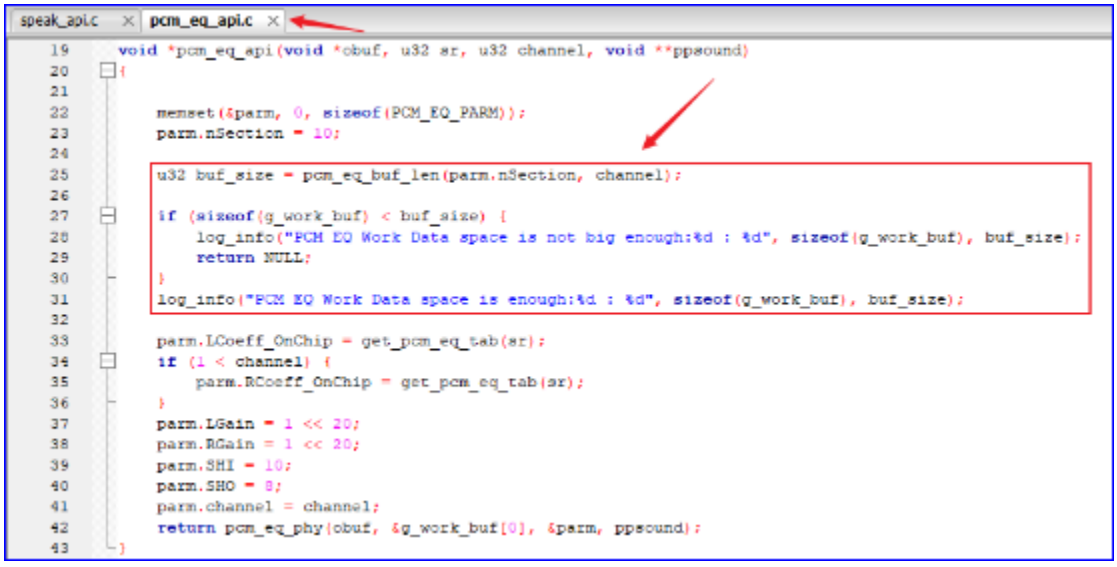


图 1 pcm_eq buff 大小获取

24.2 2 相关函数

24.2.1 2.1 函数 void *link_pcm_eq_sound(void *p_sound_out, void *p_dac_cbuf, void **pp_effect, u32 sr, u32 channel)

该函数为串联音频链路的标准接口，实现响应算法的启动，并添加到音频链路上，其中参数：

```
1、p_sound_out：当前算法的前级音频通道句柄；
2、②  p_dac_cbuf：输出的音频buff；
3、pp_effect：存放当前音效算法控制句柄指针的指针；
4、sr：当前输入音频的采样率；
5、channel：声道数，ad14n和ac104n只有单声道，传1即可；
6、返回值：启动之后最新的音频通道句柄；
```

24.2.2 2.2 函数 void *pcm_eq_api(void *obuf, u32 sr, u32 channel, void **ppsound)

该函数用于初始化 PCM_EQ 的相关参数，由函数 link_pcm_eq_sound() 调用即可，需要关注的其内部参数的初始化：

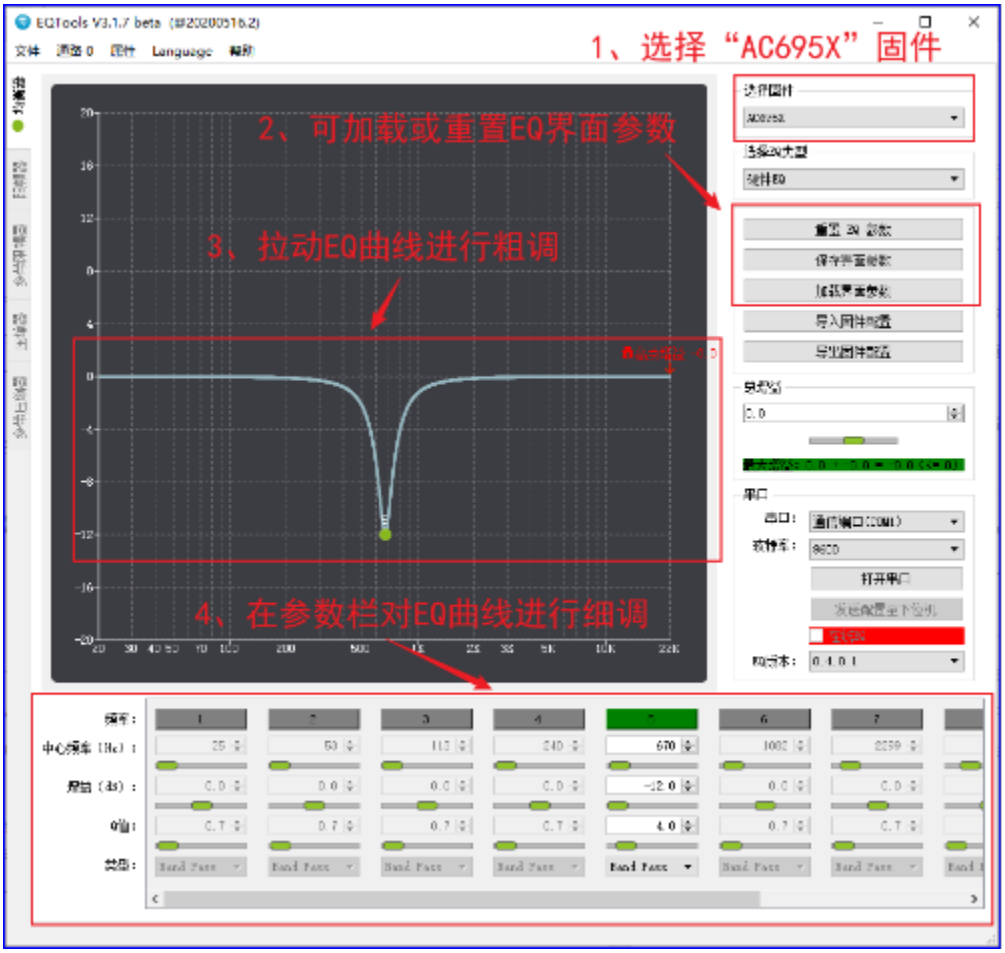
```
1、nSection：EQ频点数（段数），不可超过各采样率的eq_filt表中的段数，范围[1,␣
↪10]；
2、channal：陷波器带宽，该值越小防啸叫效果越好，但发声啸叫频点误检时音质会更差；
```

24.3 3 EQ 工具使用

AD14N / AC104N 可以使用杰理 EQ 工具进行离线 EQ 调试，调试完成后可以导出当前的 EQ 配置，并加载到程序中验证效果。

24.3.1 3.1 EQ 工具使用介绍

1、打开“EQ-3.1.7-beta-20200516.2.exe”工具并选择固件“AC695X”；2、选择需要的频点数，对于不需要的频点可以点击下方频率行对应跑按钮使其变灰；3、拉动 EQ 曲线进行粗调；4、粗调结束后，可在下方参数框对 EQ 曲线进行细调，可设置准确的中心频率、增益、Q 值以及滤波器类型；

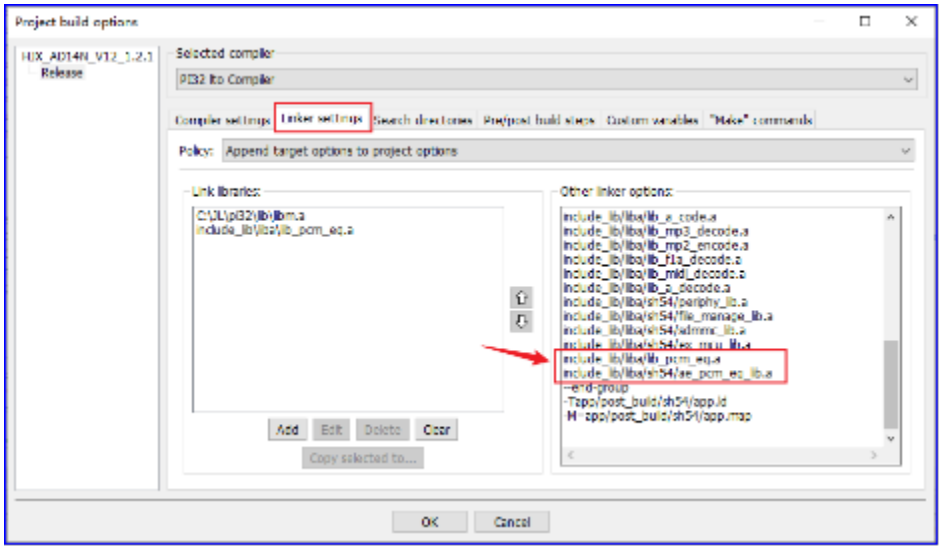


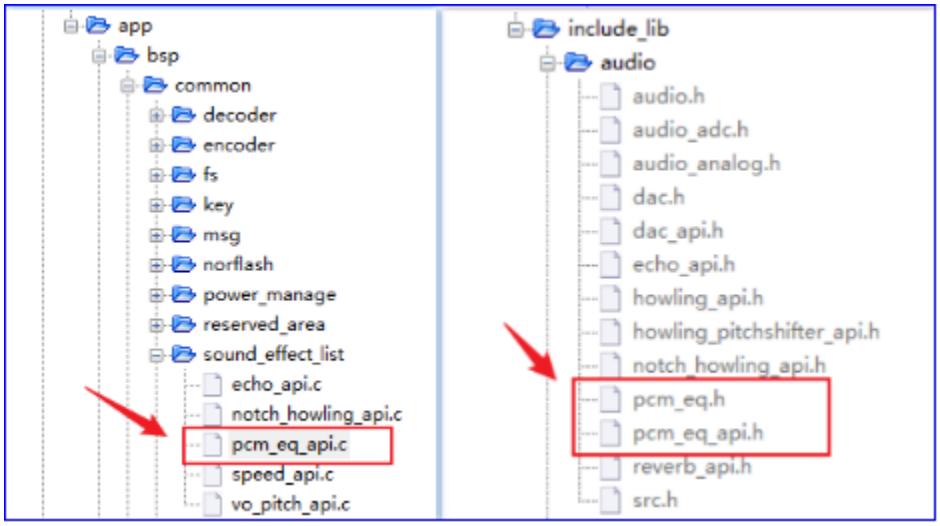
5、EQ 调节结束后，点击右边“保存界面参数”按钮对编辑的 EQ 界面进行保存；6、保存后，再点击“导出固件配置”按钮，工具根目录会生成新的“hw_eq_table.h”文件；

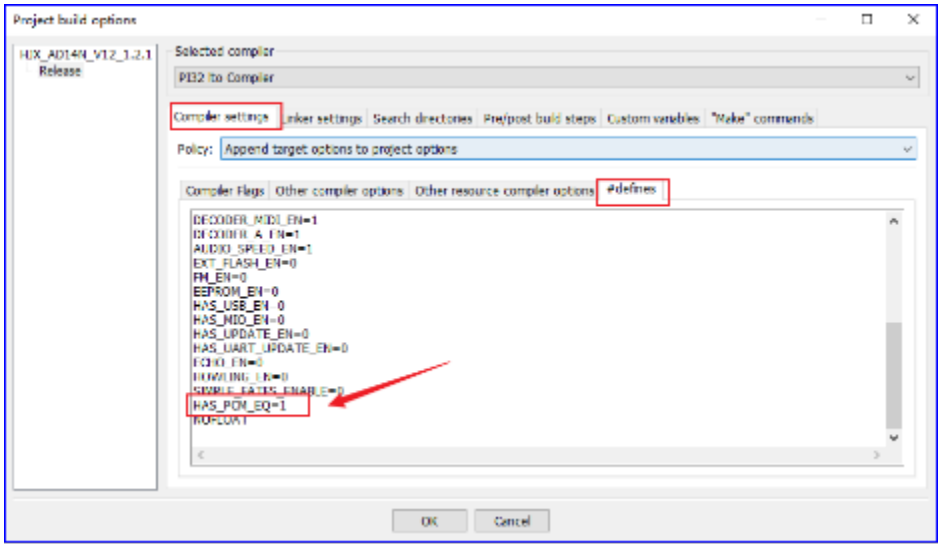


24.3.2 3.2 SDK 配置说明

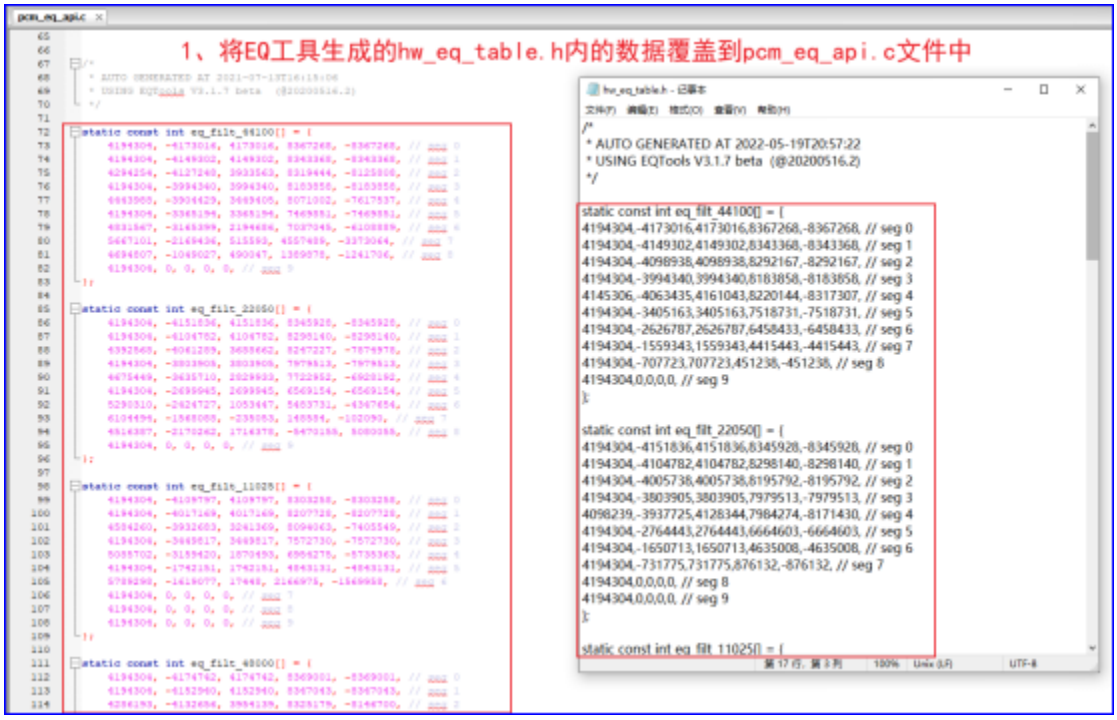
1、SDK 工程中导入 pcm_eq 相关文件；



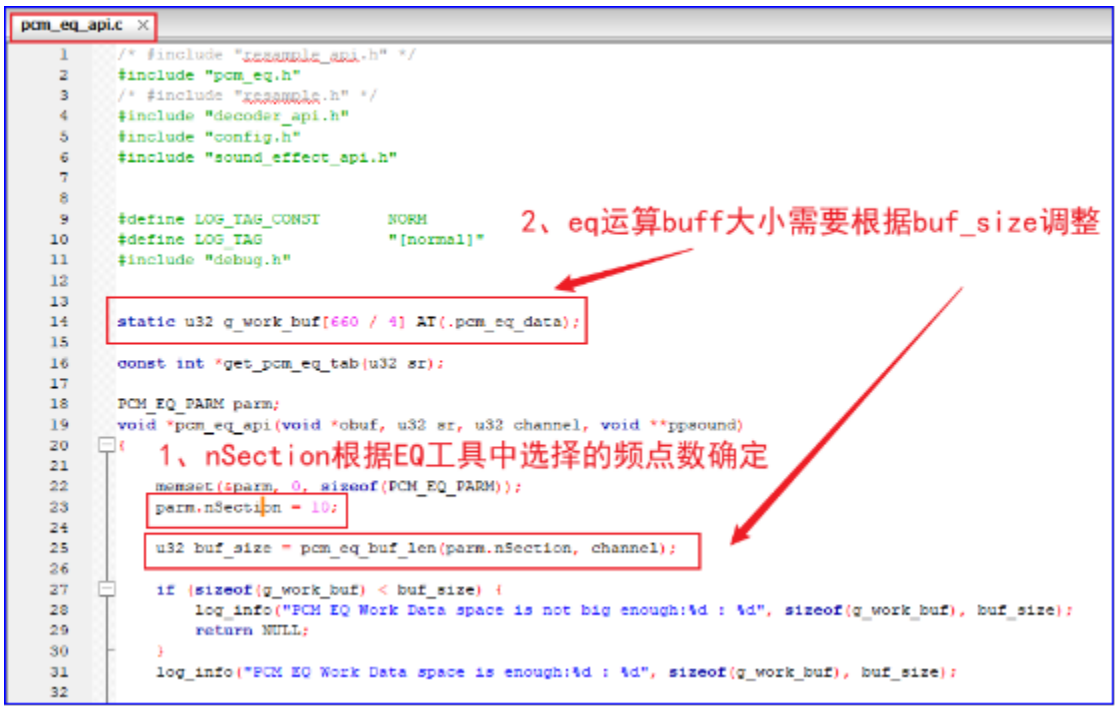




2、打开 pcm_eq_api.c 文件，并将 EQ 工具生成的 hw_eq_table.h 文件中的 eq 数据覆盖原本的 eq 数据；



3、打开 pcm_eq_api.c 文件，根据 EQ 工具选择的频点数调整 nSection 值，并根据 buf_size 打印调整 g_work_buf[] 大小；



4、打开 decoder_api.c 文件，将 EQ 串入音频解码链路中，运行测试效果；需要注意 eq 所用的 ram 是否与其他资源复用了，建议不要同时开启多种音效；



CHAPTER 25

变音音效（vo_pitch）

AD14N / AC104N SDK 中自带变音音效，可自行调整参数做到不同的效果。

25.1 可选音效

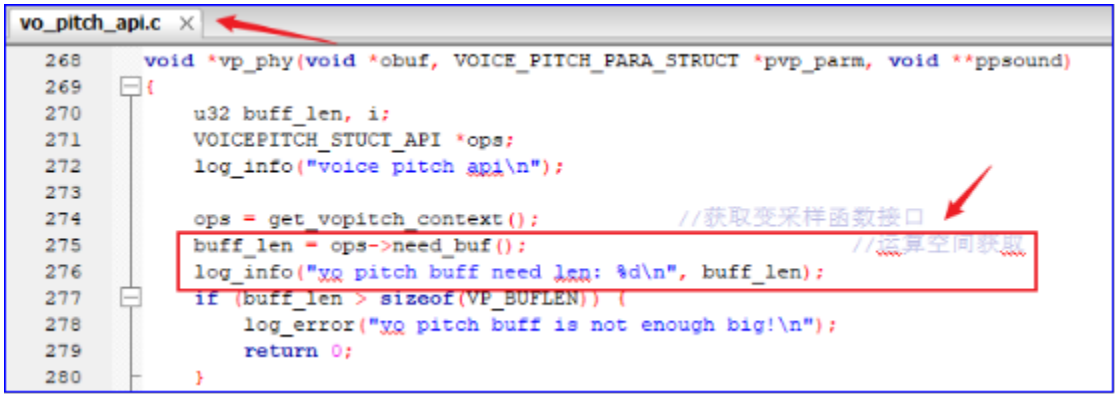
- VP_CMD_ROBOT：平调机器音模式
- VP_CMD_ROBOT2：变调机器音模式
- VP_CMD_PITCHSHIFT：变调模式
- VP_CMD_RAP：RAP 模式, 将 ADC 输入的声音按照 midi 文件的节奏进行播放
- VP_CMD_PITCHSHIFT2：变声模式 2
- VP_CMD_RAP_REALTIME：你说我唱模式，将 ADC 输入的声音按照 midi 文件的节奏进行实时播放
- VP_CMD_CARTOON：卡通模式

各音效的参数配置详见 SDK；

25.2 配置参数说明

- VC_ENABLE_FLAG：变声模块使能，该变量为 1 时，使能变声模块；
- EXTRA_DATA_SIZE：RAP 模式可以包含的音源长度，单位是 short；
- VO_RAP_LOOPEN：RAP 模式是否 repeat 音源
- VO_RAP_COMPRESS_RATE：RAP 压缩比，该值越大，RAP 音源时间压缩的越多；
- VP_DECAY_VAL：ECHO 模式的 decay 速度
- VP_HIS_LEN：ECHO 模式的 delay: 复用其他模式的 Buf
- VP_BUFLEN：变音音效运算需要的 buff

根据所选模式以及参数决定 buff 大小，可通过 ops->need_buf() 接口获取大小；



25.3 相关结构体及函数

25.3.1 结构体 VOICE_PITCH_PARA_STRUCT

该结构体用于配置各类变音音效，该结构体中部分成员的值只对特定的变音效果有效，其中成员：：

```
1、do_flag：选择的音效，参考VP_CMD枚举；
2、samplerate；输入音频的采样率，支持16K或24K；
3、noise_
↪dc：能量阈值，默认为2048，一些持续存在的噪声经过变声有可能听起来会更明显，所以该阈值是用来过滤
4、pitchrate：音调高低，默认值128代表原始音调，小于128音调升高，大于128音调降低。建议范围[40,
↪ 256]；
5、midi_file：RAP模式的输入音源，仅RAP模式有效；
6、midifile_len：RAP模式的输入音源大小，仅RAP模式有效；
7、callback：RAP模式的状态回调；
8、priv：私有参数，一般传NULL；
```

25.3.2 函数 void *link_voice_pitch_sound(sound_out_obj *p_curr_sound, void *p_dac_cbuf, void **pp_effect, u32 cmd)

该函数为串联音频链路的标准接口，实现响应算法的启动，并添加到音频链路上，其中参数：

```
1、p_sound_out：当前算法的前级音频通道句柄；
2、p_dac_cbuf：输出的音频buff；
3、pp_effect：存放当前音效算法控制句柄指针的指针；
4、cmd：选择的变音类型，参考VP_CMD枚举；
5、返回值：启动之后最新的音频通道句柄；
```

25.3.3 函数 void *voice_pitch_api(void *obuf, u32 cmd, void **ppsound)

该函数用于初始化变音音效相关参数，由函数 link_voice_pitch_sound() 调用即可；该函数内部会调用函数 vp_cmd_case()，用于配置所选的变音音效以及其参数。

25.3.4 函数 void rap_callback(void *priv, int pos)

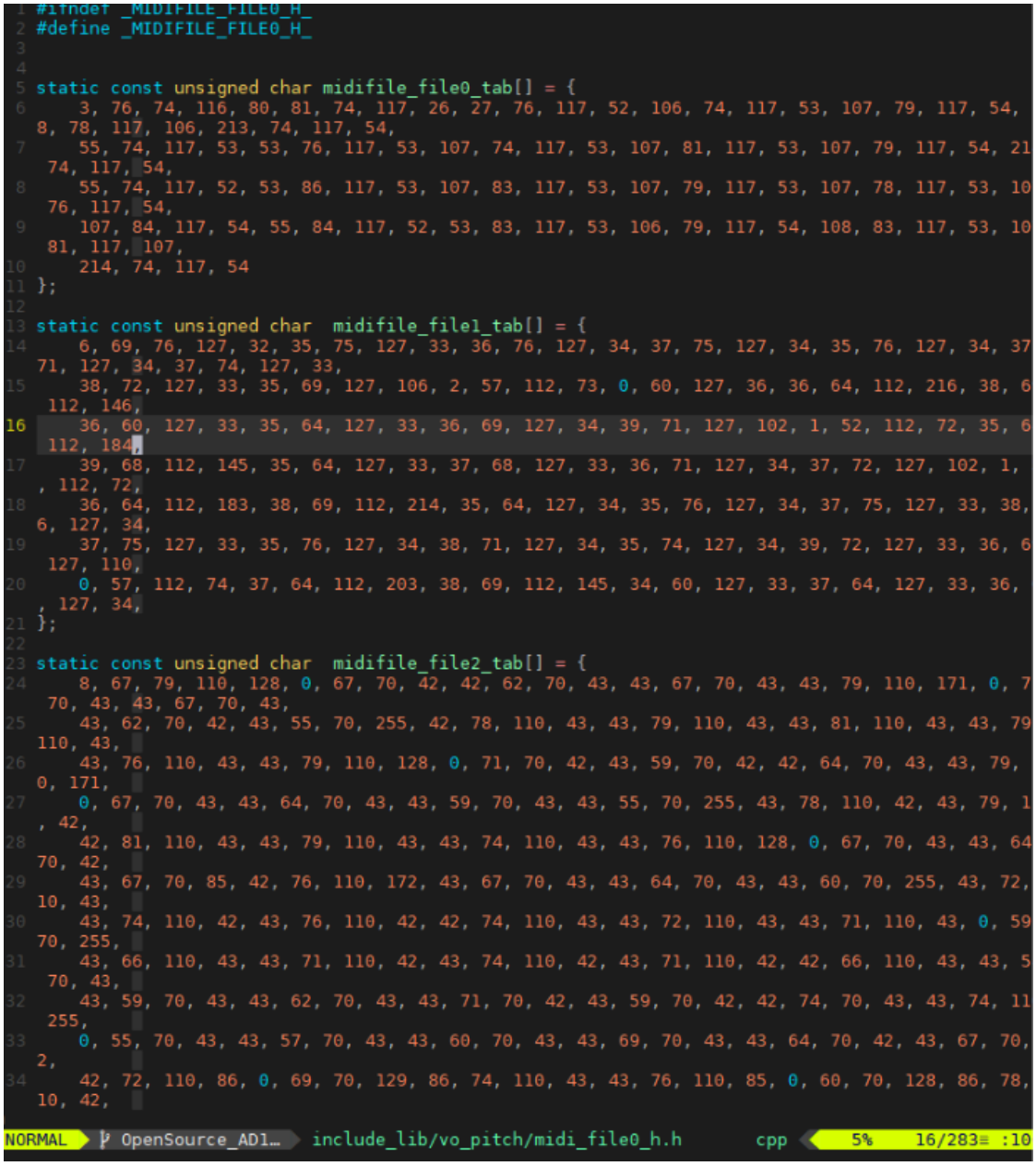
该函数为 RAP 模式的回调函数，由解码器调用。其中的参数 pos 表示 RAP 模式的状态；RAP 模式初始化后，pos 会处于 RAP_PREPARE 状态等待输入音源；当输入音源大于所设置的 noise_dc 能量阈值后，会触发 RAP 模式，此时 pos 在会 PAR_START 状态；当 RAP 播放结束后，pos 会在 RAP_END 状态循环。若需要重置 RAP 状态，需调用函数 rap_reopen()。

25.3.5 函数 void rap_reopen(void)

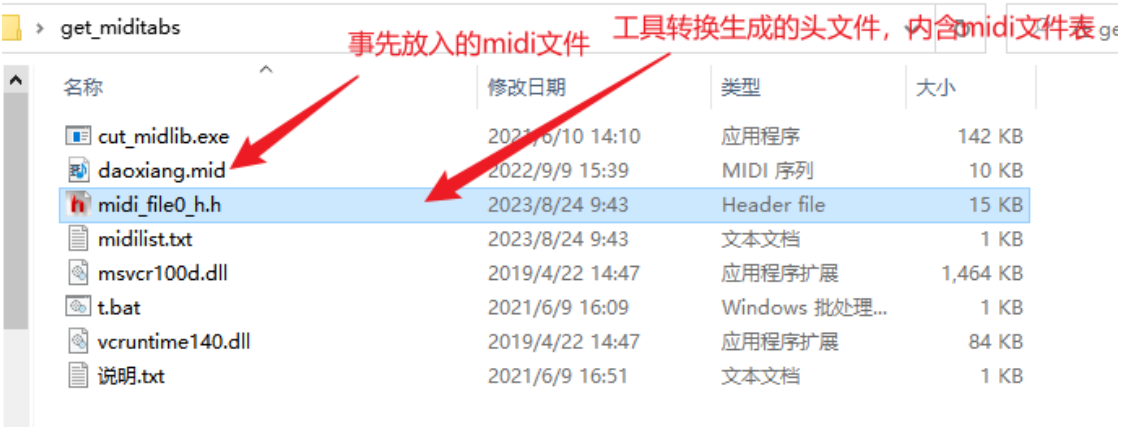
该函数用于 RAP 模式播放结束后，让 RAP 重回 RAP_PREPARE 状态。

25.4 RAP 和 RAP_REALTIME 模式文件替换说明

用户可以通过替换该音效依赖的 midi 文件表来进行播出不同的音调，midi 文件表位于头文件 midi_file0_h.h。



用户可以通过 get_miditabs 工具生成 midi 文件表，首先将需要使用的 midi 文件表放入工具目录，然后点击 t.bat 生成头文件



打开头文件可以看到生成的 midi 文件表

```
midi_file0_h.h
#ifndef _MIDIFILE_FILE0_H_
#define _MIDIFILE_FILE0_H_
static const unsigned char  midifile_file3_tab[]={
  8,61,45,79,38,42,52,64,39,43,64,69,77,0,59,72,19,22,61,64,19,21,57,65,39,43,40,71,38,
    43,52,61,38,43,64,69,77,0,56,67,77,85,42,76,39,43,52,62,39,43,64,70,77,
    0,59,73,39,43,57,62,39,43,49,71,38,43,59,62,38,43,64,71,77,0,61,66,77,
    85,47,72,39,43,54,61,39,43,64,67,77,0,57,70,77,86,40,74,38,43,52,62,38,
    43,64,69,77,0,57,71,77,85,45,71,39,43,52,58,39,43,64,69,154,0,59,71,39,
    43,57,58,19,21,59,59,39,43,57,67,20,22,52,63,38,43,59,69,38,42,64,68,43,
    0,52,63,39,43,45,81,39,1,69,76,42,42,52,62,39,0,73,80,43,43,64,60,20,
    0,76,87,129,0,59,60,19,22,61,63,19,21,57,66,39,43,40,74,38,43,52,60,38,
    0,71,81,43,43,64,59,38,0,68,74,43,0,56,59,38,43,52,60,38,0,64,71,42,
    42,42,73,39,1,69,70,42,42,49,60,39,0,73,79,43,43,59,58,39,0,76,86,129,
    0,64,58,39,43,57,62,39,43,49,70,38,43,56,62,38,0,71,83,43,43,61,58,38,
    0,68,76,43,0,64,58,38,43,59,61,38,0,64,72,42,42,47,71,39,1,69,78,115,
    42,54,66,39,43,62,59,39,0,57,59,39,43,64,67,43,0,54,65,39,43,40,70,38,
    0,69,75,116,43,52,63,38,43,64,59,38,0,57,59,38,43,64,58,42,0,52,61,38,
    42,45,70,39,1,61,64,231,10,64,64,114,11,69,64,255,21,52,63,39,43,64,60,39,
    0,59,60,39,43,57,66,19,21,59,64,39,43,57,68,20,22,52,63,38,43,59,64,38,
    21,69,81,22,22,57,58,38,0,69,78,21,21,66,73,21,21,45,79,39,1,69,77,42,
    42,52,61,39,0,66,69,22,22,69,71,43,21,57,66,39,22,66,72,21,21,52,61,39,
    0,69,76,21,22,66,72,21,21,44,79,39,0,71,75,21,22,71,74,21,21,52,58,38,
    0,71,70,21,21,71,66,22,22,56,66,38,0,73,74,43,43,52,58,38,0,69,73,38,
    42,42,76,39,1,69,75,42,42,49,61,39,0,66,69,22,22,69,66,43,21,54,67,39,
    22,66,67,21,21,49,59,39,0,69,71,21,22,66,73,21,21,40,74,39,0,71,73,21,
    22,71,71,21,21,49,58,38,0,71,69,21,21,71,67,22,22,52,65,38,0,73,72,43,
    43,49,58,38,0,69,72,38,42,47,74,39,1,69,73,42,42,54,56,39,0,66,71,22
```

最后将生成的文件表复制到 SDK 的头文件 midi_file0_h.h，然后在函数初始化设置配置文件表，图中以 rap_realtime 模式为例

```
void vp_cmd_rap_realtime(VOICE_PITCH_PARA_STRUCT *p_vc_parm)
{
    p_vc_parm->do_flag = HARMO_RAP_REALTIME ;
    p_vc_parm->noise_dc = 2048;
    p_vc_parm->samplerate = read_audio_adc_sr();
    p_vc_parm->hamorrate = 128; //改变音色，50到250
    p_vc_parm->pitchrate = 128; //改变音高，50到250
    p_vc_parm->midi_file = midifile_file3_tab; //改变旋律
    p_vc_parm->midifile_len = sizeof(midifile_file3_tab);
    //无效参数:
    p_vc_parm->callback = rap_callback;
    p_vc_parm->priv = NULL;

}
```

填入使用的文件表

CHAPTER 26

混合变音音效（voicechanger）

AD14N / AC104N SDK 中新增 voicechanger 变音音效，可自行调整参数做到不同的效果，发出不同的人声或者鸟叫声。

26.1 可选音效

- EFFECT_VC_AV_PITCHSHIFT：人声变音 1
- EFFECT_VC_F0_TD：人声变音 2
- EFFECT_VC_AV_F0_TD_AUTOFIT：人声变音 3
- EFFECT_VC_AV_PITCHSHIFT_AUTOFIT：人声变音 4
- EFFECT_VC_AV_BIRD：鸟叫声
- EFFECT_VC_AV_BIRD1：鸟叫声 1
- EFFECT_VC_AV_BIRD2：鸟叫声 2
- EFFECT_VC_AV_BIRD3：鸟叫声 3
- EFFECT_VC_AV_BIRD4：鸟叫声 4
- EFFECT_VC_AV_BIRD5：鸟叫声 5
- EFFECT_VC_AV_BIRD6：鸟叫声 6

其中 EFFECT_VC_AV_PITCHSHIFT 跟 EFFECT_VC_AV_F0_TD 是 2 种变声算法

EFFECT_VC_AV_PITCHSHIFT 中 formant_shift 参数无效；EFFECT_VC_AV_F0_TD 中可以通过组合不同的 shiftv 跟 formant_shift 调出不同的效果。

例如变女声可以设置成 formant_shift=80;shiftv=60;

EFFECT_VC_AV_F0_TD_AUTOFIT 跟 EFFECT_VC_AV_PITCHSHIFT_AUTOFIT 是会检测声音。

例如如果是童音即使 pitch 设高，也不要调高那么多。

EFFECT_VC_AV_BIRD 到 EFFECT_VC_AV_BIRD6 是把人声转换成鸟声，例如人声唱一段旋律，

然后转换出来对应的鸟叫声旋律。其中 pitch 会改变鸟声的整体音高，

鸟叫建议用 EFFECT_VC_AV_BIRD5 或 EFFECT_VC_AV_BIRD6

26.2 配置参数说明

- const int VC_NG_THRES：能量阈值，低于这个能量的声音会被 mute 掉，建议默认设成 0，根据实际底噪水平设置。

voicechanger 变音音效的参数由两个结构体管理

结构体 VOICECHANGER_AV_PARM

- shiftv 调小，音调变高，变声模式建议范围 50 到 200，鸟叫声模式建议范围 30 到 100
- formant 在 EFFECT_VC_AV_F0_TD,EFFECT_VC_AV_F0_TD_AUTOFIT 模式下，调小音色变亮，调小音色变沉。范围 50 到 200
- speedv 变速参数，范围 50 到 200. 如果是实时变声，ad 直接变声到 da 的，因为进出数据量需要一致。这时候 speedv 必须设成 100，不然会有杂音。

下面参数是 EFFECT_VC_AV_BIRD5 跟 EFFECT_VC_AV_BIRD6 模式下是调节颤音用的

结构体 VOICESYN_AV_PARM

- vibrate_rate_u 范围 0-100，非 0 代表开启颤音，0 代表关闭
- vibrate_rate_d 范围 0-100，影响颤音最大幅度，越大颤音幅度越大（BIRD6 无效）
- vibrate_lenCtrol 范围 0-100，颤音衰减速度，影响颤音持续时长，100 代表最短，0 代表最长
- randpercent 有效范围 50-100【或者 0 到 50，镜像对称】，微调音色的（BIRD6 无效）

26.3 相关函数接口

26.3.1 函数 void *link_voice_changer_sound(void *p_sound_out, void p_dac_cbuf, void **pp_effect, u32 in_sr)

该函数为串联音频链路的标准接口，实现响应算法的启动，并添加到音频链路上，其中参数：

```
1、p_sound_out：当前算法的前级音频通道句柄；
2、p_dac_cbuf：输出的音频buff；
3、pp_effect：存放当前音效算法控制句柄指针的指针；
4、in_sr：采样率；
5、返回值：启动之后最新的音频通道句柄；
```

26.3.2 函数 void *voice_changer_api(void *obuf, u32 sr, void **ppsound)

该函数用于初始化变音音效相关参数，由函数 link_voice_changer_sound() 调用即可；

26.3.3 函 数 void update_voice_changer_parm(VOICECHANGER_AV_PARM *new_vc_parm, VOICESYN_AV_PARM *new_vsyn_ctrol)

该函数用于运行 voicechanger 变音音效时，动态调整参数，使用时注意采样率要和之前一致，其中参数：

```
1、new_vc_parm：修改配置后新的参数结构体1
2、new_vsyn_ctrol：修改配置后新的参数结构体2
```

CHAPTER 27

变速变调（speed）

speed 算法在 SDK 中一般用于解码变速变调，只要在解码前置上 BIT_SPEED 位就可使用（可参考录音播放流程），该算法实时性较好，也可用于扩音。

27.1 配置参数说明

该算法的参数主要由 SPEED_PITCH_PARA_STRUCT 结构体控制，其说明如下:

```
* Insample: 输入音频采样率，与audio_adc采样率一致即可；
* Pitchrate: 变调比例，用于调节变调效果，需要与const变量搭配使用；
    | 当const变量SPITCH_PARM_PR_HResolution = 0时，pitchrate =␣
↪128为不变调；小于128时音调变高；大于128时音调变高；
    | 当const变量SPITCH_PARM_PR_HResolution = 1时，pitchrate =␣
↪32768为不变调；小于32768时音调变高；大于32768时音调变高；
* Speedin / Speedout: 变速比例，扩音应用下默认为1;
* Quality: 变调运算质量，与运算速度成正比，配置范围3~8;
```

```
20 void *speed_api(void *obuf, u32 insample, void **ppsound)
21 {
22     log_info("speed_api\n");
23
24
25     sp_parm.insample = insample;            //输入音频采样率
26     if (SPITCH_PARM_PR_HResolution) {
27         sp_parm.pitchrate = 32768;          //变调比例，32768:为不变调，<32768:音调变高，>32768:音调变低，声音更沉
28     } else {
29         sp_parm.pitchrate = 100;          //变调比例，128:为不变调，<128:音调变高，>128:音调变低，声音更沉
30     }
31     //speedout/speedin 的结果大于0.6小于1.8.变快变慢最好都不要超过2倍
32     sp_parm.speedin = 80;
33     sp_parm.speedout = 144;          //sp_parm.speedin，sp_parm.speedout为变速比例，例如speedin=1;speedout=2;则为变慢1倍
34     sp_parm.quality = 4;            //变调运算的运算质量，跟速度成正比，配置范围3-8
35     sp_parm.pitchflag = 1;      //如果pitchflag=1,就是输入输出速度只由speedin，speedout决定,调节音调的时候，速度比例保持不变
   度一直是保持放慢1倍。
36     //如果pitchflag=0，就是变速变调比例分开控制 ，这样实际变速的比例是 （speedout/speedin)*pitchrate了。
37
38     return speed_phy(obuf, &sp_parm, ppsound);
39 }
NORMAL  app/bsp/common/sound_effect_list/speed_api.c
```

图 1 变速变调参数初始化

变调限制：

1. **SPITCH_PARM_PR_HResolution=0 时，pitchrate 以 128 为原始音调；**
   - 音调最多降低为原来的 1/6，即 pitchrate=128*6；
   - 音调可升高原来的 6 倍，即 pitchrate=128/6；升高音调没有严格的限制不能超过 6 倍，但是不建议升高过多，过多效果会比较差。
2. **SPITCH_PARM_PR_HResolution=1 时，pitchrate 以 32768 为原始音调；**
   - 音调最多降低为原来的 1/2，即 pitchrate=65535；
   - 音调升高没有严格限制；

变速限制：

- speedin,speedout：没有严格限制比例。但是变快太多或者变慢太多效果都会比较差。

27.2 相关函数

27.2.1 函数 void *speed_api(void *obuf, u32 insample, void **ppsound)

该函数用于初始化变速变调（speed）的相关参数，由应用层调用即可, 其中参数：

```
1、obuf：输出的音频buff；
2、insample：当前输入音频的采样率；
3、ppsound：当前算法的前级音频通道句柄；
```

27.2.2 函数 int music_play_speed_set_para(music_play_obj *hdl, SPEED_PITCH_PARA_STRCUT *sp_parm)

该函数用于 app_mg 工程，在解码流程中动态设置变速变调参数，其中参数：

```
1、*hdl：音乐播放句柄；
2、*sp_parm：新的变速变调参数指针；
3、返回值：成功，返回0；失败，返回-1；
```

voice_toy 工程也有类似的相关函数，具体参考发声玩具应用章节。

CHAPTER 28

软关机 Soft off 和低功耗 Power down

AD14N / AD15N / AC104N / AD17N 支持 Soft off 和 Power down，同时支持多种唤醒方式；

模式	区别
Soft off	Soft off 唤醒后芯片需要从 MASKROM 开始重新跑流程；
Power down	Power down 唤醒后芯片会原地唤醒开始跑。

功耗水平：

模式 / CPU	AD14N / AC104N	AD15N	AD17N	AD18N
Soft off	1.7uA+	1.7uA+	1.4uA+	3.3uA+
Power down	27uA+	27uA+	23uA+	33uA+
支持最多唤醒口	8 个	12 个	8 个	8 个

启动速度：

- 注 *1*：启动速度测试方法为上电/触发唤醒到运行 *main.c* 第一行的时间。
- 注 *2*：软关机快速启动不会运行 *uboot* 流程，可以加快启动时间，但无法进行 *uboot* 升级操作。

启动方式	AD14N / AC104N	AD15N	AD17N	AD18N
冷启动（首次上电）	40ms	36ms	31ms	39ms
软关机唤醒（正常启动）	35ms	32ms	23ms	32ms
软关机唤醒（快速启动）	无快速启动	20ms	9ms	15ms

Soft off 及 Power down 有以下相关函数：

```
void sys_softoff(void)
void sys_power_down(u32 usec);
void board_set_soft_poweroff(void);
```

唤醒相关函数：

```
int power_wakeup_reason(void);
u8 is_wakeup_source(enum WAKEUP_REASON index)
```

28.1 关机与唤醒相关参数配置

28.1.1 A0 芯片 flash 使用 VDDIO 供电配置

- 在使用 A0 封装的芯片，即存放代码的系统 flash 由 VDDIO 供电时，需要将该宏置 1，否则可能会出现休眠功耗大的问题；

芯片	A0 芯片使用 VDDIO 给系统 flash 供电配置宏
AD14N / AC104N	TCFG_KEEP_FLASH_POWER_GATE
AD15N	TCFG_KEEP_FLASH_POWER_GATE
AD17N	FLASH_POWER_GATE_VDDIO
AD18N	FLASH_POWER_GATE_VDDIO

```
/************************************power_param*************************************************/

#define TCFG_LOWPOWER_POWER_SEL             PWR_LDO15                   //电源模式设置，可选DCDC和LDO
#define TCFG_LOWPOWER_BTOSC_DISABLE         0                           //低功耗模式下BTOSC是否保持
#define TCFG_LOWPOWER_LOWPOWER_SEL          SLEEP_EN                  //SNIFF状态下芯片是否进入powerdown
/*mvddiom: 2.0~3.4*/
#define TCFG_LOWPOWER_VDDIOM_LEVEL          VDDIOM_VOL_32V
/*wvddiom: 2.0~3.4*/
#define TCFG_LOWPOWER_VDDIOW_LEVEL          VDDIOW_VOL_28V              //弱VDDIO等级配置
#define TCFG_KEEP_FLASH_POWER_GATE          0                           //power_gate引脚由用户控制
#define TCFG_LOWPOWER_DAC_OPEN              1                           //出低功耗是否开dac电源，对于频繁进出低功耗应用建议在dac使用前开dac电源

const struct low_power_param power_param = {
    .config         = TCFG_LOWPOWER_LOWPOWER_SEL,       //0：sniff时芯片不进入低功耗  1：sniff时芯片进入powerdown
    .btosc_hz       = 16000000,                         //外接晶振频率
    .delay_us       = 16000000 / 1000000L,              //提供给低功耗模块的延时(不需要需修改)
    .btosc_disable  = TCFG_LOWPOWER_BTOSC_DISABLE,      //进入低功耗时BTOSC是否保持
    .vddiom_lev     = TCFG_LOWPOWER_VDDIOM_LEVEL,       //强VDDIO等级,可选：2.0V  2.2V  2.4V  2.6V  2.8V  3.0V  3.2V  3.6V
    .vddiow_lev     = TCFG_LOWPOWER_VDDIOW_LEVEL,       //弱VDDIO等级,可选：2.1V  2.4V  2.8V  3.2V
    .osc_type       = OSC_TYPE_LRC,
    .flash_pg       = TCFG_KEEP_FLASH_POWER_GATE,
    .vdc13_cap_en   = 0,                                //根据vdc13引脚是否有外部电容来配置，1.外挂电容 0.无外挂电容
    .vddio_keep = 0,
};

/************************************wakeup_param*************************************************/
const struct port_wakeup port0 = {
    .pullup_down_enable = ENABLE,                       //配置I/O 内部上下拉是否使能
    .edge       = FALLING_EDGE,                         //唤醒方式选择,可选：上升沿\下降沿
    .attribute  = BLUETOOTH_RESUME,                     //保留参数
app/bsp/cpu/sh54/power_api.c
```

A0 芯片 flash 使用 VDDIO 供电配置

28.1.2 VDDIO 电压档位配置

- 芯片的 VDDIO 电压会随着系统状态的变化而不同；正常运行时使用强 VDDIO 档位（MVDDIO），Powerdown/Softoff 时使用弱 VDDIO 档位（WVDDIO）；
- 修改 power_param 结构体中的 vddiom_lev 成员，可以配置芯片的强 VDDIO 电压；
- 修改 power_param 结构体中的 vddiow_lev 成员，可以配置芯片的弱 VDDIO 电压；
- power_param 的 vddio_keep 参数置为 1 时，进入 powerdown 前后的 VDDIO 电压不会发生变化，仅 AD14N / AD15N / AC104N 有效；

- AD17N 默认保持 VDDIO 电压档位；

芯片	默认强 VDDIO 档位	默认弱 VDDIO 档位
AD14N / AC104N	3.2v	2.8v
AD15N	3.2v	2.8v
AD17N	3.2v	3.2v
AD18N	3.2v	3.2v

```
/************************************power_param*************************************/

#define TCFG_LOWPOWER_POWER_SEL            PWR_LDO15                //电源模式设置，可选DCDC和LDO
#define TCFG_LOWPOWER_BTOSC_DISABLE        0                        //低功耗模式下BTOSC是否保持
#define TCFG_LOWPOWER_LOWPOWER_SEL         SLEEP_EN                 //SNIFF状态下芯片是否进入powerdown
/*mvddiom: 2.0~3.4*/
#define TCFG_LOWPOWER_VDDIOM_LEVEL         VDDIOM_VOL_32V
/*wvddiom: 2.0~3.4*/
#define TCFG_LOWPOWER_VDDIOW_LEVEL         VDDIOW_VOL_28V           //弱VDDIO等级配置
#define TCFG_KEEP_FLASH_POWER_GATE         0                        //power_gate引脚由用户控制
#define TCFG_LOWPOWER_DAC_OPEN             1                        //出低功耗是否开dac电源，对于频繁进出低功耗应用建议在dac使用前开dac电源

const struct low_power_param power_param = {
    .config         = TCFG_LOWPOWER_LOWPOWER_SEL,          //0：sniff时芯片不进入低功耗  1：sniff时芯片进入powerdown
    .btosc_hz       = 16000000,                            //外接晶振频率
    .delay_us       = 16000000 / 1000000L,                 //提供给低功耗模块的延时(不需要需修改)
    .btosc_disable  = TCFG_LOWPOWER_BTOSC_DISABLE,         //进入低功耗时BTOSC是否保持
    .vddiom_lev     = TCFG_LOWPOWER_VDDIOM_LEVEL,          //强VDDIO等级,可选：2.0V  2.2V  2.4V  2.6V  2.8V  3.0V  3.2V  3.6V
    .vddiow_lev     = TCFG_LOWPOWER_VDDIOW_LEVEL,          //弱VDDIO等级,可选：2.1V  2.4V  2.8V  3.2V
    .osc_type       = OSC_TYPE_LRC,
    .flash_pg       = TCFG_KEEP_FLASH_POWER_GATE,
    .vdc13_cap_en   = 0,                                   //根据vdc13引脚是否有外部电容来配置，1.外挂电容 0.无外挂电容
    .vddio_keep = 0,
};

/************************************wakeup_param*************************************/
const struct port_wakeup port0 = {
    .pullup_down_enable = ENABLE,                          //配置I/O 内部上下拉是否使能
    .edge       = FALLING_EDGE,                            //唤醒方式选择,可选：上升沿\下降沿
    .attribute  = BLUETOOTH_RESUME,                        //保留参数
                  app/bsp/cpu/sh54/power_api1.c
```

VDDIO 电压档位配置示意图

28.1.3 低功耗唤醒源配置

- Soft off 与 Power down 共用唤醒模块。可以在 power_api.c 的 wkup_param 中增加 port_wakeup 类型成员来增加唤醒源。（AD17N 为 power_config.c 文件）
- 唤醒前需要先配置唤醒参数结构体 wakeup_param，内容包含 PORT_FLT 唤醒滤波和 port_wakeup 单个唤醒源配置：
- port_wakeup 单个唤醒源配置可选择唤醒 I/O 口、唤醒边沿以及是否使能内部上下拉：

PORT_FLT：边沿唤醒选择过滤掉的脉冲时长

```
① PORT_FLT_NULL;
② PORT_FLT_32us;
③ PORT_FLT_64us;
④ PORT_FLT_128us;
⑤ PORT_FLT_256us;
⑥ PORT_FLT_512us;
⑦ PORT_FLT_1ms;
⑧ PORT_FLT_2ms;
```

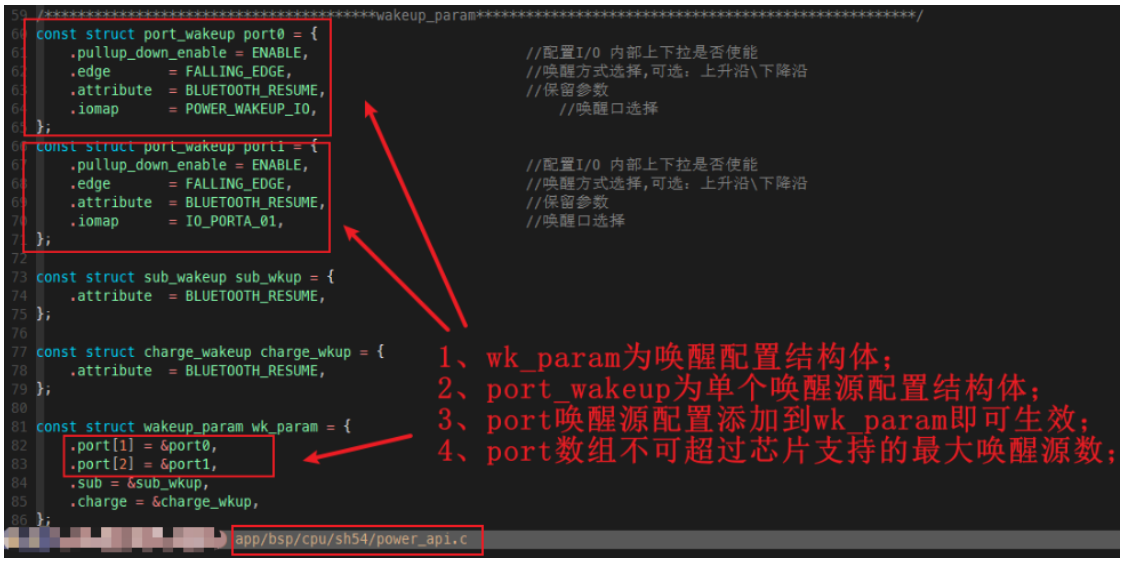
port_wakeup 结构体：唤醒端口的参数配置

```
① pullup_down_enable: 上/下拉使能;
② edge: 边沿触发方式;
③ attribute: 保留参数;
```

(续下页)

(接上页)

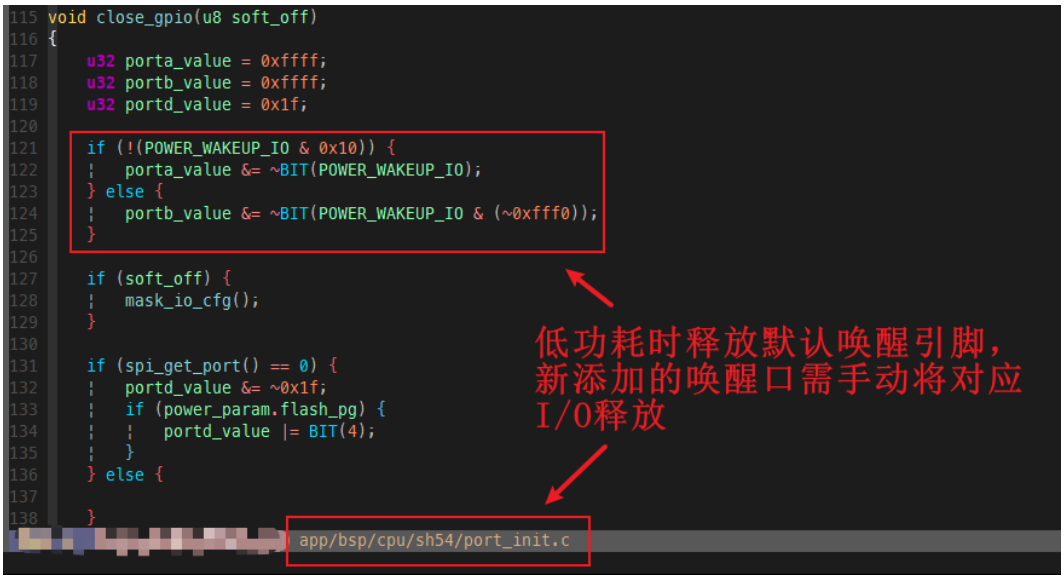
④ iomap：I/O口选择；
⑤ filter_enable：滤波使能；



唤醒源配置

注意：配置好唤醒源后，还需要在低功耗流程中释放对应 *I/O* 引脚，芯片进入低功耗时会将除默认唤醒口外的 *I/O* 设置为高阻态；新添加的唤醒口需要单独释放，否则无法触发唤醒

***AD14N / AD15N / AC104N*:**



AD14N_AD15N_AC104N 低功耗释放新增唤醒引脚

***AD17N*:**

```c
void board_set_soft_poweroff()
{
    u32 gpio_config[3] = {0xffff, 0xffff, 0xffff};
#define PORT_PROTECT(gpio)  gpio_config[gpio/IO_GROUP_NUM] &= ~BIT(gpio%IO_GROUP_NUM)

    PORT_PROTECT(POWER_WAKEUP_IO);

    //MCLR
    if (is_mclr_en()) {
        PORT_PROTECT(MCLR_PORT);
    }

    //长按复位
    if (is_pinr_en()) {
        u8 port_sel = get_pinr_port();
        if ((port_sel >= PA0_IN) && (port_sel <= PA12_IN)) {
            PORT_PROTECT(IO_PORTA_00 + (port_sel - PA0_IN));
        } else if ((port_sel >= PB0_IN) && (port_sel <= PB3_IN)) {
            PORT_PROTECT(IO_PORTB_00 + (port_sel - PB0_IN));
        }
    }

app/bsp/cpu/sh57/power_config.c
```

AD17N低功耗时释放唤醒引脚，
新增的唤醒引脚需要做相同处理

AD17N 低功耗释放新增唤醒引脚

AD18N:

```c
void board_set_soft_poweroff()
{
    /* audio_off(); */

    u32 gpio_config[6] = {0xffff, 0xffff, 0xffff, 0xffff, 0xffff, 0xffff};
    u32 usbio_config = 0x3;
    /* #define PORT_PROTECT(gpio)   gpio_config[gpio/IO_GROUP_NUM] &= ~BIT(gpio%IO_GROUP_NUM) */
#define USB_PROTECT(gpio)   usbio_config &= ~BIT(gpio-IO_PORT_DP)

    PORT_PROTECT(P33_IO_WKUP_IOMAP);

    if (is_mclr_en()) {
        PORT_PROTECT(MCLR_PORT);
    }

    //长按复位
    if (is_pinr_en()) {
        u8 port_sel = get_pinr_port();
app/bsp/cpu/ch58/power_config.c
```

AD18低功耗时释放唤醒引脚，
新增的唤醒引脚需要做相同处理

AD18N 低功耗释放新增唤醒引脚

28.2 Soft off 及 Power down 相关函数

28.2.1 函数 void sys_softoff()

此函数实现使系统进入软关机，唤醒后系统复位重新运行；

28.2.2 函数 void sys_power_down(u32 usec)

此函数实现进入系统 power down，可通过定时唤醒或 IO 触发唤醒，其中参数：

```
1、usec：唤醒时间（微秒）；
    ①. -2：静态睡眠，需要等待按键唤醒；
    ②. 非-
↪2:睡眠时间，单位us，如1000000即1s后唤醒，睡眠时间不可超过看门狗唤醒时间的一半
```

28.2.3 函数 void board_set_soft_poweroff(void)

此函数实现在进入软关机之前将 IO 口设置成高阻，若需要保留原来状态可修改该函数。

设置对应引脚之后，需要在 close_gpio() 函数中，将对应的 I/O 不设置为高阻态；

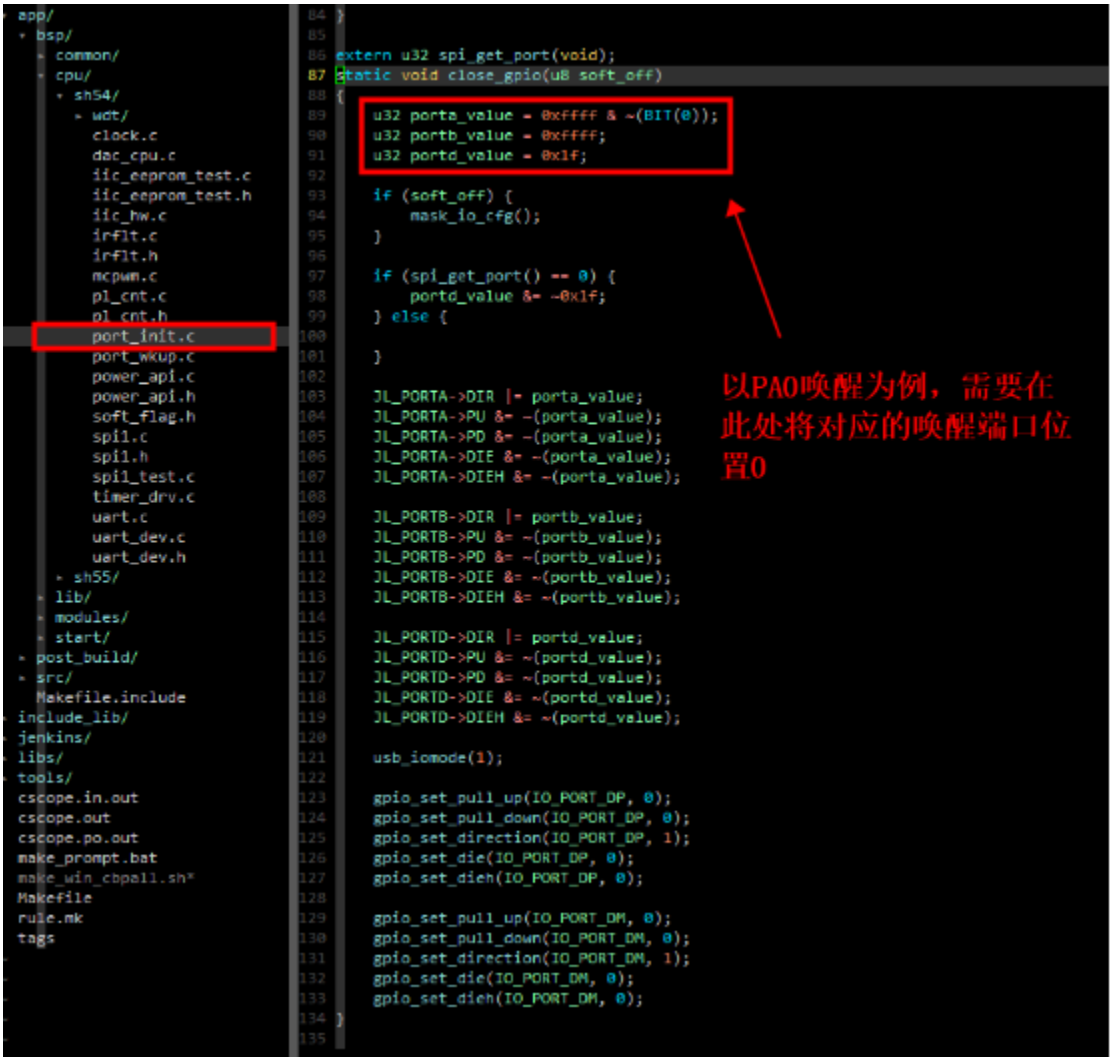


图 2 close_gpio 函数

28.3 Soft off 及 Power down 唤醒源获取及其他函数

28.3.1 AD14N / AD15N / AC104N 相关接口

函数 u32 power_wakeup_reason(void)

此函数实现获取 softoff 唤醒源信息，该接口需要在系统初始化 sys_power_init() 之前调用，否则获取的唤醒源不准确，其中参数：

1、返回值：唤醒端口（对应wakeup_param中的port成员）；

28.3.2 AD17N 与 AD18N 相关接口

函数 u8 is_wakeup_source(enum WAKEUP_REASON index)

此函数实现获取 softoff 和 softoff 唤醒源信息，外部可传参检查是否为本次唤醒的唤醒源；

获取 softoff 唤醒源需要在系统初始化 sys_power_init() 之前调用，否则获取的唤醒源不准确，其中参数：

```
1、index：唤醒源；
2、返回值：1：传入的参数时本次的唤醒源，0：传入的参数非本次唤醒源；
```

函数 void gpio_longpress_pin0_reset_config(u32 pin, u32 level, u32 time, u8 release, u8 protect)

此函数实现长按复位初始化，该接口需要在电源初始化的最后调用，其中参数：

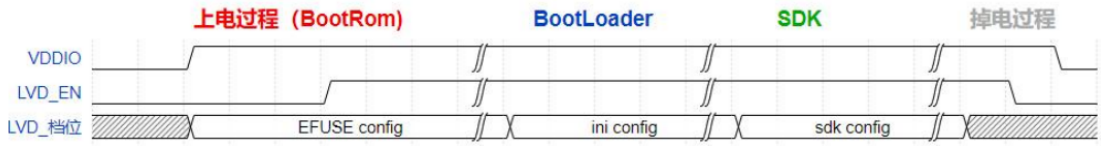
```
1、pin：复位I/O；
2、level：触发方式
    ①、0：下降沿触发；
    ②、1：上升沿触发；
3、time：时间
    ①、0：disable；
    ②、1：1s复位；
    ③、2：2s复位；
    ④、4：4s复位；
    ⑤、8：8s复位；
    ⑥、16：16s复位
3、release：系统释放方式
    ①、0：等待电平翻转后复位；
    ②、1：立即复位；
4、protect：该参数给0即可；
```

CHAPTER 29

VDDIO 电压配置说明

VDDIO : 芯片引脚出来的电压。

29.1 VDDIO 作用域



AD17 vddio 作用域

上电过程，即电源上电，可能会存在抖动，一般会设置比较高的 VDDIO 档位。

Boot-loader 和 SDK，根据 flash 运行所需的电压，VDDIO&VBAT 负载情况，重新调整 VDDIO 档位。

29.2 如何配置每个阶段的 VDDIO 档位

29.2.1 上电过程：

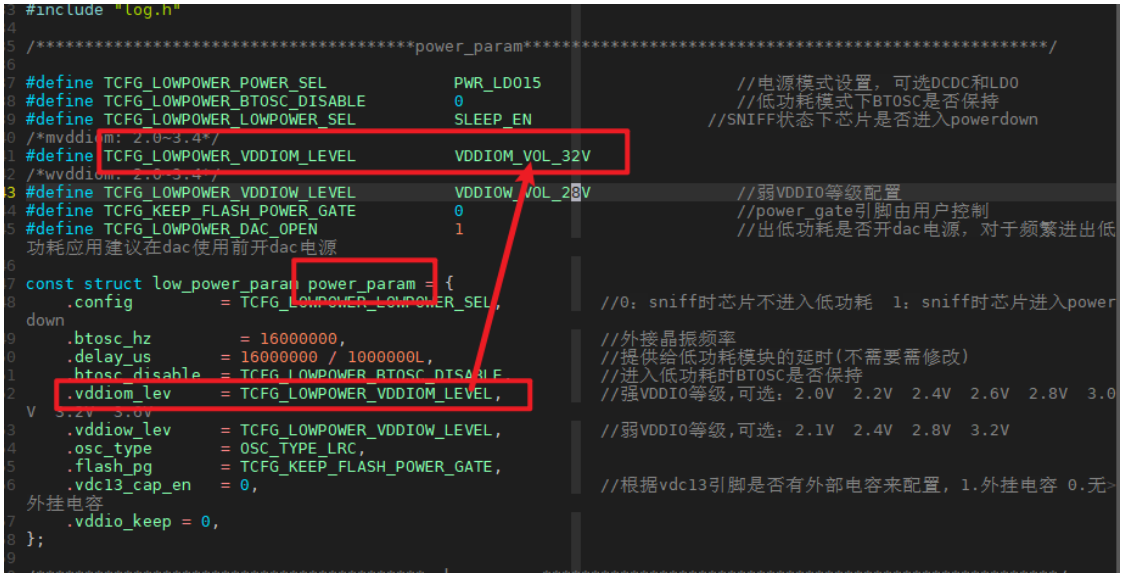
芯片 VDDIO 会有一个默认电压值；该电压值对应电源上电时刻，由于此时电源可能会存在抖动，所以一般会设置比较高的 VDDIO 档位。

29.2.2 Boot-Loader：

即 uboot 工程，会从 ini 文件里面读取 VDDIO 配置，重新调整 VDDIO 档位。

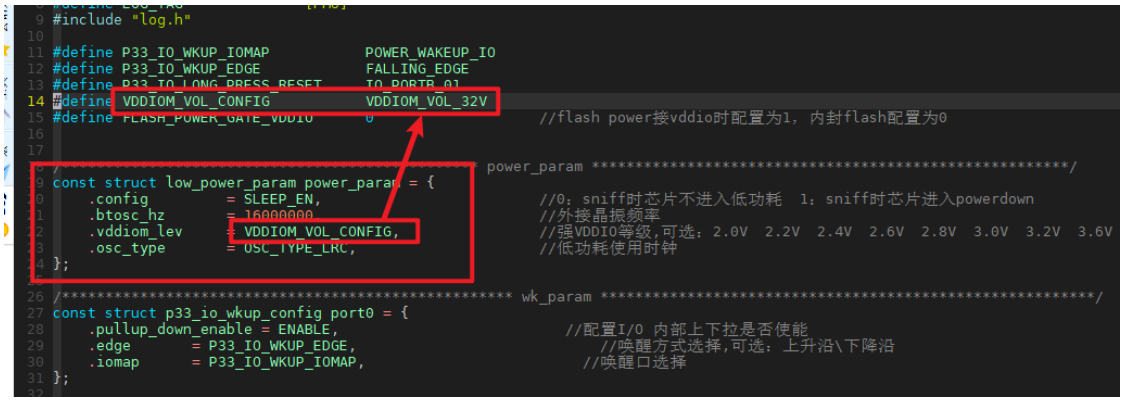
29.2.3 SDK：

AC104，AD14，AD15 在 power_api.c 里配置 power_param 结构体的 vddiom_lev 成员，即可修改 VDDIO 档位。如下图：



AC104，AD14，AD15 配置 vddio 档位

AD17，AD18 在 power_config.c 里配置 power_param 结构体的 vddiom_lev 成员，即可修改 VDDIO 档位。如下图：



AD17，AD18 配置 vddio 档位

29.3 常见问题介绍

29.3.1 VDDIO 偏差

实际值与配置值相差 0.2V 为正常现象。

偏差过大请联系助理。

CHAPTER 30

VM 掉电存储说明

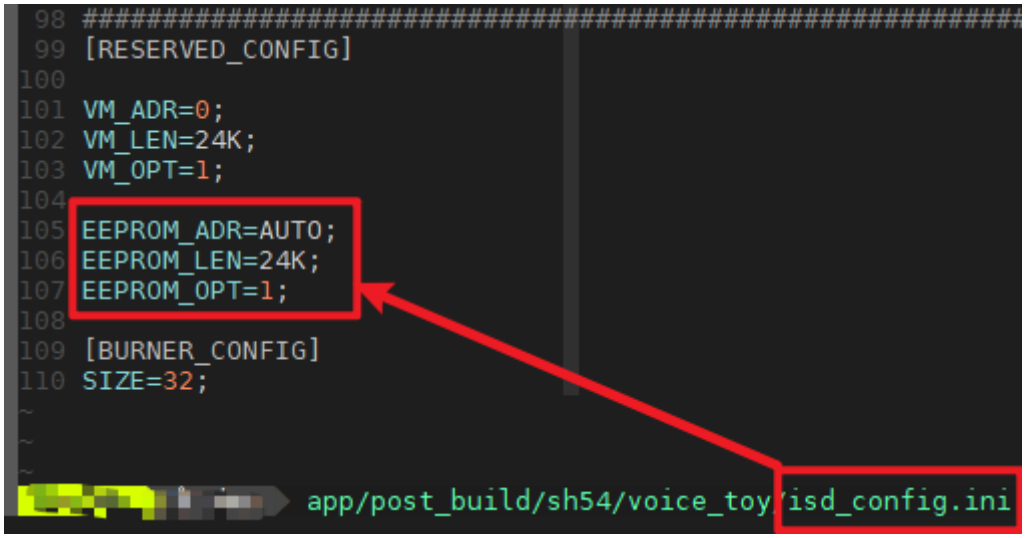
下面分为 7 个部分来介绍 VM 功能：

1. “*VM 掉电存储作用和数据存储*”
2. “*VM 掉电存储使用说明*”
3. “*VM 掉电存储版本简单说明*”
4. “*新旧 VM 文件组成及版本选择*”
5. “*VM 相关参数介绍*”
6. “*VM 掉电记忆存储接口*”
7. “*VM 常见问题介绍*”

30.1 VM 掉电存储作用和数据存储

SDK 提供了 vm 掉电记忆存储功能，该功能用于保存断电之后还需要保存的信息。

使用 VM 存储的数据会记忆在 FLASH 的” EEPROM” 区域，该区域大小可由下载目录的” isd_config.ini” 文件配置，使用 VM 功能时 EEPROM 空间不得低于 8K 字节，且需为 4K 字节的倍数（即 12K，16K，以此类推）。



VM 数据存储位置

30.2 VM 掉电存储使用说明

VM 记忆存储使用的步骤分为三步：

- 第一步：系统初始化时，调用函数 vm_init_api() 初始化 VM 功能。
- 第二步：使用 vm_write() 接口把需要保存的数据写入 FLASH。
- 第三步：使用 vm_read() 接口读取写入的数据。

可参考”*VM 简单使用 demo*：”

30.3 VM 掉电存储版本简单说明

目前 SDK 中 VM 有新旧两个版本可以选择：

备注： 旧版 VM 功能：

1. 可存储 id 号数量：128 个；
2. 单次写入最大数据长度：128 个 byte；
3. 没有 cache 管理；
4. 没有预擦除功能；

备注： 新版 VM 功能

1. 可存储 id 号数量：512 个；

2. 单次写入最大数据长度：4096 个 byte；（若 EEPROM 分配给 VM 区域只有 8K，则单次写入最大数据长度为：4092 个 byte）；
3. 增加 cache 管理，提升读取速度；
4. 增加预擦除功能。

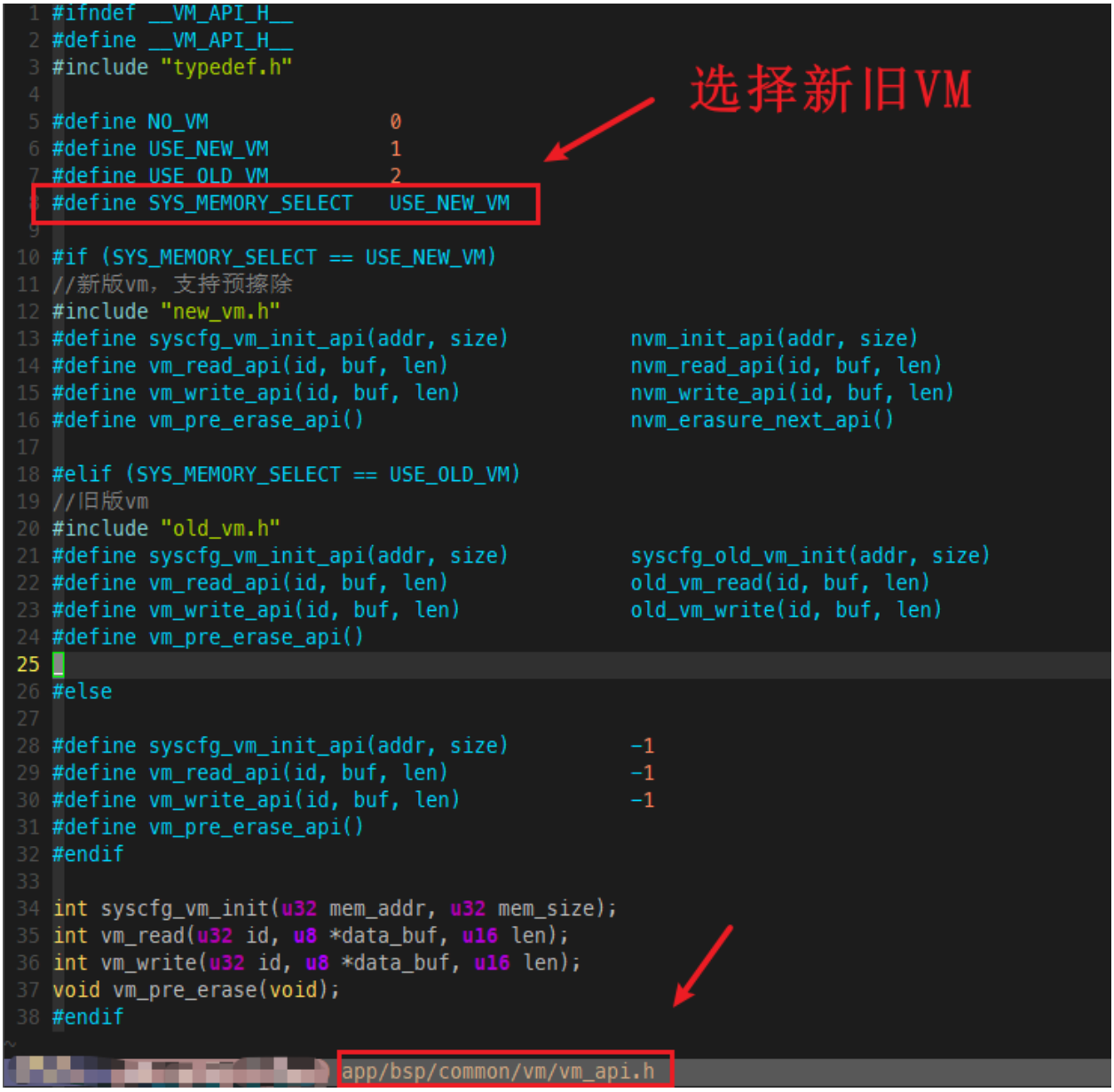
备注： 新旧版本 VM 功能对比：

1. 新版 VM id 号数量增多；
2. 新版 VM 单次写入最大数据长度增多；
3. 新版 VM 增加 cache 管理，提升 VM 读取速度；
4. 新版 VM 增加预擦除功能，可以在系统空闲时进行预擦除操作，可尽可能避免 VM 整理时关闭 flash 时间过长导致的卡音问题；
5. 新版 ID 号与旧版 index 句柄含义相同；

30.4 新旧 VM 文件组成及版本选择

vm 模块由 vm_api.c 和 vm_api.h 文件构成 API 层，包含 ovm_api.c 和 nvm_api.c 文件；

备注： 在 SDK 为 AD14/15/AC104 的 v1.7.0 版本（包括 1.7.0 版本）以前版本，用户可在 **vm_api.h** 文件里选择使用新 / 旧版 VM.




```
#ifndef __VM_API_H__
#define __VM_API_H__
#include "typedef.h"

#define NO_VM                0
#define USE_NEW_VM           1
#define USE_OLD_VM           2
#define SYS_MEMORY_SELECT    USE_NEW_VM

#if (SYS_MEMORY_SELECT == USE_NEW_VM)
//新版vm，支持预擦除
#include "new_vm.h"
#define syscfg_vm_init_api(addr, size)          nvm_init_api(addr, size)
#define vm_read_api(id, buf, len)               nvm_read_api(id, buf, len)
#define vm_write_api(id, buf, len)              nvm_write_api(id, buf, len)
#define vm_pre_erase_api()                      nvm_erasure_next_api()

#elif (SYS_MEMORY_SELECT == USE_OLD_VM)
//旧版vm
#include "old_vm.h"
#define syscfg_vm_init_api(addr, size)          syscfg_old_vm_init(addr, size)
#define vm_read_api(id, buf, len)               old_vm_read(id, buf, len)
#define vm_write_api(id, buf, len)              old_vm_write(id, buf, len)
#define vm_pre_erase_api()

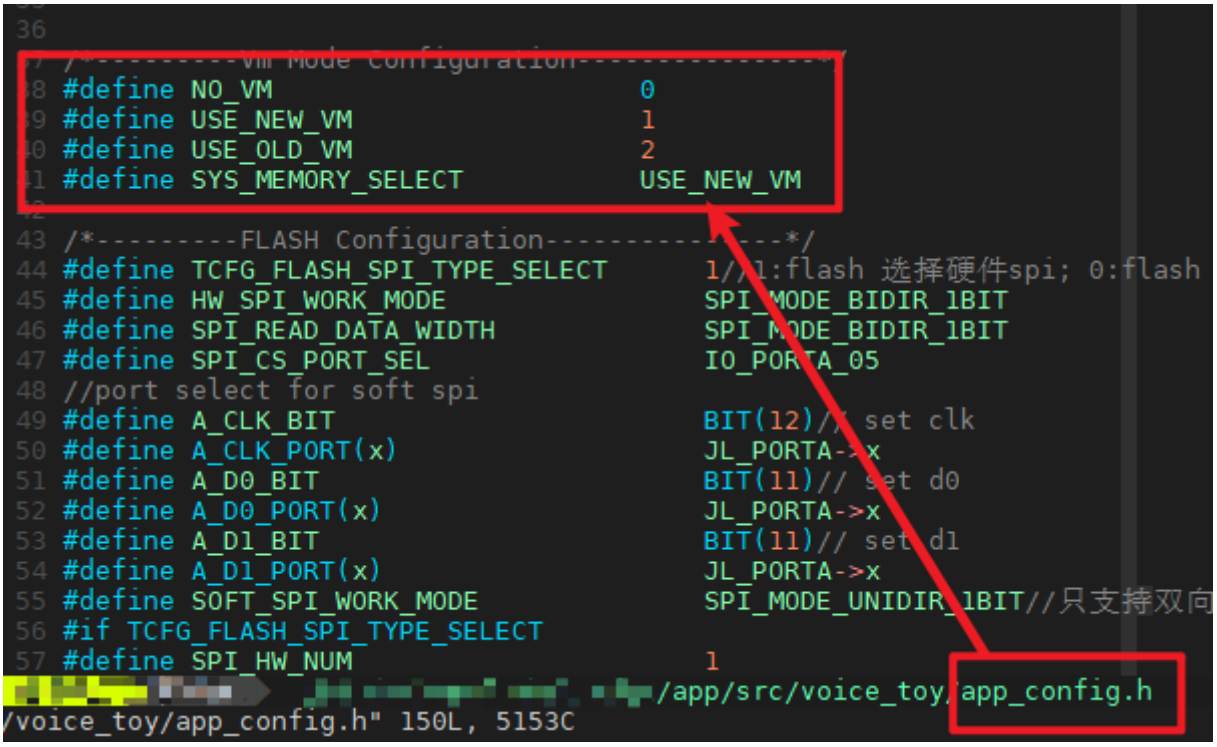
#else

#define syscfg_vm_init_api(addr, size)          -1
#define vm_read_api(id, buf, len)               -1
#define vm_write_api(id, buf, len)              -1
#define vm_pre_erase_api()
#endif

int syscfg_vm_init(u32 mem_addr, u32 mem_size);
int vm_read(u32 id, u8 *data_buf, u16 len);
int vm_write(u32 id, u8 *data_buf, u16 len);
void vm_pre_erase(void);
#endif
```

新旧 VM 功能选择

备注： 在 SDK 为 AD14/15/AC104 的 v1.7.0 版本以后，vm 配置选项更改位置，用户可在 **app_config.h** 文件里选择使用新 / 旧版 VM.



新旧 VM 功能选择

30.5 VM 相关参数介绍

对于用户，我们需要关心的参数分别为：

1. “*ID 号（vm_api.h）*”
2. “*读/写入数据长度*”
3. “*vm 存储区域大小*”

对于新版 VM

4. “*新版 VM 格式整理时缓存空间设置 new_vm_buff（new_vm.h）*”
5. “*新版 vm 格式整理后是否立即擦除旧半区区域功能*”
6. “*新版 vm 是否支持多段读功能*”

下面逐一介绍：

30.5.1 ID 号（vm_api.h）

```
}
u32 vm_write(u32 id, u8 *buf, u32 len)
{
```

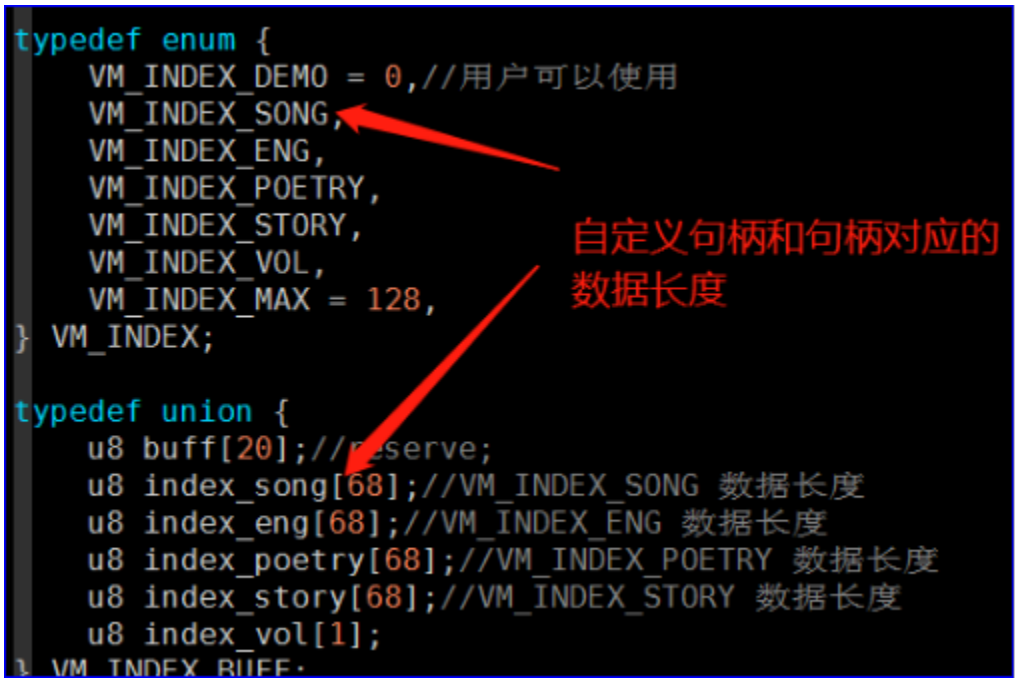
vm_write 参数

重要： 旧版 VM ID 号最多 128 个，新版 VM ID 号最多 512 个；（旧版 VM index 号在新版 vm 仍可通用）

新版 VM ID 号最多支持 512 个，且不再需要自定义对应 ID 的数据长度。

用户可在 vm_api.h 的 VM_INDEX 里添加自己的 id 号。

注意： 1~32 号 ID 为系统使用，用户不可修改其顺序



旧版 VM 掉电存储记忆项的定义 (vm_api.h)

30.5.2 读/写入数据长度

```
u32 vm_write(u32 id, u8 *buf, u32 len)
```

vm_write 参数 length

重要： 旧版 VM 读/写长度最多支持 128 个 byte；

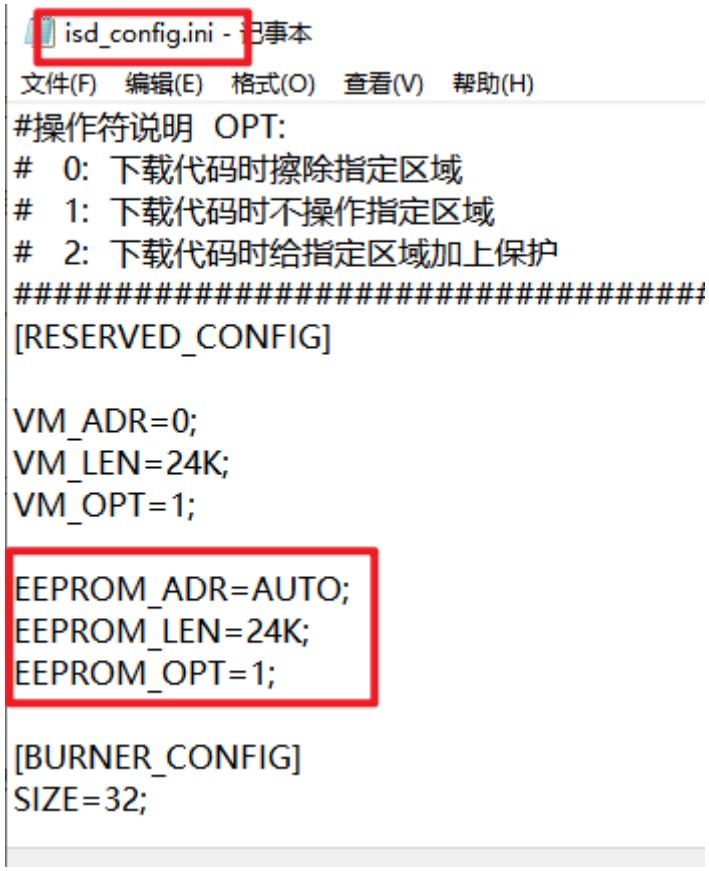
新版 VM 读/写长度最多支持 4096 个 byte；

30.5.3 vm 存储区域大小

重要： VM 记忆的数据是 EEPROM 区划分出一定空间大小给 VM 使用，该区域大小可由下载目录里的 isd_config.ini 文件配置。(该配置参数详细说明可参考”*isd_config.ini 文件介绍*”)

注意：

VM 存储区域大小分配的 EEPROM 空间不得低于 8K 字节，且需为 4K 字节的倍速，即 8K，12K，16K 等等。



VM 区域大小设置”

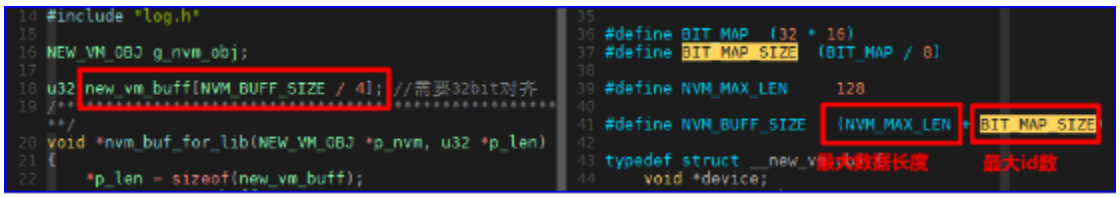
30.5.4 新版 VM 格式整理时缓存空间设置 new_vm_buff（new_vm.h）

重要： new_vm_buff，该 buff 用于 vm 格式整理时数据的缓存。

该 buff 由 **用户自定义最大 id 数量 (BIT_MAP_SIZE) + 单个 id 在整理时数据缓存的最大长度 (BIT_MAX_LEN)** 两部分组成。

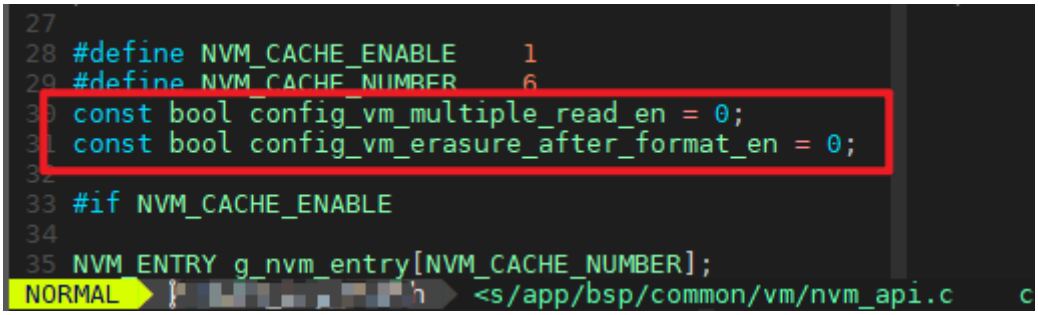
BIT_MAP_SIZE： 即 BIT_MAP：vm 最多可以存储的 id 数量，固定为 512 个。

BIT_MAX_LEN： 即 VM 格式整理时数据缓存的大小，该值越大，则整理速度越快，该值越小，则整理速度越慢。(注意：该 buf 与读/写数据最大长度为两个概念)



VM 掉电存储记忆项的定义（nvm_api.c 和 new_vm.h）

30.5.5 新版 vm 格式整理后是否立即擦除旧半区区域功能



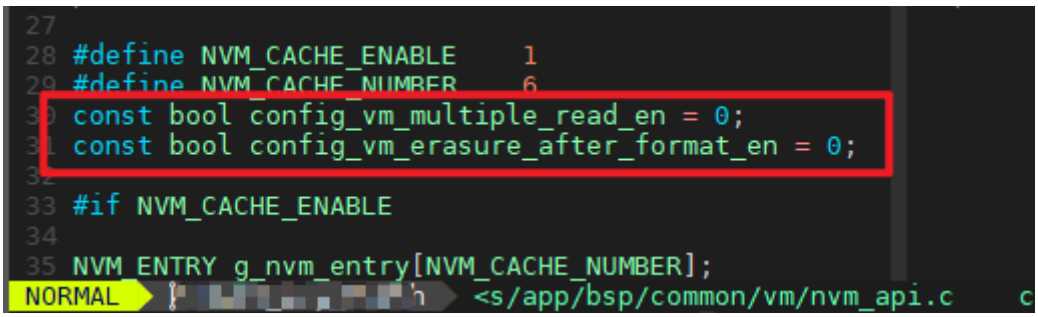
config_vm_erasure_after_format_en

config_vm_erasure_after_format_en 用于控制新版 vm 格式整理后是否立即擦除旧半区区域；
0： 不立即擦除（默认值）
1： 立即擦除

小技巧： 该功能直观影响为：打开该功能后，系统如果此时在解码，vm 进行格式整理后会进行擦除动作，进而导致出现卡音现象。

而默认关闭该功能时，系统在解码时，进行 VM 格式整理却不会进行擦除动作，而是等到系统空闲时进行擦除动作。

30.5.6 新版 vm 是否支持多段读功能



config_vm_multiple_read_en

config_vm_multiple_read_en 用于控制新版 vm 是否支持多段读功能；
0： 不支持多段读（默认值）
1： 支持多段读

30.6 VM 掉电记忆存储接口

30.6.1 函数 u32 syscfg_vm_init(u32 addr, u32 size)

此函数用来初始化 vm 虚拟存储系统，其中参数：

1. eeprom_saddr：vm 区域的起始地址。
2. eeprom_size：vm 区域的空间长度。
3. **返回值：**

   成功：0；
   失败：错误号，可参考 errno-base.h；

30.6.2 函数 u32 vm_write(u32 id,u8 *buf,u32 len)

此函数用来把对应 hdl 的数据存储到 vm 虚拟存储系统，其中参数：

1. hdl：访问 vm 的 id。
2. buf：vm 读数据 buffer。
3. len：数据长度。
4. **返回值：**

   成功：写入长度；
   失败：错误号，可参考 errno-base.h；

30.6.3 函数 u32 vm_read(u32 id ,u8 *buf,u32 len)

此函数用来把对应 hdl 的数据从 vm 虚拟存储系统读出来，其中参数：

1. id：访问 vm 的 id。
2. buf：vm 读数据 buffer。
3. len：数据长度。
4. **返回值：**

   成功：写入长度；
   失败：错误号，可参考 errno-base.h；

30.6.4 函数 u32 vm_pre_erase(void)

此函数用于预擦除设备，在系统空闲时主动调用提前擦除 flash，可尽量避免 VM 做整理时关 flash 时间过长导致卡音问题；
该函数只有在新版 VM 处生效，旧版 VM 没有该功能。
示例：音乐模式下，500MS 消息来时若系统处于空闲状态，则会进行预擦除动作：

```
case MSG_500MS:
    /* decoder_time(pctl[0].p_dec_obj); */
    UI_menu(MENU_MAIN);
    UI_menu(MENU_HALF_SEC_REFRESH);
    if (MUSIC_PLAY != get_decoder_status(pctl[0].p_dec_obj)) {
        vm_pre_erase();
        app_powerdown_deal(0);
    } else {
        app_powerdown_deal(1);
    }
```

图 2.1 系统空闲预擦除动作

30.6.5 函数 nvm_pre_erasure_next(NEW_VM_OBJ *p_nvm, u16 using_next, u16 idle_next)

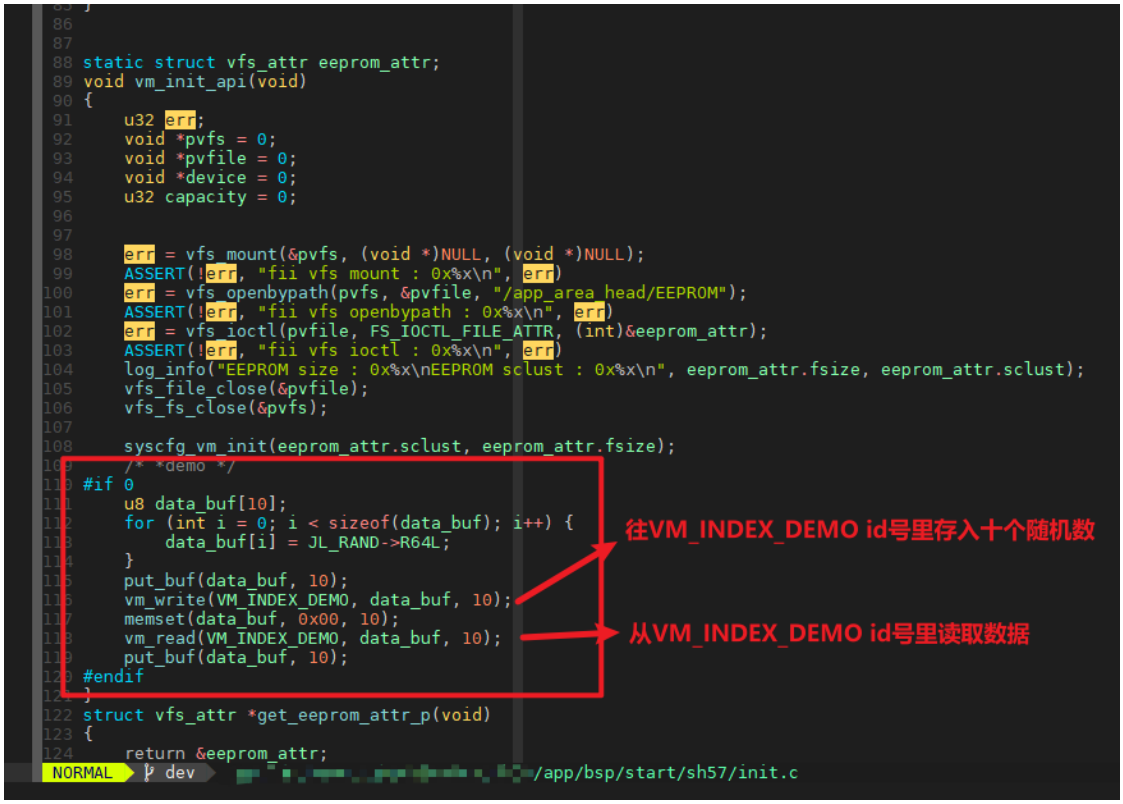
此函数为 NEW VM 中独有接口，由 2.4 函数 u32 vm_pre_erase() 调用，用于系统空闲时选择擦除当前 VM 已存储的数据往后的 flash 扇区，以及空闲 VM 需要往后擦除的 flash 扇区数，用户可自行修改需要擦除的 flash 扇区数，其中参数：

```
1、p_nvm：NEW VM句柄指针；
2、using_next：当前VM存储块需要往后擦除的flash扇区数；
3、idle_next：空闲VM存储块需要往后擦除的flash扇区数；
4、返回值：无
```

30.7 VM 常见问题介绍

30.7.1 VM 简单使用 demo：

用户可在 app/bsp/start/sh5x/init.c 里的 vm_init_api() 函数找到 demo 示例。




```
static struct vfs_attr eeprom_attr;
void vm_init_api(void)
{
    u32 err;
    void *pvfs = 0;
    void *pvfile = 0;
    void *device = 0;
    u32 capacity = 0;


    err = vfs_mount(&pvfs, (void *)NULL, (void *)NULL);
    ASSERT(!err, "fii vfs mount : 0x%x\n", err)
    err = vfs_openbypath(pvfs, &pvfile, "/app_area_head/EEPROM");
    ASSERT(!err, "fii vfs openbypath : 0x%x\n", err)
    err = vfs_ioctl(pvfile, FS_IOCTL_FILE_ATTR, (int)&eeprom_attr);
    ASSERT(!err, "fii vfs ioctl : 0x%x\n", err)
    log_info("EEPROM size : 0x%x\nEEPROM sclust : 0x%x\n", eeprom_attr.fsize, eeprom_attr.sclust);
    vfs_file_close(&pvfile);
    vfs_fs_close(&pvfs);

    syscfg_vm_init(eeprom_attr.sclust, eeprom_attr.fsize);
    /* *demo */
#if 0
    u8 data_buf[10];
    for (int i = 0; i < sizeof(data_buf); i++) {
        data_buf[i] = JL_RAND->R64L;
    }
    put_buf(data_buf, 10);
    vm_write(VM_INDEX_DEMO, data_buf, 10);
    memset(data_buf, 0x00, 10);
    vm_read(VM_INDEX_DEMO, data_buf, 10);
    put_buf(data_buf, 10);
#endif
}
struct vfs_attr *get_eeprom_attr_p(void)
{
    return &eeprom_attr;
```

vm_demo

30.7.2 调整 VM 区域大小设置：

节省 VM 占用系统空间可以从两个地方入手：

1. **减少 VM 区域大小**。
   具体参考上面”*vm 存储区域大小*”章节的 vm 存储区域大小部分
2. **新版 VM 可以减少 VM 格式整理时的缓存 new_vm_buff**
   可以适当减少 BIT_MAX_LEN 的长度。

   具体参考上面”*新版 VM 格式整理时缓存空间设置 new_vm_buff（new_vm.h）*”章节的新版 VM 格式整理时缓存空间设置部分。

30.7.3 新版 VM 在关机或者掉电后可能存在数据丢失 BUG：

该问题主要原因为系统在关机前格式整理后没有做预擦除动作。

具体原因及解决办法说明可以参考以下链接：

FAQ：AD14/15/17/AC104 new_vm 功能在关机或者掉电后可能存在数据丢失 BUG

30.7.4 VM 格式整理原理：

VM 格式整理原理：

VM 功能将 EEPROM 区域划分为 A、B 两块，A 块记录 VM 数据，B 块处于空闲状态，
当 A 块数据接近饱和时，会擦除 B 块并将 A 块的数据整合到 B 块，后续 B 块记录 VM 数据，A 块处于空闲状态，以此往复。

30.7.5 VM 卡音问题原因和解决办法：

问题原因：

数据整合过程需要关中断并擦除空闲块的 flash 扇区，时间较长可能会导致解码卡音等问题；

解决办法：

NEW VM 可以在系统空闲状态时调用 vm_pre_erase() 函数；
该函数作用是提前擦除当前已存储数据往后的 flash 扇区，以及空闲块的 flash 扇区数（内部会限制不超出 EEPROM 区域扇区数），即把格式整理的 VM 擦除提前处理。

SDK 上常见操作如：

（一）切换模式时做擦除动作；

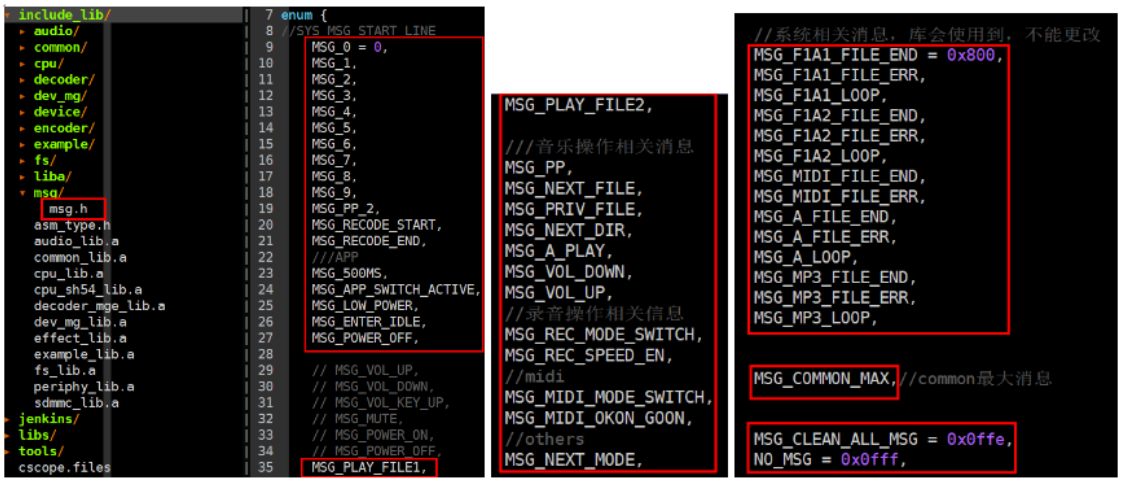
（二）500ms 时，系统空闲会做擦除动作；

（三）系统关机/睡眠前会做擦除动作；

CHAPTER 31

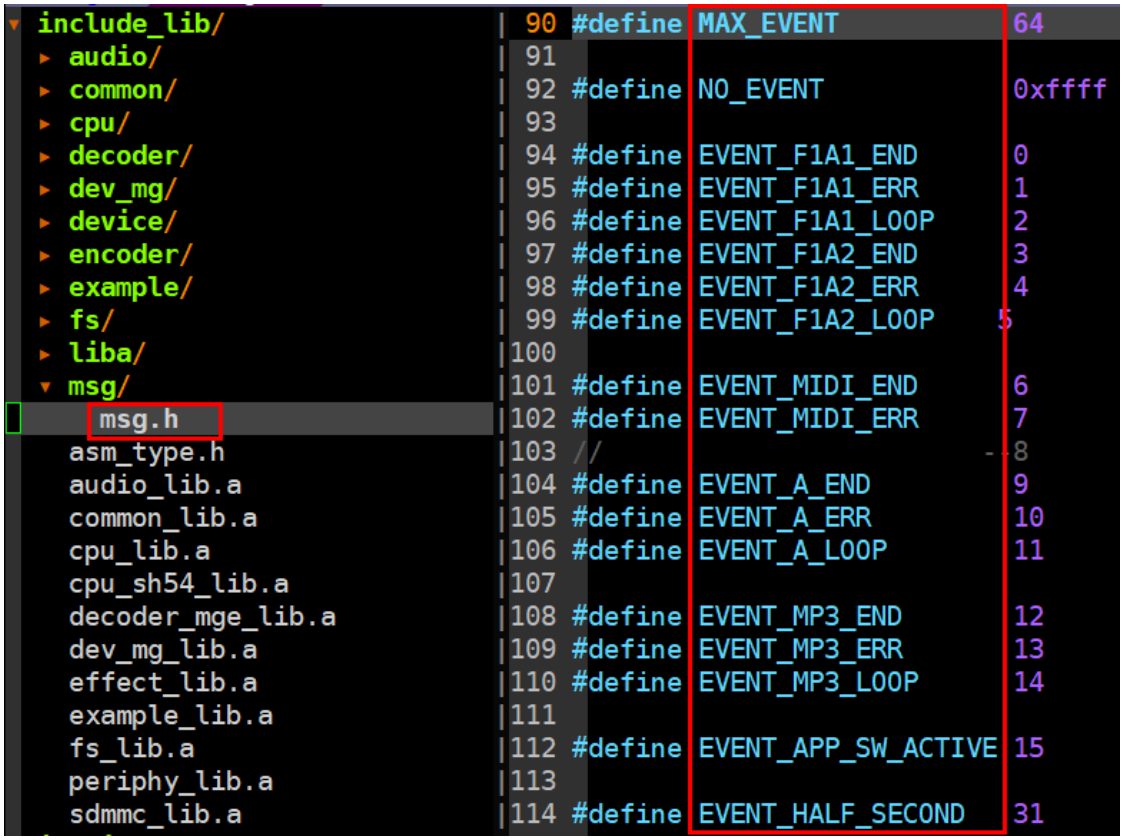
消息与事件

AD140，AD150 支持的消息见图《消息列表图》



消息列表图

AD140，AD150 目前支持的事件见下图 10.2，事件最多支持 64 种，系统已经定义的事件如图《已有事件列表图》。事件发生与不发生的状态由两个 32 位的变量 event_buf_L、event_buf_H 表示，其中变量的每个位表示一个事件的状态（1：事件发生；0：事件未发生）。当某个事件发生时对应的位将被置 1，其它位状态不变。如果用户要增加其它事件可类似定义变量和事件宏，用变量位表示其状态。



已有事件列表图

消息与事件的目录位于：APP/src/common/msg/msg.c、include_lib/msg/msg.h。其中主要的函数有：

void event_init(void);
void message_init();
u32 event2msg_api(u32 event);
int post_event(int event);
int post_msg(int argc, ⋯);
bool get_event_status(u32 event);
int get_msg(int len, int *msg);
void clear_all_message(void);

31.1 1 消息与事件使用一般流程

在开机时先调用 event_init()、message_init() 对消息系统初始化，主要初始化一些消息列表相关变量及事件状态变量。初始化后有用到消息的地方就可以设置消息或获取消息。

a. 开机调用 event_init() 及 message_init() 完成对事件和消息的初始化；

b. 调用 post_event(int event) 标记事件，调用 post_msg(int argc, ⋯) 标记消息；

c. 调用 get_msg(int len, int *msg) 获取事件消息或其他消息；

31.2 2 消息与事件函数

31.2.1 2.1 void event_init(void)

此函数会初始化事件状态信息，将 event_buf_L、event_buf_H 事件状态清空，这一步一般在开机时进行。

31.2.2 2.2 void message_init()

此函数会初始化消息信息，将 msg_cbuf 结构体消息状态清空，这一步一般在开机时进行。

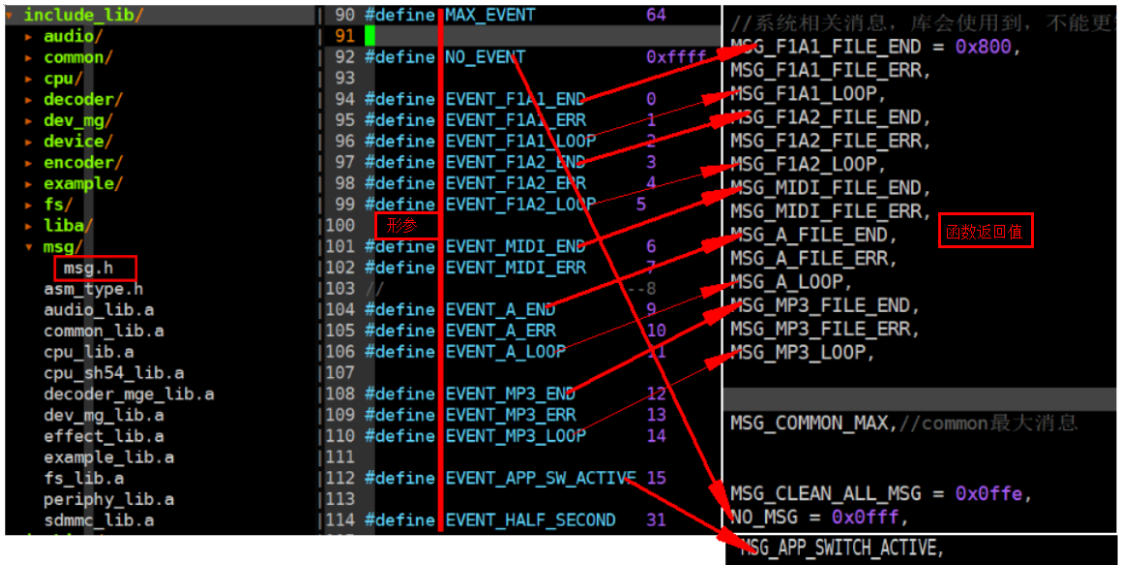
31.2.3 2.3 u32 event2msg_api(u32 event)

此函数实现根据事件 event 信息获取对应的消息信息的功能。

1、event, 事件编号（事件位置见图《事件消息对照表图》形参部分）可选事件:

① EVENT_F1A1_END, F1A1 文件播放结束事件;
② EVENT_F1A1_ERR, F1A1 文件播放错误事件;
③ EVENT_F1A1_LOOP, F1A1 文件播放循环事件;
④ EVENT_F1A2_END, F1A2 文件播放结束事件;
⑤ EVENT_F1A2_ERR, F1A2 文件播放错误事件;
⑥ EVENT_F1A2_LOOP, F1A2 文件播放循环事件;
⑦ EVENT_MIDI_END, MIDI 文件播放结束事件;
⑧ EVENT_MIDI_ERR, MIDI 文件播放错误事件;
⑨ EVENT_A_END, A 文件播放结束事件;
⑩ EVENT_A_ERR, A 文件播放错误事件;
 EVENT_A_LOOP, A 文件播放循环事件;
 EVENT_MP3_END, MP3 文件播放结束事件;
 EVENT_MP3_ERR, MP3 文件播放错误事件;
 EVENT_MP3_LOOP, MP3 文件播放循环事件;
 EVENT_APP_SW_ACTIVE, APP 切换活跃事件;
 NO_EVENT, 无事件发生;

2、返回值: 与事件对应的消息信息；| 事件与消息对照表如下图《事件消息对照表图》



事件消息对照表图

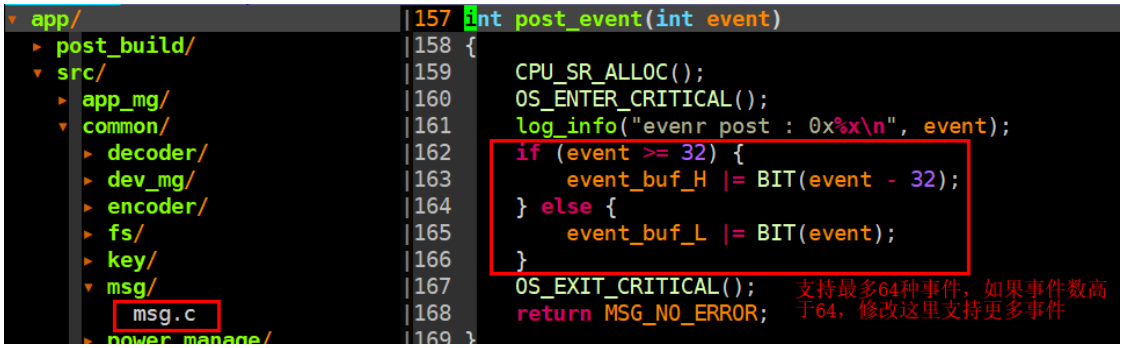
31.2.4 2.4 int post_event(int event)

当某个事件 event 发生时，本函数实现将事件 event 对应的事件状态标号设置为 1。只有事件 event 状态位被置 1 后，消息获取函数 get_msg 才能获取到与事件对应的消息。l 事件最多支持 64 种，系统已经定义的事件如图《已有事件列表图》。事件发生与不发生的状态由两个 32 位的变量 event_buf_L、event_buf_H 表示，其中变量的每个位表示一个事件的状态（1：事件发生；0：事件未发生）。当某个事件 event 发生时对应的位将被置 1，其它位状态不变。如果用户需求事件量高于 64，其事件可类似定义变量和事件宏，用变量位表示其状态, 并修改文件中本函数 post_event、get_event 函数及事件消息对照数组 event2msg[MAX_EVENT] 等使其支持高于 64 种事件的事件。

1、event：事件列表中的某个事件，位置见上图《事件消息对照表图》形参，可选事件：

① EVENT_F1A1_END，F1A1 文件播放结束事件
② EVENT_F1A1_ERR，F1A1 文件播放错误事件
③ EVENT_F1A1_LOOP，F1A1 文件播放循环事件
④ EVENT_F1A2_END，F1A2 文件播放结束事件
⑤ EVENT_F1A2_ERR，F1A2 文件播放错误事件
⑥ EVENT_F1A2_LOOP，F1A2 文件播放循环事件
⑦ EVENT_MIDI_END，MIDI 文件播放结束事件
⑧ EVENT_MIDI_ERR，MIDI 文件播放错误事件
⑨ EVENT_A_END，A 文件播放结束事件
⑩ EVENT_A_ERR，A 文件播放错误事件
 EVENT_A_LOOP，A 文件播放循环事件
 EVENT_MP3_END，MP3 文件播放结束事件
 EVENT_MP3_ERR，MP3 文件播放错误事件
 EVENT_MP3_LOOP，MP3 文件播放循环事件
 EVENT_APP_SW_ACTIVE，APP 切换活跃事件
 NO_EVENT，无事件发生

2、返回值：返回无误信息 MSG_NO_ERROR，返回值无作用；



事件状态改变函数

31.2.5 2.5 int post_msg(int argc, …)

当某个消息发生时，本函数实现将消息放入消息列表。

1、argc：后面可变参数 (…) 的数量；

2、…：可变参数，参数个数可变，可带入一个或多个文件；

可变参数 (…) 有：

(1) MSG_500MS，ticktime 时钟到 500ms 消息
(2) MSG_PRIV_FILE，播放前一个文件消息
(3) MSG_NEXT_FILE，播放下一个文件消息
(4) MSG_NEXT_DIR，播放下一个文件夹消息
(5) MSG_PLAY_FILE1，播放文件 1 消息
(6) MSG_PLAY_FILE2，播放文件 2 消息
(7) MSG_PP，播放文件 1 暂停消息
(8) MSG_PP_2，播放文件 2 暂停消息
(9) MSG_VOL_UP，播放声音增大消息
(10) MSG_VOL_DOWN，播放声音减小消息
(11) MSG_A_PLAY，播放文件 A 消息
(12) MSG_REC_MODE_SWITCH，开始录音与开始播放录音切换消息
(13) MSG_REC_SPEED_EN，录音速率模式使能消息
(14) MSG_MIDI_MODE_SWITCH，midi 模式切换消息
(15) MSG_MIDI_OKON_GOON，midi one key one note mode 下一音节消息
(16) MSG_NEXT_MODE，app 模式切换消息
(17) MSG_RECODE_START，重新编码开始消息
(18) MSG_RECODE_END，重新编码结束消息
(19) MSG_LOW_POWER，低电消息
(20) MSG_ENTER_IDLE，cpu 进入空闲消息
(21) MSG_POWER_OFF，关机消息
(22) MSG_CLEAN_ALL_MSG，清空所有消息的消息

3、返回值：MSG_NO_ERROR 无实际作用；

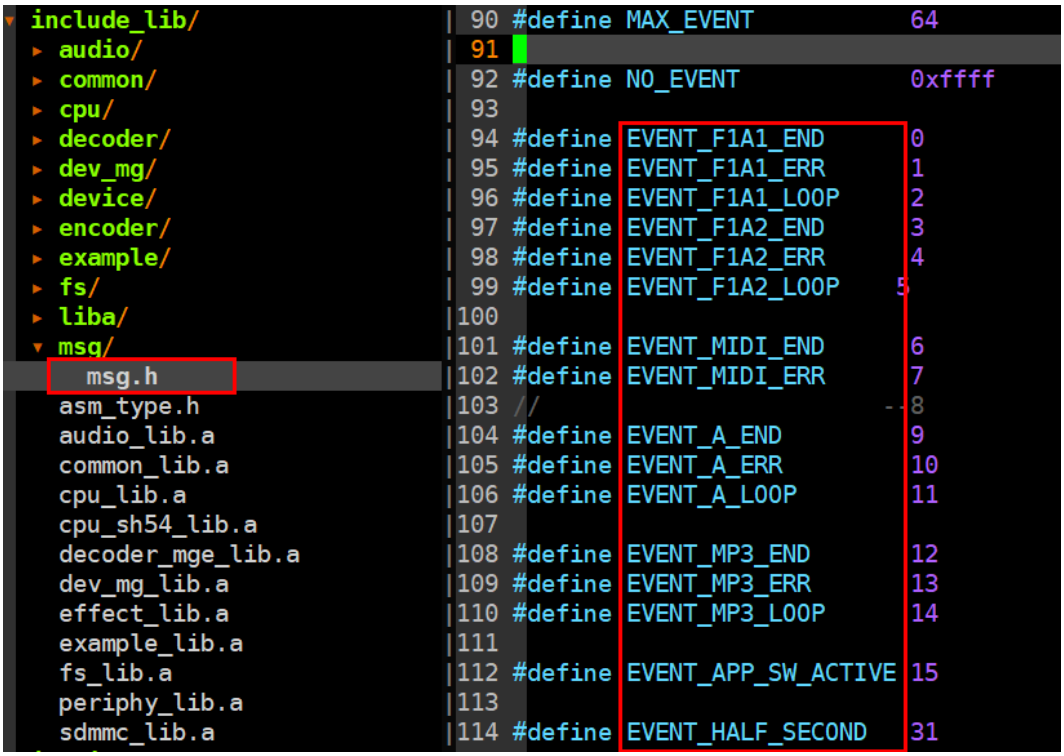
31.2.6 2.6 bool get_event_status(u32 event)

本函数实现获取事件 event 状态的功能。1、event: 事件序号, 位置见图《事件宏列表图》, 可选事件:

① EVENT_F1A1_END, F1A1 文件播放结束事件
② EVENT_F1A1_ERR，F1A1 文件播放错误事件
③ EVENT_F1A1_LOOP，F1A1 文件播放循环事件
④ EVENT_F1A2_END，F1A2 文件播放结束事件
⑤ EVENT_F1A2_ERR，F1A2 文件播放错误事件
⑥ EVENT_F1A2_LOOP，F1A2 文件播放循环事件
⑦ EVENT_MIDI_END，MIDI 文件播放结束事件
⑧ EVENT_MIDI_ERR，MIDI 文件播放错误事件
⑨ EVENT_A_END，A 文件播放结束事件
⑩ EVENT_A_ERR，A 文件播放错误事件
 EVENT_A_LOOP，A 文件播放循环事件
 EVENT_MP3_END，MP3 文件播放结束事件
 EVENT_MP3_ERR，MP3 文件播放错误事件
 EVENT_MP3_LOOP，MP3 文件播放循环事件
 EVENT_APP_SW_ACTIVE，APP 切换活跃事件
 NO_EVENT，无事件发生

2、返回值: 1：事件进行中; 0：事件未执行

可选事件 event 列表:



事件宏列表图

31.2.7 2.7 函数 int get_msg(int len, int *msg)

本函数实现获取事件对应的消息信息以及消息列表中消息的功能当获取到事件消息时，不在获取消息列表中消息，当没有事件消息时，查询消息列表，如果有消息，这保存消息信息到形参 msg。形参 msg 需要自定义。

1、len：形参 msg 的长度，长度由实际要获取的消息长度决定，msg 长度小于要获取的消息长度时返回 MSG_BUF_NOT_ENOUGH 错误信息；
2、msg：保存获取到的消息信息（可获取到的消息如图图 10.1 消息列表图）；
3、返回值：

MSG_NO_ERROR：获取消息无误
MSG_BUF_NOT_ENOUGH：形参 len 长度小于实际消息长度

可获取到的消息如图《消息列表图》

31.2.8 2.8 函数 void clear_all_message(void)

本函数实现清除所有消息的功能，清除后消息列表为空，没有消息可获取。

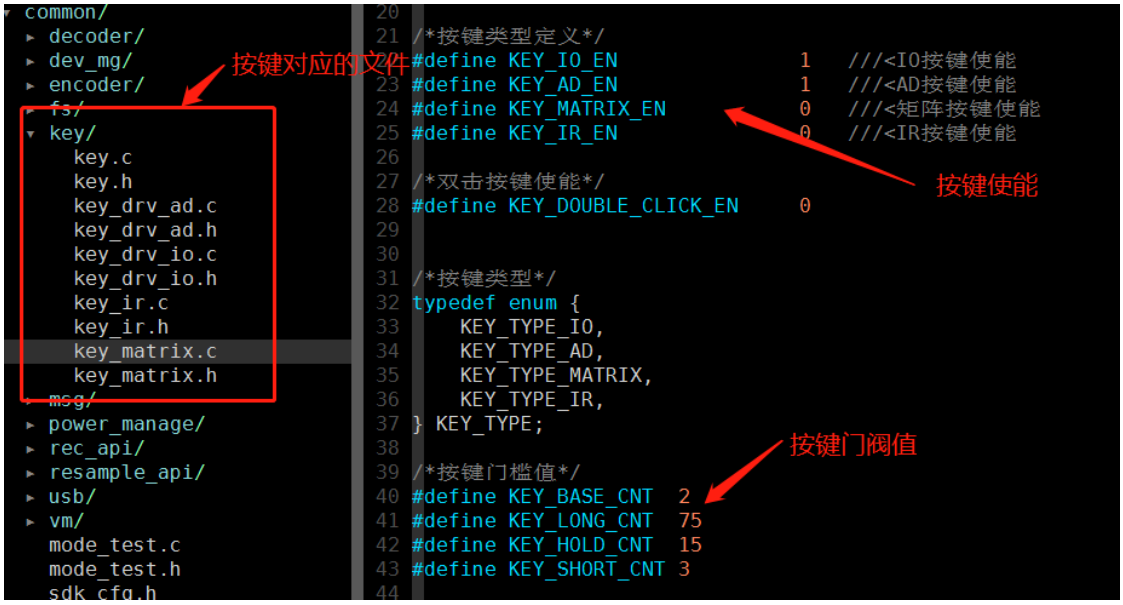
CHAPTER 32

按键系统 key

按键系统章节按照下面两个部分进行介绍：

1. 按键类型介绍
2. 按键接口介绍：

SDK 提供了系统按键 key 来进行人机交互。
SDK 包含的按键类型包括有 IO 按键、AD 按键、IR 按键、TOUCH 按键和 MATRIX 按键 5 类。
用户可以在 **key.h** 里选择单独开其中 1 类按键，也可以选择同时开启多类按键。按键可以调按键门阀值。



按键基本配置

32.1 按键类型介绍

SDK 包含的按键类型共有 5 类，分别为：

1. *IO* 按键
2. *AD* 按键
3. *IR* 按键
4. *touch* 按键：
5. *MATRIX* 按键

32.1.1 IO 按键

IO 按键的基本配置在“key_drv_io.h”文件中。
IO 按键需要配置选择的 IO 口和 IO 是否需要内部上下拉。

小技巧： SDK 中配置了 3 个 IO 按键，其配置如下：

```
#define IS_KEY0_DOWN()      (!(JL_PORTA->IN & BIT(7)))
#define IS_KEY1_DOWN()      (!(JL_PORTA->IN & BIT(8)))
#define IS_KEY2_DOWN()      (!(JL_PORTA->IN & BIT(9)))

#define KEY_INIT()          do{\
                              /**key0 init*/\
                              JL_PORTA->PU |= BIT(7),\
                              JL_PORTA->PD &= ~BIT(7),\
                              JL_PORTA->DIR |= BIT(7);\
                              /**key1 init*/\
                              JL_PORTA->PU |= BIT(8),\
                              JL_PORTA->PD &= ~BIT(8),\
                              JL_PORTA->DIR |= BIT(8);\
                              /**key2 init*/\
                              JL_PORTA->PU |= BIT(9),\
                              JL_PORTA->PD &= ~BIT(9),\
                              JL_PORTA->DIR |= BIT(9);\
                              }while(0)
```

IO 按键配置

32.1.2 AD 按键

AD 按键的基本配置在“key_drv_ad.h”中。

AD 按键需要配置选择的 ADC 通道和外上拉电阻的阻值，如果外上拉电阻选择阻值为 0，即使选择内上拉电阻。

小技巧： SDK 中配置了 1 个 AD 按键，其配置如下：

```
#define AD_KEY_CH       ADC_CH_PA0
#define EXTERN_R_UP     100//220 -> 22k,外挂上拉电阻,0使用内部上拉,内部上拉为10k

extern const key_interface_t key_ad_info;
```

AD 按键配置

AD 按键的键值基本配置在“key_drv_ad.c”中。可根据按键连接电阻阻值配置键值。

```
#if EXTERN_R_UP
#define R_UP        EXTERN_R_UP//220     //22K
#else
#define R_UP        100    //内部上拉为10K
#endif

#define ADC10_33    (0x3ffL)
#define ADC10_30    (0x3ffL*1000/(1000 + R_UP))    //100K
#define ADC10_27    (0x3ffL*510 /(510  + R_UP))    //51K
#define ADC10_23    (0x3ffL*240 /(240  + R_UP))    //24K
#define ADC10_20    (0x3ffL*150 /(150  + R_UP))    //15K
#define ADC10_17    (0x3ffL*100 /(100  + R_UP))    //10K
#define ADC10_13    (0x3ffL*68  /(68   + R_UP))    //6.8K
#define ADC10_10    (0x3ffL*47  /(47   + R_UP))    //4.7K
#define ADC10_07    (0x3ffL*22  /(22   + R_UP))    //2.2K
#define ADC10_04    (0x3ffL*10  /(10   + R_UP))    //1K
#define ADC10_00    (0)

#define AD_NOKEY        ((ADC10_33 + ADC10_30)/2)
#define ADKEY1_0        ((ADC10_30 + ADC10_27)/2)
#define ADKEY1_1        ((ADC10_27 + ADC10_23)/2)
#define ADKEY1_2        ((ADC10_23 + ADC10_20)/2)
#define ADKEY1_3        ((ADC10_20 + ADC10_17)/2)
#define ADKEY1_4        ((ADC10_17 + ADC10_13)/2)
#define ADKEY1_5        ((ADC10_13 + ADC10_10)/2)
#define ADKEY1_6        ((ADC10_10 + ADC10_07)/2)
#define ADKEY1_7        ((ADC10_07 + ADC10_04)/2)
#define ADKEY1_8        ((ADC10_04 + ADC10_00)/2)

const u16 ad_key_table[] = {
    ADKEY1_8, ADKEY1_7, ADKEY1_6, ADKEY1_5, ADKEY1_4,
    ADKEY1_3, ADKEY1_2, ADKEY1_1, ADKEY1_0
};
#endif
```

AD 按键键值配置

32.1.3 IR 按键

IR 按键的基本配置在“key_ir.h”中。
IR 按键需要配置 IR 引脚和 TIMER 源。

小技巧： SDK 中配置了 1 个 IR 按键，其配置如下：

```
#include "gpio.h"
#include "irflt.h"

#define IR_KEY_IO           IR_KEY_IO_SEL
#define IR_KEY_TIMER        SEL_TIMER2

extern const key_interface_t key_ir_info;
#endif
```

IR 按键配置

32.1.4 MATRIX 按键

MATRIX 按键的基本配置在“ke_matrix.h”中。
MATRIX 按键需要配置 X 轴的 ADC 通道和 Y 轴的 IO 口，并且需要配置 ADC 通道检测到有按键的门阀值。

```
//X轴 io 要求是AD口，详细AD口看adc_drv.h
#define X_ADC_CH_INIT {ADC_CH_PA2,ADC_CH_PA3,ADC_CH_PA5}
//Y轴 io 要求是普通IO口
#define Y_PORT_INIT {IO_PORTA_10,IO_PORTA_11}

#define MATRIX_KEY_THRESHOLD        300
```

MATRIX 按键配置

32.1.5 touch 按键：

触摸按键识别的原理：

触摸按键引脚，其网络对地存在寄生电容。
当人手接触网络时，寄生电容会变大。
常态下，引脚为输出，并输出高，保证寄生电容满电。
当引脚突然设为输入，关上拉，开下拉，则寄生电容的电会从下拉电阻放电，当程序读引脚电平为 0 时，则表示放电结束。
寄生电容越大则放电时间就越久（如人手触摸该引脚，相当于寄生电容增大）。
此时我们可以通过一个计数模块，来计算放电时间，通过时间的变化量来判断是否有人手触摸。

SDK 参数配置：

触摸按键共关心三个参数：引脚，标定周期，变化阈值。通过 key_touch.h 修改。

```
#ifndef _KEY_TOUCH_H
#define _KEY_TOUCH_H

#include "key.h"
#include "gpio.h"

#define TOUCH_KEY_INIT                  TOUCH_KEY_SEL

//滤波算法用到
#define TOUCH_VAL_CALIBRATE_CYCLE       100     //标定常态值的时间，单位为key_scand的时间
#define TOUCH_DELDA                     1000      //手按下时变化量的阈值，大于阈值说明被按下

extern const key_interface_t key_touch_info;

#endif
```

key_touch.h

TOUCH_KEY_SEL //触摸用到的引脚自行定义

```
//TOUCH KEY
#define TOUCH_KEY_SEL               {IO_PORTA_09,IO_PORTA_10,IO_PORTB_03}
```

触摸按键 IO 配置

小技巧:

TOUCH_VAL_CALIBRATE_CYCLE

//标定常态值的时间，单位为 key_scand 的时间，默认配置是 1s 左右。
//这是 SDK 参考的标定常态值的算法用到，如果用户有自己的一套算法，该参数可以不用理会。

TOUCH_DELDA

//手按下时变化量的阈值，大于阈值说明被按下。
//这个阈值是要调试出来的，先给的很大的值，保证不触发按键。下文会告诉怎么调试这个值。当然，这是 SDK 参考的处理变化量的的算法用到，如果用户有自己的一套算法，该参数可以不用理会。

触摸按键调试

key.h 文件使能触摸按键：

```
typedef int (*key_emit_t)(u8 key_status, u8 key_num, u8 key_type);

/*按键类型定义*/
#define KEY_IO_EN               0   ///<IO按键使能
#define KEY_AD_EN               0   ///<AD按键使能
#define KEY_MATRIX_EN           0   ///<矩阵按键使能
#define KEY_IR_EN               0   ///<IR按键使能
#define KEY_TOUCH_EN            1   ///<触摸按键使能

/*双击按键使能*/
```

1

key_touch.c 文件打开按键扫描的打印：

```
/*-----------------------------------------------------------------------------*/
static u8 get_touch_key_value(void)
{
    u16 key_or = 0;
    u32 cur_val = 0;
    for (u8 i = 0; i < TOUCH_KEY_NUM; i ++) {
        cur_val = get_pl_cnt_value(i);
        log_print("%d ", cur_val);
        //简单滤波
        if (touch_pre_value[i] == 0) {
            touch_pre_value[i] = cur_val;
        } else if (cur_val >= touch_pre_value[i]) {
            touch_pre_value[i] = touch_pre_value[i] + (u32)((cur_val - touch_pre_value[i]) * 0.2f);
        } else {
            touch_pre_value[i] = touch_pre_value[i] - (u32)((touch_pre_value[i] - cur_val) * 0.2f);
        }
        //处理滤波之后的值
        if (touch_pre_value[i] > (touch_normal_value[i] + TOUCH_DELDA)) {
            key_or |= BIT(i);
            touch_calibrate_cnt[i] = 0;
        } else {
            touch_calibrate_cnt[i] ++;
        }
        //定期标定常态下的基准值
        if (touch_calibrate_cnt[i] > TOUCH_VAL_CALIBRATE_CYCLE) {
            touch_normal_value[i] = touch_calibrate_tmp_value[i] / 10;
            touch_calibrate_tmp_value[i] = 0;
            touch_calibrate_cnt[i] = 0;
        } else if (touch_calibrate_cnt[i] >= (TOUCH_VAL_CALIBRATE_CYCLE / 2)) {
            if (touch_calibrate_cnt[i] < ((TOUCH_VAL_CALIBRATE_CYCLE / 2) + 10)) {
                touch_calibrate_tmp_value[i] += touch_pre_value[i];
            }
        } else {
            touch_calibrate_tmp_value[i] = 0;
        }
    }
    log_print("\n");
    static u8 pre_i = 0;
```

打开打印

下载程序，可以看到，按键的常态下的计数值，参考如下：

```
186 186 186
186 190 186
186 186 186
186 190 134
186 186 186
186 190 186
186 186 186
186 186 186
186 190 186
186 186 186
186 190 186
186 186 186
186 190 186
186 186 186
186 190 186
```

第一列是按键0

第二列是按键1

第三列是按键2

当按键被手触摸时，计数值会变大，参考如下：

```
288 186 3157
186 186 3157
186 186 3320
186 186 3186
186 186 3354
186 186 3176
186 186 3339
186 186 3176
186 186 3320
186 186 3258
186 186 3301
```

可以看出变化量有到 3000 左右，那么此时，我们可以将变化量的阈值设为 1000 左右就行，回到 key_touch.h 文件修改阈值：

```
#define TOUCH_DELDA                    1000
```

触摸按键的数据处理

SDK 提供的是参考的示例，其中包括简单的滤波，滤波后的值的处理，常态值的标定，这些用户都可以自己修改，或者完全重写重新自定义。
主要涉及 pl_cnt.c 、pl_cnt.h 、key_touch.c 、key_touch.h 文件

32.2 按键接口介绍：

32.2.1 函数 void key_init(void)

此函数根据 key 的使能类型来初始化不同的 key。

32.2.2 函数 void key_table_sel(void *msg_filter)

此函数用来注册一个 msg_filter 函数，来获取 key 事件对应的 msg。| 1、msg_filter：key 事件转换成消息的函数接口。

32.2.3 函数 void key_scan(void)

此函数在 tick_timer 中断中调用，用来扫描处理 key 事件。

CHAPTER 33

Reserved_Area 区域说明

AD14N、AD15N、AC104N 可以选择 FLASH 最后的 4K 作为一个特殊的保留区，这个保留区只有使用烧写器烧写程序才能进行擦写动作。一些希望烧写完程序后就不经常给擦写的信息可以选择放到这个区域里面。
使用这个保留区需要在 isd_config.ini 文件中把 NEED_RESERVED_4K 置成 YES。

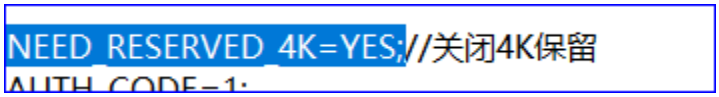


图 1 isd_config 中配置保留区

33.1 1 认证码使用说明

认证码支持的一拖二烧写器最低版本号为：2.24.4
认证码是一个烧写在保留区中的一个信息，用来给每一个产品烧写一个特殊的 ID 使用。
认证码使用的时候需要在 isd_config.ini 文件 [SYS_CFG_PARAM] 段中的如图位置，增加一个 AUTH_CODE=1; 配置项。

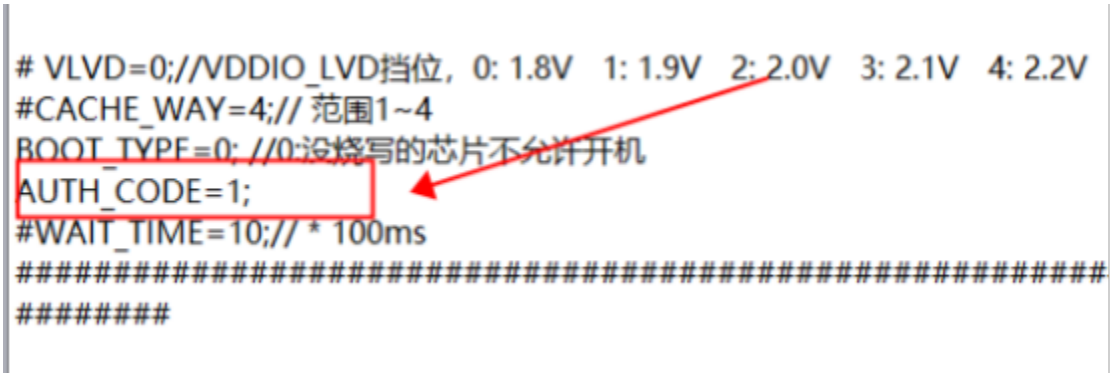


图 2 isd_config 文件 [SYS_CFG_PARAM] 段增加 AUTH 配置项

33.2 2 认证码文件说明

认证码文件的格式是：认证码格式头 + 数据。

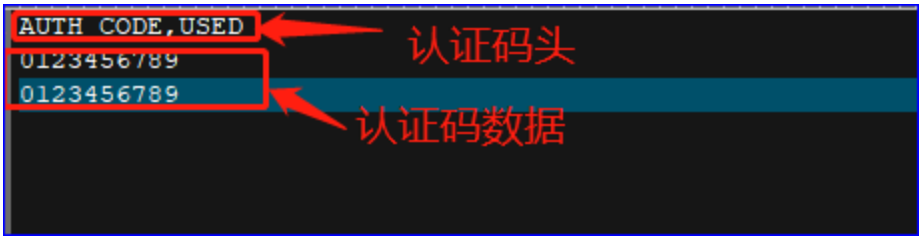


图 3 认证码文件格式

认证码的数据烧写到固件的时候会有 2 种格式

```
当文件的后缀名为“\*.csv”时候，数据格式按照ASCII方式读取烧写。
当文件的后缀名为“\*.hex”时候，数据格式按照HEX方式读取烧写。
```

CHAPTER 34

多录音分区（AD14N）

AD14N 目前支持多录音分区功能，能在各个分区中独立的进行录音操作。本章着重介绍录音的多分区功能实现流程。将会涉及到 app_record.c 以及 nor_fs_resource.c。

34.1 1 多录音分区一般流程

在开机初始化中会调用 void norfs_init_api(void)，此函数实现对录音区域的初始化，会初始化录音的起始地址、结束地址以及字节大小等变量。在分区前需要确保有足够的空间能够存放至少 256 字节。此流程中以将原本的录音地址一分为二，实现双录音分区为例。

34.1.1 1.1 创建录音分区初始化

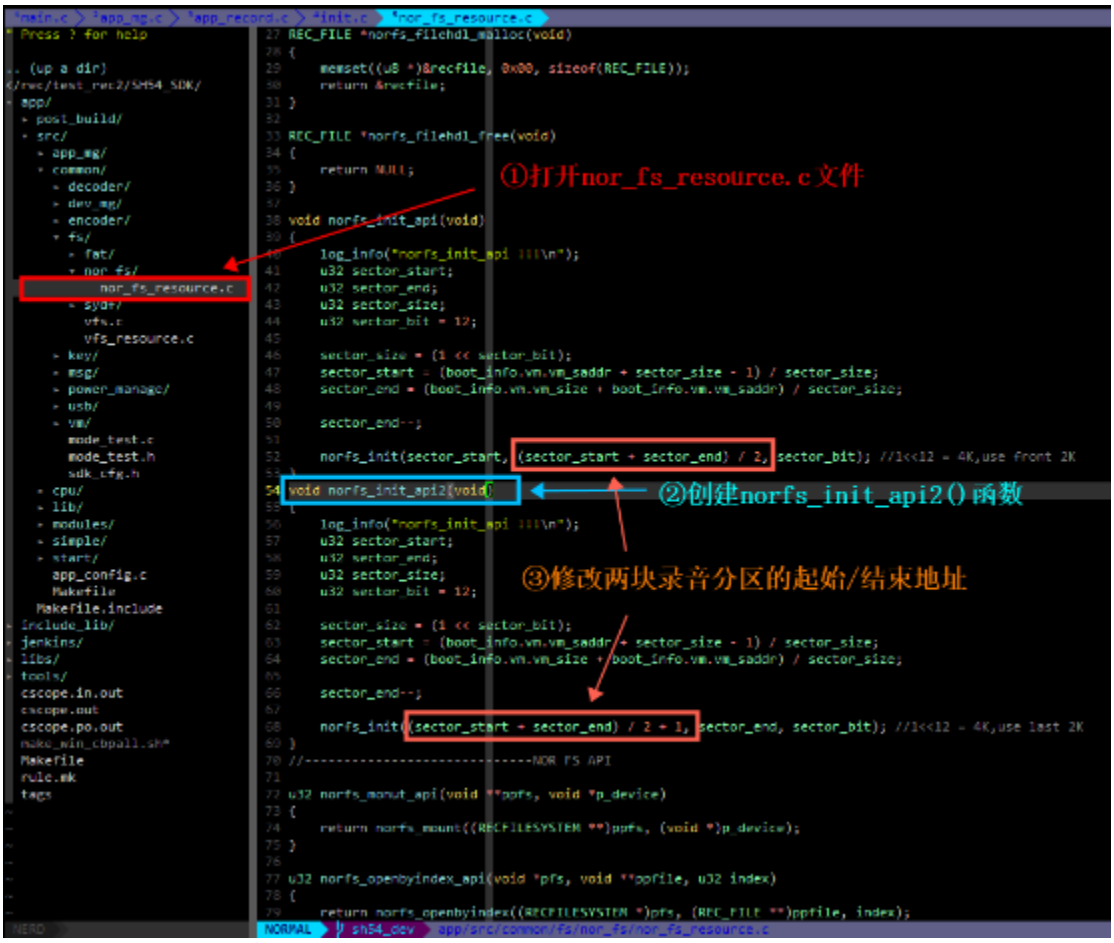


图 1 创建录音分区初始化

创建双录音分区初始化步骤：

```
① 打开nor_fs_resource.c文件；
② 仿照函数norfs_init_api()创建录音分区2的初始化函数norfs_init_api2()；
③ 修改录音分区1函数norfs_init_api()中，norfs_init()传入的结束地址。本章中将其改为原来长度的一半；
④ 修改录音分区2函数norfs_init_api2()中，norfs_init()传入的起始地址。本章中将其改为同录音分区1同长度；
```

详情步骤可参考图 1。同理，多录音分区也可根据上述步骤修改进行录音分区初始化。

若不选择默认录音地址，可在 isd_config.ini 文件中自行开辟地址。

34.1.2 1.2 创建录音切换分区消息事件并设置键位

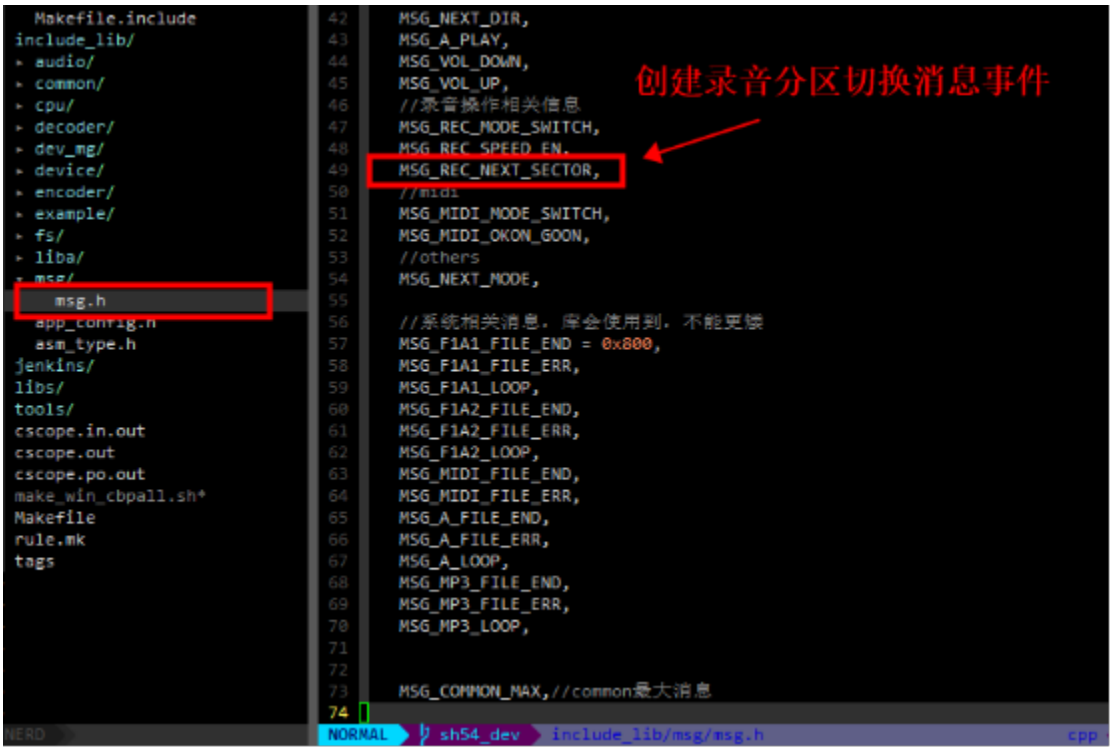


图 2 创建录音切换分区消息事件



图 3 设置录音切换分区键位

创建录音切换分区消息事件并设置键位步骤：

① 打开msg.h文件；
② 如图2，在录音操作相关信息处添加录音切换分区消息事件 MSG_REC_NEXT_

(续下页)

(接上页)

```
↪SECTOR;
③  打开app_record_key.c文件;
④  如图3, 在IOKEY_RECORD_SHORT_UP定义中添加IOKEY 3作为切换录    音分区按键;
```

可自行命名切换消息事件和选择切换的键位。

34.1.3 1.3 在 app 录音任务中实现录音分区切换操作



图 4 在 app 录音任务中实现录音分区切换操作

在 app 录音任务中实现录音分区切换操作步骤:

```
①  打开nor_fs_resource.c文件;
②  创建录音分区切换标志norfs_sector_flag;
③  关闭正在编码或者正在播放的录音任务, 防止任务干扰分区的切换;
④  根据标志选择初始化的录音分区;
    a.标志为0时, 切换至分区2, 同时将标志置1;
    b.标志为1时, 切换至分区1, 同时将标志置0;
⑤  播放所在分区最新的录音文件;
    a.初始化解码器;
    b.打开录音设备;
    c.进行录音解码, 解码会自动选择该分区最新的文件进行播放;
```

详情步骤可参考图 4。同理，多录音分区也可根据上述步骤，通过增加标志的置来选择多录音分区切换。

CHAPTER 35

虚拟 RTC

AD150 v1.7.0 版本 SDK 提供了虚拟 RTC 功能示例，其功能包含读写时钟、闹钟，可在 powerdown、softoff 情况下走时并且响应闹钟。使用时需要开启 app_modules.h 中的 rtc 功能宏，相关应用示例位于 app/bsp/common/rtc/rtc.c。

注：虚拟 rtc 在 powerdown、softoff 情况下运行会六个小时自动唤醒一次，否则时间会不准！

35.1 功能接口介绍

35.1.1 int rtc_simulate_init(const struct vir_rtc_data *arg)

该函数主要实现虚拟 rtc 功能初始化。其中参数：

```
1、vir_rtc_data：初始化的时钟、闹钟和回调信息；
```

35.1.2 void read_sys_time(struct sys_time *curr_time);

该函数实现读取当前 rtc 运行的时钟。其中参数：

```
1、curr_time：读取到的时钟信息，函数会将读取到的时钟更新到该结构体；
```

35.1.3 void write_sys_time(struct sys_time *curr_time);

该函数实现写入时钟。其中参数：

```
1、curr_time：读取到的时钟信息，函数会将读取到的时钟更新到该结构体；
```

35.1.4 void read_alarm(struct sys_time *alarm_time);

该函数实现读取闹钟。其中参数：

```
1、alarm_time：读取到的闹钟信息，函数会将读取到的时钟更新到该结构体；
```

35.1.5 void write_alarm(struct sys_time *alarm_time);

该函数实现写入闹钟。其中参数：

```
1、alarm_time：写入的闹钟信息；
```

35.1.6 void vir_set_alarm_ctrl(u8 set_alarm);

该函数实现设置闹钟使能。其中参数：

```
1、set_alarm：闹钟使能标志；1:闹钟使能， 0：闹钟失能。
```

35.1.7 u8 vir_get_alarm_ctrl(void);

该函数实现获取闹钟使能标志。其中参数：：

```
1、返回值：闹钟使能标志。1:闹钟使能， 0：闹钟失能。
```

CHAPTER 36

双备份升级

双备份升级存储结构有两个 app code 区域，当芯片正在运行 app code1，升级时将新固件写入 app code 2 区域，反之亦然。升级时对 app code N 区域数据校验成功后，复位即可运行 app code N 代码。当升级中断或者新写入固件校验不成功，不会影响原来 app code 的运行

杰理双备份升级有两种方式实现

1. 设备主动升级（由设备端主动向远端获取升级数据，通信协议需要能支持文件内容获取能力）

2. 设备被动升级（由远端主动推送升级数据，设备端被动写入的方式，远端可控制升级流程

36.1 支持双备份升级的芯片以及对应 SDK 版本

支持双备份升级的芯片：AD14/15/16/18/和 AC104

AD14/15/AC104 的 v1.7.0 SDK 和 AD16 v1.3.0SDK 需要添加补丁支持双备份设备被动升级。后续版本 SDK 会直接支持。

36.2 SDK 相关配置

首先需要在 isd_config.ini 加入以下配置:

```
[EXTRA_CFG_PARAM]
BR22_TWS_DB=YES;          //dual bank flash framework enable (only valid for␣
↪dual_bank)
FLASH_SIZE=1M;            //flash_size cfg (only valid for dual_bank)
BR22_TWS_VERSION=0; //default fw version(only valid for dual_bank)
DB_UPDATE_DATA=YES;  //generate db_update_data.bin(only valid for dual_bank)
SPECIAL_OPT=0;       //special cfg : single bin generating(only valid for␣
```

(续下页)

(接上页)

```
↪dual_bank)
FORCE_4K_ALIGN=YES;  // force aligin with 4k bytes(only valid for dual_bank)
#NEW_FLASH_FS=YES;   //enable single bank flash framework (only valid for␣
↪single_bank)
```

另外，AD14/15/16/18 以及 AC104 需要关闭所有写保护，其中软件写保护位于

```
AT_SPI_CODE/*该函数放置段不可更改*/
u32 flash_code_protect_callback(u32 offset, u32 len)
{
    return 0;  返回0即可
    u32 limit_addr = vm_attr.sclust;
    /* log_info("0x%x 0x%x", limit_addr, offset); */
    if ((offset < limit_addr) || ((offset + len) > boot_info.flash_size)) {
        /* 超过正常擦写区域，不进行擦写操作 */
        return 1;
    } else {
        /* 进行擦写操作 */
        return 0;
    }
}
```

图 1 软件写保护

如果升级使用 simple_fat 读取升级文件，需要去掉框内的限制

```
u32 smpl_fat_read_api(void *pfile, void *buff, u32 len)
{
    u32 rlen = f_read(&pfi, buff, len);
    /* if (rlen != len) { */
    /* return 0; */
    /* } */
    return rlen;
}

u32 smpl_fat_seek_api(void *pfile, u32 offset, u32 fromwhere)
{
    return f_seek(&pfi, offset);
}

u32 smpl_fat_file_close_api(void **ppfile)
{
    return (u32) * ppfile;
```

图 2 simple_fat 读函数

设备被动升级采用 db_update_data.bin 作为升级文件，升级文件会在编译下载的时候生成，在 SDK 示例中由于 fat 文件系统有 8+3 的长名限制，名字被缩短为 db_data.bin

36.3 相关接口说明

36.3.1 函数 u32 jlfs_get_idle_bank_info(u32 *bank_addr, u32 *bank_size)

此函数获取 idle_bank 的信息，其中参数：

```
1. bank_addr: 空闲的code区地址，接口内部会更新到指针；
2. bank_size: 空闲的code区大小，接口内部会更新到指针
3. 返回值：
```

(续下页)

(接上页)

```
    成功 返回0;
    失败 其他值;
```

36.3.2 函数 u32 jlfs_updata_dual_bank_info(u32 bank_addr, u16 data_crc)

此函数更新 idle_bank 的信息其中参数:

```
1. bank_addr: 空闲的code区地址;
2. data_crc: 升级文件校验值;
3. 返回值:
    成功 返回0;
    失败 其他值;
```

36.3.3 函数 u32 jlfs_check_dual_bank_info(u32 update_bank_addr)

此函数校验写入升级信息后的 idle_bank 信息, 其中参数:

```
1. update_bank_addr: 升级的code区地址;
2. 返回值:
    成功 返回0;
    失败 其他值;
```

36.4 常见客户问题

1、AD15 双备份升级后，使用强制升级工具升级失败?

AD15 的 download 工具在下载的时候不会去主动擦除 code2 区的代码，所以需要在下载目录的 download_bat.c 加入-format all 全擦除 flash 指令擦除 code2 区域代码，加入指令后需要编译一次，看到图中的全擦除模式生效即为配置成功。

```
打开设备(COM3)成功
SPI nor flash online.
Online flash id: ef4016
Online flash size: 4M
Online external flash id: ef4017
Online external flash size: 8M
Erase Flash Size is 4096
ISDdownload
----------------------------------------FLASH INFO----------------------------------------
|  PID : SH54                                                                             |
|  FLASH_BIN_SIZE : 0x16000                                                               |
|  FLASH_NEED_SIZE : 0x100000                                                             |
|  FLASH_REAL_SIZE : 0x400000                                                             |
|  VM_REAL_SIZE : 0x6000                                                                  |
|  VM_START_ADDR : 0xf4000                                                                |
|  VM_END_ADDR : 0xfa000                                                                  |
|  EEPROM_RESERVED_SIZE : 0x6000                                                          |
|  EEPROM_RESERVED_START : 0xfa000                                                        |
|  EEPROM_RESERVED_END : 0x100000                                                         |
|  CODE_BOUNDARY_LINE : 0x7a000                                                           |
|  LAVE_SIZE : 0x64000                                                                    |
|  ENTRY_ADDR : 0x100100                                                                  |
|  ERASE MODE : all                                                                       |
|  UBOOT VERSION : 2023-06-25 17:46:17,UBOOT-$-@20230625-$1242b31                         |
|  LOADER VERSION : 2023-06-30 20:11:01,LOADER-v1.0.0-$-@20230630-$a806557                |
-------------------------------------------------------------------------------------------
正在擦除FLASH[0x0-0x400000]……
...
.
```

图 3 ad15 编译擦除 flash 示意图

CHAPTER 37

Indices and tables

- genindex
- modindex
- search